
wass2s

Release 0.4.7.4

Mandela C. M. HOUNGNIBO

Jun 14, 2026

GETTING STARTED

1	Key capabilities	3
2	Indices and tables	213
	Python Module Index	215
	Index	217

wass2s is a Python library that streamlines the full pipeline of seasonal climate forecasting over West Africa and the Sahel. Starting from raw satellite, reanalysis, and GCM data, it lets you build, validate, and operationalise statistical and machine-learning-based forecasts in a reproducible way.

The library implements the *new-generation* seasonal forecasting framework promoted by the World Meteorological Organization (WMO), covering everything from predictor preparation to probabilistic tercile-probability maps (Below-Normal / Near-Normal / Above-Normal).

Note

A companion set of [Jupyter notebooks](#) is available for hands-on walkthroughs of each major workflow.

KEY CAPABILITIES

- **Automated data acquisition** — ERA5, CHIRPS, TAMSAT, NMME, C3S GCMs, and AgERA5 agro-indicators in a single unified interface.
- **Agroclimatic predictand computation** — onset, cessation, dry/wet spells, ETCCDI extreme indices, and heat-wave metrics from daily gridded or station data.
- **Bias correction and data merging** — quantile mapping (QUANT, RQUANT, SSPLIN, PTF, DIST) and station-gridded merging (Kriging, Regression Kriging, Neural-Network Kriging, Multiplicative Bias).
- **Statistical and ML models** — OLS, Ridge, Lasso, ElasticNet, MARS, SVR, MLP, stacking ensembles, EOF/PCR, CCA, and analog methods, all sharing a common `compute_model` / `compute_prob` / `forecast` interface.
- **Multi-model ensemble post-processing** — weighted averaging, BMA, NGR, Extended Logistic Regression, Random Forest / XGBoost / ELM / MLP super-ensembles.
- **Leakage-free cross-validation** — a custom leave-one-out splitter with a symmetric exclusion window, wired to every model family automatically.
- **Forecast verification** — deterministic (KGE, Pearson, NSE, RMSE, MAE) and probabilistic (GROC, RPSS, Brier, Ignorance, ROC, Reliability) metrics.

1.1 Installation

wass2s requires Python 3.9 or later. The recommended approach is to install it inside a dedicated conda environment so that all binary dependencies (GDAL, Cartopy, Dask, etc.) are resolved consistently.

1.1.1 1. Create and activate the environment

Pre-built environment files are provided for both platforms.

Linux / macOS

Download the environment file from the repository:

```
wget https://raw.githubusercontent.com/hmandela/WASS2S/main/WAS_S2S_linux.yml
```

Then create and activate the environment:

```
conda env create -f WAS_S2S_linux.yml
conda activate WASS2S
```

Windows

Download the Windows-specific file from the repository ([WAS_S2S_windows.yml](#)) and run:

```
conda env create -f WAS_S2S_windows.yml
conda activate WASS2S
```

1.1.2 2. Install or upgrade wass2s

With the environment active, install the latest stable release from PyPI:

```
pip install wass2s
```

To upgrade an existing installation:

```
pip install --upgrade wass2s
```

1.1.3 3. Configure the CDS API (required for ERA5 and C3S data)

Downloading ERA5 reanalysis and Copernicus C3S seasonal forecast data requires a free account on the [Copernicus Climate Data Store](#).

Once registered, create a credentials file at `~/.cdsapirc`:

```
url: https://cds.climate.copernicus.eu/api/v2
key: <YOUR_UID>:<YOUR_API_KEY>
```

Your UID and API key are available on your CDS profile page. See the [CDS API How-to](#) for step-by-step instructions.

Note

NMME data (NCEP CFS, GFDL, NASA, NCAR, CMC) is downloaded directly from the IRI Data Library via HTTP — no credentials are required for this source.

1.1.4 4. Verify the installation

```
import wass2s
print(wass2s.__version__)
```

1.2 Quick Start

This page walks through a minimal end-to-end forecast in five steps: download data, prepare predictors and predictand, cross-validate a model, verify the hindcast, and produce an operational forecast map.

For more detailed explanations of each step see the [User guide](#).

1.2.1 Prerequisites

Install wass2s and activate your environment (see [Installation](#)) then open a Jupyter notebook or Python script and import the package:

```
from wass2s import *
import numpy as np
```

1.2.2 Step 1 — Download data

```

downloader = WAS_Download()

# ERA5 global SST — predictor window (JFM)
downloader.WAS_Download_Reanalysis(
  dir_to_save="./data/era5/",
  center_variable=["ERA5.SST"],
  year_start=1991, year_end=2025,
  area=[45, -180, -45, 180], # [N, W, S, E]
  seas=["01", "02", "03"],
  force_download=False
)

# AgERA5 precipitation — predictand (JJAS)
downloader.WAS_Download_AgroIndicators(
  dir_to_save="./data/obs/",
  variables=["AGRO.PRCP"],
  year_start=1991, year_end=2024,
  area=[30, -25, 0, 30],
  seas=["06", "07", "08", "09"],
  force_download=False
)

```

1.2.3 Step 2 — Prepare predictors and predictand

```

clim_year_start, clim_year_end = 1991, 2020
year_start, year_end = 1991, 2024

# Observed predictand (T, Y, X)
predictand = prepare_predictand(
  "./data/obs/", ["AGRO.PRCP"],
  year_start, year_end,
  seas=["06", "07", "08", "09"],
  ds=False
)

# SST indices as predictors
sst_index_names = ["NINO34", "TNA", "TSA", "DMI"]
predictors = compute_sst_indices(
  "./data/era5/", sst_index_names,
  "ERA5.SST", year_start, year_end,
  seas=["01", "02", "03"],
  clim_year_start=clim_year_start,
  clim_year_end=clim_year_end
)
predictors = (
  predictors
  .to_array()
  .rename({"variable": "features"})
  .transpose("T", "features")
)

```

1.2.4 Step 3 — Cross-validate a Ridge regression model

```
# Optimise the regularisation parameter
ridge = WAS_Ridge_Model(
    n_clusters=6,
    alpha_range=np.logspace(-4, 0.1, 20),
    nb_cores=4
)
alpha, clusters = ridge.compute_hyperparameters(
    predictand, predictors.isel(T=slice(None, -1)),
    clim_year_start, clim_year_end
)

# Leave-one-out cross-validation
cv = WAS_Cross_Validator(
    n_splits=len(predictand.get_index("T")),
    nb_omit=2
)
hindcast_det, hindcast_prob = cv.cross_validate(
    ridge, predictand,
    predictors.isel(T=slice(None, -1)),
    clim_year_start, clim_year_end,
    alpha=alpha
)
```

1.2.5 Step 4 — Verify the hindcast

```
verifier = WAS_Verification()

pearson = verifier.compute_deterministic_score(
    verifier.pearson_corr, predictand, hindcast_det
)
groc = verifier.compute_probabilistic_score(
    verifier.calculate_groc, predictand, hindcast_prob,
    clim_year_start=clim_year_start,
    clim_year_end=clim_year_end
)

verifier.plot_model_score(pearson, "Pearson",
    dir_save_score="./scores/", figure_name="pearson_ridge")
verifier.plot_model_score(groc, "GROC",
    dir_save_score="./scores/", figure_name="groc_ridge")
```

1.2.6 Step 5 — Operational forecast for the target year

```
# The last time step of predictors is the real-time predictor
forecast_det, forecast_prob = ridge.forecast(
    predictand, clim_year_start, clim_year_end,
    predictors.isel(T=slice(None, -1)),    # hindcast predictors
    hindcast_det,
    predictors.isel(T=[-1]),              # real-time predictor
    alpha=alpha
)
```

(continues on next page)

(continued from previous page)

```

)

# Save and plot
plot_prob_forecasts(
  dir_to_save="./forecasts/",
  forecast_prob=forecast_prob,
  model_name="Ridge",
  title="JJAS 2025 Sahel Rainfall Forecast - Ridge Regression"
)

```

 Tip

Replace `WAS_Ridge_Model` with any other model class — `WAS_PCR`, `WAS_CCA`, `WAS_SVR`, `WAS_MLP`, `WAS_Analog`, or an MME class — without changing the cross-validation or forecasting calls.

1.3 Data Download

The `WAS_Download` class provides a unified interface for retrieving all climate data needed for seasonal forecasting: GCM hindcasts and real-time forecasts, ERA5/ERSST reanalysis, and satellite-based observational products (CHIRPS, TAMSAT, AgERA5).

All methods share the same behaviour:

- They check for an existing cached NetCDF file before triggering any network request. Pass `force_download=True` to override the cache.
- They aggregate monthly fields into seasonal totals internally, including cross-year seasons such as DJF.
- They return files with standardised T, Y (latitude), and X (longitude) dimensions.

```

from wass2s import WAS_Download

downloader = WAS_Download()

```

 Note

C3S data (ECMWF, UKMO, Météo-France, DWD, CMCC, JMA, ECCC) requires a `~/.cdsapirc` credentials file. See [Installation](#) for setup instructions. NMME data is downloaded over HTTP without authentication.

1.3.1 1. Seasonal GCM Forecasts

Monthly GCM Data

Method: `WAS_Download_Models`

Downloads monthly mean hindcasts (1993–2016 by default) or real-time forecasts from C3S and NMME. The method automatically adapts the download protocol (CDS API vs. IRI FTP) based on the requested centre.

Supported centres and their identifiers: .. list-table:: Supported GCM centres

header-rows

1

widths

30 70

- – Source
 - Centre identifiers
- – C3S / ECMWF
 - ECMWF_51
- – C3S / UK Met Office
 - UKMO_603, UKMO_604, UKMO_605, UKMO_610
- – C3S / Météo-France
 - METEOFRACTANCE_8, METEOFRACTANCE_9
- – C3S / DWD
 - DWD_21, DWD_22
- – C3S / CMCC
 - CMCC_35, CMCC_4
- – C3S / JMA
 - JMA_3, JMA_4
- – C3S / ECCC
 - ECCC_4, ECCC_5
- – NMME / NCEP CFS
 - NCEP_2, CFSV2_1
- – NMME
 - CMC1_1, CMC2_1, GFDL_1, NASA_1, NCAR_CCSM4_1, NCAR_CESM1_1
- – BOM
 - BOM_2

Available variables: PRCP, TEMP, TMAX, TMIN, SST, SLP, UGRD10, VGRD10, DSWR, DLWR, NOLR.

Pressure-level wind and humidity are also available: UGRD_850, VGRD_850, HUSS_850, and equivalents at 925 hPa and 1000 hPa.

```
# Download ECMWF and NCEP precipitation hindcasts (May initialisation, JAS_
→target)
downloader.WAS_Download_Models(
  dir_to_save="./data/GCM/",
  center_variable=["ECMWF_51.PRCP", "NCEP_2.PRCP"],
  month_of_initialization=5,
  lead_time=["01", "02", "03"],           # June, July, August
  year_start_hindcast=1993,
  year_end_hindcast=2016,
  area=[20, -20, 0, 10],                 # [N, W, S, E]
  ensemble_mean="mean"
```

(continues on next page)

(continued from previous page)

```

)

# Download the 2025 real-time forecast
downloader.WAS_Download_Models(
  dir_to_save="./data/GCM/",
  center_variable=["ECMWF_51.PRCP"],
  month_of_initialization=5,
  lead_time=["01", "02", "03"],
  year_start_hindcast=1993,
  year_end_hindcast=2016,
  area=[20, -20, 0, 10],
  year_forecast=2025,
  ensemble_mean="mean"
)

```

Daily GCM Data

Method: WAS_Download_Models_Daily

Downloads daily or sub-daily data for applications such as heat-wave or dry-spell analysis. Daily data volumes are large; restrict the area and lead-time range accordingly.

```

downloader.WAS_Download_Models_Daily(
  dir_to_save="./data/GCM_Daily/",
  center_variable=["ECMWF_51.TMAX"],
  month_of_initialization=5,
  day_of_initialization=1,
  leadtime_hour=["24", "48", "72", "96"],
  year_start_hindcast=2000,
  year_end_hindcast=2020,
  area=[20, -20, 0, 10]
)

```

1.3.2 2. Reanalysis Data

ERA5 and ERSST

Method: WAS_Download_Reanalysis

Downloads monthly means from ERA5 (atmosphere and ocean surface) or NOAA ERSSTv5. Cross-year seasons are handled automatically: requesting NDJ (November–December–January) downloads the correct months from adjacent calendar years and aggregates them.

```

# Global SST for predictor indices (NDJ season)
downloader.WAS_Download_Reanalysis(
  dir_to_save="./data/Reanalysis/",
  center_variable=["ERA5.SST"],
  year_start=1981,
  year_end=2024,
  area=[45, -180, -45, 180],
  seas=["11", "12", "01"],

```

(continues on next page)

(continued from previous page)

```
        force_download=False
    )

    # NOAA ERSST for Niño 3.4 calculation
    downloader.WAS_Download_Reanalysis(
        dir_to_save="./data/Reanalysis/",
        center_variable=["NOAA.SST"],
        year_start=1981,
        year_end=2024,
        area=[5, -170, -5, -120],
        seas=["11", "12", "01"]
    )
```

ERA5-Land

Methods: WAS_Download_ERA5Land (monthly) and WAS_Download_ERA5Land_daily

ERA5-Land provides higher-resolution (~9 km) land surface variables, making it particularly suited to hydrological applications.

```
# Monthly ERA5-Land soil moisture (JJA)
downloader.WAS_Download_ERA5Land(
    dir_to_save="./data/Reanalysis/",
    center_variable=["ERA5Land.SOILWATER1"],
    year_start=1981,
    year_end=2024,
    area=[15, -18, 4, 10],
    seas=["06", "07", "08"]
)
```

1.3.3 3. Observational Data

AgERA5 Agro-Climatic Indicators

Method: WAS_Download_AgroIndicators

AgERA5 is a bias-adjusted product derived from ERA5 and corrected against station observations. It provides consistent daily time series of variables widely used for agroclimatic index computation.

Available variables: AGRO.PRCP, AGRO.TMAX, AGRO.TMIN, AGRO.DSWR (solar radiation), AGRO.ETP (reference evapotranspiration).

```
downloader.WAS_Download_AgroIndicators(
    dir_to_save="./data/Obs/",
    variables=["AGRO.PRCP"],
    year_start=1991,
    year_end=2024,
    area=[30, -25, 0, 30],
    seas=["06", "07", "08", "09"],
    force_download=False
)
```

CHIRPS Precipitation

Methods: WAS_Download_CHIRPSv3_Seasonal and WAS_Download_CHIRPSv3_Daily

CHIRPS v3 provides high-resolution (0.05°) precipitation estimates. The seasonal method aggregates monthly totals; the daily method downloads individual files.

```
# CHIRPS seasonal totals (JJAS)
downloader.WAS_Download_CHIRPSv3_Seasonal(
  dir_to_save="./data/Obs/",
  variables=["PRCP"],
  year_start=1981,
  year_end=2024,
  region="africa",
  season_months=["06", "07", "08", "09"],
  area=[20, -20, 0, 15]
)

# CHIRPS daily files
downloader.WAS_Download_CHIRPSv3_Daily(
  dir_to_save="./data/Obs_Daily/",
  variables=["PRCP"],
  year_start=2020,
  year_end=2024,
  area=[20, -20, 0, 15]
)
```

TAMSAT Precipitation

Methods: WAS_Download_TAMSAT_Seasonal and WAS_Download_TAMSAT_Daily

TAMSAT provides African rainfall estimates (RFE) and soil moisture from the University of Reading.

```
# TAMSAT seasonal rainfall
downloader.WAS_Download_TAMSAT_Seasonal(
  dir_to_save="./data/Obs/",
  product="rfe",
  year_start=1983,
  year_end=2024,
  area=[20, -20, 0, 15],
  season_months=["06", "07", "08", "09"]
)

# TAMSAT daily rainfall
downloader.WAS_Download_TAMSAT_Daily(
  dir_to_save="./data/Obs_Daily/",
  product="rfe",
  year_start=2023,
  year_end=2023,
  area=[20, -20, 0, 15]
)
```

1.4 Preprocessing

Before building forecasting models, raw data must go through several preprocessing stages:

1. *Computing Agroclimatic Predictands* — compute agroclimatic indices (onset, cessation, dry/wet spells, ETCCDI extremes) from daily data.
2. *Merging Gridded Data with Station Observations* — blend coarse gridded estimates with station observations.
3. *Bias Correction* — correct systematic biases in GCM or reanalysis output.
4. *Data Transformation and Distribution Fitting* — handle skewed distributions and fit parametric distributions for probabilistic post-processing.

1.4.1 Input data format conventions

CDT — daily station data

The Climate Data Tools (CDT) format is used as input to all `compute_insitu` methods:

ID	ALLADA	APLAHOUE
LON	2.133333	1.666667
LAT	6.65	6.916667
DAILY/ELEV	92.0	153.0
19810101	0.0	0.0
19810102	0.0	0.0
...		

CPT — seasonal station data

The Climate Prediction Tools (CPT) format is used when passing station data to the merging classes or when reading seasonal aggregates:

STATION	ABEO	ABUJ	ADEK
LAT	7.2	7.6	9.0
LON	3.3	5.2	7.2
1991	514.9	715.1	934.3
1992	503.6	736.4	714.6
...			

Gridded data must be `xarray.DataArray` objects with dimensions (T, Y, X) and standard datetime coordinates on the T axis.

1.4.2 Computing Agroclimatic Predictands

The `was_compute_predictand` module computes climate indices directly from daily gridded or station data. It supports parallel execution on large grids via Dask.

Onset of the rainy season

Class: `WAS_compute_onset`

Detects the first date of reliable season onset using the AGRHYMET / Sivakumar (1988) methodology:

1. Search starts from a zone-specific calendar date.
2. A cumulative rainfall threshold must be met within a short window (e.g. 20 mm in 3 consecutive days).

3. No prolonged dry spell is allowed in the following weeks.

Default agro-ecological zones and criteria:

Zone	Search start	Cumulative	Max dry days
Sahel (0–100 mm/yr)	01 Jun	10 mm	25
Sahel (100–200 mm/yr)	15 May	15 mm	25
Sahel (200–400 mm/yr)	01 May	15 mm	20
Sahel (400–600 mm/yr)	15 Mar	20 mm	20
Soudanian	15 Mar	20 mm	10
Gulf of Guinea	01 Feb	20 mm	10

Zone boundaries are automatically assigned based on mean annual rainfall and latitude; custom criteria can be supplied via `user_criteria`.

```
from wass2s import WAS_compute_onset

# Use default zone criteria
onset_calc = WAS_compute_onset()

# Gridded computation (daily_rain is xarray DataArray T, Y, X)
onset_da = onset_calc.compute(daily_data=daily_rain, nb_cores=4)

# Station computation (CDT format)
onset_cpt = onset_calc.compute_insitu(daily_df=daily_df_cdt)
```

To override the defaults, pass a `user_criteria` dictionary:

```
criteria = {
    0: {
        "zone_name": "Sahel",
        "start_search": "06-01",
        "cumulative": 20,
        "number_dry_days": 10,
        "thrd_rain_day": 0.85,
        "end_search": "08-30"
    }
}
onset_calc = WAS_compute_onset(user_criteria=criteria)
```

Cessation of the rainy season

Class: `WAS_compute_cessation`

Detects the end of the season using a soil water-balance model. The season ends when accumulated evapotranspiration demand exceeds accumulated post-onset rainfall plus the maximum soil water retention capacity.

```
from wass2s import WAS_compute_cessation

criteria_cess = {
    0: {
        "zone_name": "Sahel",
        "date_dry_soil": "01-01",
```

(continues on next page)

(continued from previous page)

```

        "start_search": "09-01",
        "ETP": 5.0,          # mm/day
        "Cap_ret_maxi": 70,  # mm
        "end_search": "10-30"
    }
}

cess_calc = WAS_compute_cessation(user_criteria=criteria_cess)
cessation_da = cess_calc.compute(daily_data=daily_rain, nb_cores=4)

```

Dry and wet spell analysis

Classes: WAS_compute_onset_dry_spell, WAS_count_dry_spells, WAS_count_wet_spells, WAS_count_rainy_days

These classes characterise the intra-seasonal distribution of rainfall between onset and cessation.

```

from wass2s import (
    WAS_compute_onset_dry_spell,
    WAS_count_dry_spells,
    WAS_count_rainy_days
)

# Longest dry spell in the 30 days after onset (critical for seedling_
→survival)
dry_spell_calc = WAS_compute_onset_dry_spell()
max_dry_spell_da = dry_spell_calc.compute(daily_data=daily_rain, nb_cores=4)

# Total number of rainy days (> 1 mm) between onset and cessation
rain_days_calc = WAS_count_rainy_days()
nb_rainy_da = rain_days_calc.compute(
    daily_data=daily_rain,
    onset_date=onset_da,
    cessation_date=cessation_da,
    rain_threshold=1.0,
    nb_cores=4
)

# Number of dry spells ≥ 7 days between onset and cessation
ds_count_calc = WAS_count_dry_spells()
n_dry_spells_da = ds_count_calc.compute(
    daily_data=daily_rain,
    onset_date=onset_da,
    cessation_date=cessation_da,
    d_len=7,
    nb_cores=4
)

```

ETCCDI temperature extremes

Classes: ETCCDITempIndices, ETCCDIHeatWaveIndices

These factory classes implement standard ETCCDI indices using a 5-day centred bootstrapping window for percentile estimation.

```
from wass2s import ETCCDITempIndices, ETCCDIHeatWaveIndices

# TX90p - percentage of days where Tmax > 90th percentile
tx90p_calc = ETCCDITempIndices.hot_days(
    base_period=slice("1981", "2010"),
    season=[6, 7, 8]
)
tx90p_da = tx90p_calc.compute_xarray(tmax_da, parallel=True)

# WSDI - Warm Spell Duration Index
wsdi_calc = ETCCDIHeatWaveIndices.wsdi(base_period=slice("1981", "2010"))
wsdi_da = wsdi_calc.compute_xarray(tmax_da)
```

ETCCDI precipitation extremes

Class: WAS_PrecipIndices

Computes extreme precipitation indices (R95p, R99p) based on wet-day percentiles from a reference period.

```
from wass2s import WAS_PrecipIndices

r95p_calc = WAS_PrecipIndices(
    base_period=slice("1981", "2010"),
    percentile=95,
    wet_day_threshold=1.0
)
r95p_da = r95p_calc.compute_xarray(daily_rain)
```

1.4.3 Merging Gridded Data with Station Observations

The WAS_Merging class blends a coarse gridded estimate (satellite product, reanalysis, or model output) with sparse station observations to produce a spatially complete, station-corrected field.

The class is initialised with a station DataFrame in *CPT format* and a gridded xarray.DataArray:

```
from wass2s import WAS_Merging

merger = WAS_Merging(
    df=station_df,          # CPT-format DataFrame
    da=satellite_da,        # (T, Y, X) DataArray
    date_month_day="08-01"  # Aligns CPT year labels to a specific day
)
```

Available merging methods

Simple Bias Adjustment (Residual Kriging)

Computes $\text{Station} - \text{Grid}$ residuals at station locations, interpolates them over the full grid with Ordinary Kriging, and adds them back to the original field.

```
corrected_da, cv_stats = merger.simple_bias_adjustment(  
    missing_value=-999.0,  
    do_cross_validation=True  
)
```

Regression Kriging

Fits a linear regression between the grid and the stations, then Kriges the regression residuals. Preferred when there is a strong linear relationship between the gridded product and station observations.

```
corrected_da, _ = merger.regression_kriging()
```

Neural Network Kriging

Replaces the linear regression with a Multi-Layer Perceptron to capture non-linear biases, followed by Kriging of the residuals. More powerful for complex terrain but computationally heavier.

```
corrected_da, _ = merger.neural_network_kriging()
```

Multiplicative Bias

Computes the ratio $\text{Station} / \text{Grid}$ instead of the difference, interpolates it spatially, and multiplies the original grid. Appropriate for strictly positive variables like precipitation where subtraction can yield unphysical negative values.

```
corrected_da, _ = merger.multiplicative_bias()
```

Visualising the correction

```
merger.plot_merging_comparaison(  
    df_Obs=station_df,  
    da_estimated=satellite_da,  
    da_corrected=corrected_da  
)
```

This generates a two-panel scatter plot (observation vs. original estimate and observation vs. corrected result) with a 1:1 reference line.

1.4.4 Bias Correction

Two classes handle bias correction depending on the nature of the variable.

Precipitation — WAS_Qmap

WAS_Qmap implements quantile-mapping methods adapted from the R `qmap` package. They explicitly handle wet-day frequency (the drizzle effect common in GCMs) before correcting the quantile structure.

Supported methods:

- QUANT — empirical quantile mapping.

- RQUANT — robust quantile mapping using local linear fits (recommended for precipitation tails).
- SSPLIN — PCHIP smoothing-spline quantile mapping.
- PTF — parametric transformation function (power, exponential, linear).
- DIST — full distribution mapping (Bernoulli-Gamma for rain, etc.).

```
from wass2s import WAS_Qmap

# Fit correction on the historical period
fit_obj = WAS_Qmap.fitQmap(
    obs=obs_da,
    mod=mod_da,
    method="RQUANT",
    wet_day=True,      # Correct dry-day frequency first
    qstep=0.01
)

# Apply to forecast data
corrected_da = WAS_Qmap.doQmap(forecast_da, fit_obj)

# Evaluate the correction (compare dry fractions and extreme quantiles)
ds_dry_wet, ds_extreme = WAS_Qmap.evaluate_bias_correction(
    obs=obs_da,
    mod=mod_da,
    corrected=WAS_Qmap.doQmap(mod_da, fit_obj),
    wet_threshold=1.0,
    extreme_quantiles=[0.95, 0.99]
)
WAS_Qmap.plot_extreme_quantiles_group(ds_extreme)
```

Continuous variables — WAS_bias_correction

WAS_bias_correction targets temperature, wind speed, and similar continuously distributed variables.

Supported methods: MEAN (additive), VARSCALE (mean + variance), QUANT, NORM, DIST.

```
from wass2s import WAS_bias_correction

# Temperature: variance scaling (corrects mean and spread)
fit_temp = WAS_bias_correction.fitBC(
    obs=obs_temp_da,
    mod=mod_temp_da,
    method="VARSCALE"
)
corrected_temp = WAS_bias_correction.doBC(forecast_temp_da, fit_temp)

# Wind speed: distribution mapping with Weibull
fit_wind = WAS_bias_correction.fitBC(
    obs=obs_wind_da,
    mod=mod_wind_da,
    method="DIST",
    distr="weibull"
)
corrected_wind = WAS_bias_correction.doBC(forecast_wind_da, fit_wind)
```

1.4.5 Data Transformation and Distribution Fitting

The `WAS_TransformData` class handles the statistical preprocessing of geospatial time series, which is often necessary before applying parametric probabilistic post-processing to skewed rainfall data.

Initialise with an (T, Y, X) `dataArray`:

```
from wass2s import WAS_TransformData

transformer = WAS_TransformData(data=rainfall_da, n_clusters=5)
```

Skewness analysis

```
# Detect skewness class per grid cell
skew_ds, skew_summary = transformer.detect_skewness()
print("Skewness counts:", skew_summary["class_counts"])

# Get transformation recommendations
handle_ds, recommendations = transformer.handle_skewness()
```

Applying and inverting transformations

```
# Auto-apply recommended transformation per grid cell
transformed_da = transformer.apply_transformation()

# Or force a specific method
transformed_da = transformer.apply_transformation(method="yeo_johnson")

# Invert after modelling to recover original units
original_scale_da = transformer.inverse_transform()
```

Distribution fitting

Fitting distributions pixel-by-pixel can be noisy. `WAS_TransformData` uses K-Means clustering to pool data within homogeneous zones before fitting. The best distribution per zone is selected by AIC.

Supported distributions: norm, lognorm, gamma, weibull_min, expon, t, poisson, nbinom.

```
# Cluster-based fitting (faster, recommended for large domains)
best_code, best_shape, best_loc, best_scale, cluster_map = \
    transformer.fit_best_distribution(use_transformed=True, mode="cluster")

# Pixel-by-pixel fitting (slower, captures local detail)
best_code, best_shape, best_loc, best_scale, cluster_map = \
    transformer.fit_best_distribution(use_transformed=True, mode="grid")

# Visualise the winning distribution per grid cell
transformer.plot_best_fit_map(
    data_array=best_code,
    map_dict=transformer.distribution_map,
    output_file="distribution_map.png",
```

(continues on next page)

(continued from previous page)

```

title="Best Fitting Distribution",
show_plot=True
)

```

The outputs `best_code`, `best_shape`, `best_loc`, and `best_scale` are (Y, X) DataArrays that can be passed directly to any model's `compute_prob` and `forecast` methods when using `dist_method="bestfit"`.

1.5 Models and Cross-Validation

wass2s provides a unified forecasting interface: every model class implements the same three methods:

- `compute_model(X_train, y_train, X_test, y_test, **kwargs)` — fit on the training fold and predict the test period.
- `compute_prob(Predictant, clim_year_start, clim_year_end, hindcast_det, **kwargs)` — convert a deterministic hindcast into tercile probabilities.
- `forecast(Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, **kwargs)` — operational forecast for one target year.

This uniformity means that any model can be swapped into *cross-validation* without changing the surrounding code.

1.5.1 Linear Regression Models

All linear models are spatially distributed (one model per pixel or per cluster), run in parallel via Dask, and produce both deterministic and probabilistic (tercile) outputs.

Ordinary Least Squares — `WAS_LinearRegression_Model`

A baseline OLS model with no regularisation. Fast, interpretable, and a good first benchmark.

```

from wass2s import WAS_LinearRegression_Model

model = WAS_LinearRegression_Model(nb_cores=4, dist_method="nonparam")

hindcast = model.compute_model(X_train, y_train, X_test, y_test)
prob = model.compute_prob(
    Predictant=obs, clim_year_start=1991, clim_year_end=2020,
    hindcast_det=hindcast
)

```

Ridge — `WAS_Ridge_Model`

L2-regularised regression. Useful when predictors are correlated (e.g. multiple SST indices). The regularisation parameter `alpha` is optimised per spatial cluster.

```

from wass2s import WAS_Ridge_Model
import numpy as np

ridge = WAS_Ridge_Model(
    n_clusters=6,
    alpha_range=np.logspace(-4, 0.1, 20),

```

(continues on next page)

(continued from previous page)

```

        nb_cores=4,
        dist_method="nonparam"
    )

    alpha, clusters = ridge.compute_hyperparameters(
        predictand=y_train, predictor=X_train,
        clim_year_start=1991, clim_year_end=2020
    )
    hindcast = ridge.compute_model(X_train, y_train, X_test, y_test, alpha=alpha)

```

Lasso and LassoLars — WAS_Lasso_Model, WAS_LassoLars_Model

L1-regularised regression. Drives irrelevant predictor coefficients exactly to zero, performing implicit feature selection. LassoLars uses the LARS path algorithm and is more efficient when the number of features exceeds the number of samples.

```

from wass2s import WAS_LassoLars_Model

lasso = WAS_LassoLars_Model(n_clusters=10, nb_cores=4)
hindcast = lasso.compute_model(X_train, y_train, X_test, y_test)

```

ElasticNet — WAS_ElasticNet_Model

Combines L1 (feature selection) and L2 (multicollinearity handling). Often the most robust choice for climate indices predictors.

```

from wass2s import WAS_ElasticNet_Model

enet = WAS_ElasticNet_Model(
    l1_ratio_range=[0.1, 0.5, 0.9],
    nb_cores=4
)
alpha, l1_ratio, clusters = enet.compute_hyperparameters(
    y_train, X_train, 1991, 2020
)
forecast_det, forecast_prob = enet.forecast(
    Predictant=y_train, clim_year_start=1991, clim_year_end=2020,
    Predictor=X_train, hindcast_det=hindcast,
    Predictor_for_year=X_next_year,
    alpha=alpha, l1_ratio=l1_ratio
)

```

MARS — WAS_MARS_Model

Multivariate Adaptive Regression Splines. Fits piecewise linear functions (hinges) and automatically selects relevant predictors and interactions.

```

from wass2s import WAS_MARS_Model

mars = WAS_MARS_Model(nb_cores=4, max_terms=21, max_degree=2)
hindcast = mars.compute_model(X_train, y_train, X_test, y_test)

```

Probabilistic output methods

All models accept a `dist_method` parameter controlling how deterministic predictions are converted into tercile probabilities:

<code>dist_method</code>	Description
<code>nonparam</code>	Non-parametric: rank historical cross-validation errors. No distribution assumption. <i>Default and recommended.</i>
<code>normal</code>	Gaussian distribution fitted to cross-validation errors.
<code>gamma</code>	Gamma distribution — suitable for precipitation.
<code>lognormal</code>	Log-normal distribution.
<code>weibull_min</code>	Weibull distribution.
<code>bestfit</code>	Per-pixel best-fitting distribution from <code>WAS_TransformData.fit_best_distribution</code> .

To use `bestfit`, pass the distribution parameter arrays returned by `WAS_TransformData`:

```
prob = model.compute_prob(
    Predictant=obs, clim_year_start=1991, clim_year_end=2020,
    hindcast_det=hindcast,
    best_code_da=best_code,
    best_shape_da=best_shape,
    best_loc_da=best_loc,
    best_scale_da=best_scale
)
```

1.5.2 Advanced Machine-Learning Models

These models are non-linear regressors that share the same interface as the linear models above. Hyperparameters are optimised per spatial cluster.

Hyperparameter optimisation

All ML classes support three search strategies via `optimization_method`:

- "grid" — exhaustive grid search (reliable but slow).
- "random" — randomised search (faster for large spaces).
- "bayesian" — Optuna TPE sampler (recommended for MLP, stacking).

Support Vector Regression — `WAS_SVR`

```
from wass2s import WAS_SVR

svr = WAS_SVR(
    nb_cores=4, n_clusters=5,
    kernel="rbf", optimization_method="bayesian", n_trials=30
)
C_map, eps_map, deg_map, clusters, kernel_map, gamma_map = \
    svr.compute_hyperparameters(y_train, X_train, 1991, 2020)

forecast_det, forecast_prob = svr.forecast(
```

(continues on next page)

(continued from previous page)

```
Predictant=y_train, clim_year_start=1991, clim_year_end=2020,
Predictor=X_train, hindcast_det=hindcast,
Predictor_for_year=X_next_year,
epsilon=eps_map, C=C_map,
kernel_array=kernel_map, degree_array=deg_map, gamma_array=gamma_map
)
```

Multi-Layer Perceptron — WAS_MLP

```
from wass2s import WAS_MLP

mlp = WAS_MLP(
    nb_cores=4, n_clusters=4,
    optimization_method="random", n_trials=20
)
hl, act, solver, alpha, lr, maxiter, clusters = \
    mlp.compute_hyperparameters(y_train, X_train, 1991, 2020)

forecast_det, forecast_prob = mlp.forecast(
    Predictant=y_train, clim_year_start=1991, clim_year_end=2020,
    Predictor=X_train, hindcast_det=hindcast,
    Predictor_for_year=X_next_year,
    hl_array=hl, act_array=act, lr_array=lr
)
```

Stacking Ensemble — WAS_RandomForest_XGBoost_Stacking_MLP

A two-layer ensemble: Random Forest and XGBoost as base learners, with an MLP meta-learner trained on their out-of-fold predictions.

```
from wass2s import WAS_RandomForest_XGBoost_Stacking_MLP

stacking = WAS_RandomForest_XGBoost_Stacking_MLP(
    nb_cores=4, n_clusters=3,
    optimization_method="bayesian", n_trials=15
)
best_params, clusters = stacking.compute_hyperparameters(
    y_train, X_train, 1991, 2020
)
forecast_det, forecast_prob = stacking.forecast(
    Predictant=y_train, clim_year_start=1991, clim_year_end=2020,
    Predictor=X_train, hindcast_det=hindcast,
    Predictor_for_year=X_next_year,
    best_param_da=best_params
)
```

1.5.3 EOF Analysis and Principal Component Regression

EOF analysis (WAS_EOF)

Class: WAS_EOF

Performs Empirical Orthogonal Function decomposition on a spatial field. Cosine-latitude weighting corrects for area distortion; optional linear detrending removes any long-term signal before analysis.

```
from wass2s import WAS_EOF

eof_solver = WAS_EOF(
    opti_explained_variance=90,    # Retain modes explaining ≥ 90 % variance
    detrend=True,
    use_coslat=True
)
eofs, pcs, exp_var, _ = eof_solver.fit(sst_anomaly_da, dim="T")
eof_solver.plot_EOF(eofs, exp_var)
```

Principal Component Regression (WAS_PCR)

Class: WAS_PCR

A wrapper that chains an EOF decomposition with any regression model. The EOF is fitted on the training fold only, preventing leakage of test-year information into the predictor space.

```
from wass2s import WAS_PCR, WAS_ElasticNet_Model
import numpy as np

regression_model = WAS_ElasticNet_Model(
    alpha_range=np.logspace(-4, 0.1, 20),
    l1_ratio_range=[0.5, 0.9999],
    nb_cores=4, dist_method="nonparam"
)
pcr = WAS_PCR(
    regression_model=regression_model,
    n_modes=6, detrend=True, use_coslat=True
)

# Hyperparameter search in PC space
alpha, l1_ratio, clusters = regression_model.compute_hyperparameters(
    predictand, pcs.rename({"mode": "features"}).transpose("T", "features"),
    1991, 2020
)

# Cross-validate
cv = WAS_Cross_Validator(
    n_splits=len(predictand.get_index("T")), nb_omit=2
)
hindcast_det, hindcast_prob = cv.cross_validate(
    pcr, predictand,
    pcs.rename({"mode": "features"}).transpose("T", "features"),
    1991, 2020, alpha=alpha, l1_ratio=l1_ratio
)
```

(continues on next page)

(continued from previous page)

```
# Operational forecast
forecast_det, forecast_prob = pcr.forecast(
    Predictant=predictand, clim_year_start=1991, clim_year_end=2020,
    Predictor=sst_da, hindcast_det=hindcast_det,
    Predictor_for_year=sst_next_year,
    alpha=alpha, l1_ratio=l1_ratio
)
```

Using SST indices instead of full EOF fields

When working with pre-computed SST indices (e.g. NINO34, TNA, DMI), there is no need for EOF reduction — pass the index dataset directly as the predictor:

```
from wass2s import (
    compute_sst_indices, WAS_LinearRegression_Model,
    WAS_Cross_Validator, prepare_predictand
)
from statsmodels.stats.outliers_influence import variance_inflation_factor as VIF
import pandas as pd

predictand = prepare_predictand(
    dir_to_save_obs, variables_obs, year_start, year_end,
    seas=seas_obs, ds=False
)

sst_index_names = ["NINO34", "TNA", "TSA", "DMI"]
predictors = compute_sst_indices(
    dir_to_save_reanalysis, sst_index_names, "ERA5.SST",
    year_start, year_end, seas=seas_reanalysis,
    clim_year_start=clim_year_start, clim_year_end=clim_year_end
)

# Remove highly correlated indices (VIF > 5)
vif_data = pd.DataFrame({
    "feature": predictors.to_dataframe().columns,
    "VIF": [VIF(predictors.to_dataframe(), i)
            for i in range(predictors.to_dataframe().shape[1])]
})
low_vif = vif_data[vif_data["VIF"] < 5]["feature"].tolist()
filtered = (
    predictors[low_vif]
    .to_array()
    .rename({"variable": "features"})
    .transpose("T", "features")
)

model = WAS_LinearRegression_Model(nb_cores=4, dist_method="nonparam")
cv = WAS_Cross_Validator(
    n_splits=len(predictand.get_index("T")), nb_omit=2
)
hindcast_det, hindcast_prob = cv.cross_validate(
    model, predictand,
```

(continues on next page)

(continued from previous page)

```

        filtered.isel(T=slice(None, -1)),
        clim_year_start, clim_year_end
    )

```

1.5.4 Canonical Correlation Analysis

Class: WAS_CCA

CCA finds pairs of spatial patterns — one in the predictor field and one in the predictand field — that are maximally correlated. It is widely used for GCM-output recalibration.

```

from wass2s import WAS_CCA, WAS_Cross_Validator, retrieve_single_zone_for_PCR

was_cca = WAS_CCA(n_modes=3, n_pca_modes=10, dist_method="nonparam")

# Retrieve predictor data (GCM output, spatially averaged over a zone)
zone = {"A": ("A", -150, 150, -45, 45)}
predictor = retrieve_single_zone_for_PCR(
    dir_to_save_model, zone, "ECMWF_51.PRCP",
    year_start, year_end,
    clim_year_start=clim_year_start, clim_year_end=clim_year_end,
    model=True, month_of_initialization=3, lead_time=1
).isel(T=slice(None, -1))
predictor["T"] = predictand["T"]

# Visualise CCA modes
was_cca.plot_cca_results(
    X=predictor, Y=predictand,
    clim_year_start=clim_year_start, clim_year_end=clim_year_end
)

# Cross-validate
cv = WAS_Cross_Validator(
    n_splits=len(predictand.get_index("T")), nb_omit=2
)
hindcast_det, hindcast_prob = cv.cross_validate(
    was_cca, predictand, predictor, clim_year_start, clim_year_end
)

# Operational forecast
forecast_det, forecast_prob = was_cca.forecast(
    predictand, clim_year_start, clim_year_end,
    predictor, hindcast_det, predictor # last arg = real-time predictor
)

```

1.5.5 Analog Forecasting

Class: WAS_Analog

Identifies historically similar years (analog) by comparing SST or climate index patterns and composites their observed rainfall to produce a forecast.

Analog selection methods

- "som" — Self-Organising Maps (minisom).
- "cor_based" — Pearson correlation distance in index space.
- "pca_based" — PCA followed by Euclidean distance.
- "kmeans_based" — K-Means clustering.
- "agglomerative_based" — Agglomerative (Ward) clustering.

```
from wass2s import WAS_Analog, WAS_Cross_Validator

analog = WAS_Analog(
    dir_to_save="./data/analog/",
    year_start=1991,
    year_forecast=2025,
    reanalysis_name="NOAA.SST",
    model_name="ECMWF_51.SST",
    method_analog="som",
    month_of_initialization=3,
    clim_year_start=1991,
    clim_year_end=2020,
    define_extent=(-180, 180, -45, 45),
    index_compute=["NINO34", "DMI"],
    dist_method="nonparam"
)

# Download and preprocess SST data
sst_hist, sst_for = analog.download_and_process()

# Cross-validate
cv = WAS_Cross_Validator(
    n_splits=len(rainfall.get_index("T")), nb_omit=2
)
hindcast_det, hindcast_prob = cv.cross_validate(
    analog, rainfall, clim_year_start=1991, clim_year_end=2020
)

# Forecast
_, similar_years, forecast_det, forecast_prob = analog.forecast(
    predictant=rainfall,
    clim_year_start=1991, clim_year_end=2020,
    hindcast_det=hindcast_det
)

# Composite plot (observed SST pattern + rainfall composite)
analog.composite_plot(
    predictant=rainfall,
    clim_year_start=1991, clim_year_end=2020,
    hindcast_det=hindcast_det,
    plot_predictor=True
)
```

1.5.6 Cross-Validation

wass2s uses a leave-one-out scheme with a symmetric exclusion window to prevent temporal data leakage from near-neighbour years.

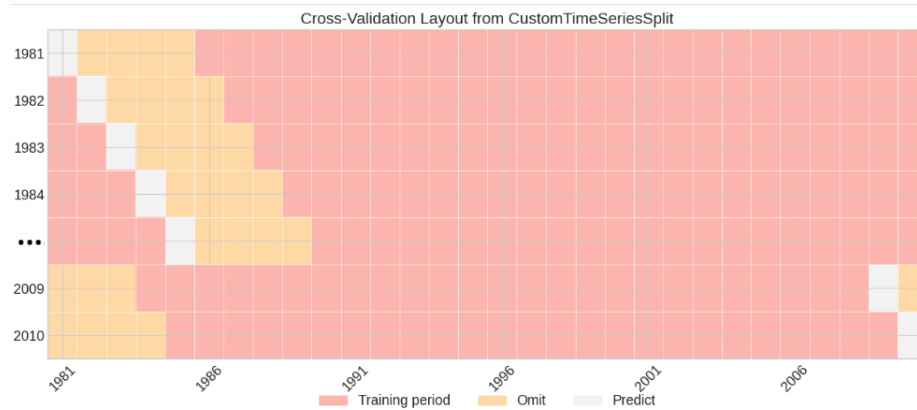


Fig. 1: Cross-validation scheme: pink = training, yellow = omitted buffer, white = predicted year.

For each test year i , a window of `nb_omit + 1` years centred on i is excluded from the training set. This prevents the model from seeing years that are strongly autocorrelated with the year being predicted.

The `WAS_Cross_Validator` class automatically selects the correct preprocessing path for each model family — standardisation, detrending, EOF fitting, and tercile classification are all applied inside the fold loop, never on the full time series.

```
from wass2s import WAS_Cross_Validator

cv = WAS_Cross_Validator(
    n_splits=len(predictand.get_index("T")),
    nb_omit=2      # Exclude the 2 years on each side of the test year
)

hindcast_det, hindcast_prob = cv.cross_validate(
    model,          # Any WAS model instance
    predictand,
    predictor,
    clim_year_start=1991,
    clim_year_end=2020,
    **model_params # Hyperparameters forwarded to compute_model
)
```

Estimating forecast uncertainty

Deterministic model outputs (the best-guess seasonal total) are converted to tercile probabilities by comparing a fitted probability density function centred on the forecast to the climatological distribution.

Classification-based models (logistic regression, extended logistic regression, SVM classifiers) estimate tercile probabilities directly without building a continuous PDF.

Important

Always pass `predictor.isel(T=slice(None, -1))` to `cross_validate` so that the real-time

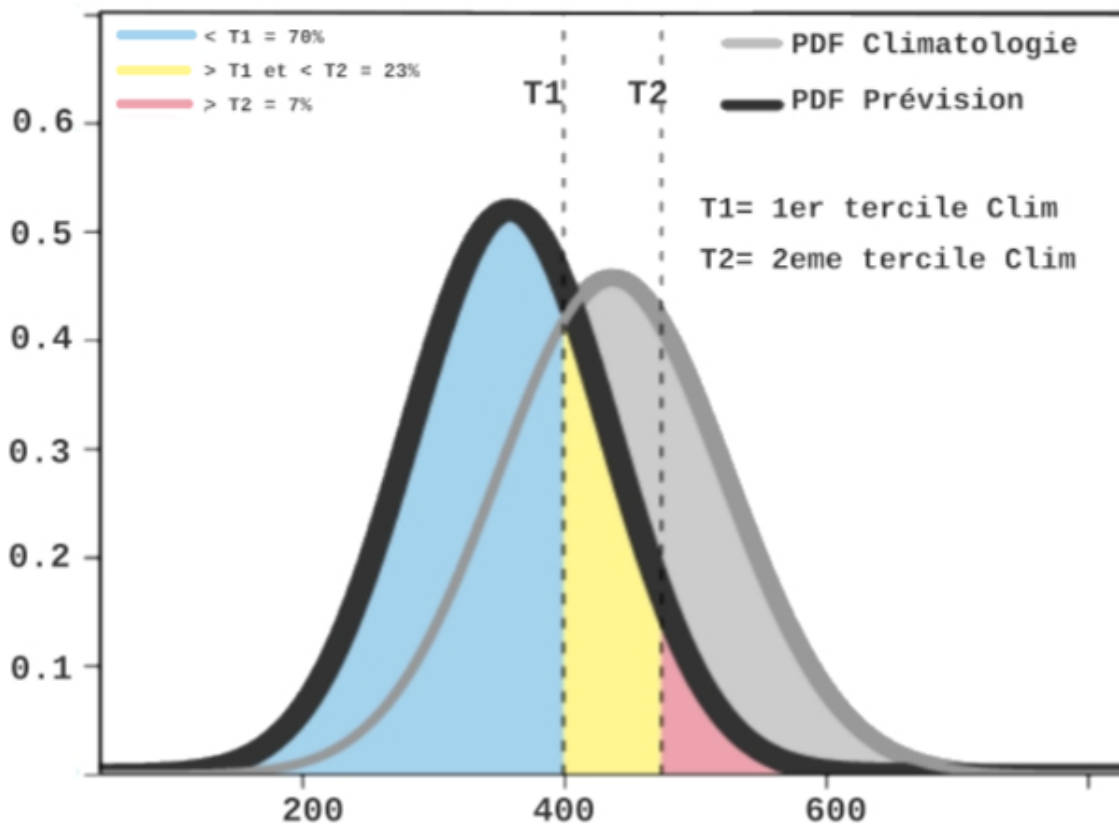


Fig. 2: Tercile probability estimation: the forecast PDF (blue) is integrated over the below-normal (<T1), near-normal (T1–T2), and above-normal (>T2) regions of the observed climatological distribution.

(last) predictor step is excluded from the validation period. Then pass `predictor.isel(T=[-1])` as `Predictor_for_year` in the forecast call.

1.6 Verification

The `WAS_Verification` class provides deterministic, probabilistic, and ensemble verification metrics for gridded seasonal forecasts. All scores are computed pixel-wise on `xarray.DataArrays` and can be visualised directly on Cartopy maps.

```
from wass2s import WAS_Verification

verifier = WAS_Verification()
```

Metadata for all available scores (name, range, colormap, type) can be retrieved as a dictionary:

```
metadata = verifier.get_scores_metadata()
```

1.6.1 Deterministic Metrics

These metrics compare point predictions to observations and are computed via `compute_deterministic_score`, which applies the chosen function spatially:

```
score_map = verifier.compute_deterministic_score(
    score_func, obs_data, model_data
)
```

Available deterministic metrics:

Example

```
pearson = verifier.compute_deterministic_score(
    verifier.pearson_corr, obs_data, hindcast_det
)
kge = verifier.compute_deterministic_score(
    verifier.kling_gupta_efficiency, obs_data, hindcast_det
)

verifier.plot_model_score(
    pearson, "Pearson",
    dir_save_score="./scores/", figure_name="Pearson_score"
)
```

To compare multiple models in a grid layout:

```
verifier.plot_models_score(
    {"Ridge": pearson_ridge, "MARS": pearson_mars},
    "Pearson",
    dir_save_score="./scores/"
)
```

1.6.2 Probabilistic Metrics

Probabilistic metrics evaluate tercile-probability forecasts (PB, PN, PA) and are computed via `compute_probabilistic_score`.

Available probabilistic metrics:

Example

```
groc = verifier.compute_probabilistic_score(
    verifier.calculate_groc, obs_data, hindcast_prob,
    clim_year_start=1991, clim_year_end=2020
)
rpss = verifier.compute_probabilistic_score(
    verifier.calculate_rpss, obs_data, hindcast_prob,
    clim_year_start=1991, clim_year_end=2020
)

verifier.plot_model_score(groc, "GROC", dir_save_score="./scores/")
```

Reliability diagrams and ROC curves

```
# Reliability diagram for each tercile category
verifier.reliability_diagram(
    modelname="Ridge",
    dir_to_save_score="./scores/",
    y_true=obs_data,
    y_probs=hindcast_prob,
    clim_year_start=1991, clim_year_end=2020
)

# ROC curves with 95 % bootstrap confidence intervals
verifier.plot_roc_curves(
    modelname="Ridge",
    dir_to_save_score="./scores/",
    y_true=obs_data,
    y_probs=hindcast_prob,
    clim_year_start=1991, clim_year_end=2020,
    n_bootstraps=1000, ci=0.95
)
```

1.6.3 Ensemble Metrics

The Continuous Ranked Probability Score (CRPS) evaluates forecasts that provide a full ensemble distribution (e.g. raw GCM ensembles).

```
crps_map = verifier.compute_crps(
    obs_data, ensemble_hindcast,
    member_dim="number", dim="T"
)
```

1.6.4 Year-by-Year Validation

Two utilities support annual verification:

```
# Classify a specific year relative to climatology
# (Well Above Average, Above Average, Near Average, ...)
verifier.ratio_to_average(
    predictant=obs_data,
    clim_year_start=1991, clim_year_end=2020,
    year=2023
)

# Compute RPSS for one year and plot the result on a map
verifier.compute_one_year_rpss(
    obs=obs_data,
    prob_pred=hindcast_prob,
    clim_year_start=1991, clim_year_end=2020,
    year=2023
)
```

1.6.5 Tercile Classification Utilities

```
# Assign each observed year to a tercile class (0=BN, 1=NN, 2=AN)
classes = verifier.compute_class(
    Predictant=obs_data,
    clim_year_start=1991, clim_year_end=2020
)

# Classify with pre-computed thresholds
classified = verifier.classify_data_into_terciles(
    y=obs_data, T1=t1_map, T2=t2_map
)
```

1.7 Multi-Model Ensemble Methods

The `was_mme` module combines hindcasts and forecasts from multiple GCMs to improve seasonal predictive skill. It provides methods ranging from simple weighted averaging to advanced machine-learning stacking and statistical calibration, all producing tercile-probability outputs.

Note

Some classes in this module are still under active development. Check the [API reference](#) for the latest status.

1.7.1 1. Data Preparation

Function: `process_datasets_for_mme`

Loads, regrids, and harmonises hindcasts and real-time forecasts from multiple models onto the common observational grid. This is always the first step before any MME method.

```
from wass2s import process_datasets_for_mme

hdcst, fcst, obs, scores = process_datasets_for_mme(
    rainfall=obs_data,
    dir_to_save_model="./data/GCM/",
    best_models=["ECMWF_51", "NCEP_2", "UKMO_604"],
    year_start=1993,
    year_end=2016,
    score_metric="GROC"
)
```

The returned `hdcst` `DataArray` has dimensions (T, M, Y, X) where M is the model dimension.

1.7.2 2. Weighted Ensemble Averaging

Class: `WAS_mme_Weighted`

Combines model hindcasts using weights derived from a historical skill score. Models whose score falls below a threshold receive zero weight.

```
from wass2s import WAS_mme_Weighted

mme = WAS_mme_Weighted(
    equal_weighted=False,
    metric="GROC",
    threshold=0.5
)
hcst_det, fcst_det = mme.compute(obs, hdcst, fcst, scores)

# Tercile probabilities from the ensemble mean
hcst_prob = mme.compute_prob(
    Predictant=obs,
    clim_year_start=1993, clim_year_end=2016,
    hindcast_det=hcst_det
)
```

Class: `WAS_ProbWeighted`

Computes weights at the probability level (per tercile category) rather than on the deterministic mean.

1.7.3 3. Min et al. (2009) Probabilistic MME

Class: `WAS_Min2009_ProbWeighted`

Implements the PMME method from Min et al. (2009):

1. Computes per-model tercile probabilities assuming a Gaussian or log-normal distribution.
2. Weights models proportionally to the square root of their ensemble size.
3. Combines the weighted probabilities.
4. Produces a chi-square combined map to flag areas of significant ensemble signal.

```

from wass2s import WAS_Min2009_ProbWeighted

pmme = WAS_Min2009_ProbWeighted(ensemble_sizes={"ECMWF_51": 25, "NCEP_2": 24})
fcst_prob = pmme.compute_combined_map(
    hindcasts=hdcast, obs=obs,
    clim_year_start=1993, clim_year_end=2016,
    new_forecasts=fcst
)

```

1.7.4 4. Machine-Learning Ensemble Methods

These methods treat the MME problem as a regression task: given GCM outputs as predictors, learn to reproduce observed rainfall. Hyperparameters are optimised per spatial cluster.

Stacking super-ensemble — WAS_mme_Stacking

A two-layer architecture:

- **Layer 0 (base)**: Random Forest, XGBoost, HP-ELM, MLP — each generates out-of-fold predictions.
- **Layer 1 (meta)**: Ridge, Lasso, or ElasticNet trained on the OOF outputs.

```

from wass2s import WAS_mme_Stacking

stacking = WAS_mme_Stacking(
    meta_learner_type="ridge",
    meta_search_method="bayesian",
    n_clusters=4
)
fcst_det, fcst_prob = stacking.forecast(
    Predictant=obs,
    clim_year_start=1993, clim_year_end=2016,
    hindcast_det=hdcast,
    hindcast_det_cross=hdcast_cv,
    Predictor_for_year=fcst
)

```

Individual machine-learning regressors

Each of these classes shares the same forecast signature as WAS_mme_Stacking:

- **Random Forest** — WAS_mme_RF
- **XGBoost** — WAS_mme_XGBoosting
- **Multi-Layer Perceptron** — WAS_mme_MLP
- **Extreme Learning Machine** — WAS_mme_hpELM

```

from wass2s import WAS_mme_RF

rf_mme = WAS_mme_RF(n_clusters=5, nb_cores=4)
fcst_det, fcst_prob = rf_mme.forecast(
    Predictant=obs,

```

(continues on next page)

(continued from previous page)

```

    clim_year_start=1993, clim_year_end=2016,
    hindcast_det=hdcast,
    hindcast_det_cross=hdcast_cv,
    Predictor_for_year=fcst
)

```

1.7.5 5. Calibration and Post-Processing

Non-Homogeneous Gaussian Regression — `WAS_mme_NGR_Gaussian`

Calibrates the ensemble mean and spread by minimising the Continuous Ranked Probability Score (CRPS). The calibrated forecast distribution is:

$$\mu = a + b \bar{x}_{\text{ens}}, \quad \sigma = \sqrt{c^2 + d^2 s_{\text{ens}}^2}$$

where a, b, c, d are fitted coefficients, $\bar{x}_{\text{ens}}$ is the ensemble mean, and s_{ens}^2 is the ensemble variance.

```

from wass2s import WAS_mme_NGR_Gaussian

ngr = WAS_mme_NGR_Gaussian()
fcst_det, fcst_prob = ngr.forecast(
    Predictant=obs,
    clim_year_start=1993, clim_year_end=2016,
    hindcast_det=hdcast,
    Predictor_for_year=fcst
)

```

Bayesian Model Averaging — `WAS_mme_FastBMA`, `WAS_mme_FullBMA`

BMA assigns posterior weights to each model's predictive distribution:

- `WAS_mme_FastBMA` — EM algorithm; fast convergence for small ensembles.
- `WAS_mme_FullBMA` — grid search over kernel bandwidths; more accurate.

```

from wass2s import WAS_mme_FastBMA

bma = WAS_mme_FastBMA()
fcst_det, fcst_prob = bma.forecast(
    Predictant=obs,
    clim_year_start=1993, clim_year_end=2016,
    hindcast_det=hdcast,
    forecast_det=fcst
)

```

Extended Logistic Regression — `WAS_mme_xcELR`, `WAS_mme_ELR`

Estimates tercile probabilities directly by fitting a logistic regression on the ensemble mean as a continuous predictor of each category boundary.

- `WAS_mme_xcELR` — wraps the xcast ELR implementation (splits on S dimension).
- `WAS_mme_ELR` — native wass2s implementation.

```
from wass2s import WAS_mme_ELR, WAS_Cross_Validator

elr = WAS_mme_ELR()
cv = WAS_Cross_Validator(n_splits=len(obs.get_index("T")), nb_omit=2)
hindcast_det, hindcast_prob = cv.cross_validate(
    elr, obs, hdcst, clim_year_start=1993, clim_year_end=2016
)
```

Logistic MME — `WAS_mme_logistic`

Multinomial logistic regression trained on all GCM outputs simultaneously. Produces direct probability estimates for each tercile category.

```
from wass2s import WAS_mme_logistic, WAS_Cross_Validator

logistic_mme = WAS_mme_logistic()
cv = WAS_Cross_Validator(n_splits=len(obs.get_index("T")), nb_omit=2)
hindcast_det, hindcast_prob = cv.cross_validate(
    logistic_mme, obs, hdcst, clim_year_start=1993, clim_year_end=2016
)
```

Mean and Variance Adjustment — `WAS_mme_MVA`

A simple bias-correction technique that rescales the forecast mean and variance to match the observational climatology.

```
from wass2s import WAS_mme_MVA

mva = WAS_mme_MVA()
mva.fit(hindcast_da=hdcst, obs_da=obs)
calibrated = mva.transform(forecast_da=fcst)
```

1.8 Implementing WAS-NextGen Approaches

The WAS-NextGen (West Africa Seasonal Forecasting Next Generation) framework defines the methodological standards for operational seasonal forecasting across the region, aligned with WMO guidelines for objective, reproducible, and scientifically rigorous seasonal outlooks.

wass2s was designed from the ground up to implement this framework end-to-end. This page describes the recommended workflow for producing a WAS-NextGen compliant seasonal forecast.

1.8.1 Overview of the WAS-NextGen pipeline

A WAS-NextGen seasonal forecast consists of five main stages:

1. **Data preparation** — download and harmonise GCM hindcasts, reanalysis, and observational products.
2. **Predictor construction** — compute SST indices or spatial EOF predictors.
3. **Model calibration and cross-validation** — fit and validate one or more statistical or ML models using leave-one-out cross-validation.
4. **Multi-model combination** — combine calibrated models into an ensemble.
5. **Verification and dissemination** — score the hindcast and produce tercile-probability maps for end users.

1.8.2 Recommended model hierarchy

The WAS-NextGen guidelines recommend applying models in order of complexity, using verification scores to select the best performers:

Category	Recommended classes	Typical use case
Baseline	WAS_LinearRegression_Model	SST indices as predictors
Regularised linear	WAS_Ridge_Model, WAS_ElasticNet_Model	Correlated SST indices or PC predictors
Spatial dimensionality reduction	WAS_PCR + any regression back-end	Full SST or SLP fields
Non-linear	WAS_SVR, WAS_MLP, stacking ensembles	When residuals show non-linear structure
GCM recalibration	WAS_CCA, WAS_mme_NGR_Gaussian, WAS_mme_ELR	GCM output as predictor
MME combination	WAS_mme_Weighted, WAS_mme_FastBMA, WAS_mme_Stacking	Multi-model ensembles

1.8.3 Running multiple models and selecting the best

A typical WAS-NextGen workflow runs several candidate models, verifies each with GROC and RPSS, selects the top performers, and combines them.

```

from wass2s import *
import numpy as np

clim_year_start, clim_year_end = 1993, 2020

# --- Predictors and predictand (already downloaded) ---
predictand = prepare_predictand(...)
predictors = compute_sst_indices(...) # SST indices
predictor_field = load_gridded_predictor(...) # Full SST field

# --- Candidate models ---
models = {
    "OLS": WAS_LinearRegression_Model(nb_cores=4),
    "Ridge": WAS_Ridge_Model(n_clusters=6, nb_cores=4),
    "ElNet": WAS_ElasticNet_Model(nb_cores=4),
    "PCR": WAS_PCR(WAS_Ridge_Model(nb_cores=4), n_modes=8, detrend=True),
}

verifier = WAS_Verification()
cv = WAS_Cross_Validator(
    n_splits=len(predictand.get_index("T")), nb_omit=2
)

hindcasts_det = {}
hindcasts_prob = {}
groc_scores = {}

for name, model in models.items():
    # Hyperparameter tuning if available

```

(continues on next page)

(continued from previous page)

```

if hasattr(model, "compute_hyperparameters"):
    alpha, _ = model.compute_hyperparameters(
        predictand, predictors, clim_year_start, clim_year_end
    )
    params = {"alpha": alpha}
else:
    params = {}

hdet, hprob = cv.cross_validate(
    model, predictand,
    predictors.isel(T=slice(None, -1)),
    clim_year_start, clim_year_end, **params
)
hindcasts_det[name] = hdet
hindcasts_prob[name] = hprob

groc_scores[name] = verifier.compute_probabilistic_score(
    verifier.calculate_groc, predictand, hprob,
    clim_year_start=clim_year_start, clim_year_end=clim_year_end
)

# Select best models (top GROC over the domain average)
best_models = get_best_models(
    center_variable="PRCP",
    scores=groc_scores,
    metric="GROC",
    top_n=3
)
print("Best models:", best_models)

```

1.8.4 Multi-model ensemble of the best performers

Once the best individual models are identified, combine them with a skill-weighted ensemble:

```

from wass2s import WAS_mme_Weighted

# Stack selected hindcasts along the model dimension
hdcst_stack = xr.concat(
    [hindcasts_det[m] for m in best_models],
    dim="M"
).assign_coords(M=best_models)

# Skill scores for weighting
scores_stack = xr.concat(
    [groc_scores[m] for m in best_models],
    dim="M"
).assign_coords(M=best_models)

mme = WAS_mme_Weighted(equal_weighted=False, metric="GROC", threshold=0.5)
mme_det, mme_prob_fcst = mme.compute(
    predictand, hdcst_stack, fcst_stack, scores_stack
)

```

1.8.5 Operational forecast dissemination

Once the ensemble forecast is ready, generate the tercile-probability map:

```
plot_prob_forecasts(
    dir_to_save="./forecasts/",
    forecast_prob=mme_prob_fcst,
    model_name="WAS-NextGen MME",
    title=(
        "JJAS 2025 Rainfall Forecast - West Africa\n"
        "Issued April 2025"
    )
)
```

The output map shows three panels (PB, PN, PA) with the dominant category highlighted and probability values encoded in the colour scale, following the standard WAS-NextGen cartographic conventions.

1.8.6 Agroclimatic forecast products

In addition to seasonal rainfall totals, wass2s can produce forecasts for agroclimatic indices that are directly useful to national agricultural extension services:

```
from wass2s import WAS_compute_onset, WAS_compute_cessation

# Compute onset and cessation dates from downscaled daily data
onset_da = WAS_compute_onset().compute(daily_data=daily_rain, nb_cores=4)
cessation_da = WAS_compute_cessation().compute(daily_data=daily_rain, nb_
    ↪ cores=4)

# Season length (days)
season_length = cessation_da - onset_da
```

These outputs can then be used as predictands in the same modelling framework, replacing seasonal totals with any agroclimatic index computed by `was_compute_predictand`.

1.9 API Reference

This page is auto-generated from the source-code docstrings. Each section corresponds to one module in the `wass2s` package.

Note

Duplicate-object warnings (methods appearing in both `automodule` and `individual autoclass` blocks) are suppressed via `suppress_warnings` in `conf.py`. If you add new modules, add them here **and** in `conf.py`.

1.9.1 Data download

Data download utilities for ERA5, CHIRPS, TAMSAT, and NMME.

Provides a unified interface for downloading and pre-processing gridded seasonal hindcast and reanalysis datasets.

Class

WAS_Download

Central download manager. Key methods:

WAS_Download_Models

NMME and C3S seasonal forecast / hindcast data.

WAS_Download_Reanalysis / WAS_Download_Reanalysis_

ERA5 single-level and pressure-level reanalysis (via CDS).

WAS_Download_ERA5Land / WAS_Download_ERA5Land_daily

ERA5-Land daily and monthly data.

WAS_Download_CHIRPSv3_Seasonal / WAS_Download_CHIRPSv3_Daily

CHIRPS v3 precipitation (seasonal aggregates and daily fields).

WAS_Download_TAMSAT_Seasonal / WAS_Download_TAMSAT_Daily

TAMSAT African rainfall estimates.

WAS_Download_AgroIndicators / WAS_Download_AgroIndicators_daily

NMME-derived agroclimatic indicator downloads.

Standalone function

plot_map

Quick Cartopy map of a geographic extent.

class wass2s.was_download.WAS_Download

Bases: `object`

`__init__()`

Initialize the WAS_Download class.

ModelsName (`centre`={`'BOM_2'`: `'bom'`, `'CFSV2_1'`: `'cfsv2'`, `'CMC1_1'`: `'cmc1'`, `'CMC2_1'`: `'cmc2'`, `'CMCC_35'`: `'cmcc'`, `'CMCC_4'`: `'cmcc'`, `'DWD_21'`: `'dwd'`, `'DWD_22'`: `'dwd'`, `'ECCC_4'`: `'eccc'`, `'ECCC_5'`: `'eccc'`, `'ECMWF_51'`: `'ecmwf'`, `'GFDL_1'`: `'gfdl'`, `'JMA_3'`: `'jma'`, `'JMA_4'`: `'jma'`, `'METEOFRACTANCE_8'`: `'meteo_france'`, `'METEOFRACTANCE_9'`: `'meteo_france'`, `'NASA_1'`: `'nasa'`, `'NCAR_CCSM4_1'`: `'ncar_ccsm4'`, `'NCAR_CESM1_1'`: `'ncar_cesm1'`, `'NCEP_2'`: `'ncep'`, `'NMME_1'`: `'nmme'`, `'UKMO_603'`: `'ukmo'`, `'UKMO_604'`: `'ukmo'`, `'UKMO_605'`: `'ukmo'`, `'UKMO_610'`: `'ukmo'`}, `variables_1`={`'DLWR'`: `'surface_thermal_radiation_downwards'`, `'DSWR'`: `'surface_solar_radiation_downwards'`, `'NOLR'`: `'top_net_thermal_radiation'`, `'PRCP'`: `'total_precipitation'`, `'RUNOFF'`: `'mean_surface_runoff_rate'`, `'SLP'`: `'mean_sea_level_pressure'`, `'SST'`: `'sea_surface_temperature'`, `'TDEW'`: `'2m_dewpoint_temperature'`, `'TEMP'`: `'2m_temperature'`, `'TMAX'`: `'maximum_2m_temperature_in_the_last_24_hours'`, `'TMIN'`: `'minimum_2m_temperature_in_the_last_24_hours'`, `'UGRD10'`: `'10m_u_component_of_wind'`, `'VGRD10'`: `'10m_v_component_of_wind'`}, `variables_2`={`'HUSS_1000'`: `'specific_humidity'`, `'HUSS_850'`: `'specific_humidity'`, `'HUSS_925'`: `'specific_humidity'`, `'UGRD_1000'`: `'u_component_of_wind'`, `'UGRD_850'`: `'u_component_of_wind'`, `'UGRD_925'`: `'u_component_of_wind'`, `'VGRD_1000'`: `'v_component_of_wind'`, `'VGRD_850'`: `'v_component_of_wind'`, `'VGRD_925'`: `'v_component_of_wind'`})

Generate a combined dictionary of model names and variables. For more information on C3S, browse the [MetaData](#). For more information on NMME, browse the [MetaData](#).

Parameters

- **centre** (*dict*) – Mapping of model identifiers to model names.
- **variables_1** (*dict*) – Mapping of variable short names to full names for category 1.

- **variables_2** (*dict*) – Mapping of variable short names to full names for category 2.

Returns

A combined dictionary with keys as model.variable combinations and values as tuples (model name, variable name).

Return type

dict

ReanalysisName (*centre*={'ERA5': 'reanalysis ERA5', 'ERA5Land': 'reanalysis ERA5Land', 'NOAA': 'NOAA ERSST'}, *variables_1*={'DLWR': 'surface_thermal_radiation_downwards', 'DSWR': 'surface_solar_radiation_downwards', 'NOLR': 'top_thermal_radiation', 'PRCP': 'total_precipitation', 'RUNOFF': 'surface_runoff', 'SLP': 'mean_sea_level_pressure', 'SST': 'sea_surface_temperature', 'TEMP': '2m_temperature', 'UGRD10': '10m_u_component_of_wind', 'VGRD10': '10m_v_component_of_wind'}, *variables_2*={'HUSS_1000': 'specific_humidity', 'HUSS_850': 'specific_humidity', 'HUSS_925': 'specific_humidity', 'UGRD_1000': 'u_component_of_wind', 'UGRD_850': 'u_component_of_wind', 'UGRD_925': 'u_component_of_wind', 'VGRD_1000': 'v_component_of_wind', 'VGRD_850': 'v_component_of_wind', 'VGRD_925': 'v_component_of_wind'})

Generate a combined dictionary of reanalysis names and variables.

Parameters

- **centre** (*dict*) – Mapping of reanalysis identifiers to reanalysis names.
- **variables_1** (*dict*) – Mapping of variable short names to full names for category 1.
- **variables_2** (*dict*) – Mapping of variable short names to full names for category 2.

Returns

A combined dictionary with keys as reanalysis.variable combinations and values as tuples (re-analysis name, variable name).

Return type

dict

AgroObsName (*variables*={'AGRO.DSWR': ('solar_radiation_flux', None), 'AGRO.ETP': ('reference_evapotranspiration', None), 'AGRO.HUMAX': ('2m_relative_humidity_derived', '24_hour_maximum'), 'AGRO.HUMIN': ('2m_relative_humidity_derived', '24_hour_minimum'), 'AGRO.PRCP': ('precipitation_flux', None), 'AGRO.TEMP': ('2m_temperature', '24_hour_mean'), 'AGRO.TMAX': ('2m_temperature', '24_hour_maximum'), 'AGRO.TMIN': ('2m_temperature', '24_hour_minimum'), 'AGRO.WFF': ('10m_wind_speed', '24_hour_mean')})

Generate a dictionary for agrometeorological observation variables.

Parameters

variables (*dict*) – Mapping of agro variable short names to full names.

Returns

A dictionary mapping agro variables to their corresponding full names.

Return type

dict

download_nmme_txt_with_progress (*url*, *file_path*, *chunk_size*=1024)

days_in_month (*year*, *month*)

parse_cpt_data_optimized (*file_path*)

WAS_Download_Models (*dir_to_save, center_variable, month_of_initialization, lead_time, year_start_hindcast, year_end_hindcast, area, year_forecast=None, ensemble_mean=None, force_download=False, nmme_source='cpt', year_init_nmmeNC=2025*)

Download seasonal forecast model data for specified center-variable combinations, initialization month, lead times, and years.

Parameters

- **dir_to_save** (*str*) – Directory to save the downloaded files.
- **center_variable** (*list*) – List of center-variable identifiers (e.g., ["ECMWF_51.PRCP", "UKMO_602.TEMP"]).
- **month_of_initialization** (*int*) – Initialization month as an integer (1-12).
- **lead_time** (*list*) – List of lead times in months.
- **year_start_hindcast** (*int*) – Start year for hindcast data.
- **year_end_hindcast** (*int*) – End year for hindcast data.
- **area** (*list*) – Bounding box as [North, West, South, East] for clipping.
- **year_forecast** (*int, optional*) – Forecast year if downloading forecast data. Defaults to None.
- **mean** (*ensemble*) – it's can be median, mean or None. Defaults to None.
- **force_download** (*bool*) – If True, forces download even if file exists.

WAS_Download_AgroIndicators_daily (*dir_to_save, variables, year_start, year_end, area, force_download=False, max_retries=3, retry_delay=5*)

Download daily agro-meteorological indicators from the Copernicus Data Store (CDS) for specified variables and years, with retries for failed downloads.

Parameters

- **dir_to_save** (*str or pathlib.Path*) – Directory path where the downloaded NetCDF files will be saved. The directory will be created if it does not exist.
- **variables** (*list of str*) – List of variable shorthand names to download. Valid options are: ["AGRO.PRCP", "AGRO.TMAX", "AGRO.TEMP", "AGRO.TMIN", "AGRO.DSWR", "AGRO.ETP", "AGRO.WFF", "AGRO.HUMAX", "AGRO.HUMIN"]. Each variable corresponds to a CDS variable and optional statistic (e.g., "AGRO.PRCP" maps to "precipitation_flux").
- **year_start** (*int*) – Start year for the data to download (inclusive).
- **year_end** (*int*) – End year for the data to download (inclusive).
- **area** (*list of float*) – Bounding box for spatial subsetting in the format [North, West, South, East]. Example: [50, -10, 40, 10] for a region in Europe.
- **force_download** (*bool, optional*) – If True, forces download even if the output file exists. Default is False.
- **max_retries** (*int, optional*) – Maximum number of retry attempts for failed downloads. Default is 3.
- **retry_delay** (*int, optional*) – Seconds to wait between retry attempts. Default is 5.

Returns

The function saves NetCDF files to *dir_to_save* but does not return a value. Output files are named as *Daily_<variable>_<year_start>_<year_end>.nc*.

Return type

None

Notes

- The function downloads data from the CDS dataset “sis-agrometeorological-indicators”.
- Data is downloaded year-by-year as ZIP files containing NetCDF files, which are extracted, concatenated, and saved as a single NetCDF file per variable.
- Temperature variables (“AGRO.TMIN”, “AGRO.TEMP”, “AGRO.TMAX”) are converted from Kelvin to Celsius.
- Solar radiation (“AGRO.DSWR”) is converted from J/m²/day to W/m².
- Coordinates are renamed to “X” (longitude), “Y” (latitude), and “T” (time), with latitude flipped to ascending order.
- The function requires a valid CDS API key configured in *~/cdsapirc*.
- Downloads are skipped for a variable if any year’s data fails to download after *max_retries* attempts to ensure data completeness.

WAS_Download_Models_Daily (*dir_to_save*, *center_variable*, *month_of_initialization*,
day_of_initialization, *leadtime_hour*, *year_start_hindcast*,
year_end_hindcast, *area*, *year_forecast=None*, *ensemble_mean=None*,
force_download=False)

Download daily/sub-daily seasonal forecast model data (original) using ‘seasonal-original-single-levels’ from the CDS.

Parameters

- **dir_to_save** (*str* or *Path*) – Directory to save the downloaded files.
- **center_variable** (*list*) – Each element e.g. “ECMWF_51.PRCP” - left side of ‘.’ is model (ECMWF_51), - right side is variable short code (PRCP).
- **month_of_initialization** (*int*) – Initialization month (1-12).
- **day_of_initialization** (*int*) – Initialization day (1-31).
- **leadtime_hour** (*list of str*) – e.g. [“24”, “48”, ..., “5160”].
- **year_start_hindcast** (*int*) – Start year for hindcast data.
- **year_end_hindcast** (*int*) – End year for hindcast data.
- **area** (*list*) – Bounding box as [North, West, South, East].
- **year_forecast** (*int*, *optional*) – If provided, downloads that single forecast year. Otherwise downloads hindcast for the specified range.
- **ensemble_mean** (*str*, *optional*) – e.g. “mean”, “median”, or None.
- **force_download** (*bool*) – Force download if True, even if file exists.

WAS_Download_AgroIndicators (*dir_to_save*, *variables*, *year_start*, *year_end*, *area*, *seas=[‘01’, ‘02’, ‘03’]*, *force_download=False*, *max_retries=3*, *retry_delay=5*)

Download agro-meteorological indicators for specified variables, years, and months, handling cross-year seasons (e.g., NDJ) with retries for failed downloads.

Parameters

- **dir_to_save** (*str*) – Directory to save the downloaded files.
- **variables** (*list*) – List of shorthand variables (e.g., ["AGRO.PRCP", "AGRO.TMAX"]).
- **year_start** (*int*) – Start year for the data.
- **year_end** (*int*) – End year for the data.
- **area** (*list*) – Bounding box as [North, West, South, East].
- **seas** (*list*) – List of months (e.g., ["11", "12", "01"] for NDJ).
- **force_download** (*bool*) – If True, forces download even if file exists.
- **max_retries** (*int*) – Maximum number of retry attempts for failed downloads (default: 3).
- **retry_delay** (*int*) – Seconds to wait between retry attempts (default: 5).

WAS_Download_Reanalysis (*dir_to_save, center_variable, year_start, year_end, area, seas=['01', '02', '03'], force_download=False, run_avg=1*)

Download reanalysis data for specified center-variable combinations, years, and months, handling cross-year seasons (e.g., NDJ).

WAS_Download_Reanalysis_ (*dir_to_save, center_variable, year_start, year_end, area, seas=['01', '02', '03'], force_download=False, run_avg=3*)

Download reanalysis data for specified center-variable combinations, years, and months, handling cross-year seasons (e.g., NDJ).

WAS_Download_ERA5Land (*dir_to_save, center_variable, year_start, year_end, area, seas=['01', '02', '03'], force_download=False*)

Download ERA5Land reanalysis data for specified variable combinations, years, and months, handling cross-year seasons (e.g., NDJ). :param dir_to_save: Directory to save the downloaded files. :type dir_to_save: str :param center_variable: List of center-variable identifiers (e.g., ["ERA5Land.PRCP", "ERA5Land.TEMP"]). :type center_variable: list :param year_start: Start year for the data. :type year_start: int :param year_end: End year for the data. :type year_end: int :param area: Bounding box as [North, West, South, East]. :type area: list :param seas: List of months (e.g., ["11", "12", "01"] for NDJ). :type seas: list :param force_download: If True, forces download even if file exists. :type force_download: bool

WAS_Download_ERA5Land_daily (*dir_to_save, center_variable, year_start, year_end, area, force_download=False, max_retries=3, retry_delay=5*)

Download daily ERA5Land reanalysis data from the Copernicus Data Store (CDS) for specified variables and years, with retries for failed downloads.

Parameters

- **dir_to_save** (*str or pathlib.Path*) – Directory path where the downloaded NetCDF files will be saved. The directory will be created if it does not exist.
- **center_variable** (*list of str*) – List of center-variable identifiers (e.g., ["ERA5Land.PRCP", "ERA5Land.TEMP"]).
- **year_start** (*int*) – Start year for the data to download (inclusive).
- **year_end** (*int*) – End year for the data to download (inclusive).
- **area** (*list of float*) – Bounding box for spatial subsetting in the format [North, West, South, East].

- **force_download** (*bool, optional*) – If True, forces download even if the output file exists. Default is False.
- **max_retries** (*int, optional*) – Maximum number of retry attempts for failed downloads. Default is 3.
- **retry_delay** (*int, optional*) – Seconds to wait between retry attempts. Default is 5.

Returns

The function saves NetCDF files to *dir_to_save* but does not return a value. Output files are named as *Daily_<variable>_<year_start>_<year_end>.nc*.

Return type

None

Notes

- The function downloads hourly data from the CDS dataset “reanalysis-era5-land” and aggregates it to daily resolution.
- Aggregation: sum for accumulative variables (e.g., PRCP, radiation), mean for others (e.g., TEMP).
- Unit conversions: PRCP to mm/day, TEMP to °C, radiation to W/m² (daily average).
- Coordinates are renamed to “X” (longitude), “Y” (latitude), and “T” (time), with latitude flipped.
- Downloads are performed month-by-month due to API limitations.
- Requires a valid CDS API key configured in *~/cdsapirc*.

WAS_Download_CHIRPSv3_Daily (*dir_to_save, year_start, year_end, blend_type='ERA5', area=None, force_download=False, max_retries=3, retry_delay=5*)

Download daily CHIRPS v3.0 precipitation (blended with ERA5 or IMERGlate-v07) from the Copernicus Data Store (CDS) for specified years, with retries for failed downloads.

Parameters

- **dir_to_save** (*str or pathlib.Path*) – Directory path where the downloaded NetCDF files will be saved. The directory will be created if it does not exist.
- **year_start** (*int*) – Start year for the data to download (inclusive).
- **year_end** (*int*) – End year for the data to download (inclusive).
- **blend_type** (*str*) – Blend type, either “ERA5” or “IMERGlate-v07” (default: “ERA5”). Note: IMERGlate-v07 availability starts from ~2000; earlier years may fail.
- **area** (*list of float*) – Bounding box for spatial subsetting in the format [North, West, South, East].
- **force_download** (*bool, optional*) – If True, forces download even if the output file exists. Default is False.
- **max_retries** (*int, optional*) – Maximum number of retry attempts for failed downloads. Default is 3.
- **retry_delay** (*int, optional*) – Seconds to wait between retry attempts. Default is 5.

Returns

The function saves NetCDF files to *dir_to_save* but does not return a value. Output files are named as *Daily_PRCP_{blend_type}_{year_start}_{year_end}.nc*.

Return type

None

Notes

- Downloads individual daily TIFF files, processes them with rioarray, clips if area specified, and combines into a single NetCDF with daily time dimension.
- Units: precipitation in mm/day.
- Coordinates renamed to “X” (longitude), “Y” (latitude), “T” (time), with Y flipped if needed.
- Deletes temporary TIFF files after processing.
- Skips invalid dates (e.g., Feb 30) automatically.

WAS_Download_CHIRPSv3_Seasonal (*dir_to_save*, *variables*, *year_start*, *year_end*, *region='africa'*,
area=None, *season_months=['03', '04', '05']*,
force_download=False)

Download CHIRPS v3.0 monthly precipitation for a specified cross-year season from *year_start* to *year_end*, optionally clipped to *area*, and aggregate them into a single NetCDF file. :param *dir_to_save*: Directory to save the downloaded files. :type *dir_to_save*: str :param *variables*: List of variables to download (e.g., ["PRCP"]). :type *variables*: list :param *year_start*: Start year for the data. :type *year_start*: int :param *year_end*: End year for the data. :type *year_end*: int :param *region*: CHIRPS region (default: "africa"). :type *region*: str :param *area*: Bounding box as [North, West, South, East] (optional). :type *area*: list :param *season_months*: List of months as strings (e.g., ["03", "04", "05"]). :type *season_months*: list :param *force_download*: If True, forces download even if file exists. :type *force_download*: bool

Returns

Saves the aggregated seasonal data to a NetCDF file.

Return type

None

WAS_Download_TAMSAT_Seasonal (*dir_to_save*: *str* | *Path*, *product*: *Literal*['rfe', 'soil_moisture'] = 'rfe',
variables: *Sequence*[*str*] | *None* = *None*, *year_start*: *int* = 1983,
year_end: *int* = 2025, *area*: *List*[*float*] | *None* = *None*,
season_months: *List*[*str*] = ['03', '04', '05'], *version*: *str* | *None* =
None, *force_download*: *bool* = *False*, *agg*: *Literal*['sum', 'mean'] |
None = *None*) → *Path*

Download and aggregate TAMSAT monthly data (RFE v3.1 or Soil Moisture v2.3.1) for a specified season.

Parameters

- **dir_to_save** (*str* | *Path*) – Directory where monthly files and the seasonal output will be saved.
- **product** ({*"rfe"*, *"soil_moisture"*}, *default* *"rfe"*) – Dataset to download: “rfe” (precipitation) or “soil_moisture”.
- **variables** (*sequence of str*, *optional*) – Names of variables to extract from NetCDF. If *None*, chosen by product. - rfe: defaults to (“rfe”), - soil_moisture: defaults to (“sm”).
- **year_start** (*int*, *default* 1983) – First seasonal year (pivot year) to include.
- **year_end** (*int*, *default* 2025) – Last year for which data is included (inclusive). For seasons spanning calendar years, the last pivot year processed will be *year_end* - 1 to ensure no data from *year_end* + 1 is fetched.

- **area** (*list[float]*, *optional*) – Bounding box [north, west, south, east] in degrees.
- **season_months** (*sequence[str]*, *default* ["03", "04", "05"]) – Months defining the season, e.g. ["11", "12", "01"] for NDJ.
- **version** (*str*, *optional*) – Product version. Defaults to: - rfe: "v3.1" - soil_moisture: "v2.3.1"
- **force_download** (*bool*, *default* *False*) – Re-download monthly files even if present locally.
- **agg** ({*"sum"*, *"mean"*}, *optional*) – Seasonal aggregation. Defaults to: - rfe: "sum" - soil_moisture: "mean"

Returns

Path to the seasonal aggregated NetCDF file.

Return type

Path

```
WAS_Download_TAMSAT_Daily (dir_to_save: str | Path, product: Literal['rfe', 'soilmoisture'] = 'rfe',  
                             variables: Sequence[str] | None = None, year_start: str = '1983',  
                             year_end: str = '2024', area: List[float] | None = None, version: str |  
                             None = None, force_download: bool = False) → Path
```

Download TAMSAT daily data (RFE v3.1 or Soil Moisture v2.3.1) and combine into a single NetCDF file.

Parameters

- **dir_to_save** (*str | Path*) – Directory where daily files and the combined output will be saved.
- **product** ({*"rfe"*, *"soil_moisture"*}, *default* *"rfe"*) – Dataset to download: "rfe" (precipitation) or "soil_moisture".
- **variables** (*sequence of str*, *optional*) – Names of variables to extract from NetCDF. If None, chosen by product. - rfe: defaults to ("rfe",) - soil_moisture: defaults to ("sm",)
- **start_date** (*str*, *default* *"1983-01-01"*) – Start date in "YYYY-MM-DD" format.
- **end_date** (*str*, *default* *"2025-10-20"*) – End date in "YYYY-MM-DD" format (inclusive).
- **area** (*list[float]*, *optional*) – Bounding box [north, west, south, east] in degrees.
- **version** (*str*, *optional*) – Product version. Defaults to: - rfe: "v3.1" - soil_moisture: "v2.3.1"
- **force_download** (*bool*, *default* *False*) – Re-download daily files even if present locally.

Returns

Path to the combined daily NetCDF file.

Return type

Path

```
wass2s.was_download.plot_map (extent, title='Map')
```

Plots a map with specified geographic extent.

Parameters: - extent: list of float, specifying [west, east, south, north] - title: str, title of the map

1.9.2 Utilities

Shared utility functions for the WASS2S pipeline.

Covers data loading, preprocessing, index computation, predictor retrieval, and forecast visualisation.

Data utilities

decode_cf

Decode CF-convention time coordinates.

fix_time_coord

Re-assign a seasonal T coordinate to a dataset.

standardize_timeseries / reverse_standardize

Climatological standardisation / de-standardisation of xarray datasets.

anomalize_timeseries

Climatological anomaly computation.

detrended_data / apply_detrend_data

Fold-safe linear detrending helpers.

predictant_mask

Binary land/ocean mask derived from a predictand DataArray.

extract_leading_eeof_component

Extract the leading extended-EOF component.

Index and predictor loaders

compute_sst_indices

Compute standard SST indices (Niño 3.4, AMM, IOD, ...) from gridded data.

compute_other_indices

Compute arbitrary spatial-average indices over user-defined boxes.

retrieve_several_zones_for_PCR

Build a multivariate predictor list (one field per zone) for PCR.

retrieve_single_zone_for_PCR

Build a single standardized predictor DataArray for PCR.

load_gridded_predictor

Load a gridded model or reanalysis predictor field.

prepare_predictand

Load, merge, and optionally aggregate an observational predictand.

Network utilities

download_file

Stream a file from a URL with progress reporting and retry logic.

build_iridl_url_ersst / to_iridl_lat / to_iridl_lon

Build IRI Data Library URLs for ERSSTv5.

parse_variable

Map user-facing variable names to CDS / NMME field names.

Geo utilities

`get_shapefile` / `get_shapefile_`

Retrieve country or sub-national shapefiles (Natural Earth / GADM / geoBoundaries).

`plot_map`

Quick map of a geographic extent with optional SST index overlays.

`get_best_models`

Select top-N forecast models by a given skill metric.

Visualisation

`plot_prob_forecasts` / `plot_prob_forecastsAlpha`

Tercile probability forecast map panels.

`wass2s.utils.decode_cf(ds, time_var)`

Decode time dimension to CFTIME standards.

Parameters

- **ds** (`xarray.Dataset`) – Input dataset containing the time variable to decode.
- **time_var** (`str`) – Name of the time variable in the dataset.

Returns

Dataset with decoded time dimension adhering to CFTIME standards.

Return type

`xarray.Dataset`

Notes

If the calendar attribute of the time variable is '360', it is converted to '360_day' to ensure compatibility with CFTIME decoding.

`wass2s.utils.to_iridl_lat(lat: float) → str`

Convert numeric latitude to IRIDL latitude string format.

Parameters

lat (`float`) – Latitude value in degrees (positive for North, negative for South).

Returns

IRIDL-formatted latitude string (e.g., '10N' for +10, '5S' for -5).

Return type

`str`

Examples

```
>>> to_iridl_lat(10)
'10N'
>>> to_iridl_lat(-5)
'5S'
```

`wass2s.utils.to_iridl_lon(lon: float) → str`

Convert numeric longitude to IRIDL longitude string format.

Parameters

lon (`float`) – Longitude value in degrees (positive for East, negative for West).

Returns

IRIDL-formatted longitude string (e.g., '15E' for +15, '15W' for -15).

Return type

str

Examples

```
>>> to_iridl_lon(15)
'15E'
>>> to_iridl_lon(-15)
'15W'
```

`wass2s.utils.build_iridl_url_ersst` (*year_start*: int, *year_end*: int, *bbox*: list, *run_avg*: int = 3, *month_start*: str = 'Jan', *month_end*: str = 'Dec')

Build a parameterized IRIDL URL for NOAA ERSST dataset.

Parameters

- **year_start** (int) – Start year for the data request.
- **year_end** (int) – End year for the data request.
- **bbox** (list) – Bounding box coordinates in the format [North, West, South, East].
- **run_avg** (int, optional) – Number of time steps for running average (default is 3). If None, no running average is applied.
- **month_start** (str, optional) – Starting month for the time range (default is 'Jan').
- **month_end** (str, optional) – Ending month for the time range (default is 'Dec').

Returns

Constructed IRIDL URL for accessing NOAA ERSST data.

Return type

str

Notes

The bounding box is reordered to match IRIDL's expected format: Y/(south)/(north)/, X/(west)/(east)/.

`wass2s.utils.fix_time_coord` (*ds*, *seas*)

Fix time coordinates by extracting year and assigning a fixed month/day.

Parameters

- **ds** (*xarray.Dataset*) – Input dataset with time dimension 'T'.
- **seas** (tuple) – Tuple containing season information, where `seas[1]` is the month (e.g., '11' for November).

Returns

Dataset with updated time coordinates in the format 'YYYY-MM-01'.

Return type

xarray.Dataset

Notes

Assumes the input dataset's time values can be converted to pandas datetime objects.

```
wass2s.utils.download_file(url, local_path, force_download=False, chunk_size=8192, timeout=120)
```

Download a file from a URL to a local path with progress tracking.

Parameters

- **url** (*str*) – URL of the file to download.
- **local_path** (*str* or *pathlib.Path*) – Local path where the file will be saved.
- **force_download** (*bool*, *optional*) – If True, overwrite existing file (default is False).
- **chunk_size** (*int*, *optional*) – Size of chunks for streaming download (default is 8192 bytes).
- **timeout** (*int*, *optional*) – Timeout for the HTTP request in seconds (default is 120).

Returns

Path to the downloaded file, or None if download fails.

Return type

pathlib.Path or None

Notes

Skips download if the file already exists and *force_download* is False.

```
wass2s.utils.parse_variable(variables_list)
```

Extract center and variable names from a variable string.

Parameters

variables_list (*str*) – Variable string in the format 'center.variable'.

Returns

A tuple containing (center, variable).

Return type

tuple

Examples

```
>>> parse_variable("NOAA.SST")
('NOAA', 'SST')
```

```
wass2s.utils.standardize_timeseries(ds, clim_year_start=None, clim_year_end=None)
```

Standardize a dataset over a specified climatology period.

Parameters

- **ds** (*xarray.Dataset* or *xarray.DataArray*) – Input dataset or data array to standardize.
- **clim_year_start** (*int*, *optional*) – Start year of the climatology period. If None, uses the full time range.
- **clim_year_end** (*int*, *optional*) – End year of the climatology period. If None, uses the full time range.

Returns

Standardized dataset or data array (z-scores).

Return type

`xarray.Dataset` or `xarray.DataArray`

Notes

Standardization is performed as $(ds - \text{mean}) / \text{std}$, where mean and std are computed over the specified climatology period or the entire time dimension.

`wass2s.utils.reverse_standardize(ds_st, ds, clim_year_start=None, clim_year_end=None)`

Reverse standardization of a dataset to original units.

Parameters

- **ds_st** (`xarray.Dataset` or `xarray.DataArray`) – Standardized dataset or data array (z-scores).
- **ds** (`xarray.Dataset` or `xarray.DataArray`) – Original dataset used to compute standardization parameters.
- **clim_year_start** (`int`, *optional*) – Start year of the climatology period. If `None`, uses the full time range.
- **clim_year_end** (`int`, *optional*) – End year of the climatology period. If `None`, uses the full time range.

Returns

Dataset or data array in original units.

Return type

`xarray.Dataset` or `xarray.DataArray`

Notes

Reverses standardization using $ds_st * \text{std} + \text{mean}$, where mean and std are computed from the original dataset over the specified period.

`wass2s.utils.anomalize_timeseries(ds, clim_year_start=None, clim_year_end=None)`

Compute anomalies by subtracting the climatological mean.

Parameters

- **ds** (`xarray.Dataset` or `xarray.DataArray`) – Input dataset or data array.
- **clim_year_start** (`int`, *optional*) – Start year of the climatology period. If `None`, uses the full time range.
- **clim_year_end** (`int`, *optional*) – End year of the climatology period. If `None`, uses the full time range.

Returns

Anomalized dataset or data array ($ds - \text{mean}$).

Return type

`xarray.Dataset` or `xarray.DataArray`

Notes

Anomalies are computed as $ds - \text{mean}$, where mean is calculated over the specified climatology period or the entire time dimension.

`wass2s.utils.predictant_mask(data)`

Create a mask for predictand data based on rainfall thresholds and latitude.

Parameters

data (*xarray.DataArray*) – Input data array with a time dimension ‘T’ and spatial dimensions ‘Y’, ‘X’.

Returns

Mask with 1 where conditions are met, NaN elsewhere.

Return type

xarray.DataArray

Notes

The mask is set to 1 where mean rainfall > 20 mm and latitude is within ± 19.5 degrees, otherwise NaN.

`wass2s.utils.extract_leading_eeof_component(data)`

extract leading eeof component using ExtendedEOF.

Parameters

data (*xarray.DataArray*) – Input data array with time dimension ‘T’.

Returns

Trended data array based on the first mode of ExtendedEOF.

Return type

xarray.DataArray

Raises

RuntimeError – If ExtendedEOF fitting fails.

Notes

Missing values are filled with the time mean before fitting. Uses ExtendedEOF with specified parameters (n_modes=2, tau=1, embedding=3, n_pca_modes=20).

`wass2s.utils.detrended_data(da, dim='T', min_valid=10)`

`wass2s.utils.apply_detrend_data(da, coeffs, meta)`

`wass2s.utils.prepare_predictand(dir_to_save_Obs, variables_obs, year_start, year_end, season=None, ds=True, daily=False, param='prcp')`

Prepare the predictand dataset from observational data.

`wass2s.utils.load_gridded_predictor(dir_to_data, variables_list, year_start, year_end, season=None, model=False, month_of_initialization=None, lead_time=None, year_forecast=None)`

Load gridded predictor data for reanalysis or model.

Parameters

- **dir_to_data** (*str*) – Directory path where predictor data is stored.
- **variables_list** (*str*) – Variable string in the format ‘center.variable’.
- **year_start** (*int*) – Start year of the data.

- **year_end** (*int*) – End year of the data.
- **season** (*list*, *optional*) – List of month numbers defining the season.
- **model** (*bool*, *optional*) – If True, load model data (hindcast/forecast); otherwise, load reanalysis data (default is False).
- **month_of_initialization** (*int*, *optional*) – Month of model initialization (required if model=True).
- **lead_time** (*list*, *optional*) – List of lead times in months (required if model=True).
- **year_forecast** (*int*, *optional*) – Forecast year (required for forecast data if model=True).

Returns

Loaded predictor data array renamed to ‘predictor’.

Return type

`xarray.DataArray`

Notes

Converts dataset to data array, drops ‘variable’ dimension, and ensures datetime64[ns] time coordinates.

```
wass2s.utils.compute_sst_indices (dir_to_data, indices, variables_list, year_start, year_end, season,
                                clim_year_start=None, clim_year_end=None, others_zone=None,
                                model=False, month_of_initialization=None, lead_time=None,
                                year_forecast=None)
```

Compute Sea Surface Temperature (SST) indices for reanalysis or model data.

Parameters

- **dir_to_data** (*str*) – Directory path where data is stored.
- **indices** (*list*) – List of SST indices to compute (e.g., ['NINO34', 'TNA']).
- **variables_list** (*str*) – Variable string in the format ‘center.variable’.
- **year_start** (*int*) – Start year of the data.
- **year_end** (*int*) – End year of the data.
- **season** (*list*) – List of month numbers defining the season.
- **clim_year_start** (*int*, *optional*) – Start year for climatology period.
- **clim_year_end** (*int*, *optional*) – End year for climatology period.
- **others_zone** (*dict*, *optional*) – Additional custom zones for SST indices with coordinates.
- **model** (*bool*, *optional*) – If True, load model data; otherwise, load reanalysis data (default is False).
- **month_of_initialization** (*int*, *optional*) – Month of model initialization.
- **lead_time** (*list*, *optional*) – List of lead times in months.
- **year_forecast** (*int*, *optional*) – Forecast year.

Returns

Dataset containing computed SST indices as variables.

Return type

`xarray.Dataset`

Notes

Derived indices like ‘TASI’ and ‘DMI’ are computed as differences of other indices.

```
wass2s.utils.compute_other_indices(dir_to_data, indices_dict, variables_list, year_start, year_end,  
                                   season, clim_year_start=None, clim_year_end=None,  
                                   model=False, month_of_initialization=None, lead_time=None,  
                                   year_forecast=None)
```

Compute indices for non-SST variables.

Parameters

- **dir_to_data** (*str*) – Directory path where data is stored.
- **indices_dict** (*dict*) – Dictionary mapping index names to coordinates (label, lon_min, lon_max, lat_min, lat_max).
- **variables_list** (*str*) – Variable string in the format ‘center.variable’.
- **year_start** (*int*) – Start year of the data.
- **year_end** (*int*) – End year of the data.
- **season** (*list*) – List of month numbers defining the season.
- **clim_year_start** (*int*, *optional*) – Start year for climatology period.
- **clim_year_end** (*int*, *optional*) – End year for climatology period.
- **model** (*bool*, *optional*) – If True, load model data; otherwise, load reanalysis data (default is False).
- **month_of_initialization** (*int*, *optional*) – Month of model initialization.
- **lead_time** (*list*, *optional*) – List of lead times in months.
- **year_forecast** (*int*, *optional*) – Forecast year.

Returns

Dataset containing computed indices as variables.

Return type

`xarray.Dataset`

```
wass2s.utils.retrieve_several_zones_for_PCR(dir_to_data, indices_dict, variables_list, year_start,  
                                             year_end, season, clim_year_start=None,  
                                             clim_year_end=None, model=False,  
                                             month_of_initialization=None, lead_time=None,  
                                             year_forecast=None)
```

Retrieve data for multiple zones for Principal Component Regression (PCR).

Parameters

- **dir_to_data** (*str*) – Directory path where data is stored.
- **indices_dict** (*dict*) – Dictionary mapping index names to coordinates (label, lon_min, lon_max, lat_min, lat_max).
- **variables_list** (*str*) – Variable string in the format ‘center.variable’.
- **year_start** (*int*) – Start year of the data.
- **year_end** (*int*) – End year of the data.
- **season** (*list*) – List of month numbers defining the season.

- **clim_year_start** (*int*, *optional*) – Start year for climatology period.
- **clim_year_end** (*int*, *optional*) – End year for climatology period.
- **model** (*bool*, *optional*) – If True, load model data (hindcast and forecast); otherwise, load reanalysis data (default is False).
- **month_of_initialization** (*int*, *optional*) – Month of model initialization.
- **lead_time** (*list*, *optional*) – List of lead times in months.
- **year_forecast** (*int*, *optional*) – Forecast year.

Returns

List of xarray.DataArray objects, each representing a standardized region.

Return type

list

```
wass2s.utils.retrieve_single_zone_for_PCR(dir_to_data, indices_dict, variables_list, year_start,
                                           year_end, season=None, clim_year_start=None,
                                           clim_year_end=None, model=False,
                                           month_of_initialization=None, lead_time=None,
                                           year_forecast=None, standardize=True)
```

Retrieve data for a single zone for Principal Component Regression (PCR) with interpolation.

skill_mask**dir_to_data**

[str] Directory path where data is stored.

indices_dict

[dict] Dictionary mapping a single index name to coordinates (label, lon_min, lon_max, lat_min, lat_max).

variables_list

[str] Variable string in the format 'center.variable'.

year_start

[int] Start year of the data.

year_end

[int] End year of the data.

season

[list, optional] List of month numbers defining the season.

clim_year_start

[int, optional] Start year for climatology period.

clim_year_end

[int, optional] End year for climatology period.

model

[bool, optional] If True, load model data (hindcast and forecast); otherwise, load reanalysis data (default is False).

month_of_initialization

[int, optional] Month of model initialization.

lead_time

[list, optional] List of lead times in months.

year_forecast

[int, optional] Forecast year.

xarray.DataArray

Standardized and interpolated data array for the specified region.

```
wass2s.utils.plot_map(extent, title='Map', sst_indices=None, fig_size=(10, 8))
```

Plot a map with specified geographic extent and optional SST index boxes.

Parameters

- **extent** (*list*) – Geographic extent in the format [west, east, south, north].
- **title** (*str*, *optional*) – Title of the map (default is 'Map').
- **sst_indices** (*dict*, *optional*) – Dictionary containing SST index information with keys as index names and values as tuples (label, lon_w, lon_e, lat_s, lat_n).
- **fig_size** (*tuple*, *optional*) – Figure size as (width, height) in inches (default is (10, 8)).

Notes

Uses Cartopy for map projection and Matplotlib for plotting.

```
wass2s.utils.get_best_models(center_variable, scores, metric='MAE', threshold=None, top_n=6,  
                             gcm=False, agroparam=False, hydro=False)
```

```
wass2s.utils.get_shapefile_(country_code, admin_level=0, source='naturalearth')
```

Downloads or loads shapefiles based on country ISO3 code.

```
wass2s.utils.get_shapefile(country_code, admin_level=0, source='naturalearth')
```

Downloads or loads shapefiles based on country ISO3 code. Automatically switches to GADM if admin_level > 1.

```
wass2s.utils.plot_prob_forecastsAlpha(dir_to_save, forecast_prob, model_name,  
                                       country_code='GHA', admin_level=0, source='naturalearth',  
                                       labels=['Below-Normal', 'Near-Normal', 'Above-Normal'],  
                                       reverse_cmap=True, hspace=None, logo=None,  
                                       logo_size=('10%', '10%'), logo_position='lower left',  
                                       sigma=None, res=None, stations_df=None, out='pdf',  
                                       dynamic_scalebar=False)
```

```
wass2s.utils.plot_prob_forecasts(dir_to_save, forecast_prob, model_name, title='Seasonal Forecast for  
Gulf of Guinea Countries \n Valid for March-April-May 2026, Issued  
February 27, 2026', country_code='GHA', admin_level=0,  
source='naturalearth', labels=['Below-Normal', 'Near-Normal',  
'Above-Normal'], reverse_cmap=True, hspace=None, logo=None,  
logo_size=('10%', '10%'), logo_position='lower left', logo_left=None,  
logo_left_size=('12%', '12%'), logo_right=None,  
logo_right_size=('12%', '12%'), sigma=None, res=None,  
stations_df=None, out='pdf', dynamic_scalebar=False)
```

```
wass2s.utils.plot_prob_forecasts_(dir_to_save, forecast_prob, model_name, labels=['Below-Normal',  
                                         'Near-Normal', 'Above-Normal'], reverse_cmap=True,  
                                   hspace=None, logo=None, logo_size=(None, None),  
                                   logo_position='lower left', sigma=None, res=None)
```

Plot probabilistic forecasts with tercile categories using discrete block colorbars.

`wass2s.utils.plot_det_forecasts` (*da2d*: *DataArray*, *title*: *str*, *outpng*: *str* = *None*)

Plot a 2D field (Y, X) on a PlateCarree map.

`wass2s.utils.plot_tercile` (*A*, *save_dir*=*None*, *colors*=*None*, *year*=*None*)

Plot a tercile map with categories: Below, Normal, Above.

Parameters

A (*xarray.DataArray*) – Data array with tercile categories (0: Below, 1: Normal, 2: Above) and dimensions 'T', 'Y', 'X'.

Notes

Uses a custom colormap and displays a legend for tercile categories.

`wass2s.utils.polygons_to_prob_ds` (*da_like*: *DataArray*, *gdf*: *GeoDataFrame*, *forecast_col*: *str* = 'Forecast', *categories*=(*'PB'*, *'PN'*, *'PA'*), *time_dim*: *str* = 'T', *y_dim*: *str* = 'Y', *x_dim*: *str* = 'X')

Rasterize polygon forecasts onto the (Y,X) grid of *da_like* and return:

xr.Dataset with dims (probability, T, Y, X) and coords probability=[*'PB'*, *'PN'*, *'PA'*].

da_like is only used for its coords (1 time step expected).

`wass2s.utils.plot_georeferenced_forecast_map` (*shapefile_zip*, *column*=*'Forecast'*, *extent_obs*=[12, -3.5, 4, 1.5], *title*=*'Georeferenced Map of 2024 consensual Forecast'*, *figsize*=(9, 7), *cmap*=*'tab20'*, *outpath*=*'georeferenced_map.pdf'*, *dpi*=300, *show*=*True*)

`wass2s.utils.process_model_for_other_params` (*agmParamModel*, *dir_to_save*, *hdcst_file_path*, *fcst_file_path*, *obs_hdcst*, *obs_fcst_year*, *month_of_initialization*, *year_start*, *year_end*, *year_forecast*, *nb_cores*=2, *agrometparam*=*'Onset'*)

Process model hindcast and forecast data for agrometeorological parameters.

Parameters

- **agmParamModel** (*object*) – Model object with a compute method for processing agrometeorological parameters.
- **dir_to_save** (*str*) – Directory path to save processed data.
- **hdcst_file_path** (*dict*) – Dictionary mapping model names to hindcast file paths.
- **fcst_file_path** (*dict*) – Dictionary mapping model names to forecast file paths.
- **obs_hdcst** (*xarray.Dataset*) – Observational hindcast dataset.
- **obs_fcst_year** (*xarray.Dataset*) – Observational forecast dataset for the specified year.
- **month_of_initialization** (*int*) – Month of model initialization.
- **year_start** (*int*) – Start year of the hindcast data.
- **year_end** (*int*) – End year of the hindcast data.
- **year_forecast** (*int*) – Forecast year.
- **nb_cores** (*int*, *optional*) – Number of CPU cores for parallel processing (default is 2).

- **agrometparam**(*str*, *optional*) – Name of the agrometeorological parameter (default is 'Onset').

Returns

Tuple of dictionaries (saved_hindcast_paths, saved_forecast_paths) mapping model names to saved file paths.

Return type

tuple

wass2s.utils.plot_date(*A*)

Plot a map of dates, interpreting values as offsets from 2024-01-01.

Parameters

A (*xarray.DataArray*) – Data array with values representing days since 2024-01-01, and dimensions 'Y', 'X'.

Notes

Converts colorbar ticks to calendar dates (e.g., '01-Jan') for readability.

wass2s.utils.verify_station_network(*df_filtered*, *extent*, *map_name*='Rain-gauge network')

Plot station locations on a map to verify the station network.

Parameters

- **df_filtered** (*pandas.DataFrame*) – DataFrame containing station data with 'STATION' column ('LAT', 'LON') and station names.
- **extent** (*list*) – Geographic extent in the format [west, east, south, north].
- **map_name** (*str*, *optional*) – Title of the map (default is 'Rain-gauge network').

Notes

Stations are plotted as red markers with labels, using Cartopy for map features.

wass2s.utils.proceed_seasonal_daily_bias_correction(*dir_to_save_model*, *observation*,
hindcast_files, *forecast_files*,
varname='PRCP', *method*='QUANT',
wet_day=True, *qstep*=0.01, *distr*=None,
transfun=None)

wass2s.utils.pre_process_biophysical_model(*dir_to_save*, *hdcst_file_path*, *fcst_file_path*, *obs_hdcst*,
obs_fcst_year, *month_of_initialization*, *year_start*,
year_end, *year_forecast*, *param*='PRCP')

Process model hindcasts and forecasts data for extended seasonal forecasts with biophysical models (Hype, SAR-RAO).

Parameters

- **dir_to_save** (*str*) – Directory path to save processed data.
- **hdcst_file_path** (*dict*) – Dictionary mapping model names to hindcast file paths.
- **fcst_file_path** (*dict*) – Dictionary mapping model names to forecast file paths.
- **obs_hdcst** (*xarray.Dataset*) – Observational hindcast dataset.
- **obs_fcst_year** (*xarray.Dataset*) – Observational forecast dataset for the specified year.

- **month_of_initialization** (*int*) – Month of model initialization.
- **year_start** (*int*) – Start year of the hindcast data.
- **year_end** (*int*) – End year of the hindcast data.
- **year_forecast** (*int*) – Forecast year.

Returns

Tuple of dictionaries (saved_hindcast_paths, saved_forecast_paths) mapping model names to saved file paths.

Return type

`tuple`

`wass2s.utils.extraterrestrial_radiation` (*lat_da, time*)

Calculate daily extraterrestrial radiation (Ra) following FAO-56 equations (21–23).

Computes the solar radiation at the top of the Earth’s atmosphere, accounting for latitude, day of the year, and solar geometry. Handles polar day/night conditions by setting radiation to zero during polar night.

Parameters

- **lat_da** (*xarray.DataArray*) – 2D latitude field in degrees, with dimensions ('Y', 'X').
- **time** (*pandas.DatetimeIndex*) – 1D array of datetime objects representing the time dimension.

Returns

Daily extraterrestrial radiation (Ra) with dimensions ('T', 'Y', 'X'), units MJ m⁻² day⁻¹, and attributes for units and long name.

Return type

`xarray.DataArray`

Notes

- Based on FAO-56 Reference Evapotranspiration methodology (Allen et al., 1998).
- Polar day/night is handled by setting Ra to 0 where the sunset hour angle is undefined.
- The solar constant used is 0.082 MJ m⁻² min⁻¹, converted to daily values.

References

Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). Crop evapotranspiration: Guidelines for computing crop water requirements. FAO Irrigation and Drainage Paper 56.

`wass2s.utils.svp` (*T*)

Calculate saturated vapor pressure (es) using FAO-56 Equation 3.

Parameters

T (*xarray.DataArray* or *numpy.ndarray*) – Air temperature in degrees Celsius.

Returns

Saturated vapor pressure in kPa, with same shape as input.

Return type

`xarray.DataArray` or `numpy.ndarray`

Notes

- Based on the Tetens equation as presented in FAO-56.
- Assumes input temperature is in Celsius.

References

Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). Crop evapotranspiration: Guidelines for computing crop water requirements. FAO Irrigation and Drainage Paper 56.

`wass2s.utils.et0_fao56_daily` (*tmax*, *tmin*, *rs*, *mlsp*, *dem*, *tdew*=None, *u10*=None, *v10*=None, *wff*=None)

Compute daily reference evapotranspiration (ET_0) using the FAO-56 Penman-Monteith equation.

Calculates ET_0 for a hypothetical reference crop (grass, 0.12 m height) based on daily meteorological data, following the standardized FAO-56 methodology.

Parameters

- **tmax** (*xarray.DataArray*) – Daily maximum temperature in °C, with dimensions ('T', 'Y', 'X').
- **tmin** (*xarray.DataArray*) – Daily minimum temperature in °C, with dimensions ('T', 'Y', 'X').
- **tdew** (*xarray.DataArray*) – Daily mean dew point temperature in °C, with dimensions ('T', 'Y', 'X').
- **u10** (*xarray.DataArray*, *optional*) – Daily mean zonal wind speed at 10 m height in $m\ s^{-1}$, with dimensions ('T', 'Y', 'X').
- **v10** (*xarray.DataArray*, *optional*) – Daily mean meridional wind speed at 10 m height in $m\ s^{-1}$, with dimensions ('T', 'Y', 'X').
- **wff** (*xarray.DataArray*, *optional*) – Daily mean wind speed at 10 m height in $m\ s^{-1}$, with dimensions ('T', 'Y', 'X').
- **rs** (*xarray.DataArray*) – Daily mean incoming solar radiation in $W\ m^{-2}$ or $MJ\ m^{-2}\ day^{-1}$, with dimensions ('T', 'Y', 'X').
- **mlsp** (*xarray.DataArray*) – Daily mean sea-level pressure in hPa, with dimensions ('T', 'Y', 'X').
- **dem** (*xarray.DataArray*) – Digital elevation model (terrain height) in meters, with dimensions ('Y', 'X').

Returns

Daily reference evapotranspiration (ET_0) in $mm\ day^{-1}$, with dimensions ('T', 'Y', 'X'), and attributes for units and long name.

Return type

xarray.DataArray

Notes

- All input arrays must be on matching grids with dimensions ('T', 'Y', 'X') except for *dem*, which is typically ('Y', 'X') and interpolated to match.
- Solar radiation input (*rs*) is automatically converted from $W\ m^{-2}$ to $MJ\ m^{-2}\ day^{-1}$ if values exceed 200 (assumed to be in $W\ m^{-2}$).
- Wind speed at 10 m is converted to 2 m height using a logarithmic profile.
- Soil heat flux (*G*) is assumed to be zero for daily calculations.

- The albedo for the reference crop is fixed at 0.23.
- Polar night conditions are handled in the extraterrestrial radiation calculation.

References

Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). Crop evapotranspiration: Guidelines for computing crop water requirements. FAO Irrigation and Drainage Paper 56.

1.9.3 Data transformation

Data transformation and distribution fitting for forecast fields.

Handles skewed precipitation distributions via Box-Cox, Yeo-Johnson, and quantile-based transformations, and fits the best parametric distribution per grid cell.

Standalone functions

inv_boxcox

Inverse Box-Cox transformation.

inv_yeojohnson

Inverse Yeo-Johnson transformation.

Class

WAS_TransformData

End-to-end transformation pipeline:

detect_skewness

Classify each grid cell as low / moderate / high skew.

handle_skewness

Recommend the appropriate transformation per skewness class.

apply_transformation

Apply Box-Cox, Yeo-Johnson, log, or square-root per grid cell.

inverse_transform

Reverse the applied transformation.

fit_best_distribution

Fit candidate parametric distributions (cluster or grid mode) and select the best-fitting one by AIC/BIC.

plot_best_fit_map

Cartopy map of the winning distribution per grid cell.

`wass2s.was_transformdata.inv_boxcox(y, lambda)`

Inverse Box-Cox transformation.

`wass2s.was_transformdata.inv_yeojohnson(y, lambda)`

Inverse Yeo-Johnson transformation.

class `wass2s.was_transformdata.WAS_TransformData` (*data*, *distribution_map=None*,
n_clusters=1000)

Bases: `object`

Optimized class for geospatial Time-Series analysis: Skewness detection, Transformation, and Distribution Fitting.

__init__ (*data*, *distribution_map=None*, *n_clusters=1000*)

detect_skewness ()

Compute and classify skewness for each grid cell.

handle_skewness ()

Recommend transformations based on skewness.

apply_transformation (*method=None*)

Apply transformations using vectorized ufuncs for speed. Stores parameters (lambdas) in `self.transform_params`.

inverse_transform (*data=None*)

Reverse transformations to recover original scale.

Parameters

data (*xarray.DataArray*, *optional*) – If provided (e.g., model predictions), this data will be inverted using the parameters learned during the *apply_transformation* step. If None, the internal *self.transformed_data* is inverted.

IMPORTANT: *data* must have the same Y and X coordinates/dimensions as the original data so the correct lambda matches the correct pixel.

fit_best_distribution (*use_transformed=False*, *mode='cluster'*)

Fit best distributions using either clustering or per-grid methods.

Parameters

mode ({*'cluster'*, *'grid'*})

plot_best_fit_map (*data_array*, *map_dict*, *output_file='map.png'*, *title='Map'*, *colors=None*, *show_plot=True*)

1.9.4 Agroclimatic predictand computation

Agroclimatic predictand computation from daily rainfall.

Implements West African rainfall season onset, cessation, and dry-spell detection from daily station or gridded precipitation data. All classes share a `compute(daily_data, nb_cores)` method that parallelises computation across grid points with `joblib`.

Classes

WAS_compute_onset

Season onset detection based on a user-configurable rainfall accumulation and dry-spell criteria.

WAS_compute_onset_dry_spell

Onset detection with an additional post-onset dry-spell check.

WAS_compute_cessation

Season cessation based on a water-balance (ETP / soil capacity) model.

WAS_compute_cessation_dry_spell

Cessation detection with a dry-spell termination criterion.

class `wass2s.was_compute_predictand.WAS_compute_onset` (*user_criteria=None*)

Bases: `object`

Class for computing agricultural seasonal onset dates based on cumulative rainfall criteria.

This class implements a widely used onset detection method in West African agrometeorology, particularly for Sahel and Sudanian zones. It supports multiple agro-climatic zones with zone-specific onset criteria (start/end search dates, cumulative rainfall threshold, maximum dry days allowed, rainy day threshold).

Key Features: - Handles both station-based (CDT format) and gridded (xarray) daily rainfall data - Automatically assigns zones based on mean annual rainfall and latitude (Sahel–Guinea) - Parallel computation using Dask for large gridded datasets - Produces onset dates in day-of-year format (1–365/366)

Standard Zones & Criteria (default): 0. Sahel100_0mm: start 01-Jun, ≥ 10 mm, ≤ 25 dry days, end 30-Aug 1. Sahel200_100mm: start 15-May, ≥ 15 mm, ≤ 25 dry days, end 15-Aug 2. Sahel400_200mm: start 01-May, ≥ 15 mm, ≤ 20 dry days, end 31-Jul 3. Sahel600_400mm: start 15-Mar, ≥ 20 mm, ≤ 20 dry days, end 31-Jul 4. Soudan: start 15-Mar, ≥ 20 mm, ≤ 10 dry days, end 31-Jul 5. Golfe Of Guinea: start 01-Feb, ≥ 20 mm, ≤ 10 dry days, end 15-Jun

References: - AGRHYMET Regional Centre (various operational guidelines) - Sivakumar (1988, 1991): Methodology for onset prediction in West Africa - Common practice in West African national meteorological services

Parameters

user_criteria (*dict*, *optional*) – Custom zone-specific onset criteria. If None, uses default dictionary. Expected format: {zone_id: {"zone_name": str, "start_search": "MM-DD", "cumulative": float, "number_dry_days": int, "thrd_rain_day": float, "end_search": "MM-DD"}}

```
default_criteria = {0: {'cumulative': 10, 'end_search': '08-30',
'number_dry_days': 25, 'start_search': '06-01', 'thrd_rain_day': 0.85,
'zone_name': 'Sahel100_0mm'}, 1: {'cumulative': 15, 'end_search': '08-15',
'number_dry_days': 25, 'start_search': '05-15', 'thrd_rain_day': 0.85,
'zone_name': 'Sahel200_100mm'}, 2: {'cumulative': 15, 'end_search':
'07-31', 'number_dry_days': 20, 'start_search': '05-01', 'thrd_rain_day':
0.85, 'zone_name': 'Sahel400_200mm'}, 3: {'cumulative': 20, 'end_search':
'07-31', 'number_dry_days': 20, 'start_search': '03-15', 'thrd_rain_day':
0.85, 'zone_name': 'Sahel600_400mm'}, 4: {'cumulative': 20, 'end_search':
'07-31', 'number_dry_days': 10, 'start_search': '03-15', 'thrd_rain_day':
0.85, 'zone_name': 'Soudan'}, 5: {'cumulative': 20, 'end_search': '06-15',
'number_dry_days': 10, 'start_search': '02-01', 'thrd_rain_day': 0.85,
'zone_name': 'Golfe_Of_Guinea'}}
```

__init__ (*user_criteria=None*)

Initialize the WAS_compute_onset class with user-defined or default criteria.

Parameters

user_criteria (*dict*, *optional*) – A dictionary containing zone-specific criteria. If not provided, the class will use the default criteria.

static adjust_duplicates (*series*, *increment=1e-05*)

If any values in the Series repeat, nudge them by a tiny increment so that all are unique (to avoid indexing collisions).

static transform_cdt (*df*)

Transform a DataFrame with:

- Row 0 = LON
- Row 1 = LAT
- Row 2 = ELEV
- Rows 3+ = daily data (or any date) with 'ID' column containing dates.

Returns an xarray DataArray with coords = (T, Y, X), variable = 'Observation'.

static day_of_year (*i*, *dem_rech1*)

Given a year 'i' and a month-day string 'dem_rech1' (e.g., '07-23'), return the day of the year (1-based).

rainf_zone (*daily_data*)

onset_function (*x*, *idebut*, *cumul*, *nbsec*, *jour_pluvieux*, *irch_fin*)

Calculate the onset date of a season based on cumulative rainfall criteria.

Parameters

- **x** (*array-like*) – Daily rainfall or similar values.
- **idebut** (*int*) – Start index to begin searching for the onset.
- **cumul** (*float*) – Cumulative rainfall threshold to trigger onset.
- **nbsec** (*int*) – Maximum number of dry days allowed in the sequence.
- **jour_pluvieux** (*float*) – Minimum rainfall to consider a day as rainy.
- **irch_fin** (*int*) – Maximum index limit for the onset.

Returns

Index of the onset date or NaN if onset not found.

Return type

int or *float*

compute_insitu (*daily_df*)

compute (*daily_data*, *nb_cores*)

Compute onset dates for each pixel in a given daily rainfall DataArray using different criteria based on isohyet zones.

Parameters

- **daily_data** (*xarray.DataArray*) – Daily rainfall data, coords = (T, Y, X).
- **nb_cores** (*int*) – Number of parallel processes to use.

Returns

Array with onset dates computed per pixel.

Return type

xarray.DataArray

class wass2s.was_compute_predictand.WAS_compute_onset_dry_spell (*user_criteria=None*)

Bases: *object*

Class for computing the **longest dry spell length after the onset** of the rainy season.

This class extends standard onset detection by calculating the maximum consecutive dry days within a user-defined window (*nbjour*) following the detected onset date.

The onset detection follows the same cumulative rainfall + dry-day tolerance method used in West African agrometeorology (AGRHYMET, national services), with zone-specific criteria.

Key Features: - Computes onset date (same logic as `WAS_compute_onset`) - Then finds the longest dry spell (consecutive days $\leq \text{thrd_rain_day}$ mm) within the next *nbjour* days - Supports station (CDT) and gridded (*xarray*) daily rainfall data - Automatically assigns agro-climatic zones based on mean annual rainfall and latitude - Parallel computation with Dask for large grids

Default Zones & Criteria: 0. Sahel100_0mm → start 01-Jun, ≥ 10 mm, ≤ 25 dry days, check 40 days after onset 1. Sahel200_100mm → start 15-May, ≥ 15 mm, ≤ 25 dry days, check 40 days 2. Sahel400_200mm → start 01-May,

≥15 mm, ≤20 dry days, check 40 days 3. Sahel600_400mm → start 15-Mar, ≥20 mm, ≤20 dry days, check 45 days 4. Soudan → start 15-Mar, ≥20 mm, ≤10 dry days, check 50 days 5. Golfe of Guinea → start 01-Feb, ≥20 mm, ≤10 dry days, check 50 days

References: - AGRHYMET Regional Centre operational methodologies - Sivakumar (1988, 1991): Onset and dry spell analysis in West Africa - Common practice in Sahelian and Sudanian national meteorological services

Parameters

user_criteria (*dict, optional*) – Custom zone-specific criteria. If None, uses default dictionary. Expected keys: zone_id → {"zone_name": str, "start_search": "MM-DD", "cumulative": float, "number_dry_days": int, "thrd_rain_day": float, "end_search": "MM-DD", "nbjour": int}

```
default_criteria = {0: {'cumulative': 10, 'end_search': '08-30', 'nbjour': 40, 'number_dry_days': 25, 'start_search': '06-01', 'thrd_rain_day': 0.85, 'zone_name': 'Sahel100_0mm'}, 1: {'cumulative': 15, 'end_search': '08-15', 'nbjour': 40, 'number_dry_days': 25, 'start_search': '05-15', 'thrd_rain_day': 0.85, 'zone_name': 'Sahel200_100mm'}, 2: {'cumulative': 15, 'end_search': '07-31', 'nbjour': 40, 'number_dry_days': 20, 'start_search': '05-01', 'thrd_rain_day': 0.85, 'zone_name': 'Sahel400_200mm'}, 3: {'cumulative': 20, 'end_search': '07-31', 'nbjour': 45, 'number_dry_days': 20, 'start_search': '03-15', 'thrd_rain_day': 0.85, 'zone_name': 'Sahel600_400mm'}, 4: {'cumulative': 20, 'end_search': '07-31', 'nbjour': 50, 'number_dry_days': 10, 'start_search': '03-15', 'thrd_rain_day': 0.85, 'zone_name': 'Soudan'}, 5: {'cumulative': 20, 'end_search': '06-15', 'nbjour': 50, 'number_dry_days': 10, 'start_search': '02-01', 'thrd_rain_day': 0.85, 'zone_name': 'Golfe_Of_Guinea'}}
```

__init__ (*user_criteria=None*)

Initialize the WAS_compute_dry_spell class with user-defined or default criteria.

Parameters

user_criteria (*dict, optional*) – A dictionary containing zone-specific criteria. If not provided, the class will use the default criteria.

static adjust_duplicates (*series, increment=1e-05*)

If any values in the Series repeat, nudge them by a tiny increment so that all are unique (to avoid indexing collisions).

static transform_cdt (*df*)

Transform a DataFrame with:

- Row 0 = LON
- Row 1 = LAT
- Row 2 = ELEV
- Rows 3+ = daily data (or any date) with 'ID' column containing dates.

Returns an xarray DataArray with coords = (T, Y, X), variable = 'Observation'.

rainf_zone (*daily_data*)

dry_spell_onset_function (*x, idebut, cumul, nbsec, jour_pluvieux, irch_fin, nbjour*)

Calculate the onset date of a season based on cumulative rainfall criteria, and determine the longest dry spell sequence within a specified period after the onset.

dry_spell_onset_function_(*x, idebut, cumul, nbsec, jour_pluvieux, irch_fin, nbjour*)

Calculate the onset date of a season based on cumulative rainfall criteria, and determine the longest dry spell sequence within a specified period after the onset.

Parameters

- **x** (*array-like*) – Daily rainfall or similar values.
- **idebut** (*int*) – Start index to begin searching for the onset.
- **cumul** (*float*) – Cumulative rainfall threshold to trigger onset.
- **nbsec** (*int*) – Maximum number of dry days allowed in the sequence.
- **jour_pluvieux** (*float*) – Minimum rainfall to consider a day as rainy.
- **irch_fin** (*int*) – Maximum index limit for the onset.
- **nbjour** (*int*) – Number of days to check for the longest dry spell after onset.

Returns

Length of the longest dry spell sequence after onset or NaN if onset not found.

Return type

float

static day_of_year(*i, dem_rech1*)

Given a year 'i' and a month-day string 'dem_rech1' (e.g., '07-23'), return the 1-based day of the year.

compute_insitu(*daily_df*)

compute(*daily_data, nb_cores*)

Compute the longest dry spell length after the onset for each pixel in a given daily rainfall DataArray, using different criteria based on isohyet zones.

Parameters

- **daily_data** (*xarray.DataArray*) – Daily rainfall data, coords = (T, Y, X).
- **nb_cores** (*int*) – Number of parallel processes to use.

Returns

Array with the longest dry spell length per pixel.

Return type

xarray.DataArray

class wass2s.was_compute_predictand.**WAS_compute_cessation**(*user_criteria=None*)

Bases: *object*

Class for computing the **cessation date** (end of rainy season) based on **soil moisture balance**.

This class uses a simple water balance model to determine when soil moisture is depleted, marking the end of the rainy season. It is widely used in West African agrometeorology (AGRHYMET, national services) for Sahel, Sudanian, and Guinean zones.

Key Features: - Soil water balance: rainfall - ETP (evapotranspiration), capped at soil retention capacity - Cessation date: first day when soil moisture reaches zero after the search start - Zone-specific criteria (start search, ETP, retention capacity, end search) - Supports station (CDT format) and gridded (*xarray*) daily rainfall data - Automatic agro-climatic zone assignment based on mean annual rainfall and latitude - Parallel computation with Dask for large grids

Default Zones & Criteria: 0. Sahel100_0mm → start search 01-Sep, ETP 5.0 mm/day, capacity 70 mm, end 30-Sep
1. Sahel200_100mm → start search 01-Sep, ETP 5.0 mm/day, capacity 70 mm, end 05-Oct 2. Sahel400_200mm

→ start search 01-Sep, ETP 5.0 mm/day, capacity 70 mm, end 10-Nov 3. Sahel600_400mm → start search 15-Sep, ETP 5.0 mm/day, capacity 70 mm, end 15-Nov 4. Soudan → start search 01-Oct, ETP 4.5 mm/day, capacity 70 mm, end 30-Nov 5. Golfe of Guinea → start search 15-Oct, ETP 4.0 mm/day, capacity 70 mm, end 01-Dec

References: - AGRHYMET Regional Centre operational methodologies - Sivakumar (1991): Soil water balance and cessation prediction in West Africa - Common practice in Sahelian and Sudanian national meteorological services

Parameters

user_criteria (*dict, optional*) – Custom zone-specific cessation criteria. If None, uses default dictionary. Expected keys: zone_id → {“zone_name”: str, “date_dry_soil”: “MM-DD”, “start_search”: “MM-DD”, “ETP”: float, “Cap_ret_maxi”: float, “end_search”: “MM-DD”}

```
default_criteria = {0: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'date_dry_soil': '01-01', 'end_search': '09-30', 'start_search': '09-01', 'zone_name': 'Sahel100_0mm'}, 1: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'date_dry_soil': '01-01', 'end_search': '10-05', 'start_search': '09-01', 'zone_name': 'Sahel200_100mm'}, 2: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'date_dry_soil': '01-01', 'end_search': '11-10', 'start_search': '09-01', 'zone_name': 'Sahel400_200mm'}, 3: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'date_dry_soil': '01-01', 'end_search': '11-15', 'start_search': '09-15', 'zone_name': 'Sahel600_400mm'}, 4: {'Cap_ret_maxi': 70, 'ETP': 4.5, 'date_dry_soil': '01-01', 'end_search': '11-30', 'start_search': '10-01', 'zone_name': 'Soudan'}, 5: {'Cap_ret_maxi': 70, 'ETP': 4.0, 'date_dry_soil': '01-01', 'end_search': '12-01', 'start_search': '10-15', 'zone_name': 'Golfe_Of_Guinea'}}
```

__init__ (*user_criteria=None*)

Initialize the WAS_compute_cessation class with user-defined or default criteria.

Parameters

user_criteria (*dict, optional*) – A dictionary containing zone-specific criteria. If not provided, the class will use the default criteria.

static adjust_duplicates (*series, increment=1e-05*)

If any values in the Series repeat, nudge them by a tiny increment so that all are unique (to avoid indexing collisions).

static day_of_year (*i, dem_rech1*)

Given a year ‘i’ and a month-day string ‘dem_rech1’ (e.g., ‘07-23’), return the 1-based day of the year.

static transform_cdt (*df*)

Transform a DataFrame with:

- Row 0 = LON
- Row 1 = LAT
- Row 2 = ELEV
- Rows 3+ = daily data (or any date) with ‘ID’ column containing dates.

Returns an xarray DataArray with coords = (T, Y, X), variable = ‘Observation’.

cessation_function (*x, ijour_dem_cal, idebut, ETP, Cap_ret_maxi, irch_fin*)

Compute cessation date using soil moisture balance criteria.

compute_insitu (*daily_df*)

rainf_zone (*daily_data*)

compute (*daily_data*, *nb_cores*)

Compute cessation dates for each pixel using criteria based on regions.

class wass2s.was_compute_predictand.**WAS_compute_cessation_dry_spell** (*user_criteria=None*)

Bases: `object`

Class for computing the **longest dry spell length** between the onset and cessation of the rainy season, using a two-stage approach:

1. Detect **onset** (start of rainy season) using cumulative rainfall criteria
2. Detect **cessation** (end of rainy season) using soil water balance
3. Calculate the maximum consecutive dry days ($\leq$ thrd_rain_day mm) within that window

This is a critical agro-meteorological indicator in West Africa, especially for assessing drought risk during the growing season after planting.

Key Features: - Uses zone-specific criteria for both onset and cessation - Automatic agro-climatic zone assignment based on mean annual rainfall and latitude - Supports station (CDT format) and gridded (xarray) daily rainfall data - Parallel computation with Dask for large domains - Returns the longest dry spell length (in days) between onset and cessation

Default Zones & Criteria: 0. Sahel100_0mm → onset: 01-May → 15-Aug | cessation: 01-Sep → 30-Sep 1. Sahel200_100mm → onset: 15-May → 15-Aug | cessation: 01-Sep → 05-Oct 2. Sahel400_200mm → onset: 01-May → 31-Jul | cessation: 01-Sep → 10-Nov 3. Sahel600_400mm → onset: 15-Mar → 31-Jul | cessation: 15-Sep → 15-Nov 4. Soudan → onset: 15-Mar → 31-Jul | cessation: 01-Oct → 30-Nov 5. Golfe of Guinea → onset: 01-Feb → 15-Jun | cessation: 15-Oct → 01-Dec

References: - AGRHYMET Regional Centre operational methodologies - Sivakumar (1991): Methodology for onset, cessation, and dry spell analysis - Common practice in West African national meteorological and agricultural services

Parameters

user_criteria (*dict*, *optional*) – Custom zone-specific criteria. If None, uses default dictionary. Expected keys: zone_id → {"zone_name": str, "start_search1": "MM-DD", "cumulative": float, "number_dry_days": int, "thrd_rain_day": float, "end_search1": "MM-DD", "nbjour": int, "date_dry_soil": "MM-DD", "start_search2": "MM-DD", "ETP": float, "Cap_ret_maxi": float, "end_search2": "MM-DD"}

```

default_criteria = {0: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'cumulative': 10,
'date_dry_soil': '01-01', 'end_search1': '08-15', 'end_search2': '09-30',
'nbjour': 40, 'number_dry_days': 25, 'start_search1': '05-01',
'start_search2': '09-01', 'thrd_rain_day': 0.85, 'zone_name':
'Sahel100_0mm'}, 1: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'cumulative': 15,
'date_dry_soil': '01-01', 'end_search1': '08-15', 'end_search2': '10-05',
'nbjour': 40, 'number_dry_days': 25, 'start_search1': '05-15',
'start_search2': '09-01', 'thrd_rain_day': 0.85, 'zone_name':
'Sahel200_100mm'}, 2: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'cumulative': 15,
'date_dry_soil': '01-01', 'end_search1': '07-31', 'end_search2': '11-10',
'nbjour': 40, 'number_dry_days': 20, 'start_search1': '05-01',
'start_search2': '09-01', 'thrd_rain_day': 0.85, 'zone_name':
'Sahel400_200mm'}, 3: {'Cap_ret_maxi': 70, 'ETP': 5.0, 'cumulative': 20,
'date_dry_soil': '01-01', 'end_search1': '07-31', 'end_search2': '11-15',
'nbjour': 45, 'number_dry_days': 20, 'start_search1': '03-15',
'start_search2': '09-15', 'thrd_rain_day': 0.85, 'zone_name':
'Sahel600_400mm'}, 4: {'Cap_ret_maxi': 70, 'ETP': 4.5, 'cumulative': 20,
'date_dry_soil': '01-01', 'end_search1': '07-31', 'end_search2': '11-30',
'nbjour': 50, 'number_dry_days': 10, 'start_search1': '03-15',
'start_search2': '10-01', 'thrd_rain_day': 0.85, 'zone_name': 'Soudan'},
5: {'Cap_ret_maxi': 70, 'ETP': 4.0, 'cumulative': 20, 'date_dry_soil':
'01-01', 'end_search1': '06-15', 'end_search2': '12-01', 'nbjour': 50,
'number_dry_days': 10, 'start_search1': '02-01', 'start_search2': '10-15',
'thrd_rain_day': 0.85, 'zone_name': 'Golfe_Of_Guinea'}}

```

__init__ (*user_criteria=None*)

Initialize the WAS_compute_cessation_dry_spell class with user-defined or default criteria.

Parameters

user_criteria (*dict, optional*) – A dictionary containing zone-specific criteria. If not provided, the class will use the default criteria.

static adjust_duplicates (*series, increment=1e-05*)

If any values in the Series repeat, nudge them by a tiny increment so that all are unique (to avoid indexing collisions).

static transform_cdt (*df*)

Transform a DataFrame with:

- Row 0 = LON
- Row 1 = LAT
- Row 2 = ELEV
- Rows 3+ = daily data (or any date) with 'ID' column containing dates.

Returns an xarray DataArray with coords = (T, Y, X), variable = 'Observation'.

rainf_zone (*daily_data*)

dry_spell_cessation_function (*x, idebut1, cumul, nbsec, jour_pluvieux, irch_fin1, idebut2, ijour_dem_cal, ETP, Cap_ret_maxi, irch_fin2, nbjour*)

Computes the longest dry spell length after the onset and determines the cessation date (when soil water returns to 0) based on water balance, then checks for a dry spell.

Parameters

- `x` (*array-like*) – Daily rainfall or similar values.
- `idebut1` (*int*) – Start index to begin searching for the onset.
- `cumul` (*float*) – Cumulative rainfall threshold to trigger onset.
- `nbsec` (*int*) – Maximum number of dry days allowed in the sequence.
- `jour_pluvieux` (*float*) – Minimum rainfall to consider a day as rainy.
- `irch_fin1` (*int*) – Maximum index limit for the onset search.
- `idebut2` (*int*) – Start index for the cessation search.
- `ijour_dem_cal` (*int*) – Start index from which the water balance is calculated.
- `ETP` (*float*) – Daily evapotranspiration (mm).
- `Cap_ret_maxi` (*float*) – Maximum soil water retention capacity (mm).
- `irch_fin2` (*int*) – Maximum index limit for the cessation search.
- `nbjour` (*int*) – Number of days after onset to check for the dry spell.

Returns

Length of the longest dry spell sequence after onset and before soil water returns to zero, or NaN if not found.

Return type

float

static `day_of_year` (*i*, *dem_rech1*)

Convert year *i* and MM-DD string *dem_rech1* (e.g., '07-23') into a 1-based day of the year.

compute_insitu (*daily_df*)

compute (*daily_data*, *nb_cores*)

Compute the longest dry spell length after the rainy season onset for each pixel in the given daily rainfall DataArray, using different criteria (both for onset and cessation) based on isohyet zones.

Parameters

- `daily_data` (*xarray.DataArray*) – Daily rainfall data, coords = (T, Y, X).
- `nb_cores` (*int*) – Number of parallel processes (workers) to use.

Returns

Array with the longest dry spell length per pixel.

Return type

xarray.DataArray

class `wass2s.was_compute_predictand.WAS_count_dry_spells`

Bases: *object*

Class for computing the **number of dry spells** of a specified length occurring between the onset and cessation dates of the rainy season.

This class is designed to quantify intra-seasonal dry spell risk — a critical indicator for agriculture in West Africa, where consecutive dry days after planting can severely impact crop establishment and yield.

Key Features: - Counts dry spells (consecutive days $\leq$ `dry_threshold` mm) between onset and cessation - Accepts onset and cessation dates as pre-computed *xarray* DataArrays or CPT DataFrames - Supports both station-based (CDT format) and gridded (*xarray*) daily rainfall data - Efficient parallel computation using Dask for large domains - Outputs in CPT-compatible wide format for station data

Typical Use Case: After computing onset (e.g., via `WAS_compute_onset`) and cessation (e.g., via `WAS_compute_cessation`), use this class to count risky dry spells (e.g., 10-day, 15-day) during the growing window.

References: - AGRHYMET Regional Centre operational dry spell monitoring - Sivakumar (1991): Methodology for dry spell analysis in Sahelian agriculture - Common practice in West African national meteorological and agricultural services

Parameters

time (None (no initialization parameters required – all criteria passed at compute))

compute_insitu (*daily_df, onset_df_cpt, cessation_df_cpt, dry_spell_length, dry_threshold=1.0*)

Compute number of dry spells for station data (CDT input, CPT output).

compute (*daily_data, onset_date, cessation_date, dry_spell_length, dry_threshold, nb_cores*)

Compute number of dry spells for gridded xarray data (parallelized).

Examples

Count number of 10-day dry spells between onset and cessation (gridded):

```
>>> count_calc = WAS_count_dry_spells()
>>> dry_spell_count = count_calc.compute(
...     daily_pr,          # xarray.DataArray daily rainfall
...     onset_da,         # xarray.DataArray of onset dates
...     cessation_da,     # xarray.DataArray of cessation dates
...     dry_spell_length=10,
...     dry_threshold=1.0,
...     nb_cores=8
... )
```

For station data (CDT rainfall, CPT onset/cessation):

```
>>> count_cpt = count_calc.compute_insitu(
...     daily_cdt_df,
...     onset_cpt_df,
...     cessation_cpt_df,
...     dry_spell_length=15,
...     dry_threshold=0.5
... )
```

static adjust_duplicates (*series, increment=1e-05*)

If any values in the Series repeat, nudge them by a tiny increment so that all are unique (to avoid indexing collisions).

static transform_cdt (*df*)

Transform a DataFrame with:

- Row 0 = LON
- Row 1 = LAT
- Row 2 = ELEV
- Rows 3+ = daily data with 'ID' column containing dates.

Returns a DataFrame with columns like:

DATE | STATION | VALUE | LON | LAT | ELEV | MEAN_ANNUAL_RAINFALL | zonename

static transform_cpt (*df*, *missing_value=None*)

Transform a DataFrame in CPT format with:

- Row 0 = LAT
- Row 1 = LON
- Rows 2+ = numeric year data in wide format (stations in columns).

Returns a DataFrame with columns like:

YEAR | STATION | VALUE | LAT | LON

static count_dry_spells (*x*, *onset*, *cessation*, *dry_spell_length*, *dry_threshold*)

Count the number of dry spells of a specific length between onset and cessation dates.

Parameters

- **x** (*array-like*) – Daily rainfall values.
- **onset** (*int*) – Start index for the calculation (onset date).
- **cessation** (*int*) – End index for the calculation (cessation date).
- **dry_spell_length** (*int*) – The length of a dry spell to count.
- **dry_threshold** (*float*) – Rainfall threshold to classify a day as “dry.”

Returns

The number of dry spells of the specified length (NaN if invalid).

Return type

int or *float*

compute_insitu (*daily_df*, *onset_df_cpt*, *cessation_df_cpt*, *dry_spell_length*, *dry_threshold=1.0*)

Compute the number of dry spells (of length = *dry_spell_length*) between the onset and cessation dates for in-situ stations (CDT format).

Returns a DataFrame in CPT format:

- Row 0: [“LAT”, lat_stn1, lat_stn2, ...]
- Row 1: [“LON”, lon_stn1, lon_stn2, ...]
- Subsequent rows: [year, station1_value, station2_value, ...]

Parameters

- **daily_df** (*pd.DataFrame*) – CDT rainfall data (ID column = date, station columns).
- **onset_df_cpt** (*pd.DataFrame*) – CPT-format DataFrame containing onset dates (as returned by some method).
- **cessation_df_cpt** (*pd.DataFrame*) – CPT-format DataFrame containing cessation dates.
- **dry_spell_length** (*int*) – The length of the dry spell to look for.
- **dry_threshold** (*float*, *optional*) – Rainfall threshold below which a day is considered “dry.” Defaults to 1.0 mm.

Returns

Final dry-spell counts in CPT pivot format.

Return type

pd.DataFrame

compute (*daily_data, onset_date, cessation_date, dry_spell_length, dry_threshold, nb_cores*)

Compute the number of dry spells for each pixel within the onset and cessation period in a daily xarray DataArray.

Parameters

- **daily_data** (*xarray.DataArray*) – Daily rainfall data, coords = (T, Y, X).
- **onset_date** (*xarray.DataArray*) – DataArray containing onset dates for each pixel.
- **cessation_date** (*xarray.DataArray*) – DataArray containing cessation dates for each pixel.
- **dry_spell_length** (*int*) – The length of a dry spell to count.
- **dry_threshold** (*float*) – Rainfall threshold to classify a day as “dry.”
- **nb_cores** (*int*) – Number of parallel processes to use.

Returns

An array with the count of dry spells per pixel.

Return type

xarray.DataArray

class wass2s.was_compute_predictand.WAS_count_wet_spellsBases: `object`

Class for computing the **number of wet spells** of a specified length occurring between the onset and cessation dates of the rainy season.

This class is designed to quantify periods of consecutive rainy days during the growing season — a key indicator for assessing moisture availability, crop growth, and potential waterlogging risk in West African agriculture.

Key Features: - Counts wet spells (consecutive days $\geq$ wet_threshold mm) between onset and cessation - Accepts pre-computed onset and cessation dates (xarray DataArrays or CPT DataFrames) - Supports both station-based (CDT format) and gridded (xarray) daily rainfall data - Efficient parallel computation using Dask for large-scale gridded analysis - Outputs in CPT-compatible wide format for station data

Typical Use Case: After calculating onset (e.g., WAS_compute_onset) and cessation (e.g., WAS_compute_cessation), use this class to count sequences of wet days (e.g., 5-day, 7-day wet spells) that support crop development or indicate flood risk.

References: - AGRHYMET Regional Centre operational wet spell monitoring - Sivakumar (1991): Rainfall variability and wet spell analysis in Sahelian agriculture - Common practice in West African national meteorological and agricultural services

Parameters

time (*None*) (None (no initialization parameters required – all criteria passed at compute))

compute_insitu (*daily_df, onset_df_cpt, cessation_df_cpt, wet_spell_length, wet_threshold=1.0*)

Compute number of wet spells for station data (CDT input, CPT output).

compute (*daily_data, onset_date, cessation_date, wet_spell_length, wet_threshold, nb_cores*)

Compute number of wet spells for gridded xarray data (parallelized).

Examples

Count number of 7-day wet spells between onset and cessation (gridded):

```
>>> wet_calc = WAS_count_wet_spells()
>>> wet_spell_count = wet_calc.compute(
...     daily_pr,          # xarray.DataArray daily rainfall (mm)
...     onset_da,         # xarray.DataArray of onset dates (day-of-
...     →year)
...     cessation_da,     # xarray.DataArray of cessation dates
...     wet_spell_length=7,
...     wet_threshold=1.0, # mm/day
...     nb_cores=8
... )
```

For station data (CDT rainfall, CPT onset/cessation):

```
>>> wet_cpt = wet_calc.compute_insitu(
...     daily_cdt_df,
...     onset_cpt_df,
...     cessation_cpt_df,
...     wet_spell_length=5,
...     wet_threshold=2.0
... )
```

static count_wet_spells (*x*, *onset_date*, *cessation_date*, *wet_spell_length*, *wet_threshold*)

Count the number of wet spells of a specific length between onset and cessation dates.

Parameters

- **x** (*array-like*) – Daily rainfall values.
- **onset_date** (*int*) – Start index for the calculation (onset date).
- **cessation_date** (*int*) – End index for the calculation (cessation date).
- **wet_spell_length** (*int*) – The length of a wet spell to count.
- **wet_threshold** (*float*) – Rainfall threshold to classify a day as “wet.”

Returns

The number of wet spells of the specified length (NaN if data is invalid).

Return type

int or *float*

static transform_cdt (*df*)

Transform a CDT-format DataFrame into a standard table.

CDT format assumptions:

- Row 0 = LON
- Row 1 = LAT
- Row 2 = ELEV
- Rows 3+ = daily data, ‘ID’ column has dates in YYYYMMDD.

Returns a DataFrame with columns:

[DATE, STATION, VALUE, LON, LAT, ELEV, MEAN_ANNUAL_RAINFALL, zonename]

compute (*daily_data*, *onset_date*, *cessation_date*, *wet_spell_length*, *wet_threshold*, *nb_cores*)

Compute the number of wet spells for each pixel within the onset and cessation period in a daily xarray DataArray.

Parameters

- **daily_data** (*xarray.DataArray*) – Daily rainfall data, coords = (T, Y, X).
- **onset_date** (*xarray.DataArray*) – DataArray containing onset dates for each pixel.
- **cessation_date** (*xarray.DataArray*) – DataArray containing cessation dates for each pixel.
- **wet_spell_length** (*int*) – The length of a wet spell to count.
- **wet_threshold** (*float*) – Rainfall threshold to classify a day as “wet.”
- **nb_cores** (*int*) – Number of parallel processes to use.

Returns

Array with the count of wet spells per pixel.

Return type

xarray.DataArray

compute_insitu (*daily_df*, *onset_df_cpt*, *cessation_df_cpt*, *wet_spell_length*, *wet_threshold=1.0*)

Compute the number of wet spells (of length = *wet_spell_length*) between onset and cessation for in-situ stations (CDT data).

Returns a DataFrame in CPT format:

- Row 0: [“LAT”, lat_station1, lat_station2, ...]
- Row 1: [“LON”, lon_station1, lon_station2, ...]
- Then one row per year: [year, station1_value, station2_value, ...]

Parameters

- **daily_df** (*pd.DataFrame*) – CDT rainfall data (ID column = date, station columns).
- **onset_df_cpt** (*pd.DataFrame*) – CPT-format DataFrame with onset dates (same station columns).
- **cessation_df_cpt** (*pd.DataFrame*) – CPT-format DataFrame with cessation dates (same station columns).
- **wet_spell_length** (*int*) – The length of a wet spell to count.
- **wet_threshold** (*float*, *optional*) – Rainfall threshold classifying a day as “wet.” Defaults to 1.0 mm.

Returns

Final wet-spell counts in CPT pivot format.

Return type

pd.DataFrame

class wass2s.was_compute_predictand.**WAS_count_rainy_days**

Bases: *object*

Class for computing the **number of rainy days** (days with precipitation $\geq$ a threshold) between the onset and cessation dates of the rainy season.

This is a key agro-meteorological indicator used in West Africa to assess seasonal moisture availability during the growing period — directly relevant for crop water requirements, yield estimation, and drought/vulnerability assessments.

Key Features: - Counts days where daily rainfall $\geq$ `rain_threshold` (default 0.85 mm) - Uses pre-computed onset and cessation dates (xarray DataArrays or CPT DataFrames) - Supports both station-based (CDT format) and gridded (xarray) daily rainfall data - Efficient parallel computation with Dask for large-scale gridded analysis - Outputs in CPT-compatible wide format for station data

Typical Use Case: After calculating onset (e.g., via `WAS_compute_onset`) and cessation (e.g., via `WAS_compute_cessation`), use this class to quantify the total number of rainy days available to crops during the growing window.

References: - AGRHYMET Regional Centre seasonal rainfall monitoring - Sivakumar (1991): Rainfall characteristics and agricultural planning in West Africa - Common practice in Sahelian and Sudanian national meteorological services

Parameters

time (None (no initialization parameters required – all criteria passed at compute))

compute_insitu (`daily_df`, `onset_df_cpt`, `cessation_df_cpt`, `rain_threshold=0.85`)

Compute number of rainy days for station data (CDT input, CPT output).

compute (`daily_data`, `onset_date`, `cessation_date`, `rain_threshold`, `nb_cores`)

Compute number of rainy days for gridded xarray data (parallelized).

Examples

Count rainy days (≥ 0.85 mm) between onset and cessation (gridded):

```
>>> rainy_calc = WAS_count_rainy_days()
>>> rainy_days_count = rainy_calc.compute(
...     daily_pr,           # xarray.DataArray daily rainfall (mm)
...     onset_da,          # xarray.DataArray of onset dates (day-of-
...     ↪year)
...     cessation_da,      # xarray.DataArray of cessation dates
...     rain_threshold=0.85,
...     nb_cores=8
... )
```

For station data (CDT rainfall, CPT onset/cessation):

```
>>> rainy_cpt = rainy_calc.compute_insitu(
...     daily_cdt_df,
...     onset_cpt_df,
...     cessation_cpt_df,
...     rain_threshold=1.0
... )
```

static transform_cdt (`df`)

Transform a DataFrame in CDT format into a standardized long DataFrame.

CDT format assumptions:

- Row 0 = LON
- Row 1 = LAT

- Row 2 = ELEV
- Rows 3+ = daily data with 'ID' column holding dates in YYYYMMDD format.

This method returns a DataFrame with columns:

DATE, STATION, VALUE, LON, LAT, ELEV, (optional) MEAN_ANNUAL_RAINFALL, zone-name

static count_rainy_days (*x*, *onset_date*, *cessation_date*, *rain_threshold*)

Count the number of rainy days between onset and cessation dates.

Parameters

- **x** (*array-like*) – Daily rainfall values.
- **onset_date** (*int*) – Start index for the calculation (onset date).
- **cessation_date** (*int*) – End index for the calculation (cessation date).
- **rain_threshold** (*float*) – Rainfall threshold to classify a day as “rainy.”

Returns

Number of rainy days (returns NaN if data is invalid).

Return type

int or *float*

compute (*daily_data*, *onset_date*, *cessation_date*, *rain_threshold*, *nb_cores*)

Compute the number of rainy days for each pixel between onset and cessation dates.

Parameters

- **daily_data** (*xarray.DataArray*) – Daily rainfall data, coords = (T, Y, X).
- **onset_date** (*xarray.DataArray*) – DataArray containing onset dates for each pixel.
- **cessation_date** (*xarray.DataArray*) – DataArray containing cessation dates for each pixel.
- **rain_threshold** (*float*) – Rainfall threshold to classify a day as “rainy.”
- **nb_cores** (*int*) – Number of parallel processes to use.

Returns

Array with the count of rainy days per pixel.

Return type

xarray.DataArray

compute_insitu (*daily_df*, *onset_df_cpt*, *cessation_df_cpt*, *rain_threshold=0.85*)

Compute, for in-situ stations (CDT data), the number of rainy days between onset and cessation, for each station and year.

Parameters

- **daily_df** (*pd.DataFrame*) – CDT precipitation data (ID column = date; columns = stations). Follows the standard CDT format.
- **onset_df_cpt** (*pd.DataFrame*) – Result of `WAS_compute_onset.compute_insitu(...)` for onset (CPT format).
- **cessation_df_cpt** (*pd.DataFrame*) – Same format for cessation (CPT format).
- **rain_threshold** (*float*, *optional*) – Precipitation threshold for counting a day as “rainy,” by default 0.85 mm.

Returns

df_final – The count of rainy days in CPT pivot format.

Return type

pd.DataFrame

```
class wass2s.was_compute_predictand.ExtremeType(*values)
```

Bases: Enum

Type of temperature extreme.

HOT = 'hot'

COLD = 'cold'

```
class wass2s.was_compute_predictand.WAS_TempPercentileIndices(base_period: slice,
                                                                percentile: float = 90,
                                                                season: List[int] | None
                                                                = None, var_type: str =
                                                                'TMAX', extreme_type:
                                                                str = 'hot',
                                                                bootstrap_samples: int =
                                                                10, min_base_years: int
                                                                = 15)
```

Bases: object

Implementation of ETCCDI temperature percentile-based extreme indices.

Computes the annual percentage of days/nights exceeding (hot) or falling below (cold) a specified percentile threshold, using a 5-day centered window for percentile calculation (standard ETCCDI methodology).

Supported ETCCDI Indices: - **TX90p / TN90p**: Percentage of hot days (TX > 90th) / hot nights (TN > 90th) - **TX10p / TN10p**: Percentage of cold days (TX < 10th) / cold nights (TN < 10th) - Custom percentiles are also supported (e.g., TX95p, TN99p, TX05p)

Key Features: - 5-day centered window for calendar-day percentiles (avoids day-of-year bias) - Proper leap-year handling (Feb 29 mapped to Feb 28) - Seasonal filtering option - Bootstrap confidence intervals (optional) - Works with both station (CDT format) and gridded (xarray) data

References: - ETCCDI Climate Change Indices (2009) - Zhang et al. (2011): Indices for monitoring changes in extremes - Sillmann et al. (2013): Climate extremes indices in the CMIP5 ensemble

Parameters

- **base_period** (*slice*) – Climatological base period for percentile calculation, e.g. slice("1961", "1990")
- **percentile** (*float*, *default=90*) – Percentile threshold: - For hot extremes: 90, 95, 99 (days above percentile) - For cold extremes: 10, 5, 1 (days below percentile)
- **season** (*list of int*, *optional*) – Months to include in analysis (e.g., [6,7,8] for JJA)
- **var_type** ({'TMAX', 'TMIN'}, *default='TMAX'*) – Temperature variable: 'TMAX' (TX - daily max) or 'TMIN' (TN - daily min)
- **extreme_type** ({'hot', 'cold'}, *default='hot'*) – Type of extreme: 'hot' (above percentile) or 'cold' (below percentile)
- **bootstrap_samples** (*int*, *default=10*) – Number of bootstrap samples for confidence interval calculation

- **min_base_years** (*int*, *default=15*) – Minimum number of years required in base period (warning if fewer)

Examples

Standard TX90p (Warm days percentage):

```
>>> calc = WAS_TempPercentileIndices(
...     base_period=slice("1961", "1990"),
...     percentile=90,
...     var_type='TMAX',
...     extreme_type='hot'
... )
>>> tx90p = calc.compute(tmax_da)
```

Cold nights (TN10p) for JJA season:

```
>>> cold_nights = WAS_TempPercentileIndices(
...     base_period=slice("1961", "1990"),
...     percentile=10,
...     var_type='TMIN',
...     extreme_type='cold',
...     season=[6, 7, 8]
... ).compute(tmin_da)
```

__init__ (*base_period: slice*, *percentile: float = 90*, *season: List[int] | None = None*, *var_type: str = 'TMAX'*, *extreme_type: str = 'hot'*, *bootstrap_samples: int = 10*, *min_base_years: int = 15*)

Parameters

- **base_period** (*slice*) – Slice for base period years, e.g., slice("1961", "1990")
- **percentile** (*float*) – Percentile value: - For hot extremes: 90, 95, 99 (days above percentile) - For cold extremes: 10, 5, 1 (days below percentile)
- **season** (*list*, *optional*) – Months to consider (e.g., [6, 7, 8] for JJA)
- **var_type** (*str*) – Temperature variable type: 'TMAX' (TX) or 'TMIN' (TN)
- **extreme_type** (*str*) – Type of extreme: 'hot' or 'cold'
- **bootstrap_samples** (*int*) – Number of bootstrap samples for confidence intervals
- **min_base_years** (*int*) – Minimum years required in base period

static transform_cdt (*df: DataFrame*) → *DataFrame*

Transform CDT format to long format *DataFrame*.

compute_insitu (*df_cdt: DataFrame*, *return_confidence: bool = False*) → *DataFrame* | *Tuple*[*DataFrame*, *DataFrame*, *DataFrame*]

Compute index for in-situ (station) data.

compute (*ds: Dataset | DataArray*, *var_name: str | None = None*, *parallel: bool = True*, *nb_cores: int = 4*) → *DataArray*

Compute index for xarray data (gridded).

get_index_definition () → *Dict*

Return index definition metadata.

```
class wass2s.was_compute_predictand.ETCCDIIndices
```

Bases: `object`

Factory for creating standard ETCCDI temperature indices.

```
static hot_days (base_period: slice, season: List[int] | None = None, percentile: float = 90) →  
                WAS_TempPercentileIndices
```

Create calculator for hot days (TX90p).

```
static hot_nights (base_period: slice, season: List[int] | None = None, percentile: float = 90) →  
                  WAS_TempPercentileIndices
```

Create calculator for hot nights (TN90p).

```
static cold_days (base_period: slice, season: List[int] | None = None, percentile: float = 10) →  
                 WAS_TempPercentileIndices
```

Create calculator for cold days (TX10p).

```
static cold_nights (base_period: slice, season: List[int] | None = None, percentile: float = 10) →  
                   WAS_TempPercentileIndices
```

Create calculator for cold nights (TN10p).

```
class wass2s.was_compute_predictand.WAS_PrecipIndices (base_period: slice, percentile: float =  
                                                         95, season: List[int] | None = None,  
                                                         wet_day_threshold: float = 1.0,  
                                                         min_base_years: int = 15)
```

Bases: `object`

Implementation of ETCCDI extreme precipitation indices, specifically R95p and R99p.

These indices measure the **annual total precipitation from very/extremely wet days**:

- **R95p**: Total precipitation from days when precipitation $\geq$ 95th percentile of wet-day precipitation in the base period
- **R99p**: Total precipitation from days when precipitation $\geq$ 99th percentile of wet-day precipitation in the base period

A **wet day** is defined as precipitation $\geq$ `wet_day_threshold` (default 1.0 mm).

Key Features: - Uses only **wet days** in the base period for percentile calculation (ETCCDI standard) - Supports seasonal filtering (e.g., JJAS season) - Handles both station-based (CDT format) and gridded (xarray) data - Proper leap-year handling and missing value treatment - Outputs in CDT-compatible format for station data

References: - ETCCDI Climate Change Indices (2009) - Zhang et al. (2011): Indices for monitoring changes in extremes - Sillmann et al. (2013): Climate extremes indices in the CMIP5 multi-model ensemble

Parameters

- **base_period** (*slice*) – Base period for percentile calculation, e.g. `slice("1991", "2020")`
- **percentile** (*float*, *default=95*) – Percentile threshold (95 → R95p, 99 → R99p)
- **season** (*list of int*, *optional*) – Months to include in analysis (e.g., [6,7,8,9] for JJAS)
- **wet_day_threshold** (*float*, *default=1.0*) – Minimum precipitation amount (mm) to consider a wet day
- **min_base_years** (*int*, *default=15*) – Minimum number of years required in base period (warning issued if fewer)

Examples

Compute R95p for full year:

```
>>> r95p_calc = WAS_PrecipIndices(
...     base_period=slice("1991", "2020"),
...     percentile=95
... )
>>> r95p = r95p_calc.compute(pr_da)  # pr_da is precipitation DataArray
```

Compute R99p for JJAS season only:

```
>>> r99p_jjas = WAS_PrecipIndices(
...     base_period=slice("1991", "2020"),
...     percentile=99,
...     season=[6, 7, 8, 9]
... ).compute(pr_da)
```

__init__ (*base_period: slice, percentile: float = 95, season: List[int] | None = None, wet_day_threshold: float = 1.0, min_base_years: int = 15*)

static transform_cdt (*df: DataFrame*) → DataFrame

Transform CDT format to long format DataFrame.

Parameters

df (*pd.DataFrame*) – Input DataFrame in CDT format

Returns

Long format DataFrame with columns: DATE, STATION, VALUE, LAT, LON

Return type

pd.DataFrame

compute_insitu (*df_cdt: DataFrame*) → DataFrame

Compute index for in-situ (station) data in CDT format.

Parameters

df_cdt (*pd.DataFrame*) – Input data in CDT format

Returns

Result in CDT format

Return type

pd.DataFrame

compute (*da: DataArray, parallel: bool = True, nb_cores: int = 4*) → DataArray

Compute index for xarray DataArray (gridded data).

Parameters

- **da** (*xr.DataArray*) – Precipitation DataArray with dimensions (time, y, x) or (time, lat, lon)
- **chunk_size** (*dict, optional*) – Chunk sizes for parallel processing, e.g., {'y': 100, 'x': 100}
- **parallel** (*bool*) – Whether to use Dask for parallel processing

Returns

Annual index values

Return type

xr.DataArray

get_index_definition() → Dict

Return index definition metadata.

class wass2s.was_compute_predictand.**HeatWaveMetric**(*values)

Bases: Enum

Heat wave / warm spell metrics (ETCCDI-style).

WSDI = 'WSDI'**HWF** = 'HWF'**HWDI** = 'HWDI'**HWN** = 'HWN'**class** wass2s.was_compute_predictand.**HeatWaveDefinition**(start_date: pandas._libs.tslibs.timestamps.Timestamp,
end_date: pandas._libs.tslibs.timestamps.Timestamp,
duration: int, max_temp: float,
mean_temp: float)

Bases: object

start_date: Timestamp**end_date**: Timestamp**duration**: int**max_temp**: float**mean_temp**: float**__init__**(start_date: Timestamp, end_date: Timestamp, duration: int, max_temp: float, mean_temp: float) → None**class** wass2s.was_compute_predictand.**WAS_HeatWaveIndices**(base_period: slice, tx_percentile: float = 90, tn_percentile: float | None = None,
min_consecutive_days: int = 6,
season: List[int] | None = None,
require_both_tx_tn: bool = False)

Bases: object

ETCCDI-like warm spell / heat wave indices for daily temperature.

Metrics: - WSDI: total number of days that belong to spells of $\geq$ min_consecutive_dayswhere $TX > TX_p$ (p -th percentile threshold based on base_period)

- HWF : number of spells/events per year (runs $\geq$ min_consecutive_days)
- HWDI: maximum run length per year (runs $\geq$ min_consecutive_days)

Threshold definition: - Standard ETCCDI practice uses a 5-day centered window for each calendar day.

Implementation: `rolling(time=5,center=True).construct("window")` then
`groupby(time.dayofyear).quantile(..., dim=["time","window"])`

Dask/xarray robustness: - quantile along time requires the 'time' dimension to be a single chunk.

We enforce `.chunk({"time": -1})` immediately before quantile.

- parallelization is across space via lat/lon chunking.

Output: - annual values on a 'T' dimension (one value per year, time stamp YYYY-01-01) - dims renamed to (T, Y, X) i.e. time->T, lat->Y, lon->X

`__init__` (*base_period*: *slice*, *tx_percentile*: *float* = 90, *tn_percentile*: *float* | *None* = *None*,
min_consecutive_days: *int* = 6, *season*: *List[int]* | *None* = *None*, *require_both_tx_tn*: *bool* = *False*)

`compute` (*ds_tx*: *Dataset* | *DataArray*, *ds_tn*: *Dataset* | *DataArray* | *None* = *None*, *metric*: *str* = 'WSDI', *parallel*:
bool = *True*, *nb_cores*: *int* = 4, *compute*: *bool* = *True*) → *DataArray*

Compute metric on gridded data.

Parameters

- **ds_tx** (*Dataset/DataArray*) – Daily max temperature (tasmax/TMAX).
- **ds_tn** (*Dataset/DataArray*, *optional*) – Daily min temperature (tasmin/TMIN) for compound events.
- **metric** (*str*) – “WSDI”, “HWF”, “HWDI” (also accepts “HWN” alias for HWF).
- **parallel** (*bool*) – Use dask parallelization across space.
- **nb_cores** (*int*) – Used to choose spatial chunk sizes.
- **compute** (*bool*) – If True, triggers compute() at the end.

Return type

`xr.DataArray` with dims (T,Y,X)

class `wass2s.was_compute_predictand.ETCCDIHeatWaveIndices`

Bases: `object`

Factory helpers for standard configurations.

static `wsdi` (*base_period*: *slice*, *tx_percentile*: *float* = 90, *min_consecutive_days*: *int* = 6, *season*: *List[int]* |
None = *None*) → *WAS_HeatWaveIndices*

static `heat_wave_frequency` (*base_period*: *slice*, *tx_percentile*: *float* = 90, *min_consecutive_days*: *int* =
3, *season*: *List[int]* | *None* = *None*) → *WAS_HeatWaveIndices*

static `heat_wave_duration_index` (*base_period*: *slice*, *tx_percentile*: *float* = 90,
min_consecutive_days: *int* = 3, *season*: *List[int]* | *None* = *None*)
→ *WAS_HeatWaveIndices*

static `compound_heat_wave` (*base_period*: *slice*, *tx_percentile*: *float* = 90, *tn_percentile*: *float* = 90,
min_consecutive_days: *int* = 3, *season*: *List[int]* | *None* = *None*) →
WAS_HeatWaveIndices

1.9.5 Bias correction

Bias correction utilities for climate variables.

Two classes are provided:

- `WAS_Qmap` : quantile-mapping methods tailored to precipitation (non-negative, possibly with many dry days). Supports QUANT, RQUANT, SSPLIN, PTF and DIST methods, with optional wet/dry day handling.
- `WAS_bias_correction` : bias-correction methods for continuous variables such as temperature or wind speed. Supports MEAN, VARSCALE, QUANT, NORM and DIST methods.

Both classes accept NumPy arrays (1D / 2D / 3D) and 3D `xarray.DataArray` inputs, and they propagate / handle NaNs.

This is a corrected version of the original implementation.

class `wass2s.was_bias_correction.WAS_Qmap`

Bases: `object`

Bias correction methods using quantile mapping techniques, adapted from the R `qmap` package.

Provides fitting and application methods for empirical quantile mapping (QUANT), robust quantile mapping (RQUANT), smoothing splines (SSPLIN), parametric transformations (PTF) and distribution-based methods (DIST). Methods optionally handle wet/dry day corrections and are designed for precipitation or other non-negative variables.

Notes

- Inputs are expected to be non-negative.
- For gridded data, computations are performed column-wise (per grid cell).
- `xarray` support preserves coordinates and attributes.
- NaNs in observed / modeled data are filtered out at fit time. NaNs in the data passed to `doQmap` propagate to NaNs in the output.

static `fitQmap(obs, mod, method, **kwargs)`

Fit a bias correction model using the specified quantile mapping method.

Parameters

- **obs** (*array_like or `xarray.DataArray`*) – Observed and modeled data of identical shape. Arrays may be 1D (time), 2D (time, grid) or 3D (T, Y, X). `xarray DataArrays` must be 3D.
- **mod** (*array_like or `xarray.DataArray`*) – Observed and modeled data of identical shape. Arrays may be 1D (time), 2D (time, grid) or 3D (T, Y, X). `xarray DataArrays` must be 3D.
- **method** (*str*) – One of 'QUANT', 'RQUANT', 'SSPLIN', 'PTF', 'DIST' (case-insensitive).
- ****kwargs** – Forwarded to the underlying `fitQmap*` method.

Returns

Fitted object with parameters, class identifier and metadata.

Return type

`dict`

static doQmap (*x*, *fobj*, ***kwargs*)

Apply the fitted bias correction to new data.

Returns the same array type as *x* and preserves its shape.

static fitQmapQUANT (*obs*, *mod*, *wet_day=False*, *qstep=0.01*, *nboot=1*)

Fit empirical quantile mapping with optional bootstrap of obs.

static doQmapQUANT (*x*, *fobj*, *type='linear'*)

Apply empirical quantile mapping (QUANT).

Three regions are handled explicitly:

- $x < th_mod \rightarrow 0$ (dry)
- $x > modq[-1] \rightarrow$ additive shift $x + (fitq[-1] - modq[-1])$ (constant-correction extrapolation for extremes)
- otherwise $\rightarrow$ interpolation between *modq* and *fitq*

NaN values in *x* are preserved as NaN in the output.

static fitQmapRQUANT (*obs*, *mod*, *wet_day=True*, *qstep=0.01*, *nlls=10*, *nboot=10*)

Robust quantile mapping with local linear fits.

static doQmapRQUANT (*x*, *fobj*, *type='linear'*, *slope_bound=(0.0, inf)*)

Apply RQUANT correction with slope-bounded extrapolation.

static fitQmapSSPLIN (*obs*, *mod*, *wet_day=False*, *qstep=0.01*)

Fit quantile mapping using PCHIP smoothing splines.

static doQmapSSPLIN (*x*, *fobj*)

Apply PCHIP-based quantile mapping correction.

static fitQmapPTF (*obs*, *mod*, *transfun='power'*, *parini=None*, *cost='RSS'*, *wet_day=False*, *qstep=None*)

Fit a parametric transformation function (PTF).

static doQmapPTF (*x*, *fobj*)

Apply parametric transformation correction.

static fitQmapDIST (*obs*, *mod*, *distr='berngamma'*, *qstep=None*, ***kwargs*)

Fit distribution-based quantile mapping.

Uses a Bernoulli for wet/dry occurrence plus a continuous distribution for the positive (wet) values.

static doQmapDIST (*x*, *fobj*)

Apply distribution-based bias correction.

static evaluate_bias_correction (*obs*, *mod*, *corrected*, *wet_threshold=0.1*,
extreme_quantiles=(0.95, 0.99), *time_dim=None*)

Evaluate bias correction performance.

Returns a 2-tuple in both code paths so callers can unpack consistently.

- xarray inputs $\rightarrow$ (*ds_dry_wet*, *ds_extreme*) of xarray Datasets.
- numpy inputs $\rightarrow$ (*dict_dry_wet*, *dict_extreme*).

Parameters

time_dim (*str*, *optional*) – Name of the time dimension for xarray inputs. If *None* it is taken from *obs.dims[0]* so any naming convention works.

static plot_fraction_group (*ds, group_prefix, extent=None, robust=True, cmap='viridis'*)

Plot dry/wet fraction variables with one shared colorbar.

static plot_mean_wet_group (*ds, extent=None, robust=True, cmap='viridis'*)

Plot mean_wet_* variables with one shared colorbar.

static plot_extreme_quantiles_group (*ds, extent=None, robust=True, cmap='viridis'*)

Plot extreme_quantiles_* variables faceted by quantile.

class wass2s.was_bias_correction.**WAS_bias_correction**

Bases: `object`

Bias correction methods for continuous climate variables (e.g. temperature, wind speed).

Supports mean adjustment, variance scaling, empirical quantile mapping and parametric distribution mapping. Inputs may include negative values; for strictly positive skewed data such as wind speed, use 'lognormal', 'gamma' or 'weibull' distributions.

Notes

- NaNs in observed / modeled data are filtered out at fit time. NaNs in doBC input propagate to NaNs in the output.
- Grid cells with fewer than 2 valid points in obs or mod are flagged (`all_nan`) and yield NaN output during application.

static fitBC (*obs, mod, method, **kwargs*)

Fit a bias correction model using the specified method.

static doBC (*x, fobj, **kwargs*)

Apply the fitted bias correction to new data.

static fitMean (*obs, mod*)

Fit mean additive bias correction.

static doMean (*x, fobj*)

Apply mean additive bias correction.

static fitVarscale (*obs, mod, std_tol=1e-06*)

Fit variance scaling bias correction.

If the modeled std is below `std_tol` the cell is flagged as `all_nan` rather than silently substituting `std_m = 1.0`; that used to hide the underlying degeneracy.

static doVarscale (*x, fobj*)

Apply variance scaling bias correction.

static fitQuant (*obs, mod, qstep=0.01, nboot=1*)

Fit empirical quantile mapping (non-parametric).

static doQuant (*x, fobj, type='linear'*)

Apply empirical quantile mapping correction.

static fitDist (*obs, mod, distr='normal'*)

Fit parametric bias correction assuming a specified distribution.

static doDist (*x, fobj*)

Apply parametric distribution bias correction.

1.9.6 Station & gridded merging

Station-gridded merging methods for observed rainfall.

Combines sparse station observations with a gridded first-guess field (e.g. CHIRPS, TAMSAT, or a model hindcast) using a suite of spatial interpolation and machine-learning techniques.

Classes

WAS_Merging

First-generation merging class (legacy interface). Implements simple bias adjustment, regression kriging, and neural-network kriging.

WAS_Merging_Extended

Refactored merging class with a richer method suite, DEM support, leave-one-out cross-validation, and a unified comparison plot.

Available methods:

simple_bias_adjustment

Additive or multiplicative mean-bias correction at station points.

regression_kriging

OLS regression on the gridded field + kriging of residuals.

conditional_merging

Preserve gridded spatial structure while matching station totals.

kriging_with_external_drift

Kriging with the gridded field as an external drift variable.

optimal_interpolation

Gaussian-covariance optimal interpolation (Gandin 1963).

barnes_interpolation

Multi-pass Barnes successive correction.

random_forest_merging / xgboost_merging

Machine-learning regression with k-NN neighbourhood features.

validate_merging_methods

Leave-one-out cross-validation across all methods.

```
class wass2s.was_merge_predictand.WAS_Merging (df: DataFrame, da: Dataset, date_month_day: str = '08-01')
```

Bases: `object`

```
__init__ (df: DataFrame, da: Dataset, date_month_day: str = '08-01')
```

```
adjust_duplicates (series: Series, increment: float = 1e-05) → Series
```

If any values in the Series repeat, nudge them by a tiny increment so that all are unique (to avoid indexing collisions).

```
transform_cpt (df: DataFrame, missing_value: float = -999.0) → DataArray
```

```
transform_cdt (df: DataFrame) → DataArray
```

```
auto_select_kriging_parameters (df: DataFrame, x_col: str = 'X', y_col: str = 'Y', z_col: str = 'residuals', variogram_models: list = None, nlags_range=range(3, 10), n_splits: int = 5, random_state: int = 42, verbose: bool = False, enable_plotting: bool = False)
```

Automatically selects the best variogram_model and nlags for Ordinary Kriging using cross-validation.

Returns

- `best_model` (str)
- `best_nlags` (int)
- `ok_best` (OrdinaryKriging object)
- `results_df` (pd.DataFrame)

simple_bias_adjustment (*missing_value: float = -999.0, do_cross_validation: bool = False*) -> (<class 'xarray.core.dataarray.DataArray'>, <class 'pandas.core.frame.DataFrame'>)

Performs spatial bias adjustment for each time slice. Optionally does leave-one-out (LOO) cross-validation to compute RMSE at each time.

Returns

- *xr.DataArray* – Concatenated bias-adjusted values along the time dimension.
- *pd.DataFrame* or *None* – A DataFrame containing the LOO RMSE for each time if `do_cross_validation=True`. Otherwise, returns *None*.

regression_kriging (*missing_value: float = -999.0, do_cross_validation: bool = False*) -> (<class 'xarray.core.dataarray.DataArray'>, <class 'pandas.core.frame.DataFrame'>)

Performs spatial bias adjustment for each time slice using:

- 1) Linear Regression
- 2) Kriging of residuals

Optionally performs LOO cross-validation for each time.

neural_network_kriging_ (*missing_value: float = -999.0, do_cross_validation: bool = False*) -> (<class 'xarray.core.dataarray.DataArray'>, <class 'pandas.core.frame.DataFrame'>)

Performs spatial bias adjustment for each time slice using:

- 1) Neural Network
- 2) Kriging of residuals

Optionally performs LOO cross-validation for each time.

neural_network_kriging (*missing_value: float = -999.0, do_cross_validation: bool = False*) -> (<class 'xarray.core.dataarray.DataArray'>, *pandas.core.frame.DataFrame* | *None*)

Performs a two-step spatial bias adjustment for each time slice, leveraging:

- 1) **Neural Network** (MLPRegressor) to capture non-linear relationships between the large-scale estimations (predictors) and in-situ observations (predictands).
- 2) **Kriging** of the residuals (observation - NN_prediction) to spatially interpolate the remaining error structure under the assumption that these residuals are stationary and exhibit spatial autocorrelation.

Parameters

- **missing_value** (*float, optional*) – Value used to fill missing data in the input station dataset, by default -999.0
- **do_cross_validation** (*bool, optional*) – Whether or not to perform cross-validation (e.g., leave-one-out or k-fold) during kriging parameter selection, by default False.

Returns

- *xr.DataArray* – Bias-adjusted field over the domain, with the same spatial dimensions as the original input (and time dimension if applicable).
- *pd.DataFrame* or *None* – If *do_cross_validation=True*, returns a *DataFrame* with RMSE records from CV; otherwise, *None*.

Notes

- The hyperparameter search for the MLP uses *GridSearchCV* with MSE-based scoring.
- Kriging assumes the residuals are reasonably stationary and spatially correlated.

multiplicative_bias (*missing_value: float = -999.0, do_cross_validation: bool = False*) -> (<class 'xarray.core.dataarray.DataArray'>, <class 'pandas.core.frame.DataFrame'>)

Apply multiplicative bias correction to gridded predictions using ground observations.

This method performs bias adjustment for each time step by comparing standardized observations and model estimates, interpolating residuals using kriging, and applying a multiplicative correction across the spatial domain.

Parameters

- **missing_value** (*float, optional*) – Value used to represent missing data in the input observational CSV, by default -999.0.
- **do_cross_validation** (*bool, optional*) – If *True*, perform Leave-One-Out Cross-Validation to estimate kriging performance, by default *False*.

Returns

- *xr.DataArray* – The bias-corrected spatial dataset over time (with dimensions: T, Y, X), masked over valid areas.
- *pd.DataFrame* or *None* – *DataFrame* containing LOOCV RMSE per time step, or *None* if cross-validation was not performed.

plot_merging_comparaison (*df_Obs, da_estimated, da_corrected, missing_value=-999.0*)

```
class wass2s.was_merge_predictand.WAS_Merging_Extended (df: DataFrame, da: DataArray | Dataset, dem: DataArray | None = None, date_month_day: str = '08-01', varname: str | None = None, projection: str = 'utm', utm_zone: int | None = None, hemisphere: str | None = None, kriging_cv: bool = True, kriging_fixed: dict | None = None, kriging_cache: bool = True, kriging_cache_key: str = 'global', max_grid_points_warn: int = 200000, verbose: bool = True)
```

Bases: *object*

Merging class for station observations with gridded estimates, using geostatistical and machine learning methods.

```
__init__ (df: DataFrame, da: DataArray | Dataset, dem: DataArray | None = None, date_month_day: str = '08-01', varname: str | None = None, projection: str = 'utm', utm_zone: int | None = None, hemisphere: str | None = None, kriging_cv: bool = True, kriging_fixed: dict | None = None, kriging_cache: bool = True, kriging_cache_key: str = 'global', max_grid_points_warn: int = 200000, verbose: bool = True)
```

adjust_duplicates (*series: Series, increment: float = 1e-05*) → Series

If any values in the Series repeat, nudge them by a tiny increment so that all are unique (to avoid indexing collisions).

transform_cpt (*df: DataFrame, missing_value: float = -999.0*) → Dataset

Convert a CPT-format station data frame to an xarray Dataset. Expects first two rows: station names, then latitudes, then longitudes. (The second row is used for LAT, the third for LON – but here we assume

the input has exactly two metadata rows: first row station names, second row LAT, third row LON. The class will handle it as in the first version.)

transform_cdt (*df: DataFrame, missing_value: float = -999.0*) → Dataset

Convert a CDT-format station data frame to an xarray Dataset. Expects first three rows: station names, longitudes, latitudes, elevations.

auto_select_kriging_parameters (*df: DataFrame, method_key: str, t=None, x_col: str = 'Xp', y_col: str = 'Yp', z_col: str = 'residuals', variogram_models=None, nlags_range=range(3, 10), n_splits=5, random_state=42*)

simple_bias_adjustment (*missing_value=-999.0*)

regression_kriging (*missing_value=-999.0, use_dem: bool = False, dem_fill: str = 'mask'*)

conditional_merging (*missing_value=-999.0*)

kriging_with_external_drift (*missing_value=-999.0, variogram_model='linear', drift_terms: tuple[str, ...] = ('Estimation',)*)

optimal_interpolation (*missing_value=-999.0, L_m: float = 150000.0, epsilon: float = 0.1, chunk_size: int | None = None*)

barnes_interpolation (*missing_value=-999.0, kappa_m: float = 200000.0, gamma: float = 0.3, chunk_size: int | None = None*)

random_forest_merging (*missing_value=-999.0, k_nearest=10*)

xgboost_merging (*missing_value=-999.0, tune=True, k_nearest=10*)

validate_merging_methods (*missing_value=-999.0, min_points_year=5, k_nearest=10*)

Leave-one-year-out validation of all merging methods at station points.

Parameters

- **missing_value** (*float*) – Missing value indicator for observations (passed to transform_cpt).
- **min_points_year** (*int*) – Minimum number of test points required to include a year.
- **k_nearest** (*int*) – Number of nearest stations to use for RF/XGB feature engineering.

Returns

Mean RMSE per method, sorted ascending.

Return type

pd.DataFrame

plot_merging_comparison (*da_corrected*, *da_estimated=None*, *missing_value=-999.0*)

Create a side-by-side scatter plot comparing observations against the original estimation and a corrected field.

Parameters

- **da_corrected** (*xr.DataArray*) – Corrected field (output from a merging method) with dimensions (T, Y, X).
- **da_estimated** (*xr.DataArray*, *optional*) – Original estimation grid. If *None*, uses *self.da*.
- **missing_value** (*float*, *optional*) – Missing value indicator for observations (passed to *transform_cpt*).

1.9.7 EOF analysis

Leakage-free EOF analysis for spatial predictors.

Wraps the *xeofs* EOF class with fold-safe preprocessing (NaN fill, optional linear detrending, optional per-field normalization) so that all statistics are estimated on the training fold and applied to the test fold without information leakage.

Class

WAS_EOF

EOF analysis for single-field or multivariate (list) predictors. Key methods: *fit*, *transform*, *inverse_transform*, *plot_EOF*.

```
class wass2s.was_eof.WAS_EOF (n_modes=None, use_coslat=True, standardize=False,  
                               opti_explained_variance=None, detrend=True, L2norm=True,  
                               normalize=None)
```

Bases: *object*

EOF analysis with leakage-free preprocessing.

Corrections relative to my previous version

1. The training-fold time-mean is STORED at fit and reused to fill NaNs in *transform()*. Previously the test fold was filled implicitly (or with its own statistics), which is the same leakage WAS_CCA_base fixed with *_fill_test_data*.
2. The linear trend is fitted on the training fold and the SAME coefficients are evaluated at the test/forecast times (already the intent, now made robust and always paired with the stored fill).
3. Variance optimization (*opti_explained_variance*) is done on the training fold only.
4. Dimension renaming is reversible (components can be mapped back to Y/X).

```
__init__ (n_modes=None, use_coslat=True, standardize=False, opti_explained_variance=None,  
          detrend=True, L2norm=True, normalize=None)
```

```
fit (predictor, dim='T', clim_year_start=None, clim_year_end=None)
```

```
transform (predictor, dim=None)
```

```
inverse_transform (pcs, return_anomalies=False)
```

plot_EOF (*s_eofs=None, s_expvar=None, cmap='RdBu_r', n_cols=3*)

Plot EOF spatial patterns and their explained variance.

Works for a single field AND for a multivariate (list) fit: in the list case *components()* returns one pattern per field, and each is drawn in its own figure (the fields may live on different grids).

Parameters

- **s_eofs** (*xarray.DataArray* or *list of DataArray, optional*) – EOF spatial patterns. If *None*, taken from the fitted model, so you can simply call *eof.plot_EOF()* after fit.
- **s_expvar** (*array-like, optional*) – Explained-variance ratio per mode (shared across fields for a combined EOF). If *None*, taken from the fitted model.
- **cmap** (*str*) – Diverging colormap (default 'RdBu_r').
- **n_cols** (*int*) – Mode columns per figure row (default 3).

1.9.8 Cross-validation

Leakage-free cross-validation for the WASS2S pipeline.

Provides a leave-one-out splitter with a configurable exclusion window and a high-level orchestrator that dispatches each fold to the correct preprocessing path for every supported model family.

Classes

CustomTimeSeriesSplit

Symmetric-window leave-one-out splitter; prevents near-neighbour temporal leakage.

WAS_Cross_Validator

Orchestrates cross-validation for CCA, Kernel CCA, PCR, linear models, machine-learning models, MME methods, and analog ensembles.

Helper functions

_pcr_isel_T

isel (*T=idx*) for a *DataArray* or *list/tuple of DataArrays*.

_pcr_T_index

Return the *T* coordinate used to drive the splitter.

class *wass2s.was_cross_validate.CustomTimeSeriesSplit* (*n_splits*)

Bases: *object*

Custom time series cross-validator for splitting data into training and test sets.

Ensures temporal ordering is maintained by generating training and test indices suitable for time series data, with an option to omit samples after the test index.

Parameters

n_splits (*int*) – Number of splits for the cross-validation.

__init__ (*n_splits*)

split (*X, nb_omit, y=None, groups=None*)

Generate indices to split data into training and test sets.

Yields training indices before the test index (excluding a specified number of samples after the test index) and test indices for each split.

Parameters

- **X** (*array-like*) – The data to be split, typically time series data.
- **nb_omit** (*int*) – Number of samples to omit from training after the test index to avoid data leakage.
- **y** (*array-like, optional*) – The target variable (ignored in this implementation).
- **groups** (*array-like, optional*) – Group labels for the samples (ignored in this implementation).

Yields

- **train_indices** (*ndarray*) – The training set indices for the current split.
- **test_indices** (*list*) – The test set indices for the current split.

get_n_splits (*X=None, y=None, groups=None*)

Return the number of splits for the cross-validation.

Parameters

- **X** (*array-like, optional*) – The data to be split (ignored in this implementation).
- **y** (*array-like, optional*) – The target variable (ignored in this implementation).
- **groups** (*array-like, optional*) – Group labels for the samples (ignored in this implementation).

Returns

The number of splits configured for the cross-validator.

Return type

int

class wass2s.was_cross_validate.**WAS_Cross_Validator** (*n_splits, nb_omit*)

Bases: *object*

Performs cross-validation for time series forecasting models using a custom time series split.

This class wraps a custom time series cross-validator to evaluate forecasting models, handling both deterministic hindcasts and probabilistic (tercile) predictions.

Parameters

- **n_splits** (*int*) – Number of splits for the cross-validation.
- **nb_omit** (*int*) – Number of samples to omit from training after the test index to prevent data leakage.

__init__ (*n_splits, nb_omit*)

cross_validate (*model, Predictant, Predictor=None, clim_year_start=None, clim_year_end=None, **model_params*)

Perform cross-validation to compute deterministic hindcasts and tercile probabilities.

Iterates over time series splits, trains the model on training data, and generates predictions for test data. Supports special handling for specific model types (e.g., CCA, Analog, ELM, ELR, and various machine learning models).

Parameters

- **model** (*object*) – The forecasting model instance to evaluate.

- **Predictant** (*xarray.DataArray*) – Target dataset with dimensions ('T', 'Y', 'X') or ('T', 'M', 'Y', 'X').
- **Predictor** (*xarray.DataArray, optional*) – Predictor dataset with dimensions ('T', 'M', 'Y', 'X') or ('T', 'features'). Required for most models except specific cases like WAS_Analog.
- **clim_year_start** (*int or str, optional*) – Start year of the climatology period for standardization and probability calculations.
- **clim_year_end** (*int or str, optional*) – End year of the climatology period for standardization and probability calculations.
- ****model_params** (*dict*) – Additional keyword arguments to pass to the model's `compute_model` method.

Returns

If the model supports probability calculations (has `compute_prob` method):

- **hindcast_det**
[*xarray.DataArray*] Deterministic hindcast results with dimensions ('T', 'Y', 'X') or ('probability', 'T', 'Y', 'X') for specific models.
- **hindcast_prob**
[*xarray.DataArray*] Tercile probabilities with dimensions ('probability', 'T', 'Y', 'X'), where 'probability' includes ['PB', 'PN', 'PA'] (below-normal, normal, above-normal).

If the model does not support probability calculations (e.g., WAS_mme_ELR):

- **hindcast_det**
[*xarray.DataArray*] Deterministic hindcast results with dimensions ('T', 'Y', 'X').

Return type

tuple or *xarray.DataArray*

1.9.9 Linear models

Linear regression models for spatial seasonal forecasting.

Each class implements a `compute_model` / `compute_prob` / `forecast` interface compatible with *WAS_Cross_Validator* and *WAS_PCR*.

Classes

WAS_LinearRegression_Model

Pixel-wise ordinary least-squares regression with parametric or non-parametric tercile probabilities.

WAS_Ridge_Model

Ridge (L2-regularised) regression with Optuna / randomized hyperparameter search.

WAS_Lasso_Model

Lasso (L1-regularised) regression with Optuna / randomized hyperparameter search.

WAS_LassoLars_Model

LassoLars regression.

WAS_ElasticNet_Model

Elastic-Net (L1+L2) regression.

```
class wass2s.was_linear_models.WAS_LinearRegression_Model (nb_cores=1,
                                                            dist_method='nonparam')
```

Bases: `object`

Simple Linear Regression model for spatiotemporal climate prediction.

This class implements ordinary least squares linear regression (via scikit-learn's *LinearRegression*) to model linear relationships between predictors and a continuous predictand (e.g., seasonal rainfall totals, temperature anomalies, agro-climatic indices).

Key features: - Basic linear modeling (no polynomial features or regularization by default) - Spatially parallel fitting and prediction across large grids using dask + xarray - Deterministic point predictions with optional error computation - Probabilistic tercile forecasting (Below/Normal/Above = PB/PN/PA) using either:

- Parametric best-fit distributions per grid cell ('bestfit')
- Non-parametric sampling of historical forecast errors ('nonparam')

Ideal as a baseline model or when relationships are expected to be approximately linear.

Parameters

- **nb_cores** (*int*, *default=1*) – Number of CPU cores for parallel processing (dask workers).
- **dist_method** (*{'bestfit', 'nonparam'}*, *default='nonparam'*) – Method for computing tercile probabilities: - 'bestfit' → uses best-fit distribution per grid cell (requires distribution fit inputs) - 'nonparam' → empirical sampling of historical forecast errors

nb_cores, dist_method

Stored initialization parameters.

fit_predict (*x, y, x_test, y_test=None*)

Fits linear regression on one grid cell and returns [error, prediction] or just prediction.

compute_model (*X_train, y_train, X_test, y_test*)

Parallel linear regression across the entire spatial domain. Returns xarray.DataArray with dims ('output'=['error','prediction'], Y, X).

compute_prob (*Predictant, clim_year_start, clim_year_end, hindcast_det,*

best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None)

Computes tercile probabilities for deterministic hindcasts.

forecast (*Predictant, clim_year_start, clim_year_end, Predictor,*
hindcast_det,

Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None,
best_scale_da=None)

Full end-to-end forecast pipeline for one target year: - deterministic linear prediction - tercile probabilities (PB, PN, PA)

Notes

- **Input data requirements:** - Target (y) should be continuous (rainfall, temperature, etc.). - Predictors (X) should be continuous or properly encoded.
- Negative predictions are automatically clipped to zero (useful for rainfall or non-negative variables).

- Linear regression assumes linear relationships; for non-linear patterns, consider higher-degree polynomial, tree-based models (XGBoost, RF), or neural approaches (MLP).
- Large spatial domains benefit significantly from higher *nb_cores*.
- For *dist_method='bestfit'*, distribution fit results from *WAS_TransformData* must be provided.
- No explicit feature or target scaling is applied (linear regression coefficients adjust automatically, but centering/scaling predictors can improve numerical stability when features have very different scales).

Warning

- Linear regression can produce unrealistic negative predictions for non-negative variables (e.g., rainfall); clipping is applied automatically but may bias results.
- Assumes homoscedasticity and no strong multicollinearity; check residuals and VIF if needed.
- Small per-grid-cell training sets can lead to unstable coefficient estimates.
- For very non-linear relationships, performance will be poor compared to tree-based or spline models.

__init__ (*nb_cores=1, dist_method='nonparam'*)

Initializes the *WAS_LinearRegression_Model* with a specified number of CPU cores.

Parameters

- **nb_cores** (*int, optional*) – Number of CPU cores to use for parallel computation, by default 1.
- **dist_method** (*str, optional*) – Distribution method to compute tercile probabilities, by default “gamma”.

fit_predict (*x, y, x_test, y_test=None*)

Fits a linear regression model to the provided training data, makes predictions on the test data, and calculates the prediction error (if *y_test* is provided).

Parameters

- **x** (*array-like, shape (n_samples, n_features)*) – Training data (predictors).
- **y** (*array-like, shape (n_samples,)*) – Training targets.
- **x_test** (*array-like, shape (n_features,) or (1, n_features)*) – Test data (predictors).
- **y_test** (*float or None*) – Test target value. If *None*, no error is computed.

Returns

If *y_test* is not *None*, returns [error, prediction]. If *y_test* is *None*, returns [prediction].

Return type

np.ndarray

compute_model (*X_train, y_train, X_test, y_test*)

Applies linear regression across a spatiotemporal dataset in parallel.

Parameters

- **X_train** (*xarray.DataArray*) – Training predictors with dims (“T”, “features”).
- **y_train** (*xarray.DataArray*) – Training targets with dims (“T”, “Y”, “X”).

- **x_test** (*xarray.DataArray*) – Test predictors, shape ('features',) or (T, features).
- **y_test** (*xarray.DataArray*) – Test targets with dims ('Y','X'), or broadcastable.

Returns

dims ('output','Y','X'), where 'output'=[error, prediction].

Return type

xarray.DataArray

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1, T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in *_ppf_terciles_from_code* - *shape, loc, scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use *best_guess[t]* to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.
- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If `dist_method == 'bestfit'`:

• Use cluster-based best-fit distributions to:

- derive terciles analytically from (`best_code_da`, `best_shape_da`, `best_loc_da`, `best_scale_da`),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=['PB','PN','PA'], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generates a single-year forecast using linear regression, then computes tercile probabilities using `self.dist_method`.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data with dims (T, Y, X).

- **clim_year_start** (*int*) – Start year for climatology
- **clim_year_end** (*int*) – End year for climatology
- **Predictor** (*xarray.DataArray*) – Historical predictor data with dims (T, features).
- **hindcast_det** (*xarray.DataArray*) – Historical deterministic forecast with dims (output=[error,prediction], T, Y, X).
- **Predictor_for_year** (*xarray.DataArray*) – Single-year predictor with shape (features,) or (1, features).

Returns

- **result_** (*xarray.DataArray*) – dims (output=2, Y, X) => [error, prediction]. For a true forecast, error is typically NaN.
- **hindcast_prob** (*xarray.DataArray*) – dims (probability=3, Y, X) => [PB, PN, PA].

```
class wass2s.was_linear_models.WAS_Ridge_Model (alpha_range=None, n_clusters=5, nb_cores=1,  
                                              dist_method='nonparam',  
                                              hyperparam_optimizer='bayesian', n_trials=50,  
                                              n_iter=50, mode='cluster')
```

Bases: `object`

Ridge regression model for spatial rainfall (or continuous climate variable) prediction.

Supports two optimization modes: - **'grid'**: Independent alpha optimization for each grid cell (pixel-wise)

→ Most local fit, but computationally expensive

- **'cluster'** (default): Spatially cluster the predictand field → one alpha per cluster → Much faster, more robust to noise, better generalization on small/sparse data

Hyperparameter search methods: - **'bayesian'** → Optuna Bayesian optimization (recommended) - **'random'** → scikit-learn RandomizedSearchCV - **'ridgecv'** → classic RidgeCV grid search

The model uses scikit-learn's Ridge regression with L2 regularization to prevent overfitting, especially useful when predictors are correlated or when per-cell training data is limited.

Parameters

- **alpha_range** (*array-like, optional*) – Range of alpha (regularization strength) values to search. Default: `np.logspace(-10, 10, 100)` — wide log-scale range.
- **n_clusters** (*int, default=5*) – Number of spatial clusters (used only when `mode='cluster'`).
- **nb_cores** (*int, default=1*) – Number of CPU cores for parallel computation (mainly used in grid mode).
- **dist_method** (*str, default='nonparam'*) – Method for computing tercile probabilities (PB/PN/PA). Currently only **'nonparam'** fully implemented; **'bestfit'** requires additional inputs.
- **hyperparam_optimizer** (*{'bayesian', 'random', 'ridgecv'}, default='bayesian'*) – Strategy for finding optimal alpha: - **'bayesian'** → Optuna Bayesian optimization (most efficient) - **'random'** → RandomizedSearchCV - **'ridgecv'** → Built-in RidgeCV grid search
- **n_trials** (*int, default=50*) – Number of trials for Bayesian optimization (Optuna).
- **n_iter** (*int, default=50*) – Number of parameter settings sampled in RandomizedSearchCV.

- **mode** ({'cluster', 'grid'}, default='cluster') – Optimization strategy: - 'cluster' → one alpha per spatial cluster (recommended for most cases) - 'grid' → independent alpha per grid cell (very slow, but maximally local)

Notes

- Cluster mode is significantly faster and usually more stable, especially for noisy or short time series data common in climate modeling.
- All NaN values in time series are properly masked.
- Requires at least 10 valid time steps per location/cluster to attempt optimization.
- Negative predictions are clipped to zero (useful for rainfall or non-negative variables).
- Large domains in 'grid' mode benefit greatly from higher *nb_cores*.
- For *dist_method='bestfit'*, additional distribution fit parameters must be provided.

compute_hyperparameters (*predictand*, *predictor*, *clim_year_start*, *clim_year_end*)

Computes spatially varying optimal alpha values (and cluster map if mode='cluster').

fit_predict (*x*, *y*, *x_test*, *y_test*, *alpha*)

Fits Ridge model on one grid cell using given alpha and makes prediction.

compute_model (*X_train*, *y_train*, *X_test*, *y_test*, *alpha*)

Parallel ridge regression across entire spatial domain using provided alpha map.

compute_prob (*Predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*,

best_code_da=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None)

Computes tercile probabilities for hindcast predictions.

forecast (*Predictant*, *clim_year_start*, *clim_year_end*, *Predictor*,
hindcast_det,

Predictor_for_year, *alpha*, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None,
best_scale_da=None)

Full end-to-end forecast pipeline for one target year: - deterministic ridge prediction - tercile probabilities (PB, PN, PA)

Examples

Typical seasonal rainfall forecasting workflow (recommended cluster mode):

```
>>> ridge_model = WAS_Ridge_Model(
...     mode='cluster',
...     n_clusters=8,
...     hyperparam_optimizer='bayesian',
...     n_trials=80,
...     nb_cores=12
... )
```

1. Compute spatially varying alpha (one per cluster) >> *alpha_map*, *cluster_map* = *ridge_model.compute_hyperparameters*(... *seasonal_rainfall*, *predictors*, 1991, 2020)

2. Train & predict on hindcast period >> *hindcast_pred* = *ridge_model.compute_model*(... *X_train*=*predictors*, ... *y_train*=*seasonal_rainfall*, ... *X_test*=*predictors_hindcast*, ... *y_test*=None, ... *alpha*=*alpha_map* ...)

```
# 3. Compute probabilistic hindcast (terciles) »> hindcast_prob = ridge_model.compute_prob( ... sea-
sonal_rainfall, 1991, 2020, hindcast_pred, ... best_code_da=dist_code_da, best_shape_da=shape_da, ...
best_loc_da=loc_da, best_scale_da=scale_da ... )
```

```
# 4. Forecast next year (e.g. 2025) »> forecast_det, forecast_prob = ridge_model.forecast( ... sea-
sonal_rainfall, 1991, 2020, ... predictors, hindcast_pred, ... predictor_2025, ... alpha=alpha_map, ...
best_code_da=dist_code_da, best_shape_da=shape_da, ... best_loc_da=loc_da, best_scale_da=scale_da ... )
```

```
__init__ (alpha_range=None, n_clusters=5, nb_cores=1, dist_method='nonparam',
hyperparam_optimizer='bayesian', n_trials=50, n_iter=50, mode='cluster')
```

compute_hyperparameters (*predictand, predictor, clim_year_start, clim_year_end*)

Compute spatially varying Ridge alpha parameter(s).

Parameters

- **predictand** (*xarray.DataArray*) – Target variable (usually rainfall) Must have dimensions: ('T', 'Y', 'X')
- **predictor** (*xarray.DataArray*) – Predictor array with dimensions: ('T', 'Y', 'X', 'features') or ('T', 'features')
- **clim_year_start** (*int*) – Start year of climatological period for standardization
- **clim_year_end** (*int*) – End year of climatological period for standardization

Returns

- **alpha_map** (*xarray.DataArray*) – Spatial field of optimal alpha values - shape (Y, X) when mode='grid' - shape (Y, X) with constant values per cluster when mode='cluster'
- **cluster_da** (*xarray.DataArray or None*) – Cluster labels (integers) when mode='cluster' Contains NaN where clustering was not possible Returns dummy array filled with 1s when mode='grid'

Raises

- **ValueError** – If mode is neither 'cluster' nor 'grid'
- **RuntimeError** – If dask client could not be created or computation failed (grid mode)

Notes

- All NaN values in time series are handled (masked)
- Minimum 10 valid time steps required per location/cluster
- Standardization is applied to predictand using specified climatology period

fit_predict (*x, y, x_test, y_test, alpha*)

Fit a ridge regression model and make predictions.

Parameters

- **x** (*ndarray*) – Training data (shape = [n_samples, n_features]).
- **y** (*ndarray*) – Target values for training data (shape = [n_samples,]).
- **x_test** (*ndarray*) – Test data (shape = [n_features,] or [1, n_features]).
- **y_test** (*float*) – Target value for test data.
- **alpha** (*float*) – Regularization strength for Ridge regression.

Returns

[error, prediction].

Return type

ndarray

compute_model (*X_train*, *y_train*, *X_test*, *y_test*, *alpha*)

Fit and predict ridge regression model for spatiotemporal data using Dask for parallel computation.

Parameters

- **X_train** (*xarray.DataArray*) – Training predictor data (dims: T, features).
- **y_train** (*xarray.DataArray*) – Training predictand data (dims: T, Y, X).
- **X_test** (*xarray.DataArray*) – Test predictor data (dims: features) or broadcastable.
- **y_test** (*xarray.DataArray*) – Test predictand data (dims: Y, X).
- **alpha** (*xarray.DataArray*) – Spatial map of alpha values for each grid cell.

Returns

dims (output=2, Y, X) => [error, prediction].

Return type

xarray.DataArray

static weibull_shape_solver (*k*, *M*, *V*)

Solver for Weibull shape parameter.

static calculate_tercile_probabilities_bestfit (*best_guess*, *error_variance*, *T1*, *T2*,
dist_code, *dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1*, *T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in `_ppf_terciles_from_code`
- *shape*, *loc*, *scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.

- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, alpha, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generates a ridge-based forecast for a single future time (year).

class wass2s.was_linear_models.**WAS_Lasso_Model** (*alpha_range=None, n_clusters=5, nb_cores=1, dist_method='nonparam', hyperparam_optimizer='bayesian', n_trials=50, n_iter=50, mode='cluster'*)

Bases: *object*

Lasso regression model for spatial rainfall (or continuous climate variable) prediction.

Lasso (L1 regularization) performs automatic feature selection by driving less important predictor coefficients exactly to zero — particularly useful when working with many atmospheric, oceanic, or teleconnection indices where some may be redundant or noisy.

Supports two optimization modes: - **'cluster'** (default): One optimal alpha per spatial cluster (fast, robust, recommended) - **'grid'**: Independent alpha optimization for each grid cell (very local fit, slow)

Hyperparameter search methods: - **'bayesian'** → Optuna Bayesian optimization (recommended, efficient) - **'random'** → scikit-learn RandomizedSearchCV - **'lassocv'** → scikit-learn LassoCV (fastest, grid-based)

Compared to Ridge, Lasso typically requires: - smaller alpha values (less aggressive regularization) - higher *max_iter* (convergence can be slower near zero coefficients)

Parameters

- **alpha_range** (*array-like, optional*) – Range of alpha (regularization strength) values to search. Default: `np.logspace(-6, 2, 100)` — suitable for standardized rainfall data.
- **n_clusters** (*int, default=5*) – Number of spatial clusters (only used when *mode='cluster'*).
- **nb_cores** (*int, default=1*) – Number of CPU cores for parallel computation (mainly used in *grid* mode).

- **dist_method**(*str*, *default*='nonparam') – Method for computing tercile probabilities (PB/PN/PA). Currently only 'nonparam' fully implemented; 'bestfit' requires additional inputs.
- **hyperparam_optimizer** ({'bayesian', 'random', 'lassocv'}, *default*='bayesian') – Strategy for finding optimal alpha: - 'bayesian' → Optuna Bayesian optimization (best quality/speed trade-off) - 'random' → Randomized search - 'lassocv' → Built-in LassoCV grid search (fastest)
- **n_trials**(*int*, *default*=50) – Number of trials for Bayesian optimization (Optuna).
- **n_iter**(*int*, *default*=50) – Number of parameter settings sampled in randomized search.
- **mode**({'cluster', 'grid'}, *default*='cluster') – Optimization strategy: - 'cluster' → one alpha per spatial cluster (fast, stable, recommended) - 'grid' → independent alpha per grid cell (slow, maximally local)

Notes

- **Cluster mode** is strongly recommended for most rainfall/climate applications: - Much faster - More robust to noise and short time series - Better generalization
- **Grid mode** should only be used when maximum spatial detail is critical and sufficient computing resources are available.
- Lasso is more sensitive to feature scaling than Ridge → predictors and predictand should be properly standardized/normalized.
- Requires at least 10 valid time steps per location/cluster to attempt optimization.
- Negative predictions are clipped to zero (useful for rainfall).
- Large domains in 'grid' mode benefit greatly from higher *nb_cores*.

compute_hyperparameters(*predictand*, *predictor*, *clim_year_start*, *clim_year_end*)

Computes spatially varying optimal alpha values (and cluster map if mode='cluster').

fit_predict(*x*, *y*, *x_test*, *y_test*, *alpha*)

Fits Lasso model on one grid cell using given alpha and makes prediction.

compute_model(*X_train*, *y_train*, *X_test*, *y_test*, *alpha*=None, *clim_year_start*=None, *clim_year_end*=None)

Parallel Lasso regression across entire spatial domain using provided alpha map.

compute_prob(*Predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*,

best_code_da=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None)

Computes tercile probabilities for hindcast predictions.

forecast(*Predictant*, *clim_year_start*, *clim_year_end*, *Predictor*,
hindcast_det,

Predictor_for_year, *alpha*, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None,
best_scale_da=None)

Full end-to-end forecast pipeline for one target year: - deterministic Lasso prediction - tercile probabilities (PB, PN, PA)

Examples

Recommended workflow (cluster mode with Bayesian optimization):

```
>>> lasso_model = WAS_Lasso_Model(
...     mode='cluster',
...     n_clusters=8,
...     hyperparam_optimizer='bayesian',
...     n_trials=80,
...     nb_cores=12
... )
```

1. Compute spatially varying alpha (one per cluster) »> alpha_map, cluster_map = lasso_model.compute_hyperparameters(... seasonal_rainfall, predictors, 1991, 2020)

2. Train & predict on hindcast period »> hindcast_pred = lasso_model.compute_model(... X_train=predictors, ... y_train=seasonal_rainfall, ... X_test=predictors_hindcast, ... y_test=None, ... alpha=alpha_map ...)

3. Compute probabilistic hindcast (terciles) »> hindcast_prob = lasso_model.compute_prob(... seasonal_rainfall, 1991, 2020, hindcast_pred, ... best_code_da=dist_code_da, best_shape_da=shape_da, ... best_loc_da=loc_da, best_scale_da=scale_da ...)

4. Forecast next year (e.g. 2025) »> forecast_det, forecast_prob = lasso_model.forecast(... seasonal_rainfall, 1991, 2020, ... predictors, hindcast_pred, ... predictor_2025, ... alpha=alpha_map, ... best_code_da=dist_code_da, best_shape_da=shape_da, ... best_loc_da=loc_da, best_scale_da=scale_da ...)

Warning

- Lasso can eliminate all predictors if alpha is too large → monitor selected features.
- Very small training sets per cell/cluster may lead to unstable results.
- For heavy-tailed rainfall distributions, consider log-transformation before modeling.
- Use 'bayesian' optimizer for best quality; 'lassocv' is fastest but least flexible.

__init__ (alpha_range=None, n_clusters=5, nb_cores=1, dist_method='nonparam', hyperparam_optimizer='bayesian', n_trials=50, n_iter=50, mode='cluster')

compute_hyperparameters (predictand, predictor, clim_year_start, clim_year_end)

Compute spatially varying optimal Lasso regularization parameter alpha.

Parameters

- **predictand** (*xarray.DataArray*) – Target variable (typically rainfall amount) Expected dimensions: ('T', 'Y', 'X')
- **predictor** (*xarray.DataArray*) – Predictor fields Expected dimensions: ('T', 'Y', 'X', 'features') or ('T', 'features')
- **clim_year_start** (*int*) – Start year of the reference climatology period for standardization
- **clim_year_end** (*int*) – End year of the reference climatology period for standardization

Returns

- **alpha_map** (*xarray.DataArray*) – Spatial field (Y, X) containing optimal alpha value(s) - In 'grid' mode: different value possible for each cell - In 'cluster' mode: same value for all cells in the same cluster

- **cluster_da** (*xarray.DataArray*) – Cluster labels (when mode='cluster') Dummy array filled with 1s when mode='grid' (for compatibility)

Raises

- **ValueError** – If unknown mode is provided
- **RuntimeError** – If dask parallel computation fails (grid mode)

Notes

- Time series with fewer than 10 valid points are skipped (return NaN)
- Lasso uses increased max_iter=10000 to help convergence
- All NaN handling is automatic (masking invalid time steps)
- Progress messages are printed during long computations

fit_predict (*x*, *y*, *x_test*, *y_test*, *alpha*)

Fit a Lasso model and make predictions.

compute_model (*X_train*, *y_train*, *X_test*, *y_test*, *alpha=None*, *clim_year_start=None*, *clim_year_end=None*)

Computes Lasso predictions for spatiotemporal data using provided alpha values.

Returns

dims (output=2, Y, X) => [error, prediction].

Return type

xarray.DataArray

static weibull_shape_solver (*k*, *M*, *V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess*, *error_variance*, *T1*, *T2*,
dist_code, *dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1*, *T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in `_ppf_terciles_from_code` - *shape*, *loc*, *scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.

- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.
- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If **dist_method** == 'bestfit':

- **Use cluster-based best-fit distributions to:**

- derive terciles analytically from (best_code_da, best_shape_da, best_loc_da, best_scale_da),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=['PB','PN','PA'], T, Y, X).

Return type

`xarray.DataArray`

forecast (*Predictant*, *clim_year_start*, *clim_year_end*, *Predictor*, *hindcast_det*, *Predictor_for_year*, *alpha*, *best_code_da=None*, *best_shape_da=None*, *best_loc_da=None*, *best_scale_da=None*)

Generate a forecast for a single time (year) using Lasso with alpha, then compute tercile probabilities from the chosen distribution method.

Parameters

- **Predictant** (`xarray.DataArray`) – Observed data (T, Y, X).
- **clim_year_start** (`int`) – Start year of the climatology period.
- **clim_year_end** (`int`) – End year of the climatology period.
- **Predictor** (`xarray.DataArray`) – Historical predictor data (T, features).
- **hindcast_det** (`xarray.DataArray`) – Historical deterministic forecast with dims (output=2, T, Y, X).
- **Predictor_for_year** (`xarray.DataArray`) – Single-year predictor data (features,).
- **alpha** (`xarray.DataArray`) – Spatial map of alpha values (Y, X).

Returns

- **result_** (`xarray.DataArray`) – dims (output=2, Y, X) => [error, prediction]. For a real forecast scenario, the error is typically NaN.
- **hindcast_prob** (`xarray.DataArray`) – dims (probability=3, Y, X) => [PB, PN, PA].

```
class wass2s.was_linear_models.WAS_ElasticNet_Model (alpha_range=None,  
                                                    l1_ratio_range=None, n_clusters=5,  
                                                    nb_cores=1, dist_method='nonparam',  
                                                    hyperparam_optimizer='bayesian',  
                                                    n_trials=50, n_iter=50,  
                                                    mode='cluster')
```

Bases: `object`

ElasticNet regression model for spatial rainfall (or continuous climate variable) prediction.

ElasticNet combines L1 (Lasso) and L2 (Ridge) penalties in a single model: - L1 penalty → feature selection (sparsity) - L2 penalty → coefficient shrinkage and stability with correlated predictors This makes ElasticNet particularly suitable for climate modeling where: - Many predictors (teleconnections, atmospheric fields) are often correlated - Some predictors may be irrelevant or redundant

Two spatial optimization strategies: - **‘cluster’** (default): One optimal (alpha, l1_ratio) pair per spatial cluster

→ Fast, robust, excellent generalization — **recommended for most applications**

- **‘grid’**: Independent optimization per grid cell → Maximum spatial detail, but **very computationally expensive**

Supported hyperparameter search methods: - ‘bayesian’ → Optuna Bayesian optimization (best quality/speed trade-off) - ‘random’ → scikit-learn RandomizedSearchCV - ‘elasticnetcv’ → scikit-learn ElasticNetCV (fastest, discrete grid)

Parameters

- **alpha_range** (*array-like, optional*) – Range of total regularization strength (alpha). Default: `np.logspace(-6, 2, 100)` — well-suited for standardized rainfall data.
- **l1_ratio_range** (*array-like, optional*) – Range of L1 penalty ratio (0 = pure Ridge, 1 = pure Lasso). Default: `[0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1.0]`
- **n_clusters** (*int, default=5*) – Number of spatial clusters (only used when `mode='cluster'`).
- **nb_cores** (*int, default=1*) – Number of CPU cores for parallel computation (mainly used in grid mode).
- **dist_method** (*str, default='nonparam'*) – Method for computing tercile probabilities (PB/PN/PA). Currently only 'nonparam' fully implemented; 'bestfit' requires additional inputs.
- **hyperparam_optimizer** (*{'bayesian', 'random', 'elasticnetcv'}, default='bayesian'*) – Strategy for finding optimal (alpha, l1_ratio): - 'bayesian' → Optuna Bayesian optimization (recommended) - 'random' → Randomized search - 'elasticnetcv' → Built-in ElasticNetCV (fastest)
- **n_trials** (*int, default=50*) – Number of trials for Bayesian optimization (Optuna).
- **n_iter** (*int, default=50*) – Number of parameter settings sampled in randomized search.
- **mode** (*{'cluster', 'grid'}, default='cluster'*) – Optimization approach: - 'cluster' → one (alpha, l1_ratio) per spatial cluster (fast & stable) - 'grid' → independent optimization per grid cell (slow, maximally local)

Notes

- **Cluster mode** is the recommended default for most rainfall/climate applications: - Much faster - More robust to noise, short time series, and data sparsity - Excellent generalization
- **Grid mode** should only be used when maximum spatial detail is critical and sufficient computing resources are available.
- ElasticNet benefits from feature/predictand standardization — ensure proper scaling.
- Requires at least 10 valid time steps per location/cluster to attempt optimization.
- Negative predictions are automatically clipped to zero (useful for rainfall).
- Large domains in 'grid' mode benefit greatly from higher `nb_cores`.

compute_hyperparameters (*predictand, predictor, clim_year_start, clim_year_end*)

Computes spatially varying optimal (alpha, l1_ratio) pairs (and cluster map if `mode='cluster'`).

fit_predict (*x, y, x_test, y_test, alpha, l1_ratio*)

Fits ElasticNet model on one grid cell using given parameters and makes prediction.

compute_model (*X_train, y_train, X_test, y_test, alpha=None, l1_ratio=None, clim_year_start=None, clim_year_end=None*)

Parallel ElasticNet regression across entire spatial domain using provided parameter maps.

compute_prob (*Predictant, clim_year_start, clim_year_end, hindcast_det,*

best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None)

Computes tercile probabilities for hindcast predictions.

```
forecast(Predictand, clim_year_start, clim_year_end, Predictor,
hindcast_det,
```

```
Predictor_for_year, alpha, l1_ratio, best_code_da=None, best_shape_da=None,
best_loc_da=None, best_scale_da=None)
```

Full end-to-end forecast pipeline for one target year: - deterministic ElasticNet prediction - tercile probabilities (PB, PN, PA)

Examples

Recommended workflow (cluster mode with Bayesian optimization):

```
>>> enet_model = WAS_ElasticNet_Model(
...     mode='cluster',
...     n_clusters=8,
...     hyperparam_optimizer='bayesian',
...     n_trials=80,
...     nb_cores=12
... )
```

1. Compute spatially varying (alpha, l1_ratio) — one pair per cluster >>> alpha_map, l1_map, cluster_map = enet_model.compute_hyperparameters(... seasonal_rainfall, predictors, 1991, 2020)

2. Train & predict on hindcast period >>> hindcast_pred = enet_model.compute_model(... X_train=predictors, ... y_train=seasonal_rainfall, ... X_test=predictors_hindcast, ... y_test=None, ... alpha=alpha_map, ... l1_ratio=l1_map ...)

3. Compute probabilistic hindcast (terciles) >>> hindcast_prob = enet_model.compute_prob(... seasonal_rainfall, 1991, 2020, hindcast_pred, ... best_code_da=dist_code_da, best_shape_da=shape_da, ... best_loc_da=loc_da, best_scale_da=scale_da ...)

4. Forecast next year (e.g. 2025) >>> forecast_det, forecast_prob = enet_model.forecast(... seasonal_rainfall, 1991, 2020, ... predictors, hindcast_pred, ... predictor_2025, ... alpha=alpha_map, ... l1_ratio=l1_map, ... best_code_da=dist_code_da, best_shape_da=shape_da, ... best_loc_da=loc_da, best_scale_da=scale_da ...)

Warning

- Very high alpha or l1_ratio=1.0 can eliminate all predictors → monitor selected features.
- Small training sets per cell/cluster may lead to unstable results.
- For heavy-tailed rainfall, consider log-transformation before modeling.
- Use 'bayesian' optimizer for best quality; 'elasticnetcv' is fastest but least flexible.

```
__init__(alpha_range=None, l1_ratio_range=None, n_clusters=5, nb_cores=1, dist_method='nonparam',
hyperparam_optimizer='bayesian', n_trials=50, n_iter=50, mode='cluster')
```

```
compute_hyperparameters(predictand, predictor, clim_year_start, clim_year_end)
```

Compute spatially varying optimal ElasticNet parameters (alpha and l1_ratio).

Parameters

- **predictand** (*xarray.DataArray*) – Target variable (typically rainfall) Expected dimensions: ('T', 'Y', 'X')

- **predictor** (*xarray.DataArray*) – Predictor fields Expected dimensions: ('T', 'Y', 'X', 'features') or ('T', 'features')
- **clim_year_start** (*int*) – Start year of climatological standardization period
- **clim_year_end** (*int*) – End year of climatological standardization period

Returns

- **alpha_map** (*xarray.DataArray*) – Spatial field (Y, X) of optimal alpha values
- **ll_ratio_map** (*xarray.DataArray*) – Spatial field (Y, X) of optimal ll_ratio values
- **cluster_da** (*xarray.DataArray*) – Cluster labels (integers) when mode='cluster' Dummy array filled with 1s when mode='grid'

Raises

- **ValueError** – If invalid mode is provided
- **RuntimeError** – If dask parallel computation fails (in grid mode)

Notes

- Locations/time series with < 10 valid points are skipped (return NaN)
- NaN handling is automatic (invalid time steps are masked)
- Progress messages are printed for long-running computations

fit_predict (*x, y, x_test, y_test, alpha, ll_ratio*)

compute_model (*X_train, y_train, X_test, y_test, alpha=None, ll_ratio=None, clim_year_start=None, clim_year_end=None*)

Performs parallelized ElasticNet modeling for spatiotemporal data. Returns [error, prediction] per grid cell.

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1, T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in `_ppf_terciles_from_code` - *shape, loc, scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.
- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If dist_method == ‘bestfit’:**•Use cluster-based best-fit distributions to:**

- derive terciles analytically from (best_code_da, best_shape_da, best_loc_da, best_scale_da),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.

- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.

Returns

hindcast_prob – Probabilities with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, alpha, l1_ratio, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generates a single-year forecast using ElasticNet with (alpha, l1_ratio), and computes tercile probabilities.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X).
- **clim_year_start** (*int*) – Climatology reference period.
- **clim_year_end** (*int*) – Climatology reference period.
- **Predictor** (*xarray.DataArray*) – Historical predictor data (T, features).
- **hindcast_det** (*xarray.DataArray*) – Historical deterministic forecast with dims (output=2, T, Y, X).
- **Predictor_for_year** (*xarray.DataArray*) – Single-year predictor data (features,).
- **alpha** (*xarray.DataArray*) – Spatial map of alpha values (Y, X).
- **l1_ratio** (*xarray.DataArray*) – Spatial map of l1_ratio values (Y, X).

Returns

- **result_** (*xarray.DataArray*) – dims (output=2, Y, X) => [error, forecast]. For a real forecast scenario, ‘error’ is NaN.
- **hindcast_prob** (*xarray.DataArray*) – dims (probability=3, Y, X) => [PB, PN, PA].

class wass2s.was_linear_models.**WAS_LassoLars_Model** (*alpha_range=None, n_clusters=5, nb_cores=1, dist_method='nonparam', hyperparam_optimizer='bayesian', n_trials=50, n_iter=50, mode='grid'*)

Bases: *object*

LassoLars regression model for spatial rainfall (or continuous climate variable) prediction.

LassoLars implements **Lasso** (L1 regularization) using the **Least-Angle Regression (LARS)** algorithm, which computes the full regularization path efficiently. It is particularly well-suited when:

- Number of samples (time steps) is small per grid cell/cluster
- High numerical stability is required (LARS is less prone to convergence issues than coordinate descent)
- Predictors are highly correlated (common in climate/teleconnection data)
- You want precise control over the regularization path

Compared to standard Lasso (coordinate descent), LassoLars is: - **More stable** for small/noisy datasets - **Slightly slower** (especially for many features) - **Better behaved** near the sparse solution

Two spatial optimization strategies are supported: - **‘cluster’** (recommended): One optimal alpha per spatial cluster → fast, robust, excellent generalization - **‘grid’**: Independent alpha optimization per grid cell → maximum spatial detail, very computationally expensive

Hyperparameter search methods: - **‘bayesian’** → Optuna Bayesian optimization (recommended — best quality/speed trade-off) - **‘random’** → scikit-learn RandomizedSearchCV - **‘lassolars_cv’** → scikit-learn LassoLarsCV (fastest, uses full LARS path)

Parameters

- **alpha_range** (*array-like, optional*) – Range of regularization strength (alpha) values to search. Default: `np.logspace(-6, 2, 100)` — well-suited for standardized rainfall data.
- **n_clusters** (*int, default=5*) – Number of spatial clusters (only used when `mode='cluster'`).
- **nb_cores** (*int, default=1*) – Number of CPU cores for parallel computation (mainly used in grid mode).
- **dist_method** (*str, default='nonparam'*) – Method for computing tercile probabilities (PB/PN/PA). Currently only `'nonparam'` fully implemented; `'bestfit'` requires additional inputs.
- **hyperparam_optimizer** (*{'bayesian', 'random', 'lassolars_cv'}, default='bayesian'*) – Strategy for finding optimal alpha: - `'bayesian'` → Optuna Bayesian optimization (recommended) - `'random'` → Randomized search - `'lassolars_cv'` → Built-in LassoLarsCV (fastest, uses full LARS path)
- **n_trials** (*int, default=50*) – Number of trials for Bayesian optimization (Optuna).
- **n_iter** (*int, default=50*) – Number of parameter settings sampled in randomized search.
- **mode** (*{'cluster', 'grid'}, default='cluster'*) – Optimization approach: - `'cluster'` → one alpha per spatial cluster (fast & robust — recommended) - `'grid'` → independent alpha per grid cell (slow, maximally local)

Notes

- **Cluster mode** is the recommended default for most rainfall/climate applications: - Much faster - More robust to noise, short time series, and data sparsity - Excellent generalization
- **Grid mode** should only be used when maximum spatial detail is critical and sufficient computing resources are available.
- LassoLars is more stable than coordinate-descent Lasso for small datasets, but may be slower for many features.
- Requires at least 10 valid time steps per location/cluster to attempt optimization.
- Negative predictions are automatically clipped to zero (useful for rainfall).
- Large domains in `'grid'` mode benefit greatly from higher `nb_cores`.

compute_hyperparameters (*predictand, predictor, clim_year_start, clim_year_end*)

Computes spatially varying optimal alpha values (and cluster map if `mode='cluster'`).

fit_predict (*x, y, x_test, y_test, alpha*)

Fits LassoLars model on one grid cell using given alpha and makes prediction.

compute_model (*X_train*, *y_train*, *X_test*, *y_test*, *alpha=None*, *clim_year_start=None*, *clim_year_end=None*)

Parallel LassoLars regression across entire spatial domain using provided alpha map.

compute_prob (*Predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*,

best_code_da=None, *best_shape_da=None*, *best_loc_da=None*, *best_scale_da=None*)

Computes tercile probabilities for hindcast predictions.

forecast (*Predictant*, *clim_year_start*, *clim_year_end*, *Predictor*,
hindcast_det,

Predictor_for_year, *alpha*, *best_code_da=None*, *best_shape_da=None*, *best_loc_da=None*,
best_scale_da=None)

Full end-to-end forecast pipeline for one target year: - deterministic LassoLars prediction - tercile probabilities (PB, PN, PA)

Examples

Recommended workflow (cluster mode with Bayesian optimization):

```
>>> lars_model = WAS_LassoLars_Model (
...     mode='cluster',
...     n_clusters=8,
...     hyperparam_optimizer='bayesian',
...     n_trials=80,
...     nb_cores=12
... )
```

1. Compute spatially varying alpha (one per cluster) >> *alpha_map*, *cluster_map* = *lars_model.compute_hyperparameters*(... *seasonal_rainfall*, *predictors*, 1991, 2020)

2. Train & predict on hindcast period >> *hindcast_pred* = *lars_model.compute_model*(... *X_train*=*predictors*, ... *y_train*=*seasonal_rainfall*, ... *X_test*=*predictors_hindcast*, ... *y_test*=None, ... *alpha*=*alpha_map* ...)

3. Compute probabilistic hindcast (terciles) >> *hindcast_prob* = *lars_model.compute_prob*(... *seasonal_rainfall*, 1991, 2020, *hindcast_pred*, ... *best_code_da*=*dist_code_da*, *best_shape_da*=*shape_da*, ... *best_loc_da*=*loc_da*, *best_scale_da*=*scale_da* ...)

4. Forecast next year (e.g. 2025) >> *forecast_det*, *forecast_prob* = *lars_model.forecast*(... *seasonal_rainfall*, 1991, 2020, ... *predictors*, *hindcast_pred*, ... *predictor_2025*, ... *alpha*=*alpha_map*, ... *best_code_da*=*dist_code_da*, *best_shape_da*=*shape_da*, ... *best_loc_da*=*loc_da*, *best_scale_da*=*scale_da* ...)

Warning

- Very high alpha can eliminate all predictors → monitor selected features.
- Small training sets per cell/cluster may still lead to unstable results.
- For heavy-tailed rainfall, consider log-transformation before modeling.
- Use 'bayesian' optimizer for best quality; 'lassolars_cv' is fastest but least flexible.

__init__ (*alpha_range=None*, *n_clusters=5*, *nb_cores=1*, *dist_method='nonparam'*,
hyperparam_optimizer='bayesian', *n_trials=50*, *n_iter=50*, *mode='grid'*)

compute_hyperparameters (*predictand, predictor, clim_year_start, clim_year_end*)

Compute spatially varying optimal LassoLars regularization parameter (alpha).

Parameters

- **predictand** (*xarray.DataArray*) – Target variable (typically rainfall) Expected dimensions: ('T', 'Y', 'X')
- **predictor** (*xarray.DataArray*) – Predictor fields Expected dimensions: ('T', 'Y', 'X', 'features') or ('T', 'features')
- **clim_year_start** (*int*) – Start year of the climatological standardization period
- **clim_year_end** (*int*) – End year of the climatological standardization period

Returns

- **alpha_map** (*xarray.DataArray*) – Spatial field (Y, X) containing optimal alpha value(s) - Different value per cell when mode='grid' - Constant per cluster when mode='cluster'
- **cluster_da** (*xarray.DataArray*) – Cluster labels (integers) when mode='cluster' Dummy array filled with 1s when mode='grid' (for API consistency)

Raises

- **ValueError** – If mode is neither 'cluster' nor 'grid'
- **RuntimeError** – If dask parallel computation fails (grid mode)

Notes

- Time series with fewer than 10 valid points are automatically skipped (return NaN)
- All NaN values in input are properly masked
- Progress messages are printed during long computations
- LassoLarsCV does **not** require pre-defined alpha grid (uses full LARS path)

fit_predict (*x, y, x_test, y_test, alpha*)

compute_model (*X_train, y_train, X_test, y_test, alpha=None, clim_year_start=None, clim_year_end=None*)

Fits and predicts the LassoLars model using Dask for parallel execution.

Parameters

- **X_train** (*xarray.DataArray*) – Training predictor data (dims: T, features).
- **y_train** (*xarray.DataArray*) – Training response variable (dims: T, Y, X).
- **X_test** (*xarray.DataArray*) – Test predictor data (dims: features,) or broadcastable to (Y, X).
- **y_test** (*xarray.DataArray*) – Test response variable (dims: Y, X).
- **alpha** (*xarray.DataArray*) – Cluster-wise optimal alpha values (dims: Y, X).

Returns

dims (output=2, Y, X) => [error, prediction].

Return type

xarray.DataArray

static weibull_shape_solver (k, M, V)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean² ratio matches the observed V/M² ratio.

static calculate_tercile_probabilities_bestfit ($best_guess, error_variance, T1, T2, dist_code, dof$)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - **best_guess** : 1D array over T (hindcast_det or forecast_det) - **T1, T2** : scalar terciles from climatological best-fit distribution - **dist_code** : int, as in `_ppf_terciles_from_code` - **shape, loc, scale** : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.
- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric ($best_guess, error_samples, first_tercile, second_tercile$)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If `dist_method == 'bestfit'`:

- **Use cluster-based best-fit distributions to:**

- derive terciles analytically from (`best_code_da`, `best_shape_da`, `best_loc_da`, `best_scale_da`),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=['PB','PN','PA'], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, alpha, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generates a forecast for a single time step using LassoLars with alpha, then computes tercile probabilities based on the chosen distribution method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X).
- **clim_year_start** (*int*) – Start year for the climatological reference period.
- **clim_year_end** (*int*) – End year for the climatological reference period.
- **Predictor** (*xarray.DataArray*) – Historical predictor data (T, features).
- **hindcast_det** (*xarray.DataArray*) – Historical deterministic forecast with dims (output=2, T, Y, X).
- **Predictor_for_year** (*xarray.DataArray*) – Single-year predictor data (features,) or shape (1, features).
- **alpha** (*xarray.DataArray*) – Spatial map of alpha values (Y, X) for LassoLars.

Returns

- **result_** (*xarray.DataArray*) – dims (output=2, Y, X) => [error, prediction]. For a true forecast scenario, the error is typically NaN.
- **hindcast_prob** (*xarray.DataArray*) – dims (probability=3, Y, X) => [PB, PN, PA] for tercile categories.

1.9.10 Principal Component Regression

```
class wass2s.was_pcr.WAS_PCR (regression_model, n_modes=None, use_coslat=True, standardize=False,  
                                detrend=True, opti_explained_variance=None, L2norm=False,  
                                normalize=None)
```

Bases: `object`

Spatial principal-component regression wrapping a WAS regression model.

This class implements a two-stage pipeline:

1. **EOF dimensionality reduction** — the spatial predictor field(s) are decomposed into principal components (PCs) by `WAS_EOF`, which handles NaN filling, optional linear detrending, and optional per-field variance normalization entirely within each cross-validation fold (no data leakage).
2. **Regression** — the resulting PCs are passed to any compatible WAS regression model (e.g. `WAS_MLR`, `WAS_RandomForest`, ...).

Single-field and multivariate predictors

Predictor may be a single `xarray.DataArray` or a **list** of DataArrays representing different variables and/or grids (e.g. SST + SLP + rainfall indices). A list triggers a **combined multivariate EOF**: each field is preprocessed and normalized independently on the training fold so that no single field dominates the joint covariance, then `WAS_EOF` builds one shared set of PCs that feed the regression.

Note

When using this class inside `WAS_Cross_Validator`, pass the **raw** predictor (do *not* pre-detrend outside the fold loop). For a list predictor, slice every field per fold:

```
X_train = [p.isel(T=train_idx) for p in Predictor]
X_test  = [p.isel(T=test_idx)  for p in Predictor]
```

param regression_model

An instantiated WAS regression model exposing at least `compute_model(X_train, y_train, X_test, ...)` and, optionally, `compute_prob(...)` and `forecast(...)`.

type regression_model

object

param n_modes

Number of EOF modes to retain. Ignored when `opti_explained_variance` is set.

type n_modes

int, optional

param use_coslat

Apply cosine-latitude area weighting in the EOF decomposition.

type use_coslat

bool, default True

param standardize

Standardize each grid point before the EOF decomposition (useful when the predictor is not expressed as anomalies).

type standardize

bool, default False

param detrend

Remove a linear trend from each grid point before the EOF decomposition. The trend is estimated on the training fold and applied to the test fold to avoid leakage.

type detrend

bool, default True

param opti_explained_variance

Target cumulative explained variance (%) used to select the optimal number of modes automatically (e.g. 90.0 for 90 %).

type opti_explained_variance

float, optional

param L2norm

Return L2-normalised EOF scores and components from *WAS_EOF*.

type L2norm

bool, default False

param normalize

Per-field scalar rescaling so that no field dominates the joint covariance in a multivariate EOF. None means *auto*: enabled for a list of fields, disabled for a single field.

type normalize

bool or None, default None

eof_model

The fitted EOF object, available after the first call to *compute_model()* or *forecast()*.

Type

WAS_EOF or None

Examples

Single-field PCR with a Ridge regression back-end:

```
>>> from wass2s import WAS_PCR, WAS_Ridge_Model
>>> pcr = WAS_PCR(WAS_Ridge_Model(alpha=1.0), n_modes=10, detrend=True)
>>> result = pcr.compute_model(X_train, y_train, X_test)
```

Multivariate PCR (SST + SLP):

```
>>> pcr_mv = WAS_PCR(WAS_MLR(), opti_explained_variance=85.0)
>>> result = pcr_mv.compute_model([sst_train, slp_train], y_train,
...                               [sst_test, slp_test])
```

__init__ (*regression_model*, *n_modes*=None, *use_coslat*=True, *standardize*=False, *detrend*=True, *opti_explained_variance*=None, *L2norm*=False, *normalize*=None)

compute_model (*X_train*, *y_train*, *X_test*, *y_test=None*, ***kwargs*)

Run the full PCR pipeline on a single train/test split.

Performs EOF dimensionality reduction on the predictor, then delegates to the wrapped regression model.

Parameters

- **X_train** (*xarray.DataArray* or *list of xarray.DataArray*) – Training predictor field(s).
- **y_train** (*xarray.DataArray*) – Training target (predictand).
- **X_test** (*xarray.DataArray* or *list of xarray.DataArray*) – Test predictor field(s).
- **y_test** (*xarray.DataArray*, *optional*) – Test target. Forwarded to the regression model only when its `compute_model` signature accepts `y_test`.
- ****kwargs** – Additional keyword arguments forwarded to the regression model.

Returns

Whatever the wrapped regression model's `compute_model` returns (typically a deterministic hindcast *DataArray*).

Return type

object

compute_prob (*Predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*, ***kwargs*)

Compute tercile probability forecasts from a deterministic hindcast.

Delegates directly to the wrapped regression model's `compute_prob` method. Returns `None` if the model does not implement it.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed predictand over the full hindcast period.
- **clim_year_start** (*int*) – First year of the climatological reference period.
- **clim_year_end** (*int*) – Last year of the climatological reference period.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast produced by `compute_model()`.
- ****kwargs** – Additional keyword arguments forwarded to the regression model.

Returns

Tercile probabilities (below-normal, near-normal, above-normal), or `None` if the regression model does not support probabilistic output.

Return type

xarray.DataArray or `None`

forecast (*Predictant*, *clim_year_start*, *clim_year_end*, *Predictor*, *hindcast_det*, *Predictor_for_year*, ***kwargs*)

Generate an operational forecast for a target year.

EOF patterns are fitted on the full hindcast predictor, and the target-year predictor is projected onto those patterns before being passed to the regression model.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed predictand over the full hindcast period.

- **clim_year_start** (*int*) – First year of the climatological reference period.
- **clim_year_end** (*int*) – Last year of the climatological reference period.
- **Predictor** (*xarray.DataArray* or *list of xarray.DataArray*) – Full hindcast predictor field(s) used to fit the EOF model.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast produced during cross-validation.
- **Predictor_for_year** (*xarray.DataArray* or *list of xarray.DataArray*) – Predictor field(s) for the target forecast year.
- ****kwargs** – Additional keyword arguments forwarded to the regression model.

Returns

Operational forecast returned by the wrapped regression model's `forecast` method.

Return type

object

1.9.11 Canonical Correlation Analysis

Canonical Correlation Analysis (CCA) for seasonal forecasting.

Two implementations are provided:

WAS_CCA

Legacy CCA built on xeoofs EOF analysis and numpy CCA. Suitable for quick exploratory use.

WAS_CCA_base

Leakage-free CCA that fits all preprocessing (NaN fill, standardization, EOF decomposition) strictly on the training fold and applies the fitted transforms to the test fold. Recommended for cross-validation.

Both classes expose `compute_model`, `compute_prob`, `forecast`, and `plot_cca_results` methods.

class `wass2s.was_cca.WAS_CCA` (*n_modes=4, n_pca_modes=8, standardize=False, use_coslat=True, use_pca=True, dist_method='nonparam'*)

Bases: *object*

__init__ (*n_modes=4, n_pca_modes=8, standardize=False, use_coslat=True, use_pca=True, dist_method='nonparam'*)

Initialize the WAS_CCA class with specified parameters.

Parameters: - *n_modes*: Number of canonical modes to compute. - *n_pca_modes*: Number of PCA modes to use before CCA. - *standardize*: Whether to standardize the data. Keep it False in our case data already standardize - *use_coslat*: Whether to use cosine latitude weighting. - *use_pca*: Whether to perform PCA before CCA. - *detrend*: Whether to apply detrending to the data. Extended EOF to detrend

fit_cca (*X_train, y_train*)

Fit the CCA model using the training data.

Parameters: - *X_train*: xarray DataArray for predictor training data. - *y_train*: xarray DataArray for predictand training data.

preprocess_data (*X, Y*)

Preprocess the data by detrending, masking, and filling missing values.

Parameters: - *X*: xarray DataArray for predictors. - *Y*: xarray DataArray for predictands.

Returns: - *X_final*: Preprocessed X data. - *Y_final*: Preprocessed Y data.

preprocess_test_data (*X_test, y_test, X_train, y_train*)

Preprocess the test data.

Parameters: - *X_test*: xarray DataArray for predictor testing data. - *y_test*: xarray DataArray for predictand testing data. - *X_train*: xarray DataArray for predictor training data. - *y_train*: xarray DataArray for predictand training data.

Returns: - *X_test_prepared*: Preprocessed X test data. - *y_test_prepared*: Preprocessed Y test data.

compute_model (*X_train, y_train, X_test, y_test*)

Compute the CCA model and generate hindcasts.

Parameters: - *X_train*: xarray DataArray for predictor training data. - *y_train*: xarray DataArray for predictand training data. - *X_test*: xarray DataArray for predictor testing data. - *y_test*: xarray DataArray for predictand testing data.

Returns: - *hindcast*: xarray DataArray containing predictions and errors.

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1, T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in `_ppf_terciles_from_code` - *shape, loc, scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use *best_guess[t]* to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

$$P(B) = F(T1), P(N) = F(T2) - F(T1), P(A) = 1 - F(T2).$$

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If *dist_method* == 'bestfit':

•Use cluster-based best-fit distributions to:

- derive terciles analytically from (*best_code_da, best_shape_da, best_loc_da, best_scale_da*),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.

Returns

hindcast_prob – Probabilities with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

plot_cca_results (*X=None, Y=None, n_modes=None, clim_year_start=None, clim_year_end=None*)

Plots the CCA modes and scores.

Parameters: - X: Optional xarray DataArray for predictors. If provided, the model will be fitted using X and Y. - Y: Optional xarray DataArray for predictands. - n_modes: Number of modes to plot. If None, plots all modes.

class wass2s.was_cca.**WAS_CCA_base** (*n_modes=4, n_pca_modes=8, standardize=False, use_coslat=True, use_pca=True, dist_method='nonparam'*)

Bases: *object*

Canonical Correlation Analysis model based on xeofs.cross.CCA.

Important:

- No detrend no extract_leading_eof_component is used here.
- The CCA basis is entirely handled by xeofs.cross.CCA.
- The predictand is standardized before CCA in forecast/cross-validation, then transformed back to the original scale with reverse_standardize.
- Missing values are handled using training-period means to avoid leakage.

__init__ (*n_modes=4, n_pca_modes=8, standardize=False, use_coslat=True, use_pca=True, dist_method='nonparam'*)

preprocess_data (*X_train: DataArray, y_train: DataArray*)

Prepare training data for xeofs CCA.

No detrending is applied. Missing values are filled using training means.

preprocess_test_data (*X_test: DataArray, y_test: DataArray, X_train: DataArray, y_train: DataArray*)

Prepare test data for xeofs CCA.

Missing values in *X_test* and *y_test* are filled using training means. *y_test* is mainly used to preserve the correct T coordinate.

preprocess_forecast_data (*Predictor_for_year: DataArray, Predictor_train: DataArray*)

Prepare real-time predictor data for forecast.

Missing values are filled using the historical predictor mean.

fit_cca (*X_train: DataArray, y_train: DataArray*)

Fit xeofs CCA on training data.

compute_model (*X_train: DataArray, y_train: DataArray, X_test: DataArray, y_test: DataArray*) → *DataArray*

Fit CCA on training data and predict the test fold.

Returns

hindcast – Deterministic hindcast with dimensions T, Y, X.

Return type

xr.DataArray

static weibull_shape_solver (*k, M, V*)

Solve Weibull shape parameter *k* from mean *M* and variance *V*.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Compute tercile probabilities using a best-fit distribution family.

Returns

out – Shape: probability, T

Return type

np.ndarray

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric probability method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

forecast (*Predictant: DataArray, clim_year_start, clim_year_end, Predictor: DataArray, hindcast_det: DataArray, Predictor_for_year: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*)

Generate deterministic and probabilistic forecast with xeofs CCA.

plot_cca_results (*X: DataArray = None, Y: DataArray = None, n_modes=None, clim_year_start=None, clim_year_end=None*)

Plot CCA spatial modes and canonical scores.

class *wass2s.was_cca.WAS_CCA_op* (*n_modes=4, n_pca_modes=8, standardize=False, use_coslat=True, use_pca=True, dist_method='bestfit'*)

Bases: *object*

__init__ (*n_modes=4, n_pca_modes=8, standardize=False, use_coslat=True, use_pca=True, dist_method='bestfit'*)

Initialize parameters. The CCA model instance is created in `fit_cca` to ensure safe CV.

fit_cca (*X_train, y_train*)

Fit the CCA model. Creates a fresh xeoofs instance every time.

preprocess_data (*X, Y*)

preprocess_test_data (*X_test, y_test, X_train, y_train*)

compute_model (*X_train, y_train, X_test, y_test*)

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

plot_cca_results (*X=None, Y=None, n_modes=None, clim_year_start=None, clim_year_end=None*)

Plots the CCA modes and scores.

Parameters: - X: Optional xarray DataArray for predictors. If provided, the model will be fitted using X and Y. - Y: Optional xarray DataArray for predictands. - n_modes: Number of modes to plot. If None, plots all modes.

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - best_guess : 1D array over T (hindcast_det or forecast_det) - T1, T2 : scalar terciles from climatological best-fit distribution - dist_code : int, as in `_ppf_terciles_from_code` - shape, loc, scale : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

$$P(B) = F(T1), P(N) = F(T2) - F(T1), P(A) = 1 - F(T2).$$

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → DataArray

Compute tercile probabilities for deterministic hindcasts.

If `dist_method == 'bestfit'`:

•Use cluster-based best-fit distributions to:

- derive terciles analytically from (`best_code_da, best_shape_da, best_loc_da, best_scale_da`),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.

Returns

hindcast_prob – Probabilities with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

Return type

xarray.DataArray

```
class wass2s.was_cca.WAS_CCA_strict (n_modes=4, n_pca_modes=8, standardize=False,
                                     use_coslat=True, use_pca=True, dist_method='bestfit')
```

Bases: *object*

```
__init__ (n_modes=4, n_pca_modes=8, standardize=False, use_coslat=True, use_pca=True,
          dist_method='bestfit')
```

Initialize parameters. The CCA model instance is created in fit_cca to ensure safe CV.

```
fit_cca (X_train, y_train)
```

Fit CCA using already prepared fields.

Important: In the CCA branch of WAS_Cross_Validator, X_train and y_train are already detrended / standardized before this method is called.

```
preprocess_data (X, Y)
```

Prepare training data for xeofs CCA. This method does not detrend or standardize; those operations are handled upstream in forecast() or in the CCA cross-validation branch.

```
preprocess_test_data (X_test, y_test=None, X_train=None, y_train=None)
```

Prepare test data for xeofs CCA.

```
compute_model (X_train, y_train, X_test, y_test=None)
```

Low-level CCA prediction.

Expected input: - X_train and X_test are already in the same transformed predictor space. - y_train is already in transformed predictand space. - y_test is used only to preserve the correct T coordinate during CV.

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Operational CCA forecast.

Historical transformation: - Predictor is linearly detrended. - Predictant is standardized, then linearly detrended.

Forecast transformation: - Predictor_for_year is filled. - The historical predictor trend is evaluated at Predictor_for_year T. - Forecast predictor anomaly is passed to CCA.

Output transformation: - CCA output is assigned the target predictand T. - Predictand trend is added back at target T. - Standardization is reversed to physical units.

plot_cca_results (*X=None, Y=None, n_modes=None, clim_year_start=None, clim_year_end=None*)

Plot CCA spatial modes and canonical scores.

If X and Y are provided, the CCA model is refitted using the same transformation logic used for the CCA workflow:

- predictor: linear detrending
- predictand: standardization + linear detrending

Parameters

- **X** (*xarray.DataArray, optional*) – Predictor field with dimensions ('T', 'Y', 'X') or ('T', 'lat', 'lon').
- **Y** (*xarray.DataArray, optional*) – Predictand field with dimensions ('T', 'Y', 'X') or ('T', 'lat', 'lon').
- **n_modes** (*int, optional*) – Number of CCA modes to plot. If None, uses self.n_modes.
- **clim_year_start** (*int or str, optional*) – Climatology period used to standardize Y.
- **clim_year_end** (*int or str, optional*) – Climatology period used to standardize Y.

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - best_guess : 1D array over T (hindcast_det or forecast_det) - T1, T2 : scalar terciles from climatological best-fit distribution - dist_code : int, as in _ppf_terciles_from_code - shape, loc, scale : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use best_guess[t] to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

$$P(B) = F(T1), P(N) = F(T2) - F(T1), P(A) = 1 - F(T2).$$

```
static calculate_tercile_probabilities_nonparametric (best_guess, error_samples,  
                                                    first_tercile, second_tercile)
```

Non-parametric method using historical error samples.

```
compute_prob (Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray,  
              best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da:  
              DataArray = None, best_scale_da: DataArray = None) → DataArray
```

Compute tercile probabilities from deterministic hindcasts.

1.9.12 Kernel CCA variants

Kernel CCA variants for seasonal forecasting.

Built on the cca-zoo library, these classes extend `WAS_CCA_base` with non-linear kernels.

Classes

WAS_KernelCCA_base

Abstract base implementing preprocessing, probabilistic post-processing, and Optuna / grid hyperparameter search for any cca-zoo kernel method.

WAS_KCCA

Kernel CCA (cca-zoo KCCA / LinearCCA for the linear kernel).

WAS_KGCCA

Kernel Generalised CCA (cca-zoo KGCCA / GCCA).

WAS_EOF_KCCA

EOF pre-reduction layer on top of `WAS_KCCA`: reduces each spatial predictor field to its leading principal components before applying kernel CCA.

```
class wass2s.was_cca_2.WAS_KernelCCA_base (n_modes=4, kernel='linear', dist_method='nonparam',  
                                           standardize=True, cv_folds=3, random_state=42,  
                                           search_method='grid', reg=1e-06)
```

Bases: `object`

Shared base for kernel CCA seasonal-forecast models (KCCA, KGCCA).

Subclasses only need to implement the model-construction hooks:

`_build_model(params)` -> a fitted-ready cca-zoo estimator `_grid_estimator()` -> estimator used by Grid-SearchCV `_prepare_param_grid()` -> dict for grid search `_objective(trial, X, Y)` -> float for Optuna (optional) `_is_linear_kernel()` -> whether interpretable feature weights exist

Conventions identical to WAS_CCA_base:

WAS dims: T, Y, X Predictand standardized before CCA, returned in original scale. Missing values filled with training-period means (no leakage).

`Q_LOW = 0.33`

`Q_HIGH = 0.67`

```
__init__ (n_modes=4, kernel='linear', dist_method='nonparam', standardize=True, cv_folds=3,  
          random_state=42, search_method='grid', reg=1e-06)
```

```
preprocess_data (X_train: DataArray, y_train: DataArray)
```

```
preprocess_test_data (X_test, y_test, X_train, y_train)
```

```
preprocess_forecast_data (Predictor_for_year, Predictor_train)
```

compute_hyperparameters (*Predictors, Predictand, clim_year_start=None, clim_year_end=None*)

Find best kernel hyperparameters; stores them in self.best_params.

set_hyperparameters (*hyperparams*)

fit_cca (*X_train, y_train, best_params=None*)

Fit the kernel CCA model and build the (X canonical scores -> Y field) regression used for deterministic prediction.

y_train is used *as given* as the regression target, so callers that want predictions in standardized space should pass a standardized predictand (see compute_model / forecast).

compute_model (*X_train, y_train, X_test, y_test, best_params=None*)

Fit on the training fold and predict the test fold. Returns the deterministic hindcast in the original predictand scale (T, Y, X).

static weibull_shape_solver (*k, M, V*)

classmethod calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

compute_prob (*Predictant, clim_year_start, clim_year_end, hindcast_det, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_params=None, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

get_components ()

Spatial patterns (linear kernel only).

get_scores ()

get_canonical_correlations ()

get_model_info ()

plot_cca_results (*X=None, Y=None, n_modes=None, clim_year_start=None, clim_year_end=None, best_params=None*)

Plot canonical scores (always) and spatial modes (linear kernel).

class wass2s.was_cca_2.**WAS_KCCA** (*n_modes=4, kernel='linear', dist_method='nonparam', standardize=True, cv_folds=3, random_state=42, search_method='grid', reg=1e-06*)

Bases: [WAS_KernelCCA_base](#)

Kernel Canonical Correlation Analysis (cca-zoo KCCA / LinearCCA).

class wass2s.was_cca_2.**WAS_KGCCA** (**args, cv_folds=5, **kwargs*)

Bases: [WAS_KernelCCA_base](#)

Kernel Generalized CCA (cca-zoo KGCCA / GCCA).

__init__ (**args, cv_folds=5, **kwargs*)

```
class wass2s.was_cca_2.WAS_EOF_KCCA (eof_n_modes=5, joint=False, eof_detrend=True,
                                     eof_standardize=False, eof_use_coslat=True, **kcca_kwargs)
```

Bases: [WAS_KCCA](#)

```
__init__ (eof_n_modes=5, joint=False, eof_detrend=True, eof_standardize=False, eof_use_coslat=True,
          **kcca_kwargs)
```

eof_n_modes PCs kept per field (joint=False) or total joint modes
(joint=True).

joint False -> one EOF per field then concat PCs (default).
True -> one multivariate EOF over all fields.

eof_detrend / eof_standardize / eof_use_coslat
passed through to WAS_EOF.

****kcca_kwargs** forwarded to **WAS_KCCA / WAS_KernelCCA_base** (n_modes, kernel,
dist_method, standardize, reg, search_method, ...).

```
compute_model (X_train, y_train, X_test, y_test, best_params=None)
```

Fit on the training fold and predict the test fold. Returns the deterministic hindcast in the original predictand scale (T, Y, X).

```
compute_hyperparameters (Predictors, Predictand, clim_year_start=None, clim_year_end=None)
```

Find best kernel hyperparameters; stores them in self.best_params.

```
forecast (Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year,
           best_params=None, best_code_da=None, best_shape_da=None, best_loc_da=None,
           best_scale_da=None)
```

1.9.13 Machine-learning models

Machine-learning regression models for seasonal forecasting.

All classes expose the standard `compute_model / compute_prob / forecast` interface and are compatible with [WAS_Cross_Validator](#).

Classes

BaseOptimizer

Unified hyperparameter optimizer supporting grid search, random search, and Optuna Bayesian optimization.

WAS_PolynomialRegression

Polynomial feature expansion followed by ridge regression.

WAS_MARS_Model

Multivariate Adaptive Regression Splines (py-earth).

WAS_LogisticRegression_Model

Pixel-wise multinomial logistic regression for direct tercile classification.

WAS_PoissonRegression

Pixel-wise Poisson GLM for count / precipitation data.

WAS_NonNeural_Base

Encompassing others non_neural regression : `WAS_RandomForest_Model` ...

```
class wass2s.was_machine_learning.BaseOptimizer (optimization_method='grid', n_trials=20,
                                                  cv=5, random_state=42)
```

Bases: `object`

Unified Optimizer. Supports SVR (multi-grid), MLP, and Stacking architectures.

`__init__` (*optimization_method='grid', n_trials=20, cv=5, random_state=42*)

`optimize` (*model, param_space, X, y, scoring='neg_mean_squared_error'*)

```
class wass2s.was_machine_learning.WAS_LogisticRegression_Model (nb_cores=1,
                                                                  dist_method='nonparam',
                                                                  n_clusters=5,
                                                                  param_grid=None,
                                                                  optimization_method='grid',
                                                                  n_trials=20, cv=5,
                                                                  random_state=42,
                                                                  x_scaler=None)
```

Bases: `object`

Logistic Regression model for **tercile classification** (Below/Normal/Above = 0/1/2) on spatiotemporal seasonal climate data (typically rainfall or temperature anomalies).

Workflow overview: 1. **Tercile classification**: Compute spatial map of classes (0=Below, 1=Normal, 2=Above) based on climatological terciles (33rd and 67th percentiles).

2. **Spatial clustering** (KMeans) on a summary statistic of the predictand (default: mean over time).
3. **Per-cluster hyperparameter optimization** of LogisticRegression using grid/random/Bayesian search.
4. **Broadcast best hyperparameters** to every grid cell (Y, X).
5. **Parallel per-grid-cell classification** using local hyperparameters.
6. **Direct probabilistic output**: `model.predict_proba()` gives [P(Below), P(Normal), P(Above)] per grid cell and time step.

This class is designed for **probabilistic seasonal forecasting** in tercile format, where the target is a categorical variable derived from climatological thresholds.

Key features: - Multiclass logistic regression (multinomial + L2 penalty via 'lbfgs' solver) - Optional feature scaling (`x_scaler`: None | 'standard' | 'robust') - No target scaling (`y` is categorical 0/1/2) - Safe handling of class imbalance via '`class_weight`' tuning - Full parallelization across spatial grid using dask

Parameters

- **nb_cores** (*int*, *default=1*) – Number of CPU cores for parallel processing (dask + joblib).
- **dist_method** (*{'bestfit', 'nonparam'}*, *default='nonparam'*) – Method for computing tercile probabilities (currently unused in this class, kept for API consistency with other models).
- **n_clusters** (*int*, *default=5*) – Number of spatial clusters for KMeans grouping of the predictand field.
- **param_grid** (*dict or None*, *default=None*) – Hyperparameter search space for LogisticRegression. If None, a safe default grid is used (compatible with multinomial + lbfgs):
 - C: [0.1, 0.5, 1.0, 2.0, 5.0] # inverse regularization strength
 - class_weight: [None, 'balanced']

- `max_iter`: [300, 600, 1000]
- **optimization_method** (`{'grid', 'random', 'bayesian'}`, `default='grid'`) – Strategy for hyperparameter search.
- **n_trials** (`int`, `default=20`) – Number of trials for ‘random’ or ‘bayesian’ optimization.
- **cv** (`int`, `default=5`) – Number of cross-validation folds during tuning.
- **random_state** (`int`, `default=42`) – Random seed for reproducibility (KMeans, CV splits, optimizer).
- **x_scaler** (`{None, 'standard', 'robust'}`, `default=None`) – Whether and how to scale input features (X): - None: no scaling (default, logistic regression is scale-invariant) - ‘standard’: StandardScaler (zero mean, unit variance) - ‘robust’: RobustScaler (median-centered, IQR-scaled, robust to outliers)

nb_cores, dist_method, n_clusters, param_grid, optimization_method, n_trials, cv, random_state, x_scaler

Stored initialization parameters.

optimizer

Internal hyperparameter optimization object.

Type

BaseOptimizer

_cw_map / _cw_inv

Internal mapping for broadcasting ‘class_weight’ (None ↔ 0, ‘balanced’ ↔ 1).

Type

dict

classify (`y, index_start, index_end`)

Static method: convert continuous values to tercile classes (0/1/2).

compute_class (`Predictant, clim_year_start, clim_year_end`)

Compute tercile class map (T, Y, X) + climatological terciles.

compute_hyperparameters (`predictand, predictor, clim_year_start, clim_year_end, scoring='neg_log_loss'`)

Cluster predictand → optimize logistic hyperparameters per cluster → return broadcast arrays of best C, class_weight (coded), max_iter, solver.

fit_predict (`x, y, x_test, C, cw_code, max_iter, solver`)

Fit logistic model on one grid cell using local hyperparameters and return probability vector [P(Below), P(Normal), P(Above)].

compute_model (`X_train, y_train, X_test, C_da, cw_code_da, maxiter_da, solver_da`)

Parallel classification across entire spatial domain using local best params. Returns xarray.DataArray with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

forecast (`Predictant, clim_year_start, clim_year_end, Predictor, Predictor_for_year,`

`C_da, cw_code_da, maxiter_da, solver_da`)

Full end-to-end forecast pipeline for one target year: - classify historical data - optimize hyperparameters - predict probabilities for the forecast year

Notes

- Input predictand should be continuous seasonal values (e.g., total rainfall, anomaly).
- Target y is internally converted to integer classes {0, 1, 2} = {Below, Normal, Above}.
- Negative predictions are **not** clipped (logistic outputs probabilities 0–1).
- Large domains benefit greatly from higher *nb_cores*.
- For very small clusters or poor separability, model may fall back to uniform probabilities.

```
__init__ (nb_cores=1, dist_method='nonparam', n_clusters=5, param_grid=None,  
          optimization_method='grid', n_trials=20, cv=5, random_state=42, x_scaler=None)
```

```
static classify (y, index_start, index_end)
```

```
compute_class (Predictant: DataArray, clim_year_start: int, clim_year_end: int)
```

```
compute_hyperparameters (predictand: DataArray, predictor: DataArray, clim_year_start: int,  
                           clim_year_end: int, scoring: str = 'neg_log_loss')
```

Returns broadcast hyperparameter arrays + cluster map.

```
fit_predict (x, y, x_test, C, cw_code, max_iter, solver)
```

Trains with local hyperparameters and returns proba for classes 0/1/2, correctly mapped via **classes_** (fix vs positional padding).

```
compute_model (X_train, y_train, X_test, C_da, cw_code_da, maxiter_da, solver_da)
```

```
forecast (Predictant: DataArray, clim_year_start: int, clim_year_end: int, Predictor: DataArray,  
           Predictor_for_year: DataArray, C_da, cw_code_da, maxiter_da, solver_da)
```

```
class wass2s.was_machine_learning.WAS_PolynomialRegression (nb_cores=1, degree=2,  
                                                            dist_method='nonparam')
```

Bases: *object*

Polynomial Regression model for spatiotemporal climate prediction.

This class implements polynomial regression (via scikit-learn's PolynomialFeatures + LinearRegression) to capture non-linear relationships between predictors and a continuous predictand (e.g., seasonal rainfall, temperature anomalies, agro-climatic indices).

Key features: - Polynomial feature expansion up to a specified degree - Spatially parallel fitting and prediction across large grids using dask + xarray - Deterministic point predictions with optional error computation - Probabilistic tercile forecasting (Below/Normal/Above = PB/PN/PA) using either:

- Parametric best-fit distributions per grid cell ('bestfit')
- Non-parametric sampling of historical forecast errors ('nonparam')

Suitable for modeling moderate non-linearities where interpretability of polynomial terms is useful.

Parameters

- **nb_cores** (*int*, *default*=1) – Number of CPU cores for parallel processing (dask workers).
- **degree** (*int*, *default*=2) – Degree of the polynomial features (e.g., 2 = quadratic, 3 = cubic).

- **dist_method** (`{'bestfit', 'nonparam'}`, `default='nonparam'`) – Method for computing tercile probabilities: - 'bestfit' → uses best-fit distribution per grid cell (requires distribution fit inputs) - 'nonparam' → empirical sampling of historical forecast errors

nb_cores, degree, dist_method

Stored initialization parameters.

fit_predict (`x, y, x_test, y_test`)

Fits polynomial regression on one grid cell and returns [error, prediction].

compute_model (`X_train, y_train, X_test, y_test`)

Parallel polynomial regression across the entire spatial domain. Returns `xarray.DataArray` with dims ('output'=['error', 'prediction'], Y, X).

compute_prob (`Predictant, clim_year_start, clim_year_end, hindcast_det,`

`best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None`)

Computes tercile probabilities for deterministic hindcasts.

forecast (`Predictant, clim_year_start, clim_year_end, Predictor,`
`hindcast_det,`

`Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None,`
`best_scale_da=None`)

Full end-to-end forecast pipeline for one target year: - deterministic polynomial prediction - tercile probabilities (PB, PN, PA)

Notes

- **Input data requirements:** - Target (y) should be continuous (rainfall, temperature, etc.). - Predictors (X) should be continuous or properly encoded.
- Negative predictions are automatically clipped to zero (useful for rainfall or non-negative variables).
- Higher polynomial degrees can lead to overfitting, especially with small training sets per grid cell.
- Large spatial domains benefit significantly from higher `nb_cores`.
- For `dist_method='bestfit'`, distribution fit results from `WAS_TransformData` must be provided.
- No explicit feature or target scaling is applied (linear regression after polynomial expansion is scale-sensitive; consider normalizing predictors externally if needed).

Warning

- High polynomial degrees (degree ≥ 4) with many features can cause numerical instability or extreme overfitting → use cautiously and validate.
- Very skewed targets (e.g., heavy-tailed rainfall) may benefit from transformation (log, square-root) before modeling (not done automatically).
- Small per-grid-cell training sets can lead to poor fits or unstable coefficients.

__init__ (`nb_cores=1, degree=2, dist_method='nonparam'`)

Initializes the `WAS_PolynomialRegression` with a specified number of CPU cores and polynomial degree.

Parameters

- **nb_cores** (*int*, *optional*) – Number of CPU cores to use for parallel computation, by default 1.
- **degree** (*int*, *optional*) – The degree of the polynomial, by default 2.
- **dist_method** (*str*, *optional*) – The method to compute tercile probabilities (“best-fit”, “nonparam”), by default “nonparam”.

fit_predict (*x*, *y*, *x_test*, *y_test*)

Fit polynomial regression model and generate predictions.

Parameters

- **x** (*array-like*, *shape* (*n_samples*,) or (*n_samples*, *n_features*)) – Training feature data. For polynomial regression, this is typically a 1D array of independent variable values.
- **y** (*array-like*, *shape* (*n_samples*,)) – Training target values (dependent variable).
- **x_test** (*array-like*, *shape* (*n_test_samples*,) or (*n_test_samples*, *n_features*)) – Test feature data for which to generate predictions.
- **y_test** (*array-like*, *shape* (*n_test_samples*,), *optional*) – Test target values used to calculate prediction error. Required for error computation.
- **degree** (*int*) – Degree of the polynomial to fit. Must be a non-negative integer.

Returns

Array containing two elements: - error : float

Prediction error metric (e.g., MSE, RMSE) computed between predicted and actual *y_test* values.**•prediction**[*np.ndarray*, *shape* (*n_test_samples*,)] Predicted values for *x_test*.**Return type***np.ndarray***Raises****ValueError** – If *x*, *y*, *x_test*, or *y_test* have incompatible shapes. If *degree* is negative. If insufficient samples for the specified polynomial degree.**Notes**

1. Polynomial features are created using *np.polyfit* for 1D data or *sklearn.preprocessing.PolynomialFeatures* for multi-dimensional data.
2. The error metric computed depends on implementation (typically mean squared error or root mean squared error).
3. For high-degree polynomials, consider regularization to prevent overfitting.

compute_model (*X_train*, *y_train*, *X_test*, *y_test*)

Computes predictions for spatiotemporal data using Polynomial Regression with parallel processing.

Parameters

- **X_train** (*xarray.DataArray*) – Training data (predictors) with dimensions (T, features). (It must be chunked properly in Dask, or at least be amenable to chunking.)
- **y_train** (*xarray.DataArray*) – Training target values with dimensions (T, Y, X).
- **X_test** (*xarray.DataArray*) – Test data (predictors) with dimensions (features,) or (T, features). Typically, you'd match time steps or have a single test.
- **y_test** (*xarray.DataArray*) – Test target values with dimensions (Y, X) or broadcastable to (T, Y, X).

Returns

An array with shape (2, Y, X) after computing, where the first index is error and the second is the prediction.

Return type

xarray.DataArray

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1, T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in *_ppf_terciles_from_code* - *shape, loc, scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use *best_guess[t]* to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.
- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type
array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If `dist_method == 'bestfit'`:

• Use cluster-based best-fit distributions to:

- derive terciles analytically from (`best_code_da`, `best_shape_da`, `best_loc_da`, `best_scale_da`),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=['PB','PN','PA'], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generate forecasts for a single time (e.g., future year) and compute tercile probabilities based on the chosen distribution method.

Parameters

- **Predictant** (*xarray.DataArray*) – Target variable with dimensions (T, Y, X).
- **clim_year_start** (*int*) – Start year of climatology period.
- **clim_year_end** (*int*) – End year of climatology period.
- **Predictor** (*xarray.DataArray*) – Historical predictor data with dimensions (T, features).
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast array that includes ‘error’ and ‘prediction’ over the historical period.
- **Predictor_for_year** (*xarray.DataArray*) – Predictor data for the forecast year, shape (features,) or (1, features).

Returns

result_ : *xarray.DataArray* or numpy array with the forecast’s [error, prediction]. **hindcast_prob** : *xarray.DataArray* of shape (probability=3, Y, X) with PB, PN, and PA.

Return type

tuple (**result_**, hindcast_prob)

```
class wass2s.was_machine_learning.WAS_PoissonRegression (nb_cores=1,
                                                         dist_method='nonparam')
```

Bases: *object*

Poisson Regression model for spatiotemporal **count data** prediction (e.g., number of rainy days, number of dry spells, extreme event counts, or discretized rainfall amounts) in climate applications.

This class implements: - Standard Poisson regression via scikit-learn’s *PoissonRegressor* - Spatially parallel fitting and prediction across large grids using dask + xarray - Deterministic point predictions (expected counts) - Optional probabilistic tercile forecasting (Below/Normal/Above) using either:

- Parametric distributions fitted per grid cell (‘bestfit’)
- Non-parametric historical error sampling (‘nonparam’)

Designed for seasonal forecasting tasks where the predictand is non-negative integer counts.

Parameters

- **nb_cores** (*int*, *default=1*) – Number of CPU cores to use for parallel computation (dask workers).
- **dist_method** (*{'bestfit', 'nonparam'}*, *default='nonparam'*) – Method for computing tercile probabilities: - ‘bestfit’ → uses best-fit distribution per grid cell (requires distribution fit inputs) - ‘nonparam’ → empirical sampling of historical forecast errors

nb_cores, dist_method

Stored initialization parameters.

fit_predict (*x*, *y*, *x_test*, *y_test*)

Fits Poisson regression on one grid cell and returns [error, prediction].

compute_model (*X_train*, *y_train*, *X_test*, *y_test*)

Parallel Poisson regression across the entire spatial domain. Returns *xarray.DataArray* with dims (‘output’=[‘error’, ‘prediction’], Y, X).

```
compute_prob(Predictant, clim_year_start, clim_year_end, hindcast_det,
             best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None)
```

Computes tercile probabilities for deterministic hindcasts.

```
forecast(Predictant, clim_year_start, clim_year_end, Predictor,
         hindcast_det,
         Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None,
         best_scale_da=None)
```

Full end-to-end forecast pipeline for one target year: - deterministic prediction (Poisson expected count) -
tercile probabilities (PB, PN, PA)

Notes

- **Input data requirements:** - y (target) must be non-negative integers (counts). Non-integer or negative values may cause fitting errors or poor performance.
 - Predictors (X) should be continuous or properly encoded.
- Predictions are clipped to ≥ 0 (Poisson rates cannot be negative).
- Large spatial domains benefit significantly from higher *nb_cores*.
- For *dist_method='bestfit'*, distribution fit results from *WAS_TransformData* must be provided.
- No target scaling is applied (Poisson regression is scale-invariant in the response).
- Error is computed as $y_{test} - prediction$ (can be negative).

Warning

- Ensure y (target) contains only non-negative integers.
- Very low counts or zero-inflated data may require zero-inflated Poisson (not implemented here).
- Small clusters or sparse data can lead to unstable fits.

```
__init__(nb_cores=1, dist_method='nonparam')
```

Initializes the WAS_PoissonRegression with a specified number of CPU cores and a default distribution method for tercile probability calculations.

Parameters

- **nb_cores** (*int*, *optional*) – Number of CPU cores to use for parallel computation, by default 1.
- **dist_method** (*str*, *optional*) – The distribution method to compute tercile probabilities, by default “gamma”.

```
fit_predict(x, y, x_test, y_test)
```

Fits a Poisson regression model to the provided training data, makes predictions on the test data, and calculates the prediction error.

Parameters

- **x** (*array-like, shape (n_samples, n_features)*) – Training data (predictors).
- **y** (*array-like, shape (n_samples,)*) – Training targets (non-negative count data).
- **x_test** (*array-like, shape (n_features,) or (1, n_features)*) – Test data (predictors).
- **y_test** (*float*) – Test target value (actual counts).

Returns

[prediction_error, predicted_value]

Return type

np.ndarray of shape (2,)

compute_model (*X_train, y_train, X_test, y_test*)

Computes predictions for spatiotemporal data using Poisson Regression with parallel processing.

Parameters

- **X_train** (*xarray.DataArray*) – Predictor data with dimensions (T, features).
- **y_train** (*xarray.DataArray*) – Training target values (count data) with dimensions (T, Y, X).
- **X_test** (*xarray.DataArray*) – Test data (predictors) with shape (features,) or (T, features), typically squeezed.
- **y_test** (*xarray.DataArray*) – Test target values (count data) with dimensions (Y, X) or broadcastable to (T, Y, X).

Returns

An array with a new dimension ('output', size=2) capturing [error, prediction].

Return type

xarray.DataArray

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1, T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in *_ppf_terciles_from_code* - *shape, loc, scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use *best_guess[t]* to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.
- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If **dist_method** == 'bestfit':

- **Use cluster-based best-fit distributions to:**

- derive terciles analytically from (best_code_da, best_shape_da, best_loc_da, best_scale_da),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.

- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.

Returns

hindcast_prob – Probabilities with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generate forecasts for a single time (e.g., future year) and compute tercile probabilities based on the chosen distribution method.

Parameters

- **Predictant** (*xarray.DataArray*) – Target variable with dimensions (T, Y, X).
- **clim_year_start** (*int*) – Start year of climatology period.
- **clim_year_end** (*int*) – End year of climatology period.
- **Predictor** (*xarray.DataArray*) – Historical predictor data with dimensions (T, features).
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast array that includes ‘error’ and ‘prediction’ over the historical period.
- **Predictor_for_year** (*xarray.DataArray*) – Predictor data for the forecast year, shape (features,) or (1, features).

Returns

result_ : *xarray.DataArray* or numpy array with the forecast’s [error, prediction]. **hindcast_prob** : *xarray.DataArray* of shape (probability=3, Y, X) with PB, PN, and PA.

Return type

tuple (**result_**, hindcast_prob)

class wass2s.was_machine_learning.**MARS** (*max_terms: int = 21, max_degree: int = 2, penalty: float = 3.0, min_span: int = 5*)

Bases: *object*

Multivariate Adaptive Regression Splines with corrected forward/backward passes.

__init__ (*max_terms: int = 21, max_degree: int = 2, penalty: float = 3.0, min_span: int = 5*)

Parameters:**max_terms**

[int] Maximum number of basis functions (including intercept)

max_degree

[int] Maximum interaction degree (1 = additive, 2 = pairwise interactions)

penalty

[float] GCV penalty per knot (typically 2-4)

min_span

[int] Minimum observations between knots

forward_pass (*X: ndarray, y: ndarray*) → *Tuple[ndarray, ndarray]*

Forward pass to add basis functions.

backward_pass (*X: ndarray, y: ndarray, B: ndarray, beta: ndarray*)

Backward pass to prune basis functions.

fit (*X: ndarray, y: ndarray*)

Fit MARS model.

predict (*X: ndarray*) → *ndarray*

Predict using fitted MARS model.

get_formula (*feature_names: List[str] | None = None*) → *str*

Get human-readable formula for the model.

class `wass2s.was_machine_learning.WAS_MARS_Model` (*nb_cores=1, dist_method='nonparam',
max_terms=21, max_degree=2, c=3*)

Bases: `object`

Multivariate Adaptive Regression Splines (MARS) model for spatiotemporal climate prediction.

MARS is a non-parametric regression technique that builds flexible models by fitting piecewise linear or cubic basis functions with knots automatically determined via a forward-backward selection process, regularized by Generalized Cross-Validation (GCV).

This class implements: - MARS regression using the pyearth library (or compatible MARS implementation) - Spatially parallel fitting and prediction across large grids using dask + xarray - Deterministic point predictions with optional error computation - Probabilistic tercile forecasting (Below/Normal/Above = PB/PN/PA) using either:

- Parametric best-fit distributions per grid cell ('bestfit')
- Non-parametric sampling of historical forecast errors ('nonparam')

Ideal for modeling non-linear relationships in seasonal climate variables (rainfall, temperature, agro-climatic indices) with good interpretability through selected basis functions.

Parameters

- **nb_cores** (*int, default=1*) – Number of CPU cores for parallel processing (dask workers).
- **dist_method** (*{'bestfit', 'nonparam'}, default='nonparam'*) – Method for computing tercile probabilities: - 'bestfit' → uses best-fit distribution per grid cell (requires distribution fit inputs) - 'nonparam' → empirical sampling of historical forecast errors
- **max_terms** (*int, default=21*) – Maximum number of basis functions (terms) allowed in the MARS model.
- **max_degree** (*int, default=2*) – Maximum degree of interaction (number of variables in a single hinge function).
- **c** (*float, default=3.0*) – Penalty cost parameter for effective number of parameters in GCV score (higher values → stronger regularization, fewer terms).

nb_cores, dist_method, max_terms, max_degree, c

Stored initialization parameters.

fit_predict (*x, y, x_test, y_test=None*)

Fits MARS on one grid cell and returns [error, prediction] or just prediction.

compute_model (*X_train*, *y_train*, *X_test*, *y_test*)

Parallel MARS regression across the entire spatial domain. Returns `xarray.DataArray` with dims ('output'=['error','prediction'], Y, X).

compute_prob (*Predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*,

best_code_da=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None)

Computes tercile probabilities for deterministic hindcasts.

forecast (*Predictant*, *clim_year_start*, *clim_year_end*, *Predictor*,
hindcast_det,

Predictor_for_year, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None,
best_scale_da=None)

Full end-to-end forecast pipeline for one target year: - deterministic MARS prediction - tercile probabilities (PB, PN, PA)

Notes

- **Input data requirements:** - Target (*y*) should be continuous (rainfall, temperature, etc.). - Predictors (*X*) can be continuous or categorical (MARS handles both).
- Negative predictions are clipped to zero (useful for rainfall or non-negative variables).
- MARS models are interpretable: final model consists of selected hinge functions.
- Large spatial domains benefit significantly from higher *nb_cores*.
- For *dist_method*='bestfit', distribution fit results from *WAS_TransformData* must be provided.
- No explicit feature or target scaling is applied (MARS is invariant to monotonic transformations).

Warning

- Very small training sets per grid cell may lead to overfitting or degenerate models.
- Extremely skewed targets (e.g., heavy-tailed rainfall) may benefit from transformation before modeling (not done automatically).
- MARS can produce piecewise-linear behavior; for very smooth functions, consider higher *max_degree* or compare with other models (e.g., XGBoost, MLP).

__init__ (*nb_cores*=1, *dist_method*='nonparam', *max_terms*=21, *max_degree*=2, *c*=3)

Initializes the *WAS_MARS_Model* with specified parameters.

Parameters

- **nb_cores** (*int*, *optional*) – Number of CPU cores to use for parallel computation, by default 1.
- **dist_method** (*str*, *optional*) – Distribution method to compute tercile probabilities, by default “gamma”.
- **max_terms** (*int*, *optional*) – Maximum number of basis functions for MARS, by default 21.
- **max_degree** (*int*, *optional*) – Maximum degree of interaction for MARS, by default 1.

- `c` (*float, optional*) – Cost parameter for GCV in MARS, by default 3.

fit_predict (*x, y, x_test, y_test=None*)

Fits a MARS model to the provided training data, makes predictions on the test data, and calculates the prediction error (if `y_test` is provided).

Parameters

- `x` (*array-like, shape (n_samples, n_features)*) – Training data (predictors).
- `y` (*array-like, shape (n_samples,)*) – Training targets.
- `x_test` (*array-like, shape (n_features,) or (1, n_features)*) – Test data (predictors).
- `y_test` (*float or None*) – Test target value. If `None`, no error is computed.

Returns

If `y_test` is not `None`, returns [error, prediction]. If `y_test` is `None`, returns [prediction].

Return type

`np.ndarray`

compute_model (*X_train, y_train, X_test, y_test*)

Applies MARS regression across a spatiotemporal dataset in parallel.

Parameters

- `X_train` (*xarray.DataArray*) – Training predictors with dims ('T','features').
- `y_train` (*xarray.DataArray*) – Training targets with dims ('T','Y','X').
- `X_test` (*xarray.DataArray*) – Test predictors, shape ('features',) or (T, features).
- `y_test` (*xarray.DataArray*) – Test targets with dims ('Y','X'), or broadcastable.

Returns

dims ('output','Y','X'), where 'output'=[error, prediction].

Return type

`xarray.DataArray`

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - `best_guess` : 1D array over T (hindcast_det or forecast_det) - `T1, T2` : scalar terciles from climatological best-fit distribution - `dist_code` : int, as in `_ppf_terciles_from_code` - `shape, loc, scale` : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

– $P(B) = F(T1)$

- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.
- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If dist_method == 'bestfit':

- **Use cluster-based best-fit distributions to:**
 - derive terciles analytically from (best_code_da, best_shape_da, best_loc_da, best_scale_da),
 - compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).

- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.

Returns

hindcast_prob – Probabilities with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generate a single-year deterministic forecast using the MARS model, then compute tercile probabilities (PB, PN, PA) based on the selected distribution method (*self.dist_method*).

The forecast is computed by: - Standardizing the predictand over the climatology period - Fitting the MARS model on historical data and predicting for the target year - Reversing standardization to get raw-scale forecast values - Computing probabilities using either best-fit distributions or nonparametric methods

Parameters

- **Predictant** (*xr.DataArray*) – Observed target variable (e.g., rainfall) with dimensions (T, Y, X).
- **clim_year_start** (*int*) – Start year of climatology period for standardization and terciles.
- **clim_year_end** (*int*) – End year of climatology period.
- **Predictor** (*xr.DataArray*) – Historical predictors with dimensions (T, features).
- **hindcast_det** (*xr.DataArray*) – Historical deterministic hindcasts with dimensions (output=[error, prediction], T, Y, X). Used for error variance estimation in nonparametric method.
- **Predictor_for_year** (*xr.DataArray*) – Predictors for the target forecast year, shape (features,) or (1, features).
- **best_code_da** (*xr.DataArray, optional*) – Distribution codes per grid cell (required for “bestfit” method).
- **best_shape_da** (*xr.DataArray, optional*) – Shape parameters per grid cell (required for “bestfit”).
- **best_loc_da** (*xr.DataArray, optional*) – Location parameters per grid cell (required for “bestfit”).
- **best_scale_da** (*xr.DataArray, optional*) – Scale parameters per grid cell (required for “bestfit”).

Returns

- **forecast_expanded** (*xr.DataArray*) – Deterministic forecast values with dimensions (T=1, Y, X).

- **forecast_prob** (*xr.DataArray*) – Probabilities with dimensions (probability=3, T=1, Y, X): - probability: ['PB', 'PN', 'PA'] (Below, Normal, Above).

Raises

ValueError – If “bestfit” method is selected but required distribution parameters are missing.
If invalid `dist_method` is set.

Notes

- For real forecasts, the “error” in the deterministic output is NaN (no verification available).
- The forecast date is set to the first day of the month matching the predictand’s time structure.
- Uses Dask for parallel computation across spatial chunks.

```
class wass2s.was_machine_learning.WAS_NegativeBinomial_Model (nb_cores=1,  
                                                             dist_method='nonparam',  
                                                             alpha_init=0.2,  
                                                             add_intercept=True)
```

Bases: `_WAS_CountProbMixin`

Negative Binomial (NB2) regression for spatiotemporal count prediction (Dask/xarray safe).

Deterministic prediction: $\mu_{\text{hat}} = E[Y|X]$ from NB2 GLM with log link:

$$\text{Var}(Y|X) = \mu + \alpha * \mu^2$$

```
__init__ (nb_cores=1, dist_method='nonparam', alpha_init=0.2, add_intercept=True)
```

```
fit_predict (x, y, x_test, y_test)
```

```
compute_model (X_train, y_train, X_test, y_test)
```

```
compute_prob (Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray,  
               best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da:  
               DataArray = None, best_scale_da: DataArray = None) → DataArray
```

```
forecast (Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year,  
          best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None)
```

```
class wass2s.was_machine_learning.WAS_ZINB_Model (nb_cores=1, dist_method='nonparam',  
                                                  alpha_init=0.2, add_intercept=True)
```

Bases: `WAS_NegativeBinomial_Model`

Zero-Inflated Negative Binomial (pragmatic two-part fit, Dask-safe).

Model:

$$P(Y=0|X) = \pi(X) + (1-\pi(X)) * \text{NB}(y=0 | \mu(X), \alpha) \quad E[Y|X] = (1 - \pi(X)) * \mu(X)$$

Fitting strategy (per grid cell):

- 1) logistic regression on $I(y==0)$ to estimate $\pi(X)$
- 2) NB2 regression on y to estimate $\mu(X)$, α

```
fit_predict (x, y, x_test, y_test)
```

```
class wass2s.was_machine_learning.WAS_HurdleNB_Model (nb_cores=1, dist_method='nonparam',  
                                                         alpha_init=0.2, add_intercept=True)
```

Bases: `WAS_NegativeBinomial_Model`

Hurdle Negative Binomial (two-part, Dask-safe).

Model:

$P(Y>0|X) = p+(X)$ via logistic regression $Y|Y>0, X \sim \text{zero-truncated NB}(\mu(X), \alpha)$ fit using NB on positives as approximation

Deterministic mean:

$$E[Y|X] = p+(X) * E[Y|Y>0, X]$$

where for NB2 (with $r=1/\alpha$, $p=r/(r+\mu)$):

$$P0_NB = (r/(r+\mu))^r E[Y|Y>0] = \mu / (1 - P0_NB)$$

Notes

- We fit NB on positive counts (not fully truncated likelihood) for operational stability.

fit_predict (*x*, *y*, *x_test*, *y_test*)

```
class wass2s.was_machine_learning.WAS_RandomForest_Model (param_space=None,  
                                                         n_clusters=5, nb_cores=1,  
                                                         dist_method='nonparam', hyper-  
                                                         param_optimizer='bayesian',  
                                                         n_trials=50, n_iter=50,  
                                                         mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

Random Forest regressor (sklearn). `max_features` is a fraction in (0, 1].

```
class wass2s.was_machine_learning.WAS_ExtraTrees_Model (param_space=None, n_clusters=5,  
                                                         nb_cores=1,  
                                                         dist_method='nonparam',  
                                                         hyperparam_optimizer='bayesian',  
                                                         n_trials=50, n_iter=50,  
                                                         mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

Extremely Randomized Trees regressor (sklearn).

```
class wass2s.was_machine_learning.WAS_GradientBoosting_Model (param_space=None,  
                                                         n_clusters=5, nb_cores=1,  
                                                         dist_method='nonparam',  
                                                         hyper-  
                                                         param_optimizer='bayesian',  
                                                         n_trials=50, n_iter=50,  
                                                         mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

Gradient Boosting regressor (sklearn).

```
class wass2s.was_machine_learning.WAS_AdaBoost_Model (param_space=None, n_clusters=5,  
                                                         nb_cores=1, dist_method='nonparam',  
                                                         hyperparam_optimizer='bayesian',  
                                                         n_trials=50, n_iter=50,  
                                                         mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

AdaBoost regressor (sklearn).

```
class wass2s.was_machine_learning.WAS_XGBoost_Model (param_space=None, n_clusters=5,  
                                                    nb_cores=1, dist_method='nonparam',  
                                                    hyperparam_optimizer='bayesian',  
                                                    n_trials=50, n_iter=50,  
                                                    mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

XGBoost regressor. Requires the `xgboost` package.

```
class wass2s.was_machine_learning.WAS_LightGBM_Model (param_space=None, n_clusters=5,  
                                                    nb_cores=1, dist_method='nonparam',  
                                                    hyperparam_optimizer='bayesian',  
                                                    n_trials=50, n_iter=50,  
                                                    mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

LightGBM regressor. Requires the `lightgbm` package.

```
class wass2s.was_machine_learning.WAS_SVR_Model (param_space=None, n_clusters=5,  
                                                    nb_cores=1, dist_method='nonparam',  
                                                    hyperparam_optimizer='bayesian',  
                                                    n_trials=50, n_iter=50, mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

Support Vector Regression (RBF kernel), auto-scaled with `StandardScaler`.

```
class wass2s.was_machine_learning.WAS_KNN_Model (param_space=None, n_clusters=5,  
                                                    nb_cores=1, dist_method='nonparam',  
                                                    hyperparam_optimizer='bayesian',  
                                                    n_trials=50, n_iter=50, mode='cluster')
```

Bases: `_WAS_NonNeural_Base`

K-Nearest-Neighbours regressor, auto-scaled with `StandardScaler`.

1.9.14 Analog forecasting

Analog-ensemble seasonal forecasting.

Identifies historically similar years (analog) by comparing climate indices or EOF scores for the target period and composites their observed rainfall to produce a forecast.

Classes

WAS_Analog

Full analog toolkit: downloads ERA5 reanalysis and NMME model data, computes SST/OLR/wind indices, performs dimensionality reduction, and applies one of several analog selection strategies.

Analog methods available via `compute_model`:

- `SOM` — Self-Organising Maps (minisom).
- `Bias_Based` — mean-bias distance in index space.
- `Corr_Based` — Pearson correlation distance.
- `Pca_Based` — PCA followed by Euclidean distance.
- `KMeans_Based` — K-Means clustering in index space.
- `Agglomerative_Based` — Agglomerative (Ward) clustering.

WAS_Analog

Archived predecessor of WAS_Analog; kept for backward compatibility.

```
class wass2s.was_analog.WAS_Analog(dir_to_save, year_start, year_forecast, predictor_vars=[{'area':
    [60, -180, -60, 180], 'model_name': 'ECMWF_51',
    'reanalysis_name': 'ERA5', 'variable': 'SST'}, {'area': [60, -180,
    -60, 180], 'model_name': 'ECMWF_51', 'reanalysis_name':
    'ERA5', 'variable': 'SP'}, {'area': [60, -180, -60, 180],
    'model_name': 'ECMWF_51', 'reanalysis_name': 'ERA5',
    'variable': 'VGRD_850'}], method_analog='som',
    month_of_initialization=None, lead_time=None,
    ensemble_mean='mean', rolling=3, standardize=True,
    detrend=False, multivariateEOF=False, eof_explained_var=0.95,
    clim_year_start=None, clim_year_end=None,
    index_compute=None, some_grid_size=(None, None),
    some_learning_rate=0.5, radius=1.0,
    some_neighborhood_function='gaussian', some_sigma=1.0,
    some_num_iteration=2000, dist_method='gamma', n_clusters=4,
    affinity='euclidean', linkage='ward')
```

Bases: `object`

Analog-based forecasting toolkit for seasonal climate applications.

This class orchestrates the end-to-end workflow required to build analog ensembles for Sea-Surface Temperature (SST) predictors and to translate them into deterministic and probabilistic rainfall forecasts over West Africa or any user-defined domain. Supports five analog-selection strategies: Self-Organizing Maps (SOM), correlation ranking, Principal-Component (EOF) similarity, K-Means clustering, and Agglomerative clustering.

Parameters

- **dir_to_save** (*str*) – Directory path to save downloaded and processed data files.
- **year_start** (*int*) – Starting year for historical data.
- **year_forecast** (*int*) – Target forecast year.
- **predictor_vars** (*list of dict, optional*) – List of dictionaries specifying predictor variables, each containing 'reanalysis_name', 'model_name', 'variable', and 'area'. Default is a list with NOAA SST and ERA5 SP, VGRD_850 variables.
- **method_analog** (*str, optional*) – Analog method to use ("som", "cor_based", "bias_based", "pca_based", "kmeans_based", "agglomerative_based"). Default is "som".
- **month_of_initialization** (*int, optional*) – Month of initialization for forecasts (1-12). If None, uses previous month.
- **lead_time** (*list, optional*) – List of lead times in months. If None, defaults to [1, 2, 3, 4, 5].
- **ensemble_mean** (*str, optional*) – Method for ensemble mean ("mean" or "median"). Default is "mean".
- **rolling** (*int, optional*) – Window size for rolling mean. Default is 3.
- **standardize** (*bool, optional*) – If True, standardize data; otherwise, compute anomalies. Default is True.
- **multivariateEOF** (*bool, optional*) – If True, use multivariate EOF analysis. Default is False.
- **eof_explained_var** (*float, optional*) – Fraction of variance to explain with EOF modes. Default is 0.95.

- **clim_year_start** (*int*, *optional*) – Start year for climatology period.
- **clim_year_end** (*int*, *optional*) – End year for climatology period.
- **index_compute** (*list*, *optional*) – List of climate indices to compute (e.g., ['NINO34', 'DMI']).
- **some_grid_size** (*tuple*, *optional*) – Grid size for SOM (rows, columns). Default is (None, None) for automatic sizing.
- **some_learning_rate** (*float*, *optional*) – Learning rate for SOM training. Default is 0.5.
- **radius** (*float*, *optional*) – Neighborhood radius for SOM analog search. Default is 1.0.
- **some_neighborhood_function** (*str*, *optional*) – Neighborhood function for SOM ("gaussian", "mexican_hat"). Default is "gaussian".
- **some_sigma** (*float*, *optional*) – Initial neighborhood radius for SOM. Default is 1.0.
- **some_num_iteration** (*int*, *optional*) – Number of iterations for SOM training. Default is 2000.
- **dist_method** (*str*, *optional*) – Method for probability calculation ("gamma", "t", "normal", "lognormal", "nonparam"). Default is "gamma".
- **n_clusters** (*int*, *optional*) – Number of clusters for K-Means and Agglomerative clustering. Default is 4.
- **affinity** (*str*, *optional*) – Distance metric for Agglomerative clustering (e.g., 'euclidean', 'manhattan'). Default is 'euclidean'.
- **linkage** (*str*, *optional*) – Linkage criterion for Agglomerative clustering (e.g., 'ward', 'complete', 'average'). Default is 'ward'.

Notes

- Performance Considerations: For large datasets (e.g., many years or high-dimensional data), the AgglomerativeClustering method (*agglomerative_based*) may be significantly slower than K-Means (*kmeans_based*) or SOM (*som*) due to its computational complexity. Users should consider testing with smaller datasets or reducing the number of clusters when using AgglomerativeClustering.

```
__init__ (dir_to_save, year_start, year_forecast, predictor_vars=[{'area': [60, -180, -60, 180], 'model_name': 'ECMWF_51', 'reanalysis_name': 'ERA5', 'variable': 'SST'}, {'area': [60, -180, -60, 180], 'model_name': 'ECMWF_51', 'reanalysis_name': 'ERA5', 'variable': 'SP'}, {'area': [60, -180, -60, 180], 'model_name': 'ECMWF_51', 'reanalysis_name': 'ERA5', 'variable': 'VGRD_850'}], method_analog='som', month_of_initialization=None, lead_time=None, ensemble_mean='mean', rolling=3, standardize=True, detrend=False, multivariateEOF=False, eof_explained_var=0.95, clim_year_start=None, clim_year_end=None, index_compute=None, some_grid_size=(None, None), some_learning_rate=0.5, radius=1.0, some_neighborhood_function='gaussian', some_sigma=1.0, some_num_iteration=2000, dist_method='gamma', n_clusters=4, affinity='euclidean', linkage='ward')
```

calc_index (*indices*, *sst*)

Calculate climate indices from SST data.

download_reanalysis_ (*force_download=False*)

Download reanalysis data (ERA5 and NOAA ERSST). Saves raw MONTHLY data (1-12) without aggregation, preserving seasonality.

download_reanalysis (*force_download=False*)

Download reanalysis data.

download_models (*force_download=False*)

Download and process seasonal forecast and hindcast data.

anomaly_timeseries (*ds*)

Compute anomalies by removing the climatological mean.

standardize_timeseries (*ds*)

Standardize the dataset by removing climatological mean and scaling by standard deviation.

compute_eofs (*data*)

Compute EOFs and retain modes explaining at least the specified variance.

arrange_indices_for_som (*ds*)

Arrange climate indices for SOM training.

identify_analogs_bmu (*target_year, bmu_dict*)

Identify analog years based on SOM Best Matching Units (BMUs).

SOM (*predictant, itrain, ireference_year*)

Identify similar years using Self-Organizing Maps (SOM).

Bias_Based (*predictant, itrain, ireference_year*)

Identify similar years using bias-based analog method.

Corr_Based (*predictant, itrain, ireference_year*)

Identify similar years using correlation-based analog method.

Pca_Based (*predictant, itrain, ireference_year*)

Identify similar years using PCA-based analog method.

KMeans_Based (*predictant, itrain, ireference_year*)

Identify similar years using K-Means clustering.

Agglomerative_Based (*predictant, itrain, ireference_year*)

Identify similar years using Agglomerative Clustering.

download_and_process ()

Download and process reanalysis and forecast data.

compute_model (*predictant, itrain, itest*)

Compute deterministic hindcast using the specified analog method.

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - `best_guess` : 1D array over T (hindcast_det or forecast_det) - T1, T2 : scalar terciles from climatological best-fit distribution - `dist_code` : int, as in `_ppf_terciles_from_code` - `shape`, `loc`, `scale` : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

$$P(B) = F(T1), P(N) = F(T2) - F(T1), P(A) = 1 - F(T2).$$

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → DataArray

Compute tercile probabilities for deterministic hindcasts.

If `dist_method == 'bestfit'`:

•Use cluster-based best-fit distributions to:

- derive terciles analytically from (`best_code_da`, `best_shape_da`, `best_loc_da`, `best_scale_da`),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=['PB','PN','PA'], T, Y, X).

Return type

xarray.DataArray

forecast (*predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None)

Generate deterministic and probabilistic forecasts for the target year.

composite_plot (*predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*, *plot_predictor*=True, *variable*='SST', *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None)

Create composite plots of predictors or predictands.

```
class wass2s.was_analog.WAS_Analog__ (dir_to_save, year_start, year_forecast, predictor_vars=[{'area':
    [60, -180, -60, 180], 'model_name': 'ECMWF_51',
    'reanalysis_name': 'ERA5', 'variable': 'SST'}, {'area': [60, -180,
    -60, 180], 'model_name': 'ECMWF_51', 'reanalysis_name':
    'ERA5', 'variable': 'SP'}, {'area': [60, -180, -60, 180],
    'model_name': 'ECMWF_51', 'reanalysis_name': 'ERA5',
    'variable': 'VGRD_850'}], method_analog='som',
    month_of_initialization=None, lead_time=None,
    ensemble_mean='mean', rolling=3, standardize=True,
    detrend=False, multivariateEOF=False,
    eof_explained_var=0.95, clim_year_start=None,
    clim_year_end=None, index_compute=None,
    some_grid_size=(None, None), some_learning_rate=0.5,
    radius=1.0, some_neighborhood_function='gaussian',
    some_sigma=1.0, some_num_iteration=2000,
    dist_method='gamma', n_clusters=4, affinity='euclidean',
    linkage='ward')
```

Bases: `object`

Analog-based forecasting toolkit for seasonal climate applications.

This class orchestrates the end-to-end workflow required to build analog ensembles for Sea-Surface Temperature (SST) predictors and to translate them into deterministic and probabilistic rainfall forecasts over West Africa or any user-defined domain. Supports five analog-selection strategies: Self-Organizing Maps (SOM), correlation ranking, Principal-Component (EOF) similarity, K-Means clustering, and Agglomerative clustering.

Parameters

- **dir_to_save** (*str*) – Directory path to save downloaded and processed data files.
- **year_start** (*int*) – Starting year for historical data.
- **year_forecast** (*int*) – Target forecast year.
- **predictor_vars** (*list of dict, optional*) – List of dictionaries specifying predictor variables, each containing 'reanalysis_name', 'model_name', 'variable', and 'area'. Default is a list with NOAA SST and ERA5 SP, VGRD_850 variables.
- **method_analog** (*str, optional*) – Analog method to use ("som", "cor_based", "bias_based", "pca_based", "kmeans_based", "agglomerative_based"). Default is "som".
- **month_of_initialization** (*int, optional*) – Month of initialization for forecasts (1-12). If None, uses previous month.
- **lead_time** (*list, optional*) – List of lead times in months. If None, defaults to [1, 2, 3, 4, 5].
- **ensemble_mean** (*str, optional*) – Method for ensemble mean ("mean" or "median"). Default is "mean".
- **rolling** (*int, optional*) – Window size for rolling mean. Default is 3.

- **standardize** (*bool, optional*) – If True, standardize data; otherwise, compute anomalies. Default is True.
- **multivariateEOF** (*bool, optional*) – If True, use multivariate EOF analysis. Default is False.
- **eof_explained_var** (*float, optional*) – Fraction of variance to explain with EOF modes. Default is 0.95.
- **clim_year_start** (*int, optional*) – Start year for climatology period.
- **clim_year_end** (*int, optional*) – End year for climatology period.
- **index_compute** (*list, optional*) – List of climate indices to compute (e.g., ['NINO34', 'DMI']).
- **some_grid_size** (*tuple, optional*) – Grid size for SOM (rows, columns). Default is (None, None) for automatic sizing.
- **some_learning_rate** (*float, optional*) – Learning rate for SOM training. Default is 0.5.
- **radius** (*float, optional*) – Neighborhood radius for SOM analog search. Default is 1.0.
- **some_neighborhood_function** (*str, optional*) – Neighborhood function for SOM ("gaussian", "mexican_hat"). Default is "gaussian".
- **some_sigma** (*float, optional*) – Initial neighborhood radius for SOM. Default is 1.0.
- **some_num_iteration** (*int, optional*) – Number of iterations for SOM training. Default is 2000.
- **dist_method** (*str, optional*) – Method for probability calculation ("gamma", "t", "normal", "lognormal", "nonparam"). Default is "gamma".
- **n_clusters** (*int, optional*) – Number of clusters for K-Means and Agglomerative clustering. Default is 4.
- **affinity** (*str, optional*) – Distance metric for Agglomerative clustering (e.g., 'euclidean', 'manhattan'). Default is 'euclidean'.
- **linkage** (*str, optional*) – Linkage criterion for Agglomerative clustering (e.g., 'ward', 'complete', 'average'). Default is 'ward'.

```
__init__(dir_to_save, year_start, year_forecast, predictor_vars=[{'area': [60, -180, -60, 180], 'model_name':
'ECMWF_51', 'reanalysis_name': 'ERA5', 'variable': 'SST'}, {'area': [60, -180, -60, 180],
'model_name': 'ECMWF_51', 'reanalysis_name': 'ERA5', 'variable': 'SP'}, {'area': [60, -180, -60,
180], 'model_name': 'ECMWF_51', 'reanalysis_name': 'ERA5', 'variable': 'VGRD_850'}],
method_analog='som', month_of_initialization=None, lead_time=None, ensemble_mean='mean',
rolling=3, standardize=True, detrend=False, multivariateEOF=False, eof_explained_var=0.95,
clim_year_start=None, clim_year_end=None, index_compute=None, some_grid_size=(None, None),
some_learning_rate=0.5, radius=1.0, some_neighborhood_function='gaussian', some_sigma=1.0,
some_num_iteration=2000, dist_method='gamma', n_clusters=4, affinity='euclidean',
linkage='ward')
```

```
calc_index (indices, sst)
```

Calculate climate indices from SST data.

download_reanalysis (*force_download=False*)

Download reanalysis data (ERA5 and NOAA ERSST). Saves raw MONTHLY data (1-12) without aggregation, preserving seasonality.

download_models (*force_download=False*)

Download and process seasonal forecast and hindcast data.

anomaly_timeseries (*ds*)

Compute anomalies by removing the climatological mean.

standardize_timeseries (*ds*)

Standardize the dataset by removing climatological mean and scaling by standard deviation.

compute_eofs (*data*)

Compute EOFs and retain modes explaining at least the specified variance.

arrange_indices_for_som (*ds*)

Arrange climate indices for SOM training.

identify_analogs_bmu (*target_year, bmu_dict*)

Identify analog years based on SOM Best Matching Units (BMUs).

SOM (*predictant, itrain, ireference_year*)

Identify similar years using Self-Organizing Maps (SOM).

Bias_Based (*predictant, itrain, ireference_year*)

Identify similar years using bias-based analog method.

Corr_Based (*predictant, itrain, ireference_year*)

Identify similar years using correlation-based analog method.

Pca_Based (*predictant, itrain, ireference_year*)

Identify similar years using PCA-based analog method.

KMeans_Based (*predictant, itrain, ireference_year*)

Identify similar years using K-Means clustering.

Agglomerative_Based (*predictant, itrain, ireference_year*)

Identify similar years using Agglomerative Clustering.

download_and_process ()

Download and process reanalysis and forecast data.

compute_model (*predictant, itrain, itest*)

Compute deterministic hindcast using the specified analog method.

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean² ratio matches the observed V/M² ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If `dist_method == 'bestfit'`:

- Use cluster-based best-fit distributions to:

- derive terciles analytically from (`best_code_da`, `best_shape_da`, `best_loc_da`, `best_scale_da`),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from `WAS_TransformData.fit_best_distribution_grid`, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=['PB','PN','PA'], T, Y, X).

Return type

xarray.DataArray

forecast (*predictant, clim_year_start, clim_year_end, hindcast_det, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generate deterministic and probabilistic forecasts for the target year.

composite_plot (*predictant, clim_year_start, clim_year_end, hindcast_det, plot_predictor=True, variable='SST', best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Create composite plots of predictors or predictands.

1.9.15 Forecast verification

Deterministic and probabilistic forecast verification.

Provides pixel-wise and domain-averaged scores compatible with the WASS2S xarray data model (dimensions T, Y, X).

Class

WAS_Verification

Container for all verification methods. Scores are applied grid-point by grid-point via `compute_deterministic_score`, or computed directly over spatial maps.

Deterministic scores

Pearson correlation, Spearman correlation, Index of Agreement, Nash–Sutcliffe Efficiency, Kling–Gupta Efficiency, MAE, RMSE, Taylor diagram.

Probabilistic scores

GPROC (Generalized ROC), RPSS, Ignorance Score, Brier Score, Brier Skill Score, Resolution Score.

Tercile utilities

`classify` — assign observations to below / near / above-normal tercile classes. `compute_class` — classify a full DataArray over a climatological reference period. `classify_data_into_terciles` — binary classification with pre-computed thresholds.

```
class wass2s.was_verification.WAS_Verification (dist_method='nonparam',
                                                mae_rmse_crps_thrsd=100)
```

Bases: `object`

Verification class for evaluating weather and climate forecasts.

Provides methods to compute deterministic, probabilistic, and ensemble-based metrics, as well as visualization tools for model performance assessment.

Parameters

dist_method (*str*, *optional*) – Distribution method for tercile probability calculations ('bestfit', 'nonparam'). Default is 'nonparam'.

__init__ (*dist_method*='nonparam', *mae_rmse_crps_thrsd*=100)

get_scores_metadata ()

Retrieve metadata for all available scoring metrics.

Returns

Dictionary containing score names as keys and tuples of metadata (description, min_value, max_value, score_type, colormap, function) as values.

Return type

`dict`

kling_gupta_efficiency (*y_true*, *y_pred*)

Compute Kling-Gupta Efficiency (KGE) metric.

KGE combines correlation, bias, and variability ratios to assess model performance.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_pred** (*array-like*) – Predicted values.

Returns

KGE score, ranging from -Inf to 1 (1 is perfect). Returns `np.nan` if insufficient valid data.

Return type

`float`

pearson_corr (*y_true*, *y_pred*)

Compute Pearson Correlation Coefficient.

Measures the linear correlation between observed and predicted values.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_pred** (*array-like*) – Predicted values.

Returns

Pearson correlation coefficient, ranging from -1 to 1. Returns np.nan if insufficient valid data.

Return type

float

spearman_corr (*y_true*, *y_pred*)

Compute Spearman rank correlation coefficient.

Parameters

- **y_true** (*array-like*) – Observed values (1D).
- **y_pred** (*array-like*) – Predicted values (1D).

Returns

Spearman's rho in [-1, 1]; np.nan if insufficient valid data.

Return type

float

index_of_agreement (*y_true*, *y_pred*)

Compute Index of Agreement (IOA).

Measures the degree of model prediction accuracy relative to observed variability.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_pred** (*array-like*) – Predicted values.

Returns

IOA score, ranging from 0 to 1 (1 is perfect). Returns np.nan if insufficient valid data.

Return type

float

mean_absolute_error (*y_true*, *y_pred*)

Compute Mean Absolute Error (MAE).

Measures the average magnitude of errors in predictions.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_pred** (*array-like*) – Predicted values.

Returns

MAE value, non-negative. Returns np.nan if insufficient valid data.

Return type

float

root_mean_square_error (*y_true*, *y_pred*)

Compute Root Mean Square Error (RMSE).

Measures the square root of the average squared errors in predictions.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_pred** (*array-like*) – Predicted values.

Returns

RMSE value, non-negative. Returns np.nan if insufficient valid data.

Return type

float

nash_sutcliffe_efficiency (*y_true*, *y_pred*)

Compute Nash-Sutcliffe Efficiency (NSE).

Measures the predictive skill of the model compared to the mean of observations.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_pred** (*array-like*) – Predicted values.

Returns

NSE score, ranging from -Inf to 1 (1 is perfect). Returns np.nan if insufficient valid data.

Return type

float

taylor_diagram (*obs*, *models*, * (*Keyword-only parameters separator (PEP 3102)*), *normalize*: *bool* = *True*, *correlation*: *str* = 'pearson', *labels*: *dict* | *None* = *None*, *title*: *str* | *None* = 'Taylor diagram', *figsize*=(7, 6), *savepath*: *str* | *None* = *None*, *hide_rms_numbers*: *bool* = *True*)

Plot a Taylor diagram comparing one or more model series against observations.

Parameters

- **obs** (*xarray.DataArray* or *np.ndarray*) – Observations. If xarray, can be (T), (T,Y,X), etc.
- **models** (*dict*[*str*, *xarray.DataArray* | *np.ndarray*]) – Mapping of model name -> forecast array (same shape as obs or broadcastable).
- **normalize** (*bool*, *optional*) – If True (default), normalize std and CRMSD by std(obs).
- **correlation** ({*"pearson"*, *"spearman"*}, *optional*) – Correlation metric used on paired, finite values.
- **labels** (*dict*[*str*, *str*], *optional*) – Custom display labels for model markers. Defaults to model keys.
- **title** (*str* or *None*, *optional*) – Figure title.
- **figsize** (*tuple*, *optional*) – Matplotlib figure size.
- **savepath** (*str* or *None*, *optional*) – If provided, save figure to this path.

Returns

(*fig*, *ax*)

Return type

matplotlib Figure and Axes

compute_deterministic_score (*score_func*, *obs*, *pred*)

Apply a deterministic scoring function over xarray DataArrays.

Computes the specified metric across the time dimension for each grid point.

Parameters

- **score_func** (*callable*) – Scoring function to apply (e.g., `pearson_corr`, `kling_gupta_efficiency`).
- **obs** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X).
- **pred** (*xarray.DataArray*) – Predicted data with dimensions (T, Y, X).

Returns

Score values with dimensions (Y, X).

Return type*xarray.DataArray***classify** (*y*, *index_start*, *index_end*)

Classify data into terciles based on climatological thresholds.

Parameters

- **y** (*array-like*) – Input data to classify.
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.

Returns• **y_class**

[array-like] Classified data (0: below-normal, 1: near-normal, 2: above-normal).

• **tercile_33**

[float] 33rd percentile threshold.

• **tercile_67**

[float] 67th percentile threshold.

Return type

tuple

compute_class (*Predictant*, *clim_year_start*, *clim_year_end*)

Compute tercile class labels for observed data.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X).
- **clim_year_start** (*int or str*) – Start year of the climatology period.
- **clim_year_end** (*int or str*) – End year of the climatology period.

Returns

Classified data with dimensions (T, Y, X), where values are 0 (below-normal), 1 (near-normal), or 2 (above-normal).

Return type*xarray.DataArray*

classify_data_into_terciles (*y*, *T1*, *T2*)

Classify data into terciles based on given thresholds.

Parameters

- **y** (*array-like*) – Input data to classify.
- **T1** (*float*) – First tercile threshold (33rd percentile).
- **T2** (*float*) – Second tercile threshold (67th percentile).

Returns

Classified data (0: below-normal, 1: near-normal, 2: above-normal).

Return type

array-like

calculate_groc (*y_true*, *y_probs*, *index_start*, *index_end*, *n_classes=3*)

Compute Generalized Discrimination (GROC) score.

Averages the Area Under the Curve (AUC) for each tercile category.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (*n_classes*, *n_samples*).
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.
- **n_classes** (*int*, *optional*) – Number of classes (default is 3 for terciles).

Returns

GROC score, ranging from 0 to 1. Returns np.nan if insufficient valid data.

Return type

float

calculate_groc (*y_true*, *y_probs*, *index_start*, *index_end*, *n_classes=3*)

Compute Generalized Discrimination (GROC) score according to Weigel and Mason (2011).

The GROC score averages the Area Under the ROC Curve (AUC) for each category, measuring the ability to distinguish occurrences from non-occurrences of each category.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (*n_classes*, *n_samples*).
- **index_start** (*int*) – Start index of the climatology period for computing terciles.
- **index_end** (*int*) – End index of the climatology period for computing terciles.
- **n_classes** (*int*, *optional*) – Number of classes (default is 3 for terciles).

Returns

- *float* – GROC score, ranging from 0 to 1. Returns np.nan if insufficient valid data.
- *Reference*
- *_____*
- *Weigel, A. P. and Mason, S. J. (2011).*

- *The generalized discrimination score for ensemble forecasts.*
- **Monthly Weather Review, 139(9) (3069-3074.)**

calculate_groc_weighted (*y_true, y_probs, index_start, index_end, n_classes=3*)

Compute Generalized Discrimination (GROC) score with category weighting.

Weighted average of AUCs by climatological frequency of each category. Alternative: Weighted by climatological frequency as in some applications :param y_true: Observed values. :type y_true: array-like :param y_probs: Forecast probabilities with shape (n_classes, n_samples). :type y_probs: array-like :param index_start: Start index of the climatology period. :type index_start: int :param index_end: End index of the climatology period. :type index_end: int :param n_classes: Number of classes (default is 3 for terciles). :type n_classes: int, optional

Returns

Weighted GROC score, ranging from 0 to 1.

Return type

float

calculate_groc_fixed (*y_true, y_probs, index_start, index_end, n_classes=3*)

Fixed from version **calculate_groc_** of GROC calculation.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (n_classes, n_samples).
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.
- **n_classes** (*int, optional*) – Number of classes (default is 3 for terciles).

Returns

GROC score, ranging from 0 to 1.

Return type

float

calculate_rpss (*y_true, y_probs, index_start, index_end*)

Compute Ranked Probability Skill Score (RPSS).

Compares the Ranked Probability Score (RPS) of the forecast to a climatological reference.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (n_classes, n_samples).
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.

Returns

RPSS score, ranging from -Inf to 1. Returns np.nan if insufficient valid data.

Return type

float

ignorance_score (*y_true, y_probs, index_start, index_end*)

Compute Ignorance Score based on Weijs (2010).

Measures the logarithmic loss of forecast probabilities for the true category.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (n_classes, n_samples).
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.

Returns

Ignorance score, non-negative. Returns np.nan if insufficient valid data.

Return type

float

brier_score (*y_true, y_probs, index_start, index_end, event='PA'*)

BS for a single tercile event (PB/PN/PA). y_probs has shape (3, N).

brier_skill_score (*y_true, y_probs, index_start, index_end, event='PA', reference='uniform'*)

Event-wise Brier Skill Score (BSS) for tercile forecasts.

Parameters

- **y_true** (*1D array-like*) – Observed values (time series for one grid point).
- **y_probs** (*2D array-like, shape (3, N)*) – Forecast probabilities for PB/PN/PA over time.
- **index_start** (*int*) – Indices delimiting the calibration slice inside the *masked* series.
- **index_end** (*int*) – Indices delimiting the calibration slice inside the *masked* series.
- **event** (*{ "PB", "PN", "PA", 0, 1, 2 }, optional*) – Event to score (default “PA” = above-normal).
- **reference** (*{ "uniform", "empirical" }, optional*) –
 - “uniform”: climatology is [1/3, 1/3, 1/3]
 - “empirical”: base rate computed from calibration slice

Returns

BSS = 1 - BS / BS_ref (np.nan if not computable).

Return type

float

resolution_score_grid (*y_true, y_probs, index_start, index_end, bins=array([0., 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.67, 0.675, 0.8, 0.85, 0.9, 0.95, 0.975, 1.])*)

Compute Resolution Score on a grid based on Weijs (2010).

Measures the ability of the forecast to distinguish between different outcomes.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (n_classes, n_samples).
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.
- **bins** (*array-like, optional*) – Probability bins for discretizing forecast probabilities.

Returns

Resolution score, non-negative. Returns np.nan if insufficient valid data.

Return type

float

reliability_score_grid (*y_true*, *y_probs*, *index_start*, *index_end*, *bins*=array([0., 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.67, 0.675, 0.8, 0.85, 0.9, 0.95, 0.975, 1.]))

Compute Reliability Score on a grid based on Weijs (2010).

Measures the calibration of forecast probabilities against observed frequencies.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (n_classes, n_samples).
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.
- **bins** (*array-like, optional*) – Probability bins for discretizing forecast probabilities.

Returns

Reliability score. Returns np.nan if insufficient valid data.

Return type

float

resolution_and_reliability_per_category_grid (*y_true*, *y_probs*, *index_start*, *index_end*, *bins*=array([0., 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.67, 0.675, 0.8, 0.85, 0.9, 0.95, 0.975, 1.]))

Compute Resolution and Reliability scores per category based on Weijs (2010).

Returns both total scores and per-category breakdowns.

Parameters

- **y_true** (*array-like*) – Observed values.
- **y_probs** (*array-like*) – Forecast probabilities with shape (n_classes, n_samples).
- **index_start** (*int*) – Start index of the climatology period.
- **index_end** (*int*) – End index of the climatology period.
- **bins** (*array-like, optional*) – Probability bins for discretizing forecast probabilities.

Returns

Dictionary containing: - 'total_resolution': float - 'total_reliability': float - 'category_resolution': list of floats for each category - 'category_reliability': list of floats for each category - 'category_base_rates': list of floats (climatological frequencies)

Return type

dict

```
resolution_and_reliability_over_all_grid (y_true_class, y_probs, bins=array([0., 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.67, 0.675, 0.8, 0.85, 0.9, 0.95, 0.975, 1.]))
```

Compute Resolution and Reliability scores over all grid points.

Based on Weijjs (2010), computes scores by aggregating across all grid points.

Parameters

- **y_true_class** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X).
- **y_probs** (*xarray.DataArray*) – Forecast probabilities with dimensions (probability, T, Y, X).
- **bins** (*array-like, optional*) – Probability bins for discretizing forecast probabilities.

Returns

- **resolution_sum**
[float] Aggregated resolution score.
- **reliability_sum**
[float] Aggregated reliability score.

Return type

tuple

```
resolution_and_reliability_over_all_grid_per_category (y_true_class, y_probs, bins=array([0., 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.67, 0.675, 0.8, 0.85, 0.9, 0.95, 0.975, 1.]))
```

Assumes y_true_class already contains valid category labels (0, 1, 2).

```
reliability_diagram (modelname, dir_to_save_score, y_true_class, y_probs, bins=array([0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.67]))
```

Plot Reliability Diagrams for probabilistic forecasts.

Visualizes the calibration of forecast probabilities against observed frequencies.

Parameters

- **modelname** (*str*) – Name of the model for labeling the plot.
- **dir_to_save_score** (*str or Path*) – Directory to save the plot.
- **y_true** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X).
- **y_probs** (*xarray.DataArray*) – Forecast probabilities with dimensions (probability, T, Y, X).
- **clim_year_start** (*int or str*) – Start year of the climatology period.
- **clim_year_end** (*int or str*) – End year of the climatology period.
- **bins** (*array-like, optional*) – Probability bins for discretizing forecast probabilities.

plot_roc_curves (*modelname*, *dir_to_save_score*, *y_true_class*, *y_probs*, *n_bootstraps*=200, *ci*=0.95)

Plot ROC Curves with Confidence Intervals for probabilistic forecasts.

Visualizes the Receiver Operating Characteristic for each tercile category.

Parameters

- **modelname** (*str*) – Name of the model for labeling the plot.
- **dir_to_save_score** (*str* or *Path*) – Directory to save the plot.
- **y_true_class** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X).
- **y_probs** (*xarray.DataArray*) – Forecast probabilities with dimensions (probability, T, Y, X).
- **clim_year_start** (*int* or *str*) – Start year of the climatology period.
- **clim_year_end** (*int* or *str*) – End year of the climatology period.
- **n_bootstraps** (*int*, *optional*) – Number of bootstrap samples for confidence intervals. Default is 200.
- **ci** (*float*, *optional*) – Confidence interval level (e.g., 0.95 for 95%). Default is 0.95.

compute_crps (*y_true*, *y_pred*, *member_dim*='number', *dim*='T')

Compute Continuous Ranked Probability Score (CRPS) for ensemble forecasts.

Measures the difference between the forecast ensemble distribution and observations.

Parameters

- **y_true** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X).
- **y_pred** (*xarray.DataArray*) – Ensemble forecast data with dimensions (T, number, Y, X).
- **member_dim** (*str*, *optional*) – Dimension name for ensemble members. Default is 'number'.
- **dim** (*str*, *optional*) – Dimension to compute CRPS over (typically time). Default is 'T'.

Returns

CRPS values with dimensions (Y, X).

Return type

xarray.DataArray

compute_probabilistic_score (*score_func*, *obs*, *prob_pred*, *clim_year_start*, *clim_year_end*, ***score_kwargs*)

Apply a probabilistic scoring function over *xarray* DataArrays.

Computes the specified probabilistic metric across the time dimension for each grid point.

Parameters

- **score_func** (*callable*) – Probabilistic scoring function to apply (e.g., *calculate_rpss*, *calculate_groc*).
- **obs** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X).
- **prob_pred** (*xarray.DataArray*) – Forecast probabilities with dimensions (probability, T, Y, X).
- **clim_year_start** (*int* or *str*) – Start year of the climatology period.

- **clim_year_end** (*int or str*) – End year of the climatology period.

Returns

Score values with dimensions (Y, X).

Return type

`xarray.DataArray`

plot_model_score (*model_metric, score, dir_save_score, figure_name='WAS_MLR'*)

Plot a deterministic score on a map.

Creates a geographical plot of the specified metric for a single model.

Parameters

- **model_metric** (*xarray.DataArray*) – Score values with dimensions (Y, X).
- **score** (*str*) – Name of the score to plot (e.g., 'Pearson', 'MAE').
- **dir_save_score** (*str or Path*) – Directory to save the plot.
- **figure_name** (*str, optional*) – Prefix for the figure filename. Default is 'WAS_MLR'.

plot_models_score (*model_metrics, score, dir_save_score*)

Plot multiple model scores on a grid of maps.

Creates a subplot grid with geographical plots for each model's score.

Parameters

- **model_metrics** (*dict*) – Dictionary of model names and their corresponding score DataArrays (Y, X).
- **score** (*str*) – Name of the score to plot (e.g., 'Pearson', 'MAE').
- **dir_save_score** (*str or Path*) – Directory to save the plot.

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - **best_guess** : 1D array over T (hindcast_det or forecast_det) - **T1, T2** : scalar terciles from climatological best-fit distribution - **dist_code** : int, as in `_ppf_terciles_from_code` - **shape, loc, scale** : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

$P(B) = F(T1)$, $P(N) = F(T2) - F(T1)$, $P(A) = 1 - F(T2)$.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

gcm_compute_prob (*Predictant*: *DataArray*, *clim_year_start*, *clim_year_end*, *hindcast_det*: *DataArray*,
best_code_da: *DataArray* = None, *best_shape_da*: *DataArray* = None, *best_loc_da*:
DataArray = None, *best_scale_da*: *DataArray* = None) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If `dist_method == 'bestfit'`:

• Use cluster-based best-fit distributions to:

- derive terciles analytically from (*best_code_da*, *best_shape_da*, *best_loc_da*, *best_scale_da*),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int* or *str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int* or *str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray*, optional) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray*, optional) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray*, optional) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray*, optional) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

Return type

xarray.DataArray

gcm_validation_compute (*models_files_path*, *Obs*, *score*, *month_of_initialization*, *clim_year_start*,
clim_year_end, *dir_to_save_roc_reliability*, *lead_time*=None,
ensemble_mean=None, *gridded*=True)

Validate multiple General Circulation Model (GCM) forecasts using specified metrics.

Computes deterministic, probabilistic, or ensemble-based scores for GCM hindcasts by processing model data from provided file paths and comparing against observations.

Parameters

- **models_files_path** (*dict*) – Dictionary mapping model identifiers (e.g., model names) to file paths of hindcast NetCDF files.
- **Obs** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X), where T is time, Y is latitude, and X is longitude.
- **score** (*str*) – Score to compute, must be a key in `self.scores` (e.g., ‘Pearson’, ‘RPSS’, ‘ROC_CURVE’, ‘RELIABILITY_DIAGRAM’).

- **month_of_initialization** (*int*) – Month of forecast initialization (1-12).
- **clim_year_start** (*int or str*) – Start year of the climatology period for computing tercile thresholds.
- **clim_year_end** (*int or str*) – End year of the climatology period for computing tercile thresholds.
- **dir_to_save_roc_reliability** (*str or Path*) – Directory to save ROC curves and reliability diagram plots.
- **lead_time** (*list of int, optional*) – Lead times in months to define the forecast season (e.g., [1, 2, 3] for a 3-month season). If None, no season is specified.
- **ensemble_mean** (*str, optional*) – Ensemble statistic to use for deterministic or probabilistic scores (e.g., 'mean'). If None, ensemble mean is computed automatically when required.
- **gridded** (*bool, optional*) – Whether to perform gridded validation (True) or non-gridded (not implemented). Default is True.

Returns

If **score_type** is 'det_score', 'prob_score', or 'ensemble_score':

Dictionary with model identifiers as keys and `xarray.DataArray` of scores (dimensions Y, X) as values.

If **score_type** is 'all_grid_prob_score' (e.g., ROC_CURVE, RELIABILITY_DIAGRAM):

None, as results are saved as plots.

If **score_type** is 'all_grid_det_score':

None (not implemented).

Return type

`dict` or `None`

Raises

ValueError – If the score is not recognized or if non-gridded validation is attempted (not implemented).

weighted_gcm_forecasts (*Obs, best_models, scores, lead_time, model_dir, clim_year_start, clim_year_end, variable='PRCP'*)

Generate weighted ensemble forecasts from selected GCMs based on their performance scores.

Combines hindcasts and forecasts from multiple models using weights derived from the GROC score, scales the results to match observed climatology, and computes tercile probabilities.

Parameters

- **Obs** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X), used for scaling and probability calculations.
- **best_models** (*dict*) – Dictionary of model identifiers (keys) to be included in the ensemble.
- **scores** (*dict*) – Dictionary containing scores (e.g., GROC) for each model, with model identifiers as keys.
- **lead_time** (*list of int*) – Lead times in months for the forecast season (e.g., [1, 2, 3] for a 3-month season).
- **model_dir** (*str or Path*) – Directory containing hindcast and forecast NetCDF files.

- **clim_year_start** (*int or str*) – Start year of the climatology period for computing tercile thresholds.
- **clim_year_end** (*int or str*) – End year of the climatology period for computing tercile thresholds.
- **variable** (*str, optional*) – Variable name in the NetCDF files (e.g., ‘PRCP’ for precipitation). Default is ‘PRCP’.

Returns

- **hindcast_det**
[xarray.DataArray] Weighted deterministic hindcast with dimensions (T, Y, X).
- **hindcast_prob**
[xarray.DataArray] Tercile probabilities for hindcasts with dimensions (probability, T, Y, X).
- **forecast_prob**
[xarray.DataArray] Tercile probabilities for forecasts with dimensions (probability, Y, X).

Return type

tuple

Notes

- Model files are expected to follow a naming convention like ‘hind-cast_{center}_{variable}_{init_month}_{season}_{lead}.nc’.
- The GROC score is used as the weighting metric.
- A scaling factor is applied based on the ratio of observed to hindcast mean.

static classify_percent (*p*)

Classify a percentage value into predefined categories based on thresholds.

Maps a percentage ratio to one of five categories representing deviation from average.

Parameters

p (*float or array-like*) – Percentage value(s) to classify, typically representing a ratio to normal (%).

Returns

Category code: - 1: Well Above Average ($\geq 150\%$) - 2: Above Average ($\geq 110\%$) - 3: Near Average ($\geq 90\%$) - 4: Below Average ($\geq 50\%$) - 5: Well Below Average ($< 50\%$)

Return type

int or array-like

ratio_to_average (*predictant, clim_year_start, clim_year_end, year*)

Compute and visualize the ratio of a specific year’s data to the climatological mean.

Calculates the percentage ratio of the predictand for a given year relative to the mean over a climatology period, classifies it into categories, and plots the result on a geographical map.

Parameters

- **predictant** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X), where T is time, Y is latitude, and X is longitude.
- **clim_year_start** (*int or str*) – Start year of the climatology period.
- **clim_year_end** (*int or str*) – End year of the climatology period.

- **year** (*int or str*) – Specific year to compute the ratio for.

Returns

Displays a geographical plot of the classified ratios with a custom colormap and legend.

Return type

None

calculate_rpss_ (*y_true, y_probs*)

Compute Ranked Probability Skill Score (RPSS) for a single grid point or sample.

Compares the Ranked Probability Score (RPS) of the forecast to a climatological reference with uniform probabilities (1/3 for each tercile).

Parameters

- **y_true** (*array-like of shape (n_samples,)*) – True class labels, already classified into terciles (0: below-normal, 1: near-normal, 2: above-normal).
- **y_probs** (*array-like of shape (3, n_samples)*) – Forecast probabilities for each tercile category.

Returns

RPSS score, ranging from -Inf to 1 (1 indicates perfect forecast skill). Returns np.nan if insufficient valid data.

Return type

float

compute_one_year_rpss_ (*obs, prob_pred, clim_year_start, clim_year_end, year*)

Compute and visualize the Ranked Probability Skill Score (RPSS) for a specific year.

Applies the RPSS calculation across a gridded dataset for a single year, using tercile classifications based on a climatology period, and plots the results on a map.

Parameters

- **obs** (*xarray.DataArray*) – Observed data with dimensions (T, Y, X), where T is time, Y is latitude, and X is longitude.
- **prob_pred** (*xarray.DataArray*) – Forecast probabilities with dimensions (probability, T, Y, X), where probability includes ['PB', 'PN', 'PA'] (below-normal, near-normal, above-normal).
- **clim_year_start** (*int or str*) – Start year of the climatology period for tercile classification.
- **clim_year_end** (*int or str*) – End year of the climatology period for tercile classification.
- **year** (*int or str*) – Specific year to compute the RPSS for.

Returns

Displays a geographical plot of the RPSS values with a colorbar.

Return type

None

compute_one_year_rpss (*obs, prob_pred, clim_year_start, clim_year_end, year, shapefile_path=None*)

Compute RPSS and display the percentage of skillful grids (RPSS > 0) in the title.

1.9.16 Multi-model ensemble methods

Multi-Model Ensemble (MME) methods for seasonal forecasting.

Provides weighted averaging, probabilistic MME schemes, ELM/MLP-based ensembles, logistic and Gaussian-process based post-processing, and wrappers for the xcast ELM/ELR framework.

Key functions

process_datasets_for_mme

Align and reindex multiple model hindcast/forecast DataArrays to a common grid.

myfill

Fill missing model forecasts using the observed climatology.

Key classes

WAS_mme_Weighted

Simple or skill-weighted ensemble averaging with parametric or non-parametric tercile probabilities.

WAS_ProbWeighted

Probabilistic skill-weighted ensemble (tercile-class based).

WAS_Min2009_ProbWeighted

Probabilistic MME following Min (2009) chi-squared weighting.

WAS_mme_MLP / WAS_mme_MLP_

MLP-based MME post-processing.

WAS_mme_hpELM / WAS_mme_hpELM_

Extreme Learning Machine MME post-processing.

WAS_mme_Stacking / WAS_mme_Stacking_

Stacking ensemble MME.

WAS_mme_XGBoosting / WAS_mme_XGBoosting_

XGBoost-based MME post-processing.

WAS_mme_RF / WAS_mme_RF_

Random Forest MME post-processing.

WAS_mme_logistic

Logistic-regression MME producing direct tercile probabilities.

WAS_mme_gaussian_process

Gaussian Process MME.

WAS_mme_FastBMA / WAS_mme_FullBMA

Bayesian Model Averaging (fast and full variants).

WAS_mme_xcELM / WAS_mme_xcELR

xcast-based ELM and ELR wrappers.

```
wass2s.was_mme.process_datasets_for_mme(rainfall, hdcsted=None, fcsted=None, gcm=False,
                                         agroparam=False, Prob=False, hydro=False,
                                         ELM_ELRL=False, dir_to_save_model=None,
                                         best_models=None, scores=None, year_start=None,
                                         year_end=None, model=False,
                                         month_of_initialization=None, lead_time=None,
                                         year_forecast=None, score_metric='GROC', var='PRCP')
```

Process hindcast and forecast datasets for a multi-model ensemble.

This function loads, interpolates, and concatenates hindcast and forecast datasets from various sources (GCMs, agroparameters, or others) to prepare them for a multi-model ensemble. It supports different score metrics and configurations for probabilistic or deterministic outputs.

Parameters

- **rainfall** (*xarray.DataArray*) – Observed rainfall data used for interpolation and masking.
- **hdcsted** (*dict*, *optional*) – Dictionary of hindcast datasets for different models.
- **fcsted** (*dict*, *optional*) – Dictionary of forecast datasets for different models.
- **gcm** (*bool*, *optional*) – If True, process data as GCM data. Default is True.
- **agroparam** (*bool*, *optional*) – If True, process data as agroparameter data. Default is False.
- **Prob** (*bool*, *optional*) – If True, process data as probabilistic forecasts. Default is False.
- **ELM_ELRL** (*bool*, *optional*) – If True, use ELM_ELRL configuration for dimension re-naming. Default is False.
- **dir_to_save_model** (*str*, *optional*) – Directory path to load model data.
- **best_models** (*list*, *optional*) – List of model names to include in the ensemble.
- **scores** (*dict*, *optional*) – Dictionary containing model scores, with the key specified by *score_metric*.
- **year_start** (*int*, *optional*) – Starting year for the data range.
- **year_end** (*int*, *optional*) – Ending year for the data range.
- **model** (*bool*, *optional*) – If True, treat data as model-based. Default is True.
- **month_of_initialization** (*int*, *optional*) – Month when the forecast is initialized.
- **lead_time** (*int*, *optional*) – Forecast lead time in months.
- **year_forecast** (*int*, *optional*) – Year for which the forecast is generated.
- **score_metric** (*str*, *optional*) – Metric used to organize scores (e.g., 'Pearson', 'MAE', 'GROC'). Default is 'GROC'.
- **var** (*str*, *optional*) – variables used (e.g., 'PRCP')

Returns

- **all_model_hdcst** (*xarray.DataArray*) – Concatenated hindcast data across models.
- **all_model_fcst** (*xarray.DataArray*) – Concatenated forecast data across models.
- **obs** (*xarray.DataArray*) – Observed rainfall data expanded with a model dimension and masked.
- **scores_organized** (*dict*) – Dictionary of organized scores for selected models.

`wass2s.was_mme.myfill (all_model_fcst, obs)`

Fill missing values in forecast data using random samples from observations.

This function fills NaN values in the forecast data by randomly sampling values from the observed rainfall data along the time dimension.

Parameters

- **all_model_fcst** (*xarray.DataArray*) – Forecast data with dimensions (T, M, Y, X) containing possible NaN values.
- **obs** (*xarray.DataArray*) – Observed rainfall data with dimensions (T, Y, X) used for filling NaNs.

Returns

da_filled_random – Forecast data with NaN values filled using random samples from observations.

Return type

xarray.DataArray

class wass2s.was_mme.WAS_mme_Weighted (*equal_weighted=False, dist_method='nonparam', metric='GROC', threshold=0.5*)

Bases: *object*

Weighted Multi-Model Ensemble (MME) for climate forecasting.

This class implements a weighted ensemble approach for combining multiple climate models, supporting both equal weighting and score-based weighting. It also provides methods for computing tercile probabilities using various statistical distributions.

Parameters

- **equal_weighted** (*bool, optional*) – If True, use equal weights for all models; otherwise, use score-based weights. Default is False.
- **dist_method** (*str, optional*) – Statistical distribution for probability calculations ('t', 'gamma', 'normal', 'lognormal', 'weibull_min', 'nonparam'). Default is 'gamma'.
- **metric** (*str, optional*) – Performance metric for weighting ('MAE', 'Pearson', 'GROC'). Default is 'GROC'.
- **threshold** (*float, optional*) – Threshold for score transformation. Default is 0.5.

__init__ (*equal_weighted=False, dist_method='nonparam', metric='GROC', threshold=0.5*)

Parameters

- **equal_weighted** (*bool*) – If True, use a simple unweighted mean.
- **dist_method** (*str*) – Distribution method (kept for compatibility).
- **metric** (*str*) – Score metric name (e.g., 'MAE', 'Pearson', 'GROC').
- **threshold** (*numeric*) – Threshold value for masking the score.

transform_score (*score_array*)

Transform score array based on the chosen metric and threshold.

For 'MAE', scores below the threshold are set to 1, others to 0. For 'Pearson' or 'GROC', scores above the threshold are set to 1, others to 0.

Parameters

score_array (*xarray.DataArray*) – Score array to transform.

Returns

transformed_score – Transformed score array with binary weights.

Return type

xarray.DataArray

compute (*rainfall, hdcst, fcst, scores, complete=False*)

Compute weighted hindcast and forecast using model scores.

This method calculates weighted averages of hindcast and forecast data based on model scores. If *complete* is True, missing values are filled with unweighted averages.

Parameters

- **rainfall** (*xarray.DataArray*) – Observed rainfall data with dimensions (T, Y, X, M).
- **hdcst** (*xarray.DataArray*) – Hindcast data with dimensions (T, M, Y, X).
- **fcst** (*xarray.DataArray*) – Forecast data with dimensions (T, M, Y, X).
- **scores** (*dict*) – Dictionary mapping model names to score arrays.
- **complete** (*bool, optional*) – If True, fill missing values with unweighted averages. Default is False.

Returns

- **hindcast_det** (*xarray.DataArray*) – Weighted hindcast data with dimensions (T, Y, X).
- **forecast_det** (*xarray.DataArray*) – Weighted forecast data with dimensions (T, Y, X).

static weibull_shape_solver (*k, M, V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess, error_variance, T1, T2, dist_code, dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1, T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in `_ppf_terciles_from_code` - *shape, loc, scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use `best_guess[t]` to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

- $P(B) = F(T1)$
- $P(N) = F(T2) - F(T1)$
- $P(A) = 1 - F(T2)$

Parameters

- **best_guess** (*array-like*) – Forecast/hindcast best estimates.
- **error_variance** (*float*) – Variance of prediction errors.
- **T1** (*float*) – Lower tercile threshold.
- **T2** (*float*) – Upper tercile threshold.
- **dist_code** (*int*) – Distribution code (1-8).
- **shape** (*float*) – Shape parameter.

- **loc** (*float*) – Location parameter.
- **scale** (*float*) – Scale parameter.

Returns

Probabilities [P(B), P(N), P(A)].

Return type

array-like

Notes

- Uses F as the CDF of the predictive distribution.

static calculate_tercile_probabilities_nonparametric (*best_guess, error_samples, first_tercile, second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, best_code_da: DataArray = None, best_shape_da: DataArray = None, best_loc_da: DataArray = None, best_scale_da: DataArray = None*) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If **dist_method == 'bestfit'**:

• Use cluster-based best-fit distributions to:

- derive terciles analytically from (best_code_da, best_shape_da, best_loc_da, best_scale_da),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for 'bestfit'.

Returns

hindcast_prob – Probabilities with dims (probability=['PB','PN','PA'], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, hindcast_det, forecast_det, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

class wass2s.was_mme.WAS_ProbWeighted

Bases: `object`

Probability-Weighted Multi-Model Ensemble for seasonal rainfall forecasting.

Implements a threshold-based weighting scheme for combining multiple climate model outputs (hindcasts and forecasts). Model weights are derived from performance. It's currently for only GROC scores. With a stepwise transformation: $-\leq \text{threshold} \rightarrow \text{weight} = 0$ (excluded) - $\text{threshold} < \text{score} \leq 0.6 \rightarrow \text{weight} = 0.6$ - $0.6 < \text{score} \leq 0.8 \rightarrow \text{weight} = 0.8$ - $> 0.8 \rightarrow \text{weight} = 1.0$

This approach aims to emphasize better-performing models while maintaining some contribution from moderately skilled ones, and completely excluding very poor models.

Parameters

None

Notes

- All input data (hindcast, forecast, scores) are interpolated to the rainfall grid using nearest-neighbor interpolation with extrapolation.
- A spatial mask is derived from the first time step of rainfall observations.
- Supports an optional 'complete' mode that fills missing values with simple unweighted ensemble mean.
- Designed primarily for seasonal climate forecasting applications.

__init__ ()

compute (*rainfall, hdcst, fcst, scores, threshold=0.5, complete=False*)

Compute probability-weighted multi-model ensemble mean for hindcast and forecast.

Parameters

- **rainfall** (*xarray.DataArray*) – Observed rainfall used as reference grid and for masking. Expected dimensions: (T, Y, X, M) or (T, Y, X). The M dimension (if present) is ignored.
- **hdcst** (*xarray.DataArray*) – Multi-model hindcast dataset. Expected dimensions: (T, M, Y, X)
- **fcst** (*xarray.DataArray*) – Multi-model forecast dataset (single lead time). Expected dimensions: (T, M, Y, X)
- **scores** (*dict*) – Dictionary mapping model names (str) to performance score arrays. Each score array should be an *xarray.DataArray* with spatial coordinates (Y, X).
- **threshold** (*float, default=0.5 for GROC*) – Minimum score value below which a model is completely excluded (weight = 0).
- **complete** (*bool, default=False*) – If True, areas where weighted mean is NaN are filled with the simple unweighted ensemble mean.

Returns

- **hindcast_weighted** (*xarray.DataArray*) – Weighted hindcast ensemble mean. Dimensions: (T, Y, X)
- **forecast_weighted** (*xarray.DataArray*) – Weighted forecast ensemble mean. Dimensions: (T, Y, X)

Notes

- Time coordinate of forecast is adjusted to match the expected seasonal month based on the first timestep of rainfall observations.
- All interpolations use ‘nearest’ method with extrapolation for boundary points.
- Final output is masked using the spatial coverage of observed rainfall.
- Models with NaN scores or failed interpolation will contribute zero weight.
- The stepwise weighting is currently hardcoded (0.6 / 0.8 / 1.0 breakpoints).
- It’s only for GROC currently.

Warning

- Make sure that model names in *hdcst.M*, *fcst.M* and *scores.keys()* match exactly.
- Performance may degrade if score grids have very different resolution/spatial extent from the rainfall target grid.

```
class wass2s.was_mme.WAS_Min2009_ProbWeighted (distribution: Literal['gaussian', 'lognormal',
                                                                    'empirical'] = 'gaussian', cv_method:
                                                                    Literal['leave_one_out', 'rolling_window'] | None
                                                                    = None, rolling_window_size: int = 15,
                                                                    n_samples_for_chisq: Literal['total_ensemble',
                                                                    'effective_sample_size'] | float | int =
                                                                    'total_ensemble', eps_lognormal: float = 0.01,
                                                                    sigma_floor: float = 1e-12)
```

Bases: `object`

Min et al. (2009) Probability-Weighted Multi-Model Ensemble (PMME).

What this class does

- 1) For each model compute tercile probabilities (PB / PN / PA) at every grid point from its ensemble forecast using one of three distributions:
 - ‘gaussian’
[Gaussian approximation.] Boundaries : $T_{lo/hi} = \mu_{hcst} \pm 0.4307 * \sigma_{hcst}$ Probability : $\Phi((T_{lo/hi} - f_{mean}) / \sigma_{FORECAST})$ $\sigma_{FORECAST} = \text{std of the current forecast ensemble}$.
 - ‘lognormal’
[Same logic in log-space (better for precipitation).] eps_lognormal is added before taking logs.
 - ‘empirical’
[Category boundaries are the hindcast q33/q66.] Probability = fraction of forecast ensemble members below q33 / above q66.
- 2) Combine per-model probabilities with weights $w_m \propto \sqrt{N_m}$ (Min 2009 eq. 1), normalised to sum to 1.
- 3) Optionally compute a combined categorical map with a chi-square significance test (Min 2009 section 3b, $df = 2$).

Assumed dims

forecasts[m] : at minimum dim 'M' (ensemble members); spatial dims optional. hindcasts[m] : dims 'T' (years) and 'M' (members); same spatial dims.

param distribution

type distribution

'gaussian' | 'lognormal' | 'empirical'

param cv_method

How to compute climatological mu / sigma from the hindcast pool. None -> use the full hindcast pool (no cross-validation).

type cv_method

None | 'leave_one_out' | 'rolling_window'

param rolling_window_size

Half-width of the rolling window when cv_method='rolling_window'.

type rolling_window_size

int

param n_samples_for_chisq

How to compute N for the chi-square test.

type n_samples_for_chisq

'total_ensemble' | 'effective_sample_size' | float

param eps_lognormal

Small offset added before log transform (lognormal path only).

type eps_lognormal

float

param sigma_floor

Any sigma <= this value is replaced by the full-hindcast raw sigma.

type sigma_floor

float

__init__ (distribution: *Literal*['gaussian', 'lognormal', 'empirical'] = 'gaussian', cv_method: *Literal*['leave_one_out', 'rolling_window'] | *None* = None, rolling_window_size: *int* = 15, n_samples_for_chisq: *Literal*['total_ensemble', 'effective_sample_size'] | *float* | *int* = 'total_ensemble', eps_lognormal: *float* = 0.01, sigma_floor: *float* = 1e-12)

compute_pmme_probabilities (forecasts: *Dict*[*str*, *DataArray*], hindcasts: *Dict*[*str*, *DataArray*], climatology: *DataArray* | *None* = None, ensemble_sizes: *Dict*[*str*, *int*] | *None* = None, strict_models: *bool* = True) → *Dict*[*str*, *DataArray*]

Compute PMME probabilities (PB / PN / PA) across models.

Parameters

- **forecasts** ({model_name: *DataArray*(M, [spatial...])})
- **hindcasts** ({model_name: *DataArray*(T, M, [spatial...])})
- **climatology** (unused (kept for pipeline compatibility))
- **ensemble_sizes** ({model_name: *int*} defaults to forecasts[m]. sizes['M'])
- **strict_models** (if True, require forecasts and hindcasts to have the) – same keys.

Return type

dict with keys 'PB', 'PN', 'PA' (each a DataArray over spatial dims).

compute_combined_map (*pmme_probs: Dict[str, DataArray], ensemble_sizes: Dict[str, int], model_names: List[str], significance_level: float = 0.05*) → Tuple[DataArray, DataArray]

Combined categorical map with chi-square significance test. [Min 2009, section 3b]

Returns

- **combined_map** (*DataArray*) – 0 = no significant deviation from equal probabilities, 1 = PB dominant, 2 = PN dominant, 3 = PA dominant.
- **chi_square** (*DataArray*) – chi-square statistic (df = 2).

class wass2s.was_mme.WAS_mme_xcELR (*elm_kwargs=None*)

Bases: *object*

Extended Logistic Regression (ELR) for Multi-Model Ensemble (MME) forecasting derived from xcast package.

This class implements an Extended Logistic Regression for probabilistic forecasting, directly computing tercile probabilities without requiring separate probability calculations.

Parameters

elm_kwargs (*dict, optional*) – Keyword arguments to pass to the xcast ELR model. If None, an empty dictionary is used. Default is None.

__init__ (*elm_kwargs=None*)

compute_model (*X_train, y_train, X_test*)

Compute probabilistic hindcast using the ELR model.

Fits the ELR model on training data and predicts tercile probabilities for the test data. Applies regridding and drymasking to ensure data consistency.

Parameters

- **X_train** (*xarray.DataArray*) – Training predictor data with dimensions (T, M, Y, X).
- **y_train** (*xarray.DataArray*) – Training predictand data with dimensions (T, Y, X).
- **X_test** (*xarray.DataArray*) – Testing predictor data with dimensions (T, M, Y, X).

Returns

result_ – Tercile probabilities with dimensions (probability, T, Y, X), where probability includes ['PB', 'PN', 'PA'] (below-normal, normal, above-normal).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, hindcast_det, Predictor_for_year*)

Generate probabilistic forecast for a target year using the ELR model.

Fits the ELR model on hindcast data and predicts tercile probabilities for the target year. Applies regridding and drymasking to ensure data consistency.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed predictand data with dimensions (T, Y, X) or (T, M, Y, X).
- **clim_year_start** (*int or str*) – Start year of the climatology period (not used in this method).

- **clim_year_end** (*int or str*) – End year of the climatology period (not used in this method).
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast data with dimensions (T, M, Y, X).
- **Predictor_for_year** (*xarray.DataArray*) – Predictor data for the target year with dimensions (T, M, Y, X).

Returns

hindcast_prob – Tercile probabilities with dimensions (probability, T, Y, X), where probability includes ['PB', 'PN', 'PA'] (below-normal, normal, above-normal).

Return type

xarray.DataArray

```
class wass2s.was_mme.WAS_mme_logistic (optimization_method='grid', C_range=[0.1, 1.0, 10.0, 100.0],
                                       solver_options=['newton-cg', 'lbfgs', 'sag', 'saga'],
                                       random_state=42, cv_folds=5, n_clusters=4,
                                       n_iter_search=20, n_trials=50, timeout=None)
```

Bases: *object*

Multinomial Logistic Regression MME with spatial clustering and three hyperparameter-optimization strategies (grid / random / bayesian).

Corrections relative to the previous version

1. Bayesian objective now scores by NEGATIVE LOG LOSS (a proper score for probability forecasts), consistent with the grid/random searches which use `scoring='neg_log_loss'`. The old code used `model.score` (accuracy) despite the comment, so the three methods optimized different criteria.
2. Random search no longer samples invalid penalty/solver pairs: 11 is only paired with 'saga' (like the grid). RandomizedSearchCV accepts a LIST of distributions, mirroring the grid construction.
3. Spatial clustering is kept as in the original: KMeans directly on the predictand values (each cell labelled by its first non-NaN year). Note this is an amplitude binning rather than a regime clustering, but it is retained here by preference.
4. Degenerate clusters are guarded: a cluster with <2 observed classes is skipped during tuning and fitting (LogisticRegression needs >=2 classes).
5. `predict_proba` is mapped onto the fixed [0,1,2] tercile columns via **`model.classes_`**, so a cluster missing a class no longer raises a shape error and never misaligns PB/PN/PA.

Recommended usage / framework

Call `compute_hyperparameters` ONCE on the RAW predictors/predictand, then pass (`best_params`, `cluster_da`) into `compute_model` so they are not recomputed (and so the predictand is not re-classified) inside every CV fold. In `WAS_Cross_Validator` the existing `WAS_mme_logistic` branch already passes the class-coded predictand and the standardized predictor per fold.

```
__init__ (optimization_method='grid', C_range=[0.1, 1.0, 10.0, 100.0], solver_options=['newton-cg', 'lbfgs',
                                             'sag', 'saga'], random_state=42, cv_folds=5, n_clusters=4, n_iter_search=20, n_trials=50,
          timeout=None)
```

```
compute_hyperparameters (Predictors, Predictand, clim_year_start, clim_year_end)
```

Compute best hyperparameters per spatial cluster.

Predictors : (T, M, Y, X) RAW predictor cube. Predictand : (T, Y, X) RAW predictand (classes are computed here).

compute_model (*X_train, y_train, X_test, y_test, clim_year_start, clim_year_end, best_params=None, cluster_da=None*)

Train per-cluster logistic models and predict the test fold.

X_train / X_test : standardized predictor cubes (T, M, Y, X). *y_train* : class-coded predictand (T, Y, X) for training. *best_params, cluster_da* : pass the precomputed ones (recommended). If

omitted they are derived here – but note that requires a RAW predictand for `compute_class`, so prefer passing precomputed values inside cross-validation.

forecast (*Predictant, clim_year_start, clim_year_end, Predictors_train, Predictor_for_year, best_params=None, cluster_da=None*)

Operational forecast for a single target year (classes + tercile probabilities).

get_optimization_results ()

class `wass2s.was_mme.WAS_mme_ELR` (*predictors='mean', threshold_transform='cbrt', l2=0.001, n_iter=50, tol=1e-07, min_obs=5, random_state=42*)

Bases: `object`

Extended Logistic Regression (ELR) multi-model-ensemble tercile forecaster.

Standard logistic-regression MOS fits one model per threshold, which can produce crossing (non-monotone) cumulative probabilities -> negative interval probabilities. ELR (Wilks, 2009, *Meteorol. Appl.*) fixes this by fitting a SINGLE logistic regression whose predictors are augmented with a function of the threshold *q* itself:

$$P(Y \leq q | x) = \text{sigmoid}(b_0 + \beta \cdot x + \alpha \cdot g(q))$$

Because the SAME coefficient *alpha* multiplies *g(q)* for every threshold, the predicted non-exceedance probabilities are monotone in *q* (provided *alpha* > 0, which the data almost always enforce), so the two tercile boundaries yield a coherent below / normal / above split from one fitted model, and probabilities for any threshold can be read off the same fit.

For tercile forecasts the two thresholds are the climatological lower and upper terciles $T_1 < T_2$ of the predictand, and

$$PB = P(Y \leq T_1), PN = P(Y \leq T_2) - P(Y \leq T_1), PA = 1 - P(Y \leq T_2).$$

Predictor at each grid cell

The (T, Y, X, M) predictor cube is reduced per cell to:

predictors='mean' -> multi-model mean (1 feature) [default] *predictors='mean_std'* -> multi-model mean and spread (2 features) *predictors='models'* -> every model kept as its own feature (M features)

Thresholds

Below/normal/above are defined by the climatological terciles. By default they are estimated fold-safe from the TRAINING predictand; pass *clim_terciles=(t1, t2)* (DataArrays on Y, X, same units as *y_train*) to use fixed climatological boundaries identical to those your verification uses (recommended for exact consistency with `WAS_Verification.compute_class`).

Fitting

One logistic regression per cell, but ALL cells are fitted simultaneously with a vectorized, ridge-stabilized Newton/IRLS solver (no per-cell sklearn loop), so it scales to large grids. Rows with missing predictor/predictand are down-weighted to zero, so partial gaps never break the batch. Cells with too few valid years fall back to climatology [1/3, 1/3, 1/3].

param predictors

type predictors

{‘mean’, ‘mean_std’, ‘models’}

param threshold_transform

$g(q)$ applied to the tercile boundaries (default identity). On standardized anomalies the identity is appropriate; for raw precipitation one might pass e.g. `np.cbrt`.

type threshold_transform

callable or None

param l2

Ridge penalty on the coefficients (NOT the intercept) for numerical stability / quasi-separation (default $1e-3$).

type l2

float

param n_iter

Maximum IRLS iterations (default 50).

type n_iter

int

param tol

Early-stop tolerance on the max coefficient update (default $1e-7$).

type tol

float

param min_obs

Minimum number of valid training years for a cell to be fitted; otherwise that cell is set to climatology (default 5).

type min_obs

int

__init__ (*predictors=‘mean’, threshold_transform=‘cbrt’, l2=0.001, n_iter=50, tol=1e-07, min_obs=5, random_state=42*)

compute_model (*X_train, y_train, X_test, y_test=None, clim_year_start=None, clim_year_end=None, clim_terciles=None, best_params=None, **kwargs*)

Fit ELR on the training fold and return the test-fold tercile probabilities (dims probability, T, Y, X; probability=[‘PB’, ‘PN’, ‘PA’]).

X_train / *X_test* : standardized predictor cubes (T, Y, X[, M]). *y_train* : predictand (T, Y, X). Used to set the tercile

thresholds (fold-safe) unless *clim_terciles* is given. Keep it RAW (un-standardized): the fit is invariant to predictand scaling, and raw values let $g(q)$ act in physical units (so $g=\text{‘sqrt’}$ is valid for precipitation).

clim_terciles

[optional (t1, t2) DataArrays (Y, X) of fixed] climatological boundaries in the SAME units as *y_train* (i.e. physical/raw) – pass the same terciles your verification uses.

best_params

[optional {‘l2’: value, ‘g’: name}. If given, overrides] the penalty and/or $g(q)$ transform for this call; otherwise *self.l2* / *self.threshold_transform* are used (which *compute_hyperparameters* updates in place).

compute_hyperparameters (*Predictor, Predictand, clim_year_start, clim_year_end, clim_terciles=None, l2_grid=None, g_grid=None, score='rps', cv_folds=5, leave_one_year_out=False*)

Jointly tune the ridge penalty $l2$ AND the threshold transform $g(q)$ by year-grouped cross-validation. The winners are stored in `self.l2` and `self.threshold_transform` (and `self.best_params`) so the existing CV branch / `compute_model` use them with no further plumbing.

Why tune $g(q)$: Wilks (2009) shows the FORM of $g(q)$ drives ELR quality and must be chosen per setting ($g(q)=b2*\sqrt{q}$ was substantially better than $g(q)=b2*q$ for raw precipitation). Here we let the data pick g among non-decreasing options, so the cumulative $p(q)=Pr\{V\leq q\}$ stays monotone.

Observed truth = tercile CLASSES (like the logistic class)

Forecast probabilities are scored against `WAS_Verification.compute_class` (0=below= $Pr\{V\leq q/3\}$, 1=normal, 2=above) – the categorical target the RPSS verification uses. RPS by default, or multiclass log-loss.

Inputs / standardization: only the PREDICTOR is standardized (over the clim window) for $f(x)$; the PREDICTAND stays RAW so the thresholds and $g(q)$ are in physical units (the fit is invariant to predictand scaling). `compute_class` is computed on the raw predictand.

param l2_grid

type l2_grid

sequence of float, optional. Default `np.logspace(-4, 1, 6)`.

param g_grid

Default ['identity', 'cbrt'] (+ 'sqrt' only when all thresholds are non-negative, i.e. a raw non-negative predictand).

type g_grid

sequence of {'identity', 'cbrt', 'sqrt'} or callables, optional.

param score

type score

{'rps', 'log_loss'} – proper score MINIMIZED over the folds.

param cv_folds

type cv_folds

year-grouped CV controls.

param leave_one_year_out

type leave_one_year_out

year-grouped CV controls.

param clim_terciles

the same fixed boundaries your CV branch passes to `compute_model`. If None, fold-safe raw terciles are estimated per training fold.

type clim_terciles

optional (t1, t2) in PHYSICAL (raw predictand) units –

returns

best_params

rtype

dict {'l2': best_l2, 'g': best_g_name}

forecast (*Predictand, clim_year_start, clim_year_end, Predictor, Predictor_for_year, clim_terciles=None, **kwargs*)

Operational ELR tercile forecast for a single target year.

Fits ELR on all available years and returns the tercile probabilities for the target year (dims probability, T, Y, X). Only the PREDICTOR is standardized (historical climatology, same stats applied to the forecast year); the PREDICTAND is kept RAW so the thresholds and g(q) stay in physical units (the fit is invariant to predictand scaling).

```
class wass2s.was_mme.WAS_mme_gaussian_process (random_state=42, cv_folds=5, n_clusters=4,
                                                kernel_options=None,
                                                n_restarts_optimizer_options=(0, 2),
                                                max_iter_predict_options=(100, 200),
                                                max_train_samples=1500, hpo_method='grid',
                                                n_random_iter=10, n_trials=20, timeout=None,
                                                n_jobs=-1, warm_start=False)
```

Bases: `object`

Gaussian Process Classifier MME (multiclass via One-vs-Rest in sklearn).

Rigorous scientific implementation:

- Spatial clustering based on historical predictand coherence.
- Spatio-temporal subsampling.
- **Grouped Cross-Validation (by Time)** to prevent spatial-autocorrelation data leakage.

```
__init__ (random_state=42, cv_folds=5, n_clusters=4, kernel_options=None,
          n_restarts_optimizer_options=(0, 2), max_iter_predict_options=(100, 200),
          max_train_samples=1500, hpo_method='grid', n_random_iter=10, n_trials=20, timeout=None,
          n_jobs=-1, warm_start=False)
```

```
compute_hyperparameters (Predictors, Predictand, clim_year_start, clim_year_end)
```

Computes best hyperparameters. Uses rigorous GroupKFold by Time.

```
compute_model (X_train, y_train, X_test, y_test, clim_year_start, clim_year_end, best_params=None,
               cluster_da=None)
```

Fit models on train set and predict on test set.

```
forecast (Predictant, clim_year_start, clim_year_end, Predictors_train, Predictor_for_year,
          best_params=None, cluster_da=None)
```

Train on entire history, forecast for 1 new time slice.

```
get_hpo_summary ()
```

```
class wass2s.was_mme.WAS_mme_xcELM (elm_kwargs=None, dist_method='gamma')
```

Bases: `object`

Extreme Learning Machine (ELM) for Multi-Model Ensemble (MME) forecasting derived from xcast.

This class implements an Extreme Learning Machine model for deterministic forecasting, with optional tercile probability calculations using various statistical distributions.

Parameters

- **elm_kwargs** (*dict*, *optional*) – Keyword arguments to pass to the xcast ELM model. If None, default parameters are used: {'regularization': 10, 'hidden_layer_size': 5, 'activation': 'lin', 'preprocessing': 'none', 'n_estimators': 5}. Default is None.
- **dist_method** (*str*, *optional*) – Distribution method for tercile probability calculations ('t', 'gamma', 'nonparam', 'normal', 'lognormal', 'weibull_min'). Default is 'gamma'.

```
__init__ (elm_kwargs=None, dist_method='gamma')
```

compute_model (*X_train*, *y_train*, *X_test*)

Compute deterministic hindcast using the ELM model.

Fits the ELM model on training data and predicts deterministic values for the test data. Applies regridding and drymasking to ensure data consistency.

Parameters

- **X_train** (*xarray.DataArray*) – Training predictor data with dimensions (T, M, Y, X).
- **y_train** (*xarray.DataArray*) – Training predictand data with dimensions (T, Y, X).
- **X_test** (*xarray.DataArray*) – Testing predictor data with dimensions (T, M, Y, X).

Returns

result_ – Deterministic hindcast with dimensions (T, Y, X).

Return type

xarray.DataArray

static weibull_shape_solver (*k*, *M*, *V*)

Function to find the root of the Weibull shape parameter 'k'. We find 'k' such that the theoretical variance/mean^2 ratio matches the observed V/M^2 ratio.

static calculate_tercile_probabilities_bestfit (*best_guess*, *error_variance*, *T1*, *T2*, *dist_code*, *dof*)

Generic tercile probabilities using best-fit family per grid cell.

Inputs (per grid cell): - *best_guess* : 1D array over T (hindcast_det or forecast_det) - *T1*, *T2* : scalar terciles from climatological best-fit distribution - *dist_code* : int, as in *_ppf_terciles_from_code* - *shape*, *loc*, *scale* : scalars from climatology fit

Strategy: - For each time step, build a predictive distribution of the same family:

- Use *best_guess[t]* to adjust mean / location;
- Keep shape parameters from climatology.

•Then compute probabilities:

$P(B) = F(T1)$, $P(N) = F(T2) - F(T1)$, $P(A) = 1 - F(T2)$.

static calculate_tercile_probabilities_nonparametric (*best_guess*, *error_samples*, *first_tercile*, *second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant*: *DataArray*, *clim_year_start*, *clim_year_end*, *hindcast_det*: *DataArray*, *best_code_da*: *DataArray* = None, *best_shape_da*: *DataArray* = None, *best_loc_da*: *DataArray* = None, *best_scale_da*: *DataArray* = None) → *DataArray*

Compute tercile probabilities for deterministic hindcasts.

If *dist_method* == 'bestfit':

•Use cluster-based best-fit distributions to:

- derive terciles analytically from (*best_code_da*, *best_shape_da*, *best_loc_da*, *best_scale_da*),
- compute predictive probabilities using the same family.

Otherwise:

- Use empirical terciles from Predictant climatology and the selected parametric / nonparametric method.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed data (T, Y, X) or (T, Y, X, M).
- **clim_year_start** (*int or str*) – Climatology period (inclusive) for thresholds.
- **clim_year_end** (*int or str*) – Climatology period (inclusive) for thresholds.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast (T, Y, X).
- **best_code_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_shape_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_loc_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.
- **best_scale_da** (*xarray.DataArray, optional*) – Output from WAS_TransformData.fit_best_distribution_grid, required for ‘bestfit’.

Returns

hindcast_prob – Probabilities with dims (probability=[‘PB’, ‘PN’, ‘PA’], T, Y, X).

Return type

xarray.DataArray

forecast (*Predictant, clim_year_start, clim_year_end, hindcast_det, hindcast_det_cross_val, Predictor_for_year, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Generate deterministic and probabilistic forecast for a target year using the ELM model.

Fits the ELM model on hindcast data, predicts deterministic values for the target year, and computes tercile probabilities. Applies regridding, drymasking, and standardization.

Parameters

- **Predictant** (*xarray.DataArray*) – Observed predictand data with dimensions (T, Y, X) or (T, M, Y, X).
- **clim_year_start** (*int or str*) – Start year of the climatology period.
- **clim_year_end** (*int or str*) – End year of the climatology period.
- **hindcast_det** (*xarray.DataArray*) – Deterministic hindcast data for training with dimensions (T, M, Y, X).
- **hindcast_det_cross_val** (*xarray.DataArray*) – Deterministic hindcast data for error estimation with dimensions (T, Y, X).
- **Predictor_for_year** (*xarray.DataArray*) – Predictor data for the target year with dimensions (T, M, Y, X).

Returns

- **result_** (*xarray.DataArray*) – Deterministic forecast with dimensions (T, Y, X).
- **hindcast_prob** (*xarray.DataArray*) – Tercile probabilities with dimensions (probability, T, Y, X), where probability includes [‘PB’, ‘PN’, ‘PA’] (below-normal, normal, above-normal).

`wass2s.was_mme.censored_normal_mean_var(mu, var, dry_threshold=0.0)`

Mean and variance of $Y = \max(X, c)$ with $X \sim N(\mu, \text{var})$, $c = \text{dry_threshold}$.

Correct for ANY threshold (the original code assumed $c = 0$). Validated against $4e6$ -sample Monte-Carlo to $<1e-2$ absolute in mean and var over μ in $[-1, 5]$, sigma in $[1, 2]$, c in $[0, 2]$.

$$E[Y] = c * \Phi(\alpha) + \mu * (1 - \Phi(\alpha)) + \sigma * \phi(\alpha) \quad E[Y^2] = c^2 * \Phi(\alpha) + (\mu^2 + \sigma^2) * (1 - \Phi(\alpha))$$

$$\bullet \sigma * (\mu + c) * \phi(\alpha)$$

$$\alpha = (c - \mu) / \sigma$$

`wass2s.was_mme.truncated_normal_mean_var(mu, sigma, dry_threshold=0.0)`

Mean and variance of $X \mid X > c$, $X \sim N(\mu, \sigma^2)$. (Unchanged from your code — it was already correct; included for completeness.)

`wass2s.was_mme.crps_gaussian(y, mu, sigma)`

Analytic CRPS for $N(\mu, \sigma^2)$ (Gneiting et al. 2005). Validated to $<1e-4$ vs quadrature.

`wass2s.was_mme.crps_censored_normal(y, mu, sigma, c=0.0)`

CRPS for $Y = \max(X, c)$, $X \sim N(\mu, \sigma^2)$, observation $y \geq c$.

Closed form (Jordan, Krüger & Lerch 2019, eq. for `crps_cnorm`, lower= c , upper= $+\infty$). Validated against brute-force quadrature to $<2e-5$.

`wass2s.was_mme.crps_truncated_normal(y, mu, sigma, c=0.0)`

CRPS for $X \mid X > c$, $X \sim N(\mu, \sigma^2)$, observation $y > c$.

Closed form (Thorarinsdottir & Gneiting 2010; `crps_tnorm`, lower= c , upper= $+\infty$). Validated against brute-force quadrature to $<2e-5$.

`wass2s.was_mme.fit_one_grid_min_crps(fbar, svar, y, family='censored_normal', dry_threshold=0.0, var_link='linear', init=None, bounds=None)`

Minimum-CRPS EMOS fit at a single grid point.

Parameters

- **fbar** (1D arrays over time (ensemble mean, ensemble variance, obs))
- **svar** (1D arrays over time (ensemble mean, ensemble variance, obs))
- **y** (1D arrays over time (ensemble mean, ensemble variance, obs))
- **family** ('gaussian' | 'censored_normal' | 'truncated_normal')
- **dry_threshold** (censoring/truncation point c (precip families))
- **var_link** ('linear' ($\sigma^2 = c + d * \text{svar}$) or 'softplus')
- **(params[a** (Returns)
- **b**
- **c**
- **d]**
- **wet** (info_dict). For truncated_normal only the)
- **score** (observations ($y > c$) enter the)

- **likelihood** (*matching the conditional*)
- **(2010)** . (*used in Thorarinsdottir & Gneiting*)

```
class wass2s.was_mme.WAS_mme_NGR_Gaussian (apply_to='all', alpha=0.1, test_direction='two-sided',
                                             bounds=None)
```

Bases: `object`

Canonical Gaussian EMOS / NGR with gridwise fitting.

Model

$$Y_t | f_t \sim N(\mu_t, \sigma_t^2)$$
$$\mu_t = a + b * \bar{f}_t \quad \sigma_t^2 = c + d * s_t^2$$

where

$\bar{f}_t$ = ensemble mean s_t^2 = ensemble variance

This class is intended for predictands that are approximately Gaussian after anomaly/standardization preprocessing. For nonnegative or zero-inflated variables such as precipitation totals, prefer a truncated, censored, gamma, or transformed variant instead of plain Gaussian EMOS.

```
__init__ (apply_to='all', alpha=0.1, test_direction='two-sided', bounds=None)
```

```
fit (hcst_grid, obs_grid, clim_terciles=False, member_dim='M', time_dim='T', lat_dim='Y', lon_dim='X')
```

```
predict (fcst_grid, quantiles=None, clim_terciles=False, return_synthetic_ensemble=False, member_dim='M',
          time_dim='T', lat_dim='Y', lon_dim='X')
```

```
transform (fcst_grid, **kwargs)
```

```
compute_model (X_train, y_train, X_test, obs_for_terciles=None, quantiles=None, clim_terciles=False,
                 member_dim='M', time_dim='T', lat_dim='Y', lon_dim='X',
                 return_synthetic_ensemble=False)
```

```
forecast (fcst_grid, quantiles=None, clim_terciles=False, return_synthetic_ensemble=False, member_dim='M',
           time_dim='T', lat_dim='Y', lon_dim='X')
```

```
get_fit_stats ()
```

```
summary ()
```

```
class wass2s.was_mme.WAS_mme_NGR_NonGaussian (family='censored_normal',
                                              occurrence_model='auto', dry_threshold=0.0,
                                              transform='boxcox', transform_shift=1e-06,
                                              alpha=0.1, apply_to='all', bounds=None)
```

Bases: `object`

Precipitation-oriented EMOS-style postprocessor with optional predictive families:

- family='censored_normal'
- family='truncated_normal'
- family='gamma'
- family='transformed_gaussian'

Notes

- 1) Designed for nonnegative predictands such as precipitation.
- 2) For families without an intrinsic atom at zero, a hurdle occurrence model is used by default:

$$P(Y = 0) = p_0 \quad Y \mid Y > 0 \sim \text{positive family}$$
- 3) Fitting uses log-likelihood rather than analytic CRPS, so one coherent implementation can cover all families.

```
VALID_FAMILIES = {'censored_normal', 'gamma', 'transformed_gaussian',
                  'truncated_normal'}
```

```
VALID_OCC = {'auto', 'climatology', 'logistic', 'none'}
```

```
__init__(family='censored_normal', occurrence_model='auto', dry_threshold=0.0, transform='boxcox',
          transform_shift=1e-06, alpha=0.1, apply_to='all', bounds=None)
```

```
fit(hcst_grid, obs_grid, clim terciles=False, member_dim='M', time_dim='T', lat_dim='Y', lon_dim='X')
```

```
predict(fcst_grid, quantiles=None, clim terciles=False, return_synthetic_ensemble=False, member_dim='M',
         time_dim='T', lat_dim='Y', lon_dim='X')
```

```
forecast(fcst_grid, quantiles=None, clim terciles=False, return_synthetic_ensemble=False, member_dim='M',
          time_dim='T', lat_dim='Y', lon_dim='X')
```

```
compute_model(X_train, y_train, X_test, obs_for_terciles=None, quantiles=None, clim terciles=False,
               member_dim='M', time_dim='T', lat_dim='Y', lon_dim='X',
               return_synthetic_ensemble=False)
```

```
get_fit_stats()
```

```
summary()
```

```
class wass2s.was_mme.WAS_mme_MVA(dist_method: Literal['bestfit', 'nonparam'] = 'nonparam')
```

Bases: `object`

Method 1 of Torralba et al. (2017) (MVA): rescales ensemble to match obs mean & std. Assumes both reference and predicted distributions are approximately Gaussian.

Expected dims:

hindcast: (M, T, Y, X) obs : (T, Y, X) forecast: (M, T, Y, X)

```
- fit(hindcast, obs)
```

```
- transform(forecast)
```

```
- fit_transform_loocv(hindcast, obs)
```

```
- compute_prob(Predictant, clim_year_start, clim_year_end, hindcast_det, ..
               .)
```

```
- forecast(Predictant, clim_year_start, clim_year_end, hindcast_det,
           forecast_det, ...)
```

```
__init__(dist_method: Literal['bestfit', 'nonparam'] = 'nonparam')
```

```
fit(hindcast: DataArray, obs: DataArray) → WAS_mme_MVA
```

```
transform(forecast: DataArray) → DataArray
```

fit_transform_loocv (*hindcast: DataArray, obs: DataArray*) → DataArray

predict_mean (*forecast: DataArray*) → DataArray

Calibrate members then return ensemble mean over M: (T,Y,X).

compute_prob (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_cross: DataArray, best_code_da: DataArray | None = None, best_shape_da: DataArray | None = None, best_loc_da: DataArray | None = None, best_scale_da: DataArray | None = None*) → DataArray

Compute tercile probabilities for deterministic hindcasts.

If dist_method == 'bestfit':

- derive climatological terciles analytically from (best_code/shape/loc/scale) and compute predictive probabilities with the same family.

Else ('nonparam'):

- use empirical terciles and historical error samples.

forecast (*Predictant: DataArray, clim_year_start, clim_year_end, hindcast_det: DataArray, hindcast_cross: DataArray, forecast_det: DataArray, best_code_da: DataArray | None = None, best_shape_da: DataArray | None = None, best_loc_da: DataArray | None = None, best_scale_da: DataArray | None = None*)

Calibrate forecast and return (calibrated_deterministic_mean, tercile_probs).

class wass2s.was_mme.WAS_mme_FastBMA (*model_type: str = 'normal', tol: float = 0.001*)

Bases: object

Grid-based Bayesian Model Averaging — SLSQP maximum-likelihood fitting.

Supported model_type values

'normal'

Gaussian BMA (Raftery et al. 2005). Component: $N(f_k, \sigma^2)$ where f_k is the k -th raw ensemble member and σ is a single shared standard deviation fitted by maximum likelihood.

'gamma'

Gamma mixture for strictly positive quantities (wind speed, etc.). Parameterisation: $\Gamma(\text{shape}, \text{scale} = f_k / \text{shape})$ so that $E[Y | k] = f_k$. A single shared shape parameter is estimated.

'gamma0'

Zero-adjusted Gamma mixture for precipitation (Sloughter et al. 2007, §2). The marginal model is

$$P(Y = 0) = 1 - \text{POP} \quad P(Y > 0) = \text{POP} \cdot \Gamma\text{-mixture}(\text{shape}, f_k / \text{shape})$$

where POP is estimated via logistic regression on three predictors: intercept, $\sqrt[3]{\text{ens_mean}}$, and fraction of wet ensemble members (Sloughter et al. 2007 Eq. 1).

Tercile thresholds

Provide either `clim_terciles` (dims: ('tercile', Y, X)) or `obs_for_terciles` (dims include time_dim, Y, X); terciles are then computed per gridpoint.

__init__ (*model_type: str = 'normal', tol: float = 0.001*) → None

static compute_gridpoint_terciles_from_obs (*obs: DataArray, *, time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', q: Tuple[float, float] = (0.3333333333333333, 0.6666666666666666)*) → DataArray

Compute per-gridpoint tercile thresholds from a climatological sample.

Returns

`dims: ('tercile', lat_dim, lon_dim); tercile ∈ ['lower', 'upper']`.

Return type

`xr.DataArray`

fit (*hcst_grid: DataArray, obs_grid: DataArray, *, member_dim: str = 'M', time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', min_samples: int = 10*) → *WAS_mme_FastBMA*

Fit BMA parameters at every gridpoint via SLSQP maximum-likelihood.

Parameters

- **hcst_grid** (*dims (member_dim, time_dim, lat_dim, lon_dim)*)
- **obs_grid** (*dims (time_dim, lat_dim, lon_dim)*)

predict_probabilistic (*new_forecasts: DataArray, *, clim_terciles: DataArray | None = None, obs_for_terciles: DataArray | None = None, tercile_q: Tuple[float, float] = (0.3333333333333333, 0.6666666666666666), quantiles: Sequence[float] = (0.1, 0.5, 0.9), member_dim: str = 'M', time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X'*) → *Dataset*

Generate probabilistic forecasts from the fitted BMA mixture.

Parameters

- **new_forecasts** (*dims (member_dim, time_dim, lat_dim, lon_dim)*)
- **clim_terciles** (*dims ('tercile', lat_dim, lon_dim) [optional]*)
- **obs_for_terciles** (*dims (time_dim, lat_dim, lon_dim) [optional]*)

Returns

`predictive_mean` — (T, Y, X) `predictive_quantiles` — ('quantile', T, Y, X) `tercile_probability` — ('probability', T, Y, X) [if terciles given] `tercile_thresholds` — ('tercile', Y, X) [if terciles given]

Return type

`xr.Dataset`

class `wass2s.was_mme.WAS_mme_FullBMA` (*mode: str = 'auto', eps: float = 1e-06, tol: float = 1e-05, maxiter: int = 300, draws: int = 1000, tune: int = 1000, chains: int = 2, target_accept: float = 0.92, random_seed: int = 42, progressbar: bool = True, verbose: bool = True*)

Bases: `object`

Hybrid Bayesian seasonal ensemble postprocessor — per-gridpoint fitting.

Distribution families (per gridpoint, supplied via `dist_map`)

Code 1 → 'normal' Gaussian BMA with bias-correction Code 4 → 'gamma' Gamma BMA (positive-definite variables) Code 2 → 'lognormal' Lognormal BMA (heavy-tailed precipitation, etc.)

Each component distribution has its own intercept `a_k` and slope `b_k` so that the component mean is a linear (or log-linear for lognormal) function of the member forecast — a mixture-EMOS formulation.

Fitting modes

'fast' — Penalised L-BFGS-B optimisation (always available). 'full' — Full MCMC via PyMC (requires pymc + arviz). 'auto' — Tries MCMC first, falls back to L-BFGS-B on failure.

Notes

- Gridpoints where `dist_map` is NaN are skipped.
- Positive-definite families (gamma, lognormal) are fitted on $y > 0$ only.
- `predict_probabilistic` spatially aligns stored parameter maps to the forecast grid via nearest-neighbour selection, so training and test grids need not be identical.

```
DIST_CODE_TO_NAME = {1: 'normal', 2: 'lognormal', 4: 'gamma'}
```

```
DIST_NAME_TO_CODE = {'gamma': 4, 'lognormal': 2, 'normal': 1}
```

```
MODE_CODE = {'failed': -1, 'fast': 1, 'full': 0}
```

```
__init__(mode: str = 'auto', eps: float = 1e-06, tol: float = 1e-05, maxiter: int = 300, draws: int = 1000, tune: int = 1000, chains: int = 2, target_accept: float = 0.92, random_seed: int = 42, progressbar: bool = True, verbose: bool = True) → None
```

```
static compute_gridpoint_terciles_from_obs(obs: DataArray, *, time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', q: Tuple[float, float] = (0.3333333333333333, 0.6666666666666666)) → DataArray
```

```
fit(hcst_grid: DataArray, obs_grid: DataArray, dist_map: DataArray, *, member_dim: str = 'M', time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', min_samples: int = 12, lam_w: float = 1.0, lam_a: float = 0.1, lam_b: float = 0.1, lam_disp: float = 0.1, alpha0: float = 1.0, coef_scale_mult: float = 1.0, sigma_scale_mult: float = 1.0, shape_scale: float = 5.0) → WAS_mme_FullBMA
```

```
predict_probabilistic(new_forecasts: DataArray, *, clim_terciles: DataArray | None = None, obs_for_terciles: DataArray | None = None, tercile_q: Tuple[float, float] = (0.3333333333333333, 0.6666666666666666), quantiles: Sequence[float] = (0.1, 0.5, 0.9), n_pp_samples: int = 2000, member_dim: str = 'M', time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X') → Dataset
```

```
compute_model(X_train: DataArray, y_train: DataArray, X_test: DataArray, dist_map: DataArray, *, clim_terciles: DataArray | None = None, obs_for_terciles: DataArray | None = None, tercile_q: Tuple[float, float] = (0.3333333333333333, 0.6666666666666666), quantiles: Sequence[float] = (0.1, 0.5, 0.9), n_pp_samples: int = 2000, member_dim: str = 'M', time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', min_samples: int = 12, lam_w: float = 1.0, lam_a: float = 0.1, lam_b: float = 0.1, lam_disp: float = 0.1, alpha0: float = 1.0, coef_scale_mult: float = 1.0, sigma_scale_mult: float = 1.0, shape_scale: float = 5.0) → Dataset
```

Fit on (X_train, y_train) then predict on X_test — one-call interface.

```
class wass2s.was_mme.WAS_mme_RF (search_method: str = 'random', n_estimators_range: List[int] \ object =
    [100, 200, 300, 500], max_depth_range: List[int \ None] \ object =
    [None, 8, 12, 16], min_samples_split_range: List[int] \ object = [2, 5,
    10], min_samples_leaf_range: List[int] \ object = [1, 2, 4],
    max_features_range: List[str \ float] = ['sqrt', 0.3, 0.5, 0.7],
    bootstrap_range: List[bool] = [True, False], max_samples_range:
    List[float \ None] = [None, 0.7, 0.8], min_impurity_decrease_range:
    List[float] = [0.0, 0.001, 0.01], ccp_alpha_range: List[float] = [0.0,
    0.001, 0.01], warm_start: bool = False, random_state: int = 42,
    dist_method: str = 'nonparam', n_iter_search: int = 20, cv_folds: int =
    5, n_clusters: int = 6, scoring: str = 'neg_mean_squared_error',
    leave_one_year_out: bool = False, verbose: int = 0)
```

Bases: `object`

Enhanced Random Forest-based Multi-Model Ensemble (MME) forecasting for West Africa.

Parameters

- **search_method** (*str*, *optional*) – Hyperparameter optimization method: 'grid', 'random', or 'bayesian' (default: 'random').
- **n_estimators_range** (*list of int*, *optional*) – List of `n_estimators` values to tune (default is [100, 200, 300, 500]).
- **max_depth_range** (*list of int or None*, *optional*) – List of max depths to tune (default is [None, 8, 12, 16]).
- **min_samples_split_range** (*list of int*, *optional*) – List of minimum samples required to split (default is [2, 5, 10]).
- **min_samples_leaf_range** (*list of int*, *optional*) – List of minimum samples required at leaf node (default is [1, 2, 4]).
- **max_features_range** (*list of str or float*, *optional*) – List of max features to tune (default is ['sqrt', 0.3, 0.5, 0.7]).
- **bootstrap_range** (*list of bool*, *optional*) – List of bootstrap options to tune (default is [True, False]).
- **max_samples_range** (*list of float or None*, *optional*) – List of max samples as fraction of dataset (default is [None, 0.7, 0.8]).
- **min_impurity_decrease_range** (*list of float*, *optional*) – List of min impurity decrease values (default is [0.0, 0.001, 0.01]).
- **max_leaf_nodes_range** (*list of int or None*, *optional*) – List of max leaf nodes (default is [None, 50, 100]).
- **ccp_alpha_range** (*list of float*, *optional*) – List of complexity parameters for pruning (default is [0.0, 0.001, 0.01]).
- **min_weight_fraction_leaf_range** (*list of float*, *optional*) – List of minimum weighted fraction of leaves (default is [0.0, 0.1, 0.2]).
- **warm_start** (*bool*, *optional*) – Whether to reuse solution of previous call to fit (default is False).
- **random_state** (*int*, *optional*) – Seed for reproducibility (default is 42).
- **dist_method** (*str*, *optional*) – Distribution method for tercile probabilities ('bestfit', 'nonparam', etc.) (default is 'nonparam').

- **n_iter_search** (*int*, *optional*) – Number of iterations for randomized/bayesian search (default is 20).
- **cv_folds** (*int*, *optional*) – Number of cross-validation folds (default is 5).
- **n_clusters** (*int*, *optional*) – Number of clusters for homogenized zones (default is 6 for West Africa).
- **scoring** (*str*, *optional*) – Scoring metric for hyperparameter optimization (default is 'neg_mean_squared_error').
- **verbose** (*int*, *optional*) – Verbosity level (default is 0).

__init__ (*search_method*: *str* = 'random', *n_estimators_range*: *List[int]* | *object* = [100, 200, 300, 500], *max_depth_range*: *List[int | None]* | *object* = [None, 8, 12, 16], *min_samples_split_range*: *List[int]* | *object* = [2, 5, 10], *min_samples_leaf_range*: *List[int]* | *object* = [1, 2, 4], *max_features_range*: *List[str | float]* = ['sqrt', 0.3, 0.5, 0.7], *bootstrap_range*: *List[bool]* = [True, False], *max_samples_range*: *List[float | None]* = [None, 0.7, 0.8], *min_impurity_decrease_range*: *List[float]* = [0.0, 0.001, 0.01], *ccp_alpha_range*: *List[float]* = [0.0, 0.001, 0.01], *warm_start*: *bool* = False, *random_state*: *int* = 42, *dist_method*: *str* = 'nonparam', *n_iter_search*: *int* = 20, *cv_folds*: *int* = 5, *n_clusters*: *int* = 6, *scoring*: *str* = 'neg_mean_squared_error', *leave_one_year_out*: *bool* = False, *verbose*: *int* = 0)

compute_hyperparameters (*Predictors*: *DataArray*, *Predictand*: *DataArray*, *clim_year_start*: *int* | *None* = *None*, *clim_year_end*: *int* | *None* = *None*) → *Tuple[Dict, DataArray]*

Compute best hyperparameters per zone. Regionalization is KMeans on the predictand values (kept by preference); CV is year-grouped (2).

If *clim_year_start*/*clim_year_end* are given, predictors and predictand are standardized here. If they are *None*, the inputs are assumed already standardized – e.g. when called from *WAS_Cross_Validator*, which standardizes before the fold loop. This removes the old hard-coded 1970-2000 window AND the double-standardization that happened when the framework had already standardized the data.

compute_model (*X_train*: *DataArray*, *y_train*: *DataArray*, *X_test*: *DataArray*, *y_test*: *DataArray* | *None* = *None*, *clim_year_start*: *int* | *None* = *None*, *clim_year_end*: *int* | *None* = *None*, *best_params*: *Dict* | *None* = *None*, *cluster_da*: *DataArray* | *None* = *None*) → *DataArray*

Deterministic hindcast using *RandomForestRegressor* with optimized hyperparameters.

static weibull_shape_solver (*k*, *M*, *V*)

Root function for the Weibull shape parameter 'k'.

static calculate_tercile_probabilities_bestfit (*best_guess*, *error_variance*, *T1*, *T2*, *dist_code*, *dof*)

Generic tercile probabilities using best-fit family per grid cell.

static calculate_tercile_probabilities_nonparametric (*best_guess*, *error_samples*, *first_tercile*, *second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant*: *DataArray*, *clim_year_start*, *clim_year_end*, *hindcast_det*: *DataArray*, *best_code_da*: *DataArray* = *None*, *best_shape_da*: *DataArray* = *None*, *best_loc_da*: *DataArray* = *None*, *best_scale_da*: *DataArray* = *None*) → *DataArray*

Tercile probabilities for deterministic hindcasts.

NOTE (3): for honest, well-calibrated probabilities, pass the CROSS-VALIDATED hindcast here as *hindcast_det* (out-of-sample residuals). Feeding the in-sample hindcast makes the predictive distribution too sharp and the reliability diagram over-confident.

forecast (*Predictant, clim_year_start, clim_year_end, hindcast_det, hindcast_det_cross, Predictor_for_year, best_params=None, cluster_da=None, best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None*)

Forecast using the Random Forest model with optimized hyperparameters.

```
class wass2s.was_mme.WAS_mme_XGBoosting (search_method='random', n_estimators_range=[50, 100, 150, 200], learning_rate_range=[0.005, 0.01, 0.03, 0.05, 0.1], max_depth_range=[3, 5, 7], min_child_weight_range=[1, 3, 5], subsample_range=[0.6, 0.8, 1.0], colsample_bytree_range=[0.6, 0.8, 1.0], gamma_range=[0, 0.1, 0.2, 0.5, 1], reg_alpha_range=[0, 0.01, 0.1], reg_lambda_range=[1, 1.5, 2], early_stopping_rounds_range=[None, 10, 20], es_validation_fraction=0.15, random_state=42, dist_method='nonparam', n_iter_search=10, cv_folds=3, n_clusters=4, leave_one_year_out=False, optuna_n_jobs=1, optuna_timeout=None, verbose=0)
```

Bases: `object`

XGBoost-based Multi-Model Ensemble (MME) forecasting for seasonal rainfall. Enhanced with additional regularization hyperparameters and time-series CV.

Parameters

- **search_method** (*str, optional*) – Hyperparameter optimization method: 'grid', 'random', or 'bayesian' (default: 'random').
- **n_estimators_range** (*list of int or scipy.stats distribution, optional*) – List of n_estimators values to tune (optimized for climate data).
- **learning_rate_range** (*list of float or scipy.stats distribution, optional*) – List of learning rates to tune (optimized for climate data).
- **max_depth_range** (*list of int or scipy.stats distribution, optional*) – List of max depths to tune (optimized for climate data).
- **min_child_weight_range** (*list of float or scipy.stats distribution, optional*) – List of minimum child weights to tune (optimized for climate data).
- **subsample_range** (*list of float or scipy.stats distribution, optional*) – List of subsample ratios to tune.
- **colsample_bytree_range** (*list of float or scipy.stats distribution, optional*) – List of column sampling ratios to tune.
- **gamma_range** (*list of float or scipy.stats distribution, optional*) – List of gamma (min_split_loss) values for regularization.
- **reg_alpha_range** (*list of float or scipy.stats distribution, optional*) – List of L1 regularization (alpha) values.
- **reg_lambda_range** (*list of float or scipy.stats distribution, optional*) – List of L2 regularization (lambda) values.
- **random_state** (*int, optional*) – Seed for reproducibility (default is 42).
- **dist_method** (*str, optional*) – Distribution method for tercile probabilities ('bestfit', 'nonparam', etc.) (default is 'nonparam').

- **n_iter_search** (*int*, *optional*) – Number of iterations for randomized/bayesian search or points to sample for grid search (default is 10).
- **cv_folds** (*int*, *optional*) – Number of cross-validation folds (default is 3).
- **cv_method** (*str*, *optional*) – Cross-validation method: ‘timeseries’ or ‘kfold’ (default: ‘timeseries’). Use ‘timeseries’ for temporal data to prevent data leakage.
- **n_clusters** (*int*, *optional*) – Number of clusters for homogenized zones (default is 4).
- **optuna_n_jobs** (*int*, *optional*) – Number of parallel jobs for Optuna (default is 1).
- **optuna_timeout** (*int*, *optional*) – Timeout in seconds for Optuna optimization (default is None).

__init__ (*search_method*=‘random’, *n_estimators_range*=[50, 100, 150, 200], *learning_rate_range*=[0.005, 0.01, 0.03, 0.05, 0.1], *max_depth_range*=[3, 5, 7], *min_child_weight_range*=[1, 3, 5], *subsample_range*=[0.6, 0.8, 1.0], *colsample_bytree_range*=[0.6, 0.8, 1.0], *gamma_range*=[0, 0.1, 0.2, 0.5, 1], *reg_alpha_range*=[0, 0.01, 0.1], *reg_lambda_range*=[1, 1.5, 2], *early_stopping_rounds_range*=[None, 10, 20], *es_validation_fraction*=0.15, *random_state*=42, *dist_method*=‘nonparam’, *n_iter_search*=10, *cv_folds*=3, *n_clusters*=4, *leave_one_year_out*=False, *optuna_n_jobs*=1, *optuna_timeout*=None, *verbose*=0)

compute_hyperparameters (*Predictors*, *Predictand*, *clim_year_start*=None, *clim_year_end*=None)

Best hyperparameters per zone.

Regionalization: KMeans on the predictand VALUES (kept by preference). Standardization is applied here only if clim years are given; when None, the inputs are assumed already standardized (framework path).

compute_model (*X_train*, *y_train*, *X_test*, *y_test*=None, *clim_year_start*=None, *clim_year_end*=None, *best_params*=None, *cluster_da*=None)

Deterministic hindcast with one XGBRegressor per zone.

>>> A: clim years are now **in** the signature **and** tuning runs **in** dual mode when *best_params*/*cluster_da* are not supplied.

static weibull_shape_solver (*k*, *M*, *V*)

Root function for the Weibull shape parameter *k* (match V/M^2).

static calculate_tercile_probabilities_bestfit (*best_guess*, *error_variance*, *T1*, *T2*, *dist_code*, *dof*)

Generic tercile probabilities using best-fit family per grid cell.

static calculate_tercile_probabilities_nonparametric (*best_guess*, *error_samples*, *first_tercile*, *second_tercile*)

Non-parametric method using historical error samples.

compute_prob (*Predictant*: *DataArray*, *clim_year_start*, *clim_year_end*, *hindcast_det*: *DataArray*, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None) → *DataArray*

Tercile probabilities for deterministic hindcasts.

For honest calibration pass the CROSS-VALIDATED hindcast as *hindcast_det* (out-of-sample residuals); the in-sample hindcast yields over-confident probabilities.

forecast (*Predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*, *hindcast_det_cross*, *Predictor_for_year*, *best_params*=None, *cluster_da*=None, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None)

Operational forecast for a target year (deterministic + tercile probabilities).

```
class wass2s.was_mme.HPELMWrapper (neurons=10, activation='sigm', norm=1.0, random_state=42)
```

Bases: BaseEstimator, RegressorMixin

Wrapper for HPELM to make it compatible with scikit-learn's hyperparameter optimization.

```
__init__ (neurons=10, activation='sigm', norm=1.0, random_state=42)
```

```
fit (X, y)
```

```
predict (X)
```

```
score (X, y)
```

Return the coefficient of determination of the prediction.

The coefficient of determination R^2 is defined as $(1 - \frac{u}{v})$, where u is the residual sum of squares $((y_{\text{true}} - y_{\text{pred}})^2).sum()$ and v is the total sum of squares $((y_{\text{true}} - y_{\text{true}.mean()})^2).sum()$. The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected value of y , disregarding the input features, would get a R^2 score of 0.0.

Parameters

- **X** (*array-like of shape (n_samples, n_features)*) – Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape $(n_{\text{samples}}, n_{\text{samples_fitted}})$, where $n_{\text{samples_fitted}}$ is the number of samples used in the fitting for the estimator.
- **y** (*array-like of shape (n_samples,) or (n_samples, n_outputs)*) – True values for X .
- **sample_weight** (*array-like of shape (n_samples,)*, *default=None*) – Sample weights.

Returns

score – R^2 of `self.predict(X)` w.r.t. y .

Return type

float

Notes

The R^2 score used when calling `score` on a regressor uses `multioutput='uniform_average'` from version 0.23 to keep consistent with default value of `r2_score()`. This influences the `score` method of all the multioutput regressors (except for `MultiOutputRegressor`).

```
get_params (deep=True)
```

Get parameters for this estimator.

Parameters

deep (*bool*, *default=True*) – If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns

params – Parameter names mapped to their values.

Return type

dict

set_params (***params*)

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as `Pipeline`). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Parameters

****params** (*dict*) – Estimator parameters.

Returns

self – Estimator instance.

Return type

estimator instance

```
class wass2s.was_mme.WAS_mme_hpELM (neurons_range=[10, 20, 50, 100], activation_options=['sigm',
'tanh', 'lin', 'rbf_l1', 'rbf_l2', 'rbf_linf'], norm_range=[0.1, 1.0,
10.0, 100.0], random_state=42, dist_method='nonparam',
search_method='random', n_iter_search=10, cv_folds=3,
n_clusters=4, leave_one_year_out=False, n_trials_bayesian=50,
bayesian_sampler='tpe', scoring='neg_mean_squared_error',
verbose=0)
```

Bases: `object`

Extreme Learning Machine (ELM) based Multi-Model Ensemble (MME) forecasting using hpelm library. This class implements a single-model forecasting approach using HPELM for deterministic predictions, with optional tercile probability calculations using various statistical distributions. Implements hyperparameter optimization via multiple methods.

Parameters

- **neurons_range** (*list of int, optional*) – List of neuron counts to tune for HPELM (default is [10, 20, 50, 100]).
- **activation_options** (*list of str, optional*) – Activation functions to tune for HPELM (default is ['sigm', 'tanh', 'lin', 'rbf_l1', 'rbf_l2', 'rbf_linf']).
- **norm_range** (*list of float, optional*) – Regularization parameters to tune for HPELM (default is [0.1, 1.0, 10.0, 100.0]).
- **random_state** (*int, optional*) – Seed for reproducibility (default is 42).
- **dist_method** (*str, optional*) – Distribution method for tercile probabilities.
- **search_method** (*str, optional*) – Hyperparameter optimization method: 'grid', 'random', or 'bayesian' (default: 'random').
- **n_iter_search** (*int, optional*) – Number of iterations for randomized search (default is 10).
- **cv_folds** (*int, optional*) – Number of cross-validation folds (default is 3).
- **n_clusters** (*int, optional*) – Number of clusters for homogenized zones (default is 4).
- **n_trials_bayesian** (*int, optional*) – Number of trials for Bayesian optimization (default=50).
- **bayesian_sampler** (*str, optional*) – Sampler for Bayesian optimization: 'tpe' or 'random' (default='tpe').

- **scoring** (*str*, optional) – Scoring metric for optimization (default='neg_mean_squared_error').

__init__ (*neurons_range*=[10, 20, 50, 100], *activation_options*=['sigm', 'tanh', 'lin', 'rbf_l1', 'rbf_l2', 'rbf_linf'], *norm_range*=[0.1, 1.0, 10.0, 100.0], *random_state*=42, *dist_method*='nonparam', *search_method*='random', *n_iter_search*=10, *cv_folds*=3, *n_clusters*=4, *leave_one_year_out*=False, *n_trials_bayesian*=50, *bayesian_sampler*='tpe', *scoring*='neg_mean_squared_error', *verbose*=0)

compute_hyperparameters (*Predictors*, *Predictand*, *clim_year_start*=None, *clim_year_end*=None)

Best hyperparameters per zone.

Regionalization: KMeans on the predictand VALUES (kept by preference). Standardization is applied here only if clim years are given (dual mode).

compute_model (*X_train*, *y_train*, *X_test*, *y_test*=None, *clim_year_start*=None, *clim_year_end*=None, *best_params*=None, *cluster_da*=None)

Deterministic hindcast with one HPELM per zone.

>>> A: clim years now **in** the signature; tuning runs **in** dual mode when *best_params*/*cluster_da* are not supplied.

static weibull_shape_solver (*k*, *M*, *V*)

static calculate_tercile_probabilities_bestfit (*best_guess*, *error_variance*, *T1*, *T2*, *dist_code*, *dof*)

static calculate_tercile_probabilities_nonparametric (*best_guess*, *error_samples*, *first_tercile*, *second_tercile*)

compute_prob (*Predictant*: *DataArray*, *clim_year_start*, *clim_year_end*, *hindcast_det*: *DataArray*, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None) → *DataArray*

Tercile probabilities for deterministic hindcasts (pass the CV hindcast for calibration).

forecast (*Predictant*, *clim_year_start*, *clim_year_end*, *hindcast_det*, *hindcast_det_cross*, *Predictor_for_year*, *best_params*=None, *cluster_da*=None, *best_code_da*=None, *best_shape_da*=None, *best_loc_da*=None, *best_scale_da*=None)

Operational forecast for a target year (deterministic + tercile probabilities).

get_optimization_results ()

class wass2s.was_mme.**WAS_mme_MLP** (*search_method*='random', *hidden_layer_sizes_range*=[(2,), (4,), (2, 4), (8,), (8, 16), (16,)], *activation_options*=['relu', 'tanh', 'identity'], *learning_rate_init_range*=<scipy.stats._distn_infrastructure.rv_continuous_frozen object>, *solver_options*=['adam', 'sgd', 'lbfgs'], *alpha_range*=[0.0001, 0.001, 0.01, 0.1], *max_iter*=200, *random_state*=42, *dist_method*='nonparam', *n_iter_search*=10, *cv_folds*=3, *n_clusters*=4, *early_stopping_options*=[True, False], *validation_fraction*=0.1, *n_iter_no_change*=10, *leave_one_year_out*=False, *optuna_n_jobs*=1, *optuna_timeout*=None)

Bases: *object*

Multi-Layer Perceptron (MLPRegressor) MME forecaster with per-zone hyperparameter tuning and tercile-probability dressing. Deterministic hindcast via `compute_model`; tercile probabilities via `compute_prob` ('non-

param' or 'bestfit'). Mirrors the WAS_mme_RF / WAS_mme_XGBoosting contract so it plugs into the same_kind_model2 branch of WAS_Cross_Validator.

```
__init__ (search_method='random', hidden_layer_sizes_range=[(2,), (4,), (2, 4), (8,), (8, 16), (16,)],
          activation_options=['relu', 'tanh', 'identity'],
          learning_rate_init_range=<scipy.stats._distn_infrastructure.rv_continuous_frozen object>,
          solver_options=['adam', 'sgd', 'lbfgs'], alpha_range=[0.0001, 0.001, 0.01, 0.1], max_iter=200,
          random_state=42, dist_method='nonparam', n_iter_search=10, cv_folds=3, n_clusters=4,
          early_stopping_options=[True, False], validation_fraction=0.1, n_iter_no_change=10,
          leave_one_year_out=False, optuna_n_jobs=1, optuna_timeout=None)
```

compute_hyperparameters (Predictors, Predictand, clim_year_start=None, clim_year_end=None)

Per-zone hyperparameter tuning. KMeans on predictand VALUES (kept).

A dual-mode: clim years given -> standardize Predictors / Predictand internally; both None -> assume already standardized (framework path), so no double-standardization when called from compute_model.

```
compute_model (X_train, y_train, X_test, y_test=None, clim_year_start=None, clim_year_end=None,
                best_params=None, cluster_da=None)
```

Deterministic hindcast per zone. Inputs are assumed standardized (the same_kind_model2 branch standardizes once). A: clim_year_start/end are in the signature, so the best_params=None path no longer raises.

static weibull_shape_solver (k, M, V)

Root function for the Weibull shape k matching observed V/M^2 .

```
static calculate_tercile_probabilities_bestfit (best_guess, error_variance, T1, T2,
                                                dist_code, dof)
```

Tercile probabilities from a forecast-adjusted best-fit family.

```
static calculate_tercile_probabilities_nonparametric (best_guess, error_samples,
                                                       first_tercile, second_tercile)
```

Non-parametric tercile probabilities from historical error samples.

```
compute_prob (Predictant, clim_year_start, clim_year_end, hindcast_det, best_code_da=None,
               best_shape_da=None, best_loc_da=None, best_scale_da=None)
```

Tercile probabilities for a (cross-validated) deterministic hindcast.

```
forecast (Predictant, clim_year_start, clim_year_end, hindcast_det, hindcast_det_cross, Predictor_for_year,
           best_params=None, cluster_da=None, best_code_da=None, best_shape_da=None,
           best_loc_da=None, best_scale_da=None)
```

Deterministic + probabilistic forecast for a single target year.

```
class wass2s.was_mme.WAS_mme_PCR (n_modes=5, use_coslat=True, combine='mean', detrend=False,
                                   dist_method='nonparam', L2norm=False, random_state=42)
```

Bases: `object`

Principal-component-regression MME baseline.

Predictor field = the multi-model mean (over M) on the predictand grid. EOFs are fitted on the training fold (via the fold-safe WAS_EOF), the predictand is regressed on the leading PCs, and the held-out fold is projected out-of-sample before applying the regression. The deterministic output is in the same (standardized) space the framework hands in, so WAS_Cross_Validator's same_kind_model2 branch reverse-standardizes it and calls compute_prob exactly as it does for WAS_mme_RF.

Register it: add WAS_mme_PCR to the same_kind_model2 list.

Parameters

- **n_modes** (*int*) – Number of principal components retained (default 5).

- **use_coslat** (*bool*) – Cosine-latitude weighting in the EOF (default True).
- **detrend** (*bool*) – Detrend the predictor field inside each fold (default False; the framework already standardizes, and the RF baseline does not detrend, so leave False for a like-for-like comparison).
- **dist_method** (*str*) – ‘nonparam’ (default) or ‘bestfit’.
- **combine** (*str*) –

How the M models enter the EOF:

‘mean’ (default) – predictor field = multi-model mean (one channel). ‘meof’ – keep every model as its own channel and run a

single combined / multivariate EOF over the joint (Y, X, M) feature space. Because the framework has already standardized each (cell, model) to unit variance, the channels share a scale, so the stacked EOF is a proper MEOF (no per-field normalization needed). The leading PCs then encode the joint inter-model variability, and the predictand is regressed on them.

```
__init__ (n_modes=5, use_coslat=True, combine='mean', detrend=False, dist_method='nonparam',
          L2norm=False, random_state=42)
```

```
static weibull_shape_solver (k, M, V)
```

```
static calculate_tercile_probabilities_bestfit (best_guess, error_variance, T1, T2,
                                                dist_code, dof)
```

```
static calculate_tercile_probabilities_nonparametric (best_guess, error_samples,
                                                       first_tercile, second_tercile)
```

```
compute_model (X_train, y_train, X_test, y_test=None, **kwargs)
```

```
compute_prob (Predictant, clim_year_start, clim_year_end, hindcast_det, best_code_da=None,
               best_shape_da=None, best_loc_da=None, best_scale_da=None)
```

```
forecast (Predictant, clim_year_start, clim_year_end, Predictor, hindcast_det, Predictor_for_year,
           best_code_da=None, best_shape_da=None, best_loc_da=None, best_scale_da=None)
```

Operational PCR forecast for a single target year.

```
class wass2s.was_mme.WAS_mme_ExpertKnowledge (expert_bounds: Dict[str, Tuple[float, float]] | None
                                              = None, expert_prior: Dict[str, float] | None =
                                              None, shrinkage: float | str = 'cv', shrinkage_grid:
                                              Sequence[float] = (0.0, 0.02, 0.05, 0.1, 0.2, 0.5),
                                              spatial_pooling: str = 'global', min_train_years: int
                                              = 8)
```

Bases: `object`

Expert-bounded, RPS-optimal multi-model tercile-probability blend.

Parameters

- **expert_bounds** (*dict {model_name: (lo, hi)}* or *None*) – Hard box constraints on each model’s weight. Defaults to (0, 1). Example: {“ECMWF”: (0.25, 0.6), “DWD”: (0.0, 0.1)}.
- **expert_prior** (*dict {model_name: float}* or *None*) – Relative trust weights; normalised internally to sum to 1 and used as the shrinkage target (and the warm start). Defaults to equal trust.

- **shrinkage** (*float* or *"cv"*) – L2 shrinkage strength toward *expert_prior*. “cv” selects it by leave-one-year-out CV-RPS from *shrinkage_grid*.
- **shrinkage_grid** (*sequence of floats*) – Candidate lambdas when shrinkage=“cv”.
- **spatial_pooling** (*"global"* or *"gridpoint"*) – “global” fits ONE weight vector for the whole domain (robust, the usual seasonal choice with few years). “gridpoint” fits weights per grid cell (more flexible, needs more years / spatial smoothing).
- **min_train_years** (*int*) – Minimum valid years required to fit; otherwise falls back to the prior.

__init__ (*expert_bounds: Dict[str, Tuple[float, float]] | None = None, expert_prior: Dict[str, float] | None = None, shrinkage: float | str = 'cv', shrinkage_grid: Sequence[float] = (0.0, 0.02, 0.05, 0.1, 0.2, 0.5), spatial_pooling: str = 'global', min_train_years: int = 8*)

fit (*model_probs: Dict[str, DataArray] | DataArray, obs_class: DataArray, time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', prob_dim: str = 'probability'*) → *WAS_mme_ExpertKnowledge*

predict (*model_probs: Dict[str, DataArray] | DataArray, time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', prob_dim: str = 'probability'*) → *DataArray*

Blend the per-model tercile probabilities with the fitted weights. Returns a DataArray with dims (probability, T, Y, X).

compute_model (*X_train: Dict[str, DataArray] | DataArray, obs_class_train: DataArray, X_test: Dict[str, DataArray] | DataArray, time_dim: str = 'T', lat_dim: str = 'Y', lon_dim: str = 'X', prob_dim: str = 'probability'*) → *DataArray*

fit + predict in one call (cross-validator friendly).

summary ()

1.9.17 CEAC agroclimatic suite

CEAC agroclimatic indicator computation.

Refactored, composable implementation of onset, cessation, and spell counting for both station (CPT/CDT format) and gridded data.

Base class

CAF_AgroClimateBase

Shared helpers for date-index arithmetic, zone-specific criteria mapping, CPT/CDT input parsing, and output formatting.

Indicator classes

CEAC_compute_onset

Season onset detection.

CEAC_compute_cessation

Season cessation detection.

CEAC_compute_onset_dry_spell

Onset with a post-onset dry-spell check.

CEAC_compute_cessation_dry_spell

Cessation with an additional dry-spell criterion.

CEAC_count_dry_spells

Count of dry spells of at least *d_len* days within the growing season.

CEAC_count_wet_spells

Count of wet spells of at least *w_len* days within the growing season.

```
class wass2s.ceac_agro.CAF_AgroClimateBase (user_criteria=None)
```

Bases: `object`

SHIFT_OFFSET = 244

__init__ (user_criteria=None)

static day_of_year (y, mm_dd)

get_index_for_station (year, mm_dd, z)

output_format_value (v, z)

revert_to_index (v, z)

shift_gridded_data (daily_data, map_reclassified)

format_grid_output (res_xr, mask)

revert_grid_index (res_xr, mask)

transform_and_shift_cdt (df_raw, map_reclassified)

build_cpt_output (res_df, val_col)

```
class wass2s.ceac_agro.CEAC_compute_onset (user_criteria=None)
```

Bases: `CAF_AgroClimateBase`

static onset_function (x, idebut, cumul, nbsec, jour_pluvieux, irch_fin)

compute_insitu (daily_df_raw, map_rec)

compute (daily_data, map_rec, nb_cores)

```
class wass2s.ceac_agro.CEAC_compute_cessation (user_criteria=None)
```

Bases: `CAF_AgroClimateBase`

static cessation_function (x, ijour, idebut, ETP, Cap, irch_fin)

compute_insitu (daily_df_raw, map_rec)

compute (daily_data, map_rec, nb_cores)

```
class wass2s.ceac_agro.CEAC_compute_onset_dry_spell (user_criteria=None)
```

Bases: `CAF_AgroClimateBase`

static ds_onset_func (x, idebut, cumul, nbsec, jp, irch_fin, nbjour)

compute_insitu (daily_df_raw, map_rec)

compute (daily_data, map_rec, nb_cores)

```
class wass2s.ceac_agro.CEAC_compute_cessation_dry_spell (user_criteria=None)
```

Bases: `CAF_AgroClimateBase`

```
static ds_cess_func (x, id1, cum, nbs, jp, ir1, id2, ijd, ETP, Cap, ir2, nbj)

compute_insitu (daily_df_raw, map_rec)

compute (daily_data, map_rec, nb_cores)

class wass2s.ceac_agro.CEAC_count_dry_spells (user_criteria=None)
    Bases: CAF_AgroClimateBase
    static count_dry_spells (x, onset, cessation, d_len, thresh)

    compute_insitu (daily_raw, on_cpt, cess_cpt, map_rec, d_len, thresh=1.0)

    compute (daily_data, on_da, cess_da, map_rec, d_len, thresh, nb_cores)

class wass2s.ceac_agro.CEAC_count_wet_spells (user_criteria=None)
    Bases: CAF_AgroClimateBase
    static count_wet_spells (x, onset, cessation, w_len, thresh)

    compute_insitu (daily_raw, on_cpt, cess_cpt, map_rec, w_len, thresh=1.0)

    compute (daily_data, on_da, cess_da, map_rec, w_len, thresh, nb_cores)
```

1.9.18 Seasonal diagnostic tools

Seasonal diagnostic tools for ERA5 data.

This module provides:

- `VAR_CONFIG` — per-variable download and plotting configuration.
- `download_data()` — ERA5 monthly download via earthkit.
- `process_variable()` — compute anomalies/ratios and dispatch to plots.
- `plot_maps()` — 6-panel monthly map layout.
- `plot_hovmoller()` — time–longitude Hovmöller diagram.
- `main_driver()` — end-to-end entry point.
- `C3SViewer` — interactive Copernicus C3S seasonal forecast viewer.
- `BOMViewer` — interactive BOM MJO viewer.

```

wass2s.was_seasonal_analysis.VAR_CONFIG = {'olr': {'cds_name':
'top_net_thermal_radiation', 'cmap': 'BrBG', 'extent': [40, -180, -40, 180],
'hov_band': [10, -10], 'hov_cmap': 'BrBG', 'hov_levels': array([-60, -55, -50,
-45, -40, -35, -30, -25, -20, -15, -10, -5, 0, 5, 10, 15, 20, 25, 30, 35, 40,
45, 50, 55, 60]), 'level_type': 'single', 'levels_anom': array([-50, -45, -40,
-35, -30, -25, -20, -15, -10, -5, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50]),
'levels_contour': array([-320, -300, -280, -260, -240, -220, -200, -180]),
'plot_type': 'contour_overlay', 'unit_func': <function <lambda>>,
'unit_label': 'W/m2'}, 'precip': {'cds_name': 'total_precipitation', 'cmap':
'BrBG', 'extent': [35, -25, -15, 55], 'level_type': 'single', 'levels': [0,
25, 50, 75, 90, 110, 125, 150, 200, 300], 'plot_type': 'ratio', 'unit_func':
<function <lambda>>, 'unit_label': 'mm/mois'}, 'slp': {'cds_name':
'mean_sea_level_pressure', 'cmap': 'RdBu_r', 'extent': [50, -60, -45, 130],
'highlight_black': [1015], 'highlight_green': [1012], 'level_type': 'single',
'levels_anom': array([-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]),
'levels_contour': array([ 990, 992, 994, 996, 998, 1000, 1002, 1004, 1006,
1008, 1010, 1012, 1014, 1016, 1018, 1020, 1022, 1024, 1026, 1028, 1030, 1032,
1034, 1036]), 'plot_type': 'contour_overlay', 'unit_func': <function
<lambda>>, 'unit_label': 'hPa'}, 'sst': {'cds_name':
'sea_surface_temperature', 'cmap': 'RdBu_r', 'compute_seasonal': True,
'extent': [40, -180, -40, 180], 'level_type': 'single', 'levels': array([-3. ,
-2.5, -2. , -1.5, -1. , -0.5, 0. , 0.5, 1. , 1.5, 2. , 2.5, 3. ]), 'plot_type':
'anomaly_only', 'unit_func': <function <lambda>>, 'unit_label': 'Anomalie SST
(°C)'}, 'wind_850': {'cds_name': ['u_component_of_wind',
'v_component_of_wind', 'specific_humidity'], 'cmap': 'GnBu', 'extent': [40,
-60, -20, 60], 'level_type': 'pressure', 'levels': array([ 0, 2, 4, 6, 8, 10,
12, 14, 16]), 'plot_type': 'stream_humidity', 'pressure_level': '850',
'unit_func': <function <lambda>>, 'unit_label': 'Humidité Spécifique (g/kg)'},
'wind_surface': {'cds_name': ['10m_u_component_of_wind',
'10m_v_component_of_wind'], 'cmap': 'YlOrRd', 'extent': [35, -30, -15, 50],
'level_type': 'single', 'levels': array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
11, 12, 13, 14]), 'plot_type': 'streamlines', 'unit_func': <function
<lambda>>, 'unit_label': 'Vitesse du Vent (m/s)')}

```

Per-variable ERA5 download and plotting configuration.

Each key is a short variable identifier. The associated dict contains:

- `cds_name` — CDS variable name(s).
- `level_type` — "single" or "pressure".
- `unit_func` — callable that converts raw ERA5 units.
- `unit_label` — axis / colorbar label.
- `plot_type` — rendering strategy used by `plot_maps()`.
- `cmap`, `levels`, `extent` — Matplotlib / Cartopy parameters.
- `hov_band`, `hov_cmap`, `hov_levels` — Hovmöller parameters (OLR only).

Type

dict

`wass2s.was_seasonal_analysis.download_data` (*dir_to_save*, *clim_start*, *clim_end*, *var_key*, *extent*, *target_date*)

Download ERA5 monthly data for the six months preceding *target_date*.

Files are saved as NetCDF and cached on disk; if the file already exists the download is skipped.

Parameters

- **dir_to_save** (*str*) – Directory where the NetCDF file will be written.
- **clim_start** (*int*) – First year of the climatological reference period.
- **clim_end** (*int*) – Last year of the climatological reference period.
- **var_key** (*str*) – Variable key defined in *VAR_CONFIG*.
- **extent** (*list of float*) – Bounding box [north, west, south, east] passed to the CDS API.
- **target_date** (*datetime.date*) – The target (forecast) date. Data are downloaded for the six months *preceding* this date (i.e. T-6 to T-1).

Returns

Absolute path to the downloaded NetCDF file, or *None* if the download failed.

Return type

str or *None*

`wass2s.was_seasonal_analysis.plot_hovmoller(data_anom, data_abs, var_key)`

Plot a time–longitude Hovmöller diagram for a supported variable.

The diagram is only produced for variables that define a *hov_band* in *VAR_CONFIG* (currently OLR). For vector fields the U-component is used.

Parameters

- **data_anom** (*xarray.Dataset* or *xarray.DataArray*) – Anomaly field, already averaged over the configured latitude band.
- **data_abs** (*xarray.Dataset* or *xarray.DataArray*) – Absolute (non-anomaly) field for overlaid contours.
- **var_key** (*str*) – Variable key defined in *VAR_CONFIG*.

`wass2s.was_seasonal_analysis.plot_maps(data_main, data_overlay, var_key, title_prefix='Monthly')`

Render a 6-row map panel for the six preceding months.

Each row displays one month. The rendering strategy is controlled by `VAR_CONFIG[var_key]["plot_type"]`:

- "anomaly_only" — filled contours of the anomaly (SST).
- "contour_overlay" — filled anomaly + absolute contour lines with optional highlighted isopleths (SLP, OLR).
- "ratio" — precipitation ratio relative to climatology.
- "vector" — quiver plot of wind vectors.
- "streamlines" / "stream_humidity" — streamlines with specific humidity (850 hPa) or wind speed (surface) as the colour background.

Parameters

- **data_main** (*xarray.DataArray* or *xarray.Dataset*) – Primary data field (anomaly, ratio, or absolute for vector fields).
- **data_overlay** (*xarray.DataArray* or *None*) – Absolute field used for overlaid contour lines; *None* for types that do not need it (ratio, vector, streamlines).

- **var_key** (*str*) – Variable key defined in *VAR_CONFIG*.
- **title_prefix** (*str*, default "Monthly") – String prepended to the figure super-title.

`wass2s.was_seasonal_analysis.process_variable` (*fpath*, *var_key*, *clim_start*, *clim_end*, *target_date*)

Load, pre-process, and plot one ERA5 variable.

Computes climatology from the reference period, selects the six months preceding *target_date*, derives anomalies or ratios, and delegates to *plot_maps()* and *plot_hovmoller()*.

Parameters

- **fpath** (*str*) – Path to the ERA5 NetCDF file (typically returned by *download_data()*).
- **var_key** (*str*) – Variable key defined in *VAR_CONFIG*.
- **clim_start** (*int*) – First year of the climatological reference period.
- **clim_end** (*int*) – Last year of the climatological reference period.
- **target_date** (*datetime.date*) – Forecast target date. The six months T-6 to T-1 are processed.

`wass2s.was_seasonal_analysis.main_driver` (*dir_save*, *clim_start*, *clim_end*, *target_date_str*, *variables_list=None*)

Run the full seasonal diagnostic pipeline for a set of variables.

For each variable: download ERA5 data (if not already cached), compute anomalies relative to the climatological period, and produce maps and Hovmöller diagrams for the six months preceding the target date.

Parameters

- **dir_save** (*str*) – Root directory for cached NetCDF files.
- **clim_start** (*int*) – First year of the climatological reference period.
- **clim_end** (*int*) – Last year of the climatological reference period.
- **target_date_str** (*str*) – ISO-format target date string ("YYYY-MM-DD"). Data for the six months *before* this date are processed and visualised.
- **variables_list** (*list of str, optional*) – Subset of keys from *VAR_CONFIG* to process. Defaults to all configured variables.

class `wass2s.was_seasonal_analysis.C3SViewer`

Bases: `object`

Interactive Jupyter widget for Copernicus C3S seasonal forecast maps.

Displays C3S probabilistic seasonal forecast products (SST, MSLP, precipitation, temperature, wind, and Niño plumes) inside an `IFrame`, with dropdown controls for product, start year/month, area, and map type.

Parameters

None

Examples

```
>>> viewer = C3SViewer()
>>> viewer.show()
```

`__init__()`

show()

Display the widget interface in the current Jupyter cell.

class wass2s.was_seasonal_analysis.**BOMViewer**

Bases: `object`

Interactive Jupyter widget for BOM MJO monitoring charts.

Embeds the Australian Bureau of Meteorology MJO page inside an IFrame with a dropdown to navigate between tabs (Outlooks, MJO Phase Diagram, Monitoring, Cloudiness, Regional Cloudiness, Time-Longitude, Tropical Update).

Parameters

None

Examples

```
>>> viewer = BOMViewer()
>>> viewer.show()
```

__init__()

show()

Display the widget interface in the current Jupyter cell.

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

W

- wass2s.ceac_agro, [206](#)
- wass2s.utils, [47](#)
- wass2s.was_analog, [151](#)
- wass2s.was_bias_correction, [84](#)
- wass2s.was_cca, [122](#)
- wass2s.was_cca_2, [129](#)
- wass2s.was_compute_predictand, [62](#)
- wass2s.was_cross_validate, [92](#)
- wass2s.was_download, [38](#)
- wass2s.was_eof, [91](#)
- wass2s.was_linear_models, [94](#)
- wass2s.was_machine_learning, [131](#)
- wass2s.was_merge_predictand, [87](#)
- wass2s.was_mme, [175](#)
- wass2s.was_pcr, [119](#)
- wass2s.was_seasonal_analysis, [208](#)
- wass2s.was_transformdata, [61](#)
- wass2s.was_verification, [159](#)

INDEX

Symbols

- `__init__()` (*wass2s.ceac_agro.CAF_AgroClimateBase method*), 207
- `__init__()` (*wass2s.was_analog.WAS_Analog method*), 153
- `__init__()` (*wass2s.was_analog.WAS_Analog method*), 157
- `__init__()` (*wass2s.was_cca.WAS_CCA method*), 122
- `__init__()` (*wass2s.was_cca.WAS_CCA_base method*), 124
- `__init__()` (*wass2s.was_cca.WAS_CCA_op method*), 125
- `__init__()` (*wass2s.was_cca.WAS_CCA_strict method*), 127
- `__init__()` (*wass2s.was_cca_2.WAS_EOF_KCCA method*), 131
- `__init__()` (*wass2s.was_cca_2.WAS_KGCCA method*), 130
- `__init__()` (*wass2s.was_cca_2.WAS_KernelCCA_base method*), 129
- `__init__()` (*wass2s.was_compute_predictand.HeatWaveDefinition method*), 82
- `__init__()` (*wass2s.was_compute_predictand.WAS_HeatWaveIndices method*), 83
- `__init__()` (*wass2s.was_compute_predictand.WAS_PrecipIndices method*), 81
- `__init__()` (*wass2s.was_compute_predictand.WAS_TempPercentileIndices method*), 79
- `__init__()` (*wass2s.was_compute_predictand.WAS_compute_cessation method*), 67
- `__init__()` (*wass2s.was_compute_predictand.WAS_compute_cessation_dry_spell method*), 69
- `__init__()` (*wass2s.was_compute_predictand.WAS_compute_onset method*), 63
- `__init__()` (*wass2s.was_compute_predictand.WAS_compute_onset_dry_spell method*), 65
- `__init__()` (*wass2s.was_cross_validate.CustomTimeSeriesSplit method*), 92
- `__init__()` (*wass2s.was_cross_validate.WAS_Cross_Validator method*), 93
- `__init__()` (*wass2s.was_download.WAS_Download method*), 39
- `__init__()` (*wass2s.was_eof.WAS_EOF method*), 91
- `__init__()` (*wass2s.was_linear_models.WAS_ElasticNet_Model method*), 110
- `__init__()` (*wass2s.was_linear_models.WAS_LassoLars_Model method*), 115
- `__init__()` (*wass2s.was_linear_models.WAS_Lasso_Model method*), 105
- `__init__()` (*wass2s.was_linear_models.WAS_LinearRegression_Model method*), 96
- `__init__()` (*wass2s.was_linear_models.WAS_Ridge_Model method*), 101
- `__init__()` (*wass2s.was_machine_learning.BaseOptimizer method*), 132
- `__init__()` (*wass2s.was_machine_learning.MARS method*), 143
- `__init__()` (*wass2s.was_machine_learning.WAS_LogisticRegression_Model method*), 134
- `__init__()` (*wass2s.was_machine_learning.WAS_MARS_Model method*), 145
- `__init__()` (*wass2s.was_machine_learning.WAS_NegativeBinomial_Model method*), 149
- `__init__()` (*wass2s.was_machine_learning.WAS_PoissonRegression method*), 140
- `__init__()` (*wass2s.was_machine_learning.WAS_PolynomialRegression method*), 135
- `__init__()` (*wass2s.was_merge_predictand.WAS_Merging method*), 87
- `__init__()` (*wass2s.was_merge_predictand.WAS_Merging_Extended method*), 89
- `__init__()` (*wass2s.was_mme.HPELMWrapper method*), 201
- `__init__()` (*wass2s.was_mme.WAS_Min2009_ProbWeighted method*), 182
- `__init__()` (*wass2s.was_mme.WAS_ProbWeighted method*), 180
- `__init__()` (*wass2s.was_mme.WAS_mme_ELR method*), 186
- `__init__()` (*wass2s.was_mme.WAS_mme_ExpertKnowledge method*), 206
- `__init__()` (*wass2s.was_mme.WAS_mme_FastBMA method*), 194
- `__init__()` (*wass2s.was_mme.WAS_mme_FullBMA method*), 194

<code>__init__()</code> (wass2s.was_mme.WAS_mme_MLP method), 196	<code>(wass2s.was_merge_predictand.WAS_Merging method)</code> , 87
<code>__init__()</code> (wass2s.was_mme.WAS_mme_MVA method), 193	<code>adjust_duplicates()</code> (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90
<code>__init__()</code> (wass2s.was_mme.WAS_mme_NGR_Gaussian method), 192	<code>Agglomerative_Based()</code> (wass2s.was_analog.WAS_Analog method), 154
<code>__init__()</code> (wass2s.was_mme.WAS_mme_NGR_NonGaussian method), 193	<code>Agglomerative_Based()</code> (wass2s.was_analog.WAS_Analog method), 158
<code>__init__()</code> (wass2s.was_mme.WAS_mme_PCR method), 205	<code>AgroObsName()</code> (wass2s.was_download.WAS_Download method), 40
<code>__init__()</code> (wass2s.was_mme.WAS_mme_RF method), 198	<code>anomalize_timeseries()</code> (in module <code>wass2s.utils</code>), 51
<code>__init__()</code> (wass2s.was_mme.WAS_mme_Weighted method), 177	<code>anomaly_timeseries()</code> (wass2s.was_analog.WAS_Analog method), 154
<code>__init__()</code> (wass2s.was_mme.WAS_mme_XGBoosting method), 200	<code>anomaly_timeseries()</code> (wass2s.was_analog.WAS_Analog method), 158
<code>__init__()</code> (wass2s.was_mme.WAS_mme_gaussian_process method), 188	<code>apply_detrend_data()</code> (in module <code>wass2s.utils</code>), 52
<code>__init__()</code> (wass2s.was_mme.WAS_mme_hpELM method), 203	<code>apply_transformation()</code> (wass2s.was_transformdata.WAS_TransformData method), 62
<code>__init__()</code> (wass2s.was_mme.WAS_mme_logistic method), 184	<code>arrange_indices_for_som()</code> (wass2s.was_analog.WAS_Analog method), 154
<code>__init__()</code> (wass2s.was_mme.WAS_mme_xcELM method), 188	<code>arrange_indices_for_som()</code> (wass2s.was_analog.WAS_Analog method), 158
<code>__init__()</code> (wass2s.was_mme.WAS_mme_xcELR method), 183	<code>auto_select_kriging_parameters()</code> (wass2s.was_merge_predictand.WAS_Merging method), 87
<code>__init__()</code> (wass2s.was_pcr.WAS_PCR method), 120	<code>auto_select_kriging_parameters()</code> (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90
<code>__init__()</code> (wass2s.was_seasonal_analysis.BOMViewer method), 212	
<code>__init__()</code> (wass2s.was_seasonal_analysis.C3SViewer method), 211	
<code>__init__()</code> (wass2s.was_transformdata.WAS_TransformData method), 61	
<code>__init__()</code> (wass2s.was_verification.WAS_Verification method), 160	

A

[adjust_duplicates\(\)](#) [\(wass2s.was_compute_predictand.WAS_compute_cessation_dry_spell static method\), 67](#) [\(wass2s.was_machine_learning.MARS static method\), 144](#)
[adjust_duplicates\(\)](#) [barnes_interpolation\(\)](#)
[\(wass2s.was_compute_predictand.WAS_compute_cessation_dry_spell static method\), 69](#) [\(wass2s.was_merge_predictand.WAS_Merging_Extended static method\), 90](#)
[adjust_duplicates\(\)](#) [BaseOptimizer \(class in \(wass2s.was_compute_predictand.WAS_compute_onset_wass2s.was_machine_learning\), 131 static method\), 63](#) [Bias_Based\(\) \(wass2s.was_analog.WAS_Analog method\), 154](#)
[adjust_duplicates\(\)](#) [\(wass2s.was_compute_predictand.WAS_compute_onset_dry_spell static method\), 65](#) [\(wass2s.was_analog.WAS_Analog__ method\), 158](#)
[adjust_duplicates\(\)](#) [BOMViewer \(class in wass2s.was_seasonal_analysis\), 212](#)
[\(wass2s.was_compute_predictand.WAS_count_dry_spell_score\(\) \(wass2s.was_verification.WAS_Verification static method\), 71](#) [method\), 166](#)
[adjust_duplicates\(\)](#) [brier_skill_score\(\)](#)

B

`cessationward_pass()` (`wass2s.was_machine_learning.MARS`
`method`), 144
`barnes_interpolation()`
`cessation_dry_spell.was_merge_predictand.WAS_Merging_Extended`
`method`), 90
`BaseOptimizer` (class in
`onset wass2s.was_machine_learning`), 131
`Bias_Based()` (`wass2s.was_analog.WAS_Analog`
`method`), 154
`Bias_dry_spell()` (`wass2s.was_analog.WAS_Analog_`
`method`), 158
`BOMViewer` (class in `wass2s.was_seasonal_analysis`), 212
`spell_score()` (`wass2s.was_verification.WAS_Verification`
`method`), 166
`brier_skill_score()`

(*wass2s.was_verification.WAS_Verification* method), 166

build_cpt_output() (*wass2s.ceac_agro.CAF_AgroClimateBase* method), 207

build_iridl_url_ersst() (in module *wass2s.utils*), 49

C

C3SViewer (class in *wass2s.was_seasonal_analysis*), 211

CAF_AgroClimateBase (class in *wass2s.ceac_agro*), 207

calc_index() (*wass2s.was_analog.WAS_Analog* method), 153

calc_index() (*wass2s.was_analog.WAS_Analog__* method), 157

calculate_groc() (*wass2s.was_verification.WAS_Verification* method), 164

calculate_groc() (*wass2s.was_verification.WAS_Verification* method), 164

calculate_groc_fixed() (*wass2s.was_verification.WAS_Verification* method), 165

calculate_groc_weighted() (*wass2s.was_verification.WAS_Verification* method), 165

calculate_rpss() (*wass2s.was_verification.WAS_Verification* method), 165

calculate_rpss_() (*wass2s.was_verification.WAS_Verification* method), 174

calculate_tercile_probabilities_bestfit() (*wass2s.was_analog.WAS_Analog* static method), 154

calculate_tercile_probabilities_bestfit() (*wass2s.was_analog.WAS_Analog__* static method), 158

calculate_tercile_probabilities_bestfit() (*wass2s.was_cca.WAS_CCA* static method), 123

calculate_tercile_probabilities_bestfit() (*wass2s.was_cca.WAS_CCA_base* static method), 125

calculate_tercile_probabilities_bestfit() (*wass2s.was_cca.WAS_CCA_op* static method), 126

calculate_tercile_probabilities_bestfit() (*wass2s.was_cca.WAS_CCA_strict* static method), 128

calculate_tercile_probabilities_bestfit() (*wass2s.was_cca_2.WAS_KernelCCA_base* class method), 130

calculate_tercile_probabilities_bestfit() (*wass2s.was_linear_models.WAS_ElasticNet_Model* static method), 111

calculate_tercile_probabilities_bestfit() (*wass2s.was_linear_models.WAS_Lasso_Model* static method), 106

calculate_tercile_probabilities_bestfit() (*wass2s.was_linear_models.WAS_LassoLars_Model* static method), 117

calculate_tercile_probabilities_bestfit() (*wass2s.was_linear_models.WAS_LinearRegression_Model* static method), 97

calculate_tercile_probabilities_bestfit() (*wass2s.was_linear_models.WAS_Ridge_Model* static method), 102

calculate_tercile_probabilities_bestfit() (*wass2s.was_machine_learning.WAS_MARS_Model* static method), 146

calculate_tercile_probabilities_bestfit() (*wass2s.was_machine_learning.WAS_PoissonRegression* static method), 141

calculate_tercile_probabilities_bestfit() (*wass2s.was_machine_learning.WAS_PolynomialRegression* static method), 137

calculate_tercile_probabilities_bestfit() (*wass2s.was_mme.WAS_mme_hpELM* static method), 203

calculate_tercile_probabilities_bestfit() (*wass2s.was_mme.WAS_mme_MLP* static method), 204

calculate_tercile_probabilities_bestfit() (*wass2s.was_mme.WAS_mme_PCR* static method), 205

calculate_tercile_probabilities_bestfit() (*wass2s.was_mme.WAS_mme_RF* static method), 198

calculate_tercile_probabilities_bestfit() (*wass2s.was_mme.WAS_mme_Weighted* static method), 178

calculate_tercile_probabilities_bestfit() (*wass2s.was_mme.WAS_mme_xcELM* static method), 189

calculate_tercile_probabilities_bestfit() (*wass2s.was_mme.WAS_mme_XGBoosting* static method), 200

calculate_tercile_probabilities_bestfit() (*wass2s.was_verification.WAS_Verification* static method), 170

calculate_tercile_probabilities_nonparametric() (*wass2s.was_analog.WAS_Analog* static method), 155

calculate_tercile_probabilities_nonparametric() (*wass2s.was_analog.WAS_Analog__* static method), 158

calculate_tercile_probabilities_nonparametric() (*wass2s.was_cca.WAS_CCA* static method), 123

`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_cca.WAS_CCA_base` static method), 125
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_cca.WAS_CCA_op` static method), 126
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_cca.WAS_CCA_strict` static method), 128
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_cca_2.WAS_KernelCCA_base` static method), 130
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_linear_models.WAS_ElasticNet_Model` static method), 112
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_linear_models.WAS_Lasso_Model` static method), 107
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_linear_models.WAS_LassoLars_Model` static method), 117
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_linear_models.WAS_LinearRegression_Model` static method), 98
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_linear_models.WAS_Ridge_Model` static method), 103
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_machine_learning.WAS_MARS_Model` static method), 147
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_machine_learning.WAS_PoissonRegression` static method), 142
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_machine_learning.WAS_PolynomialRegression` static method), 138
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_mme.WAS_mme_hpELM` static method), 203
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_mme.WAS_mme_MLP` static method), 204
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_mme.WAS_mme_PCR` static method), 205
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_mme.WAS_mme_RF` static method), 198
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_mme.WAS_mme_Weighted` static method), 179
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_mme.WAS_mme_xcELM` static method), 189
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_mme.WAS_mme_XGBoosting` static method), 200
`calculate_tercile_probabilities_nonparametric()`
 (`wass2s.was_verification.WAS_Verification` static method), 170
`CEAC_compute_cessation` (class in `wass2s.ceac_agro`), 207
`CEAC_compute_cessation_dry_spell` (class in `wass2s.ceac_agro`), 207
`CEAC_compute_onset` (class in `wass2s.ceac_agro`), 207
`CEAC_compute_onset_dry_spell` (class in `wass2s.ceac_agro`), 207
`CEAC_count_dry_spells` (class in `wass2s.ceac_agro`), 208
`CEAC_count_wet_spells` (class in `wass2s.ceac_agro`), 208
`compute_normal_mean_var` (in module `wass2s.was_mme`), 190
`cessation_function` (`wass2s.ceac_agro.CEAC_compute_cessation` static method), 207
`cessation_function` (`wass2s.was_compute_predictand.WAS_compute_cessation` static method), 67
`classify` (`wass2s.was_machine_learning.WAS_LogisticRegression_Model` static method), 133
`classify` (`wass2s.was_machine_learning.WAS_LogisticRegression_Model` static method), 134
`classify` (`wass2s.was_verification.WAS_Verification` static method), 163
`classify_data_into_terciles` (`wass2s.was_verification.WAS_Verification` static method), 163
`classify_percent` (`wass2s.was_verification.WAS_Verification` static method), 173
`COLD` (`wass2s.was_compute_predictand.ExtremeType` attribute), 78
`cold_days` (`wass2s.was_compute_predictand.ETCCDIIndices` static method), 80
`cold_nights` (`wass2s.was_compute_predictand.ETCCDIIndices` static method), 80
`composite_plot` (`wass2s.was_analog.WAS_Analog` static method), 156
`composite_plot` (`wass2s.was_analog.WAS_Analog` static method), 159
`compute_heat_wave` (`wass2s.was_compute_predictand.ETCCDIHeatWaveIndices` static method), 83
`compute` (`wass2s.ceac_agro.CEAC_compute_cessation` static method), 207
`compute` (`wass2s.ceac_agro.CEAC_compute_cessation_dry_spell` static method), 207

method), 208

compute () (wass2s.ceac_agro.CEAC_compute_onset method), 207

compute () (wass2s.ceac_agro.CEAC_compute_onset_dry_spell method), 207

compute () (wass2s.ceac_agro.CEAC_count_dry_spells method), 208

compute () (wass2s.ceac_agro.CEAC_count_wet_spells method), 208

compute () (wass2s.was_compute_predictand.WAS_compute_cessation method), 68

compute () (wass2s.was_compute_predictand.WAS_compute_cessation_dry_spells method), 70

compute () (wass2s.was_compute_predictand.WAS_compute_onset method), 64

compute () (wass2s.was_compute_predictand.WAS_compute_onset_dry_spell method), 66

compute () (wass2s.was_compute_predictand.WAS_count_dry_spells method), 71, 73

compute () (wass2s.was_compute_predictand.WAS_count_rainy_days method), 76, 77

compute () (wass2s.was_compute_predictand.WAS_count_wet_spells method), 73, 74

compute () (wass2s.was_compute_predictand.WAS_HeatWaveIndices method), 83

compute () (wass2s.was_compute_predictand.WAS_PrecipIndices method), 81

compute () (wass2s.was_compute_predictand.WAS_TempPercentileIndices method), 79

compute () (wass2s.was_mme.WAS_mme_Weighted method), 177

compute () (wass2s.was_mme.WAS_ProbWeighted method), 180

compute_class () (wass2s.was_machine_learning.WAS_LogisticRegression_Model method), 133, 134

compute_class () (wass2s.was_verification.WAS_Verification method), 163

compute_combined_map () (wass2s.was_mme.WAS_Min2009_ProbWeighted method), 183

compute_crps () (wass2s.was_verification.WAS_Verification method), 169

compute_deterministic_score () (wass2s.was_verification.WAS_Verification method), 163

compute_eof () (wass2s.was_analog.WAS_Analog method), 154

compute_eof () (wass2s.was_analog.WAS_Analog__ method), 158

compute_gridpoint_terciles_from_obs () (wass2s.was_mme.WAS_mme_FastBMA static method), 194

compute_gridpoint_terciles_from_obs () (wass2s.was_mme.WAS_mme_FullBMA static method), 196

compute_hyperparameters () (wass2s.was_cca_2.WAS_EOF_KCCA method), 131

compute_hyperparameters () (wass2s.was_cca_2.WAS_KernelCCA_base method), 129

compute_hyperparameters () (wass2s.was_linear_models.WAS_ElasticNet_Model method), 109, 110

compute_hyperparameters () (wass2s.was_linear_models.WAS_Lasso_Model method), 104, 105

compute_hyperparameters () (wass2s.was_linear_models.WAS_LassoLars_Model method), 114, 115

compute_hyperparameters () (wass2s.was_linear_models.WAS_Ridge_Model method), 100, 101

compute_hyperparameters () (wass2s.was_machine_learning.WAS_LogisticRegression_Model method), 133, 134

compute_hyperparameters () (wass2s.was_mme.WAS_mme_ELR method), 186

compute_hyperparameters () (wass2s.was_mme.WAS_mme_gaussian_process method), 188

compute_hyperparameters () (wass2s.was_mme.WAS_mme_hpELM method), 203

compute_hyperparameters () (wass2s.was_mme.WAS_mme_logistic method), 204

compute_hyperparameters () (wass2s.was_mme.WAS_mme_MLP method), 204

compute_hyperparameters () (wass2s.was_mme.WAS_mme_RF method), 198

compute_hyperparameters () (wass2s.was_mme.WAS_mme_XGBoosting method), 200

compute_insitu () (wass2s.ceac_agro.CEAC_compute_cessation method), 207

compute_insitu () (wass2s.ceac_agro.CEAC_compute_cessation_dry_spell method), 208

compute_insitu () (wass2s.ceac_agro.CEAC_compute_onset method), 207

compute_insitu () (wass2s.ceac_agro.CEAC_compute_onset_dry_spell method), 207

compute_insitu () (wass2s.ceac_agro.CEAC_count_dry_spells method), 208

compute_insitu () (wass2s.ceac_agro.CEAC_count_wet_spells method), 208

`method`), 208
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 67
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 70
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 64
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 66
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 71, 72
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 76, 77
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 73, 75
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 81
`compute_insitu()` (wass2s.was_compute_predictand.WAS_compute_insitu_ensemble method), 79
`compute_model()` (wass2s.was_analog.WAS_Analog method), 154
`compute_model()` (wass2s.was_analog.WAS_Analog method), 158
`compute_model()` (wass2s.was_cca.WAS_CCA method), 123
`compute_model()` (wass2s.was_cca.WAS_CCA_base method), 125
`compute_model()` (wass2s.was_cca.WAS_CCA_op method), 126
`compute_model()` (wass2s.was_cca.WAS_CCA_strict method), 127
`compute_model()` (wass2s.was_cca_2.WAS_EOF_KCCA method), 131
`compute_model()` (wass2s.was_cca_2.WAS_KernelCCA_base method), 130
`compute_model()` (wass2s.was_linear_models.WAS_ElasticNet_Model method), 109, 111
`compute_model()` (wass2s.was_linear_models.WAS_Lasso_Model method), 104, 106
`compute_model()` (wass2s.was_linear_models.WAS_Lasso_Model method), 114, 116
`compute_model()` (wass2s.was_linear_models.WAS_LinearRegression_Model method), 95, 96
`compute_model()` (wass2s.was_linear_models.WAS_Ridge_Model method), 100, 102
`compute_model()` (wass2s.was_machine_learning.WAS_LogisticRegression_Model method), 133, 134
`compute_model()` (wass2s.was_machine_learning.WAS_MARS_Model method), 144, 146
`compute_model()` (wass2s.was_machine_learning.WAS_NegativeBinomial_Model method), 149
`compute_model()` (wass2s.was_machine_learning.WAS_PoissonRegression_Model method), 139, 141
`compute_model()` (wass2s.was_machine_learning.WAS_PolynomialRegression method), 135, 136
`compute_model()` (wass2s.was_mme.WAS_mme_ELR method), 186
`compute_model()` (wass2s.was_mme.WAS_mme_ExpertKnowledge method), 206
`compute_model()` (wass2s.was_mme.WAS_mme_FullBMA method), 196
`compute_model()` (wass2s.was_mme.WAS_mme_gaussian_process method), 188
`compute_model()` (wass2s.was_mme.WAS_mme_hpELM method), 203
`compute_model()` (wass2s.was_mme.WAS_mme_logistic method), 184
`compute_model()` (wass2s.was_mme.WAS_mme_MLP method), 204
`compute_model()` (wass2s.was_mme.WAS_mme_NGR_Gaussian method), 192
`compute_model()` (wass2s.was_mme.WAS_mme_NGR_NonGaussian method), 193
`compute_model()` (wass2s.was_mme.WAS_mme_PCR method), 205
`compute_model()` (wass2s.was_mme.WAS_mme_RF method), 198
`compute_model()` (wass2s.was_mme.WAS_mme_xcELM method), 188
`compute_model()` (wass2s.was_mme.WAS_mme_xcELR method), 183
`compute_model()` (wass2s.was_mme.WAS_mme_XGBoosting method), 200
`compute_model()` (wass2s.was_pcr.WAS_PCR method), 120
`compute_one_year_rpss()` (wass2s.was_verification.WAS_Verification method), 174
`compute_one_year_rpss_()` (wass2s.was_verification.WAS_Verification method), 174
`compute_other_indices()` (in module `wass2s.utils`), 54
`compute_mme_probabilities()` (wass2s.was_mme.WAS_Min2009_ProbWeighted method), 182
`compute_prob()` (wass2s.was_analog.WAS_Analog method), 155
`compute_prob()` (wass2s.was_analog.WAS_Analog method), 123
`compute_prob()` (wass2s.was_cca.WAS_CCA method), 123
`compute_prob()` (wass2s.was_cca.WAS_CCA_base method), 126
`compute_prob()` (wass2s.was_cca.WAS_CCA_op method), 126
`compute_prob()` (wass2s.was_cca.WAS_CCA_strict method), 129

compute_prob() (wass2s.was_cca_2.WAS_KernelCCA_base method), 130

compute_prob() (wass2s.was_linear_models.WAS_ElasticNet_Model static method), 112

compute_prob() (wass2s.was_linear_models.WAS_Lasso_Model static method), 107

compute_prob() (wass2s.was_linear_models.WAS_LassoLars_Model static method), 117

compute_prob() (wass2s.was_linear_models.WAS_LinearRegression_Model static method), 98

compute_prob() (wass2s.was_linear_models.WAS_Ridge_Model static method), 103

compute_prob() (wass2s.was_machine_learning.WAS_MARS_Model static method), 147

compute_prob() (wass2s.was_machine_learning.WAS_NegativeBinomial_Model static method), 149

compute_prob() (wass2s.was_machine_learning.WAS_PoissonRegression static method), 142

compute_prob() (wass2s.was_machine_learning.WAS_PolynomialRegression static method), 138

compute_prob() (wass2s.was_mme.WAS_mme_hpELM static method), 203

compute_prob() (wass2s.was_mme.WAS_mme_MLP static method), 204

compute_prob() (wass2s.was_mme.WAS_mme_MVA static method), 194

compute_prob() (wass2s.was_mme.WAS_mme_PCR static method), 205

compute_prob() (wass2s.was_mme.WAS_mme_RF static method), 198

compute_prob() (wass2s.was_mme.WAS_mme_Weighted static method), 179

compute_prob() (wass2s.was_mme.WAS_mme_xcELM static method), 189

compute_prob() (wass2s.was_mme.WAS_mme_XGBoosting static method), 200

compute_prob() (wass2s.was_pcr.WAS_PCR static method), 121

compute_probabilistic_score() (wass2s.was_verification.WAS_Verification static method), 169

compute_sst_indices() (in module wass2s.utils), 53

conditional_merging() (wass2s.was_merge_predictand.WAS_Merging_Extended static method), 90

Corr_Based() (wass2s.was_analog.WAS_Analog static method), 154

Corr_Based() (wass2s.was_analog.WAS_Analog static method), 158

count_dry_spells() (wass2s.ceac_agro.CEAC_count_dry_spells static method), 208

count_dry_spells() (wass2s.was_compute_predictand.WAS_count_dry_spells static method), 72

count_rainy_days() (wass2s.was_compute_predictand.WAS_count_rainy_days static method), 77

count_wet_spells() (wass2s.ceac_agro.CEAC_count_wet_spells static method), 208

cross_validate() (wass2s.was_compute_predictand.WAS_count_wet_spells static method), 74

cross_validate() (wass2s.was_cross_validate.WAS_Cross_Validator static method), 93

crps_censored_normal() (in module wass2s.mme), 191

crps_gaussian() (in module wass2s.mme), 191

crps_truncated_normal() (in module wass2s.mme), 191

cross_validate() (in module wass2s.was_cross_validate), 92

day_of_year() (wass2s.ceac_agro.CAF_AgroClimateBase static method), 207

day_of_year() (wass2s.was_compute_predictand.WAS_compute_cessation static method), 67

day_of_year() (wass2s.was_compute_predictand.WAS_compute_cessation static method), 70

day_of_year() (wass2s.was_compute_predictand.WAS_compute_onset static method), 63

day_of_year() (wass2s.was_compute_predictand.WAS_compute_onset static method), 66

days_in_month() (wass2s.was_download.WAS_Download static method), 40

encode_cf() (in module wass2s.utils), 48

default_criteria(wass2s.was_compute_predictand.WAS_compute_cessation attribute), 67

default_criteria(wass2s.was_compute_predictand.WAS_compute_cessation attribute), 68

default_criteria(wass2s.was_compute_predictand.WAS_compute_onset attribute), 63

default_criteria(wass2s.was_compute_predictand.WAS_compute_onset attribute), 65

detect_skewness() (wass2s.was_transformdata.WAS_TransformData static method), 62

detrended_data() (in module wass2s.utils), 52

DIST_CODE_TO_NAME (wass2s.was_mme.WAS_mme_FullBMA attribute), 196

DIST_NAME_TO_CODE (wass2s.was_mme.WAS_mme_FullBMA attribute), 196

doBC () (wass2s.was_bias_correction.WAS_bias_correction method), 65
 static method), 86 dry_spell_onset_function ()
 doDist () (wass2s.was_bias_correction.WAS_bias_correction (wass2s.was_compute_predictand.WAS_compute_onset_dry_spell
 static method), 86 method), 65
 doMean () (wass2s.was_bias_correction.WAS_bias_correction cess_func () (wass2s.ceac_agro.CEAC_compute_cessation_dry_spell
 static method), 86 static method), 207
 doQmap () (wass2s.was_bias_correction.WAS_Qmap ds_onset_func () (wass2s.ceac_agro.CEAC_compute_onset_dry_spell
 static method), 84 static method), 207
 doQmapDIST () (wass2s.was_bias_correction.WAS_Qmap duration (wass2s.was_compute_predictand.HeatWaveDefinition
 static method), 85 attribute), 82
 doQmapPTF () (wass2s.was_bias_correction.WAS_Qmap
 static method), 85
E
 doQmapQUANT () (wass2s.was_bias_correction.WAS_Qmap end_date (wass2s.was_compute_predictand.HeatWaveDefinition
 static method), 85 attribute), 82
 doQmapRQUANT () (wass2s.was_bias_correction.WAS_Qmap eof_model (wass2s.was_pcr.WAS_PCR attribute), 120
 static method), 85 et0_fao56_daily () (in module wass2s.utils), 60
 doQmapSSPLIN () (wass2s.was_bias_correction.WAS_Qmap ETCCDIHeatWaveIndices (class in
 static method), 85 wass2s.was_compute_predictand), 83
 doQuant () (wass2s.was_bias_correction.WAS_bias_correction ETCCDITempIndices (class in
 static method), 86 wass2s.was_compute_predictand), 79
 doVarscale () (wass2s.was_bias_correction.WAS_bias_correction evaluate_bias_correction ()
 static method), 86 (wass2s.was_bias_correction.WAS_Qmap
 static method), 85
 download_and_process ()
 (wass2s.was_analog.WAS_Analog method), extract_leading_eof_component () (in mod-
 154 ule wass2s.utils), 52
 download_and_process ()
 (wass2s.was_analog.WAS_Analog method), extraterrestrial_radiation () (in module
 158 wass2s.utils), 59
 download_data () (in module ExtremeType (class in
 wass2s.was_seasonal_analysis), 209 wass2s.was_compute_predictand), 78
 download_file () (in module wass2s.utils), 50
F
 download_models ()
 (wass2s.was_analog.WAS_Analog method), fit () (wass2s.was_eof.WAS_EOF method), 91
 154 fit () (wass2s.was_machine_learning.MARS method),
 144
 download_models ()
 (wass2s.was_analog.WAS_Analog method), fit () (wass2s.was_mme.HPELMWrapper method), 201
 158 fit () (wass2s.was_mme.WAS_mme_ExpertKnowledge
 method), 206
 download_nmme_txt_with_progress ()
 (wass2s.was_download.WAS_Download fit () (wass2s.was_mme.WAS_mme_FastBMA method),
 method), 40 195
 download_reanalysis ()
 (wass2s.was_analog.WAS_Analog method), fit () (wass2s.was_mme.WAS_mme_FullBMA method),
 154 196
 download_reanalysis ()
 (wass2s.was_analog.WAS_Analog method), fit () (wass2s.was_mme.WAS_mme_MVA method), 193
 157 fit () (wass2s.was_mme.WAS_mme_NGR_Gaussian
 method), 192
 download_reanalysis ()
 (wass2s.was_analog.WAS_Analog method), fit () (wass2s.was_mme.WAS_mme_NGR_NonGaussian
 method), 193
 153 fit_best_distribution ()
 (wass2s.was_transformdata.WAS_TransformData
 method), 62
 dry_spell_cessation_function ()
 (wass2s.was_compute_predictand.WAS_compute_cessation_dry_spell fit_cca () (wass2s.was_cca.WAS_CCA method), 122
 method), 69 fit_cca () (wass2s.was_cca.WAS_CCA_base method),
 125
 dry_spell_onset_function ()
 (wass2s.was_compute_predictand.WAS_compute_onset_dry_spell fit_cca () (wass2s.was_cca.WAS_CCA_op method),
 126

`fit_cca()` (wass2s.was_cca.WAS_CCA_strict method), 86
`fit_cca()` (wass2s.was_cca_2.WAS_KernelCCA_base method), 130
`fit_one_grid_min_crps()` (in module wass2s.was_mme), 191
`fit_predict()` (wass2s.was_linear_models.WAS_ElasticNet_Model method), 109, 111
`fit_predict()` (wass2s.was_linear_models.WAS_Lasso_Model method), 104, 106
`fit_predict()` (wass2s.was_linear_models.WAS_LassoLars_Model method), 114, 116
`fit_predict()` (wass2s.was_linear_models.WAS_LinearRegression_Model method), 95, 96
`fit_predict()` (wass2s.was_linear_models.WAS_Ridge_Model method), 100, 101
`fit_predict()` (wass2s.was_machine_learning.WAS_HurdleNB_Model method), 150
`fit_predict()` (wass2s.was_machine_learning.WAS_LogisticRegression_Model method), 133, 134
`fit_predict()` (wass2s.was_machine_learning.WAS_MARS_Model method), 144, 146
`fit_predict()` (wass2s.was_machine_learning.WAS_NegativeBinomial_Model method), 149
`fit_predict()` (wass2s.was_machine_learning.WAS_PoissonRegression_Model method), 139, 140
`fit_predict()` (wass2s.was_machine_learning.WAS_PolynomialRegression_Model method), 135, 136
`fit_predict()` (wass2s.was_machine_learning.WAS_ZINB_Model method), 149
`fit_transform_loocv()` (wass2s.was_mme.WAS_mme_MVA method), 193
`fitBC()` (wass2s.was_bias_correction.WAS_bias_correction static method), 86
`fitDist()` (wass2s.was_bias_correction.WAS_bias_correction static method), 86
`fitMean()` (wass2s.was_bias_correction.WAS_bias_correction static method), 86
`fitQmap()` (wass2s.was_bias_correction.WAS_Qmap static method), 84
`fitQmapDIST()` (wass2s.was_bias_correction.WAS_Qmap static method), 85
`fitQmapPTF()` (wass2s.was_bias_correction.WAS_Qmap static method), 85
`fitQmapQUANT()` (wass2s.was_bias_correction.WAS_Qmap static method), 85
`fitQmapRQUANT()` (wass2s.was_bias_correction.WAS_Qmap static method), 85
`fitQmapSSPLIN()` (wass2s.was_bias_correction.WAS_Qmap static method), 85
`fitQuant()` (wass2s.was_bias_correction.WAS_bias_correction static method), 86
`fitVarscale()` (wass2s.was_bias_correction.WAS_bias_correction static method), 86
`fix_time_coord()` (in module wass2s.utils), 49
`forecast()` (wass2s.was_analog.WAS_Analog method), 155
`forecast()` (wass2s.was_analog.WAS_Analog__ method), 159
`forecast()` (wass2s.was_cca.WAS_CCA method), 124
`forecast()` (wass2s.was_cca.WAS_CCA_base method), 125
`forecast()` (wass2s.was_cca.WAS_CCA_op method), 126
`forecast()` (wass2s.was_cca.WAS_CCA_strict method), 127
`forecast()` (wass2s.was_cca_2.WAS_EOF_KCCA method), 131
`forecast()` (wass2s.was_cca_2.WAS_KernelCCA_base method), 130
`forecast()` (wass2s.was_linear_models.WAS_ElasticNet_Model method), 108
`forecast()` (wass2s.was_linear_models.WAS_Lasso_Model method), 108
`forecast()` (wass2s.was_linear_models.WAS_LassoLars_Model method), 108
`forecast()` (wass2s.was_linear_models.WAS_LinearRegression_Model method), 98
`forecast()` (wass2s.was_linear_models.WAS_Ridge_Model method), 103
`forecast()` (wass2s.was_machine_learning.WAS_LogisticRegression_Model method), 134
`forecast()` (wass2s.was_machine_learning.WAS_MARS_Model method), 148
`forecast()` (wass2s.was_machine_learning.WAS_NegativeBinomial_Model method), 149
`forecast()` (wass2s.was_machine_learning.WAS_PoissonRegression_Model method), 143
`forecast()` (wass2s.was_machine_learning.WAS_PolynomialRegression_Model method), 138
`forecast()` (wass2s.was_mme.WAS_mme_ELR method), 187
`forecast()` (wass2s.was_mme.WAS_mme_gaussian_process method), 188
`forecast()` (wass2s.was_mme.WAS_mme_hpELM method), 203
`forecast()` (wass2s.was_mme.WAS_mme_logistic method), 185
`forecast()` (wass2s.was_mme.WAS_mme_MLP method), 204
`forecast()` (wass2s.was_mme.WAS_mme_MVA method), 194
`forecast()` (wass2s.was_mme.WAS_mme_NGR_Gaussian method), 192
`forecast()` (wass2s.was_mme.WAS_mme_NGR_NonGaussian method), 193

`method`), 205
`forecast()` (`wass2s.was_mme.WAS_mme_RF`
`method`), 198
`forecast()` (`wass2s.was_mme.WAS_mme_Weighted`
`method`), 179
`forecast()` (`wass2s.was_mme.WAS_mme_xcELM`
`method`), 190
`forecast()` (`wass2s.was_mme.WAS_mme_xcELR`
`method`), 183
`forecast()` (`wass2s.was_mme.WAS_mme_XGBoosting`
`method`), 200
`forecast()` (`wass2s.was_pcr.WAS_PCR` `method`), 121
`format_grid_output()`
(`wass2s.ceac_agro.CAF_AgroClimateBase`
`method`), 207
`forward_pass()` (`wass2s.was_machine_learning.MARS`
`method`), 143

G

`gcm_compute_prob()`
(`wass2s.was_verification.WAS_Verification`
`method`), 170
`gcm_validation_compute()`
(`wass2s.was_verification.WAS_Verification`
`method`), 171
`get_best_models()` (in module `wass2s.utils`), 56
`get_canonical_correlations()`
(`wass2s.was_cca_2.WAS_KernelCCA_base`
`method`), 130
`get_components()` (`wass2s.was_cca_2.WAS_KernelCCA_base`
`method`), 130
`get_fit_stats()` (`wass2s.was_mme.WAS_mme_NGR_Gaussian`
`method`), 192
`get_fit_stats()` (`wass2s.was_mme.WAS_mme_NGR_NonGaussian`
`method`), 193
`get_formula()` (`wass2s.was_machine_learning.MARS`
`method`), 144
`get_hpo_summary()`
(`wass2s.was_mme.WAS_mme_gaussian_process`
`method`), 188
`get_index_definition()`
(`wass2s.was_compute_predictand.WAS_PrecipIndices`
`method`), 82
`get_index_definition()`
(`wass2s.was_compute_predictand.WAS_TempPercentileIndices`
`method`), 79
`get_index_for_station()`
(`wass2s.ceac_agro.CAF_AgroClimateBase`
`method`), 207
`get_model_info()` (`wass2s.was_cca_2.WAS_KernelCCA_base`
`method`), 130
`get_n_splits()` (`wass2s.was_cross_validate.CustomTimeSeriesSplit`
`method`), 93
`get_optimization_results()`

(`wass2s.was_mme.WAS_mme_hpELM` `method`),
203
`get_optimization_results()`
(`wass2s.was_mme.WAS_mme_logistic` `method`),
185
`get_params()` (`wass2s.was_mme.HPELMWrapper`
`method`), 201
`get_scores()` (`wass2s.was_cca_2.WAS_KernelCCA_base`
`method`), 130
`get_scores_metadata()`
(`wass2s.was_verification.WAS_Verification`
`method`), 160
`get_shapefile()` (in module `wass2s.utils`), 56
`get_shapefile_()` (in module `wass2s.utils`), 56

H

`handle_skewness()`
(`wass2s.was_transformdata.WAS_TransformData`
`method`), 62
`heat_wave_duration_index()`
(`wass2s.was_compute_predictand.ETCCDIHeatWaveIndices`
`static method`), 83
`heat_wave_frequency()`
(`wass2s.was_compute_predictand.ETCCDIHeatWaveIndices`
`static method`), 83
`HeatWaveDefinition` (class in
`wass2s.was_compute_predictand`), 82
`HeatWaveMetric` (class in
`wass2s.was_compute_predictand`), 82
`HeatWaveMetric` (`wass2s.was_compute_predictand.ExtremeType`
attribute), 78
`HeatWaveIndices` (`wass2s.was_compute_predictand.ETCCDIHeatWaveIndices`
`static method`), 80
`HeatWaveMetrics` (`wass2s.was_compute_predictand.ETCCDIHeatWaveIndices`
`static method`), 80
`HPELMWrapper` (class in `wass2s.was_mme`), 201
`HWDI` (`wass2s.was_compute_predictand.HeatWaveMetric`
attribute), 82
`HWF` (`wass2s.was_compute_predictand.HeatWaveMetric` at-
tribute), 82
`HWN` (`wass2s.was_compute_predictand.HeatWaveMetric` at-
tribute), 82

I

`identify_analogs_bmu()`
(`wass2s.was_analog.WAS_Analog` `method`),
154
`identify_analogs_bmu()`
(`wass2s.was_analog.WAS_Analog__` `method`),
158
`ignorance_score()`
(`wass2s.was_verification.WAS_Verification`
`method`), 165

index_of_agreement() (wass2s.was_verification.WAS_Verification method), 161

inv_boxcox() (in module wass2s.was_transformdata), 61

inv_yeojohnson() (in module wass2s.was_transformdata), 61

inverse_transform() (wass2s.was_eof.WAS_EOF method), 91

inverse_transform() (wass2s.was_transformdata.WAS_TransformData method), 62

K

kling_gupta_efficiency() (wass2s.was_verification.WAS_Verification method), 160

KMeans_Based() (wass2s.was_analog.WAS_Analog method), 154

KMeans_Based() (wass2s.was_analog.WAS_Analog__ method), 158

kriging_with_external_drift() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90

L

load_gridded_predictor() (in module wass2s.utils), 52

M

main_driver() (in module wass2s.was_seasonal_analysis), 211

MARS (class in wass2s.was_machine_learning), 143

max_temp (wass2s.was_compute_predictand.HeatWaveDefinition attribute), 82

mean_absolute_error() (wass2s.was_verification.WAS_Verification method), 161

mean_temp (wass2s.was_compute_predictand.HeatWaveDefinition attribute), 82

MODE_CODE (wass2s.was_mme.WAS_mme_FullBMA attribute), 196

ModelsName() (wass2s.was_download.WAS_Download method), 39

module

- wass2s.ceac_agro, 206
- wass2s.utils, 47
- wass2s.was_analog, 151
- wass2s.was_bias_correction, 84
- wass2s.was_cca, 122
- wass2s.was_cca_2, 129
- wass2s.was_compute_predictand, 62
- wass2s.was_cross_validate, 92
- wass2s.was_download, 38

- wass2s.was_eof, 91
- wass2s.was_linear_models, 94
- wass2s.was_machine_learning, 131
- wass2s.was_merge_predictand, 87
- wass2s.was_mme, 175
- wass2s.was_pcr, 119
- wass2s.was_seasonal_analysis, 208
- wass2s.was_transformdata, 61
- wass2s.was_verification, 159

multiplicative_bias() (wass2s.was_merge_predictand.WAS_Merging method), 89

myfill() (in module wass2s.was_mme), 176

N

nash_sutcliffe_efficiency() (wass2s.was_verification.WAS_Verification method), 162

neural_network_kriging() (wass2s.was_merge_predictand.WAS_Merging method), 88

neural_network_kriging_() (wass2s.was_merge_predictand.WAS_Merging method), 88

O

onset_function() (wass2s.ceac_agro.CEAC_compute_onset static method), 207

onset_function() (wass2s.was_compute_predictand.WAS_compute_onset method), 64

optimal_interpolation() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90

optimize() (wass2s.was_machine_learning.BaseOptimizer method), 132

optimizer (wass2s.was_machine_learning.WAS_LogisticRegression_Mode attribute), 133

output_format_value() (wass2s.ceac_agro.CAF_AgroClimateBase method), 207

P

parse_cpt_data_optimized() (wass2s.was_download.WAS_Download method), 40

parse_variable() (in module wass2s.utils), 50

Pca_Based() (wass2s.was_analog.WAS_Analog method), 154

Pca_Based() (wass2s.was_analog.WAS_Analog__ method), 158

pearson_corr() (wass2s.was_verification.WAS_Verification method), 160

`plot_best_fit_map()`
 (*wass2s.was_transformdata.WAS_TransformData*
 method), 62
`plot_cca_results()` (*wass2s.was_cca.WAS_CCA*
 method), 124
`plot_cca_results()`
 (*wass2s.was_cca.WAS_CCA_base* *method*),
 125
`plot_cca_results()`
 (*wass2s.was_cca.WAS_CCA_op* *method*),
 126
`plot_cca_results()`
 (*wass2s.was_cca.WAS_CCA_strict* *method*),
 128
`plot_cca_results()`
 (*wass2s.was_cca_2.WAS_KernelCCA_base*
 method), 130
`plot_date()` (*in module wass2s.utils*), 58
`plot_det_forecasts()` (*in module wass2s.utils*), 56
`plot_EOF()` (*wass2s.was_eof.WAS_EOF method*), 91
`plot_extreme_quantiles_group()`
 (*wass2s.was_bias_correction.WAS_Qmap*
 static method), 86
`plot_fraction_group()`
 (*wass2s.was_bias_correction.WAS_Qmap*
 static method), 85
`plot_georeferenced_forecast_map()` (*in*
 module wass2s.utils), 57
`plot_hovmoller()` (*in* *module*
 wass2s.was_seasonal_analysis), 210
`plot_map()` (*in module wass2s.utils*), 56
`plot_map()` (*in module wass2s.was_download*), 46
`plot_maps()` (*in* *module*
 wass2s.was_seasonal_analysis), 210
`plot_mean_wet_group()`
 (*wass2s.was_bias_correction.WAS_Qmap*
 static method), 86
`plot_merging_comparaison()`
 (*wass2s.was_merge_predictand.WAS_Merging*
 method), 89
`plot_merging_comparison()`
 (*wass2s.was_merge_predictand.WAS_Merging_Extended*
 method), 90
`plot_model_score()`
 (*wass2s.was_verification.WAS_Verification*
 method), 170
`plot_models_score()`
 (*wass2s.was_verification.WAS_Verification*
 method), 170
`plot_prob_forecasts()` (*in module wass2s.utils*),
 56
`plot_prob_forecasts_()` (*in module wass2s.utils*),
 56
`plot_prob_forecastsAlpha()` (*in* *module*
 wass2s.utils), 56
`plot_roc_curves()`
 (*wass2s.was_verification.WAS_Verification*
 method), 168
`plot_tercile()` (*in module wass2s.utils*), 57
`polygons_to_prob_ds()` (*in module wass2s.utils*),
 57
`pre_process_biophysical_model()` (*in module*
 wass2s.utils), 58
`predict()` (*wass2s.was_machine_learning.MARS*
 method), 144
`predict()` (*wass2s.was_mme.HPELMWrapper*
 method), 201
`predict()` (*wass2s.was_mme.WAS_mme_ExpertKnowledge*
 method), 206
`predict()` (*wass2s.was_mme.WAS_mme_NGR_Gaussian*
 method), 192
`predict()` (*wass2s.was_mme.WAS_mme_NGR_NonGaussian*
 method), 193
`predict_mean()` (*wass2s.was_mme.WAS_mme_MVA*
 method), 194
`predict_probabilistic()`
 (*wass2s.was_mme.WAS_mme_FastBMA*
 method), 195
`predict_probabilistic()`
 (*wass2s.was_mme.WAS_mme_FullBMA*
 method), 196
`predictant_mask()` (*in module wass2s.utils*), 52
`prepare_predictand()` (*in module wass2s.utils*), 52
`preprocess_data()` (*wass2s.was_cca.WAS_CCA*
 method), 122
`preprocess_data()`
 (*wass2s.was_cca.WAS_CCA_base* *method*),
 124
`preprocess_data()` (*wass2s.was_cca.WAS_CCA_op*
 method), 126
`preprocess_data()`
 (*wass2s.was_cca.WAS_CCA_strict* *method*),
 127
`preprocess_data()`
 (*wass2s.was_cca_2.WAS_KernelCCA_base*
 method), 129
`preprocess_forecast_data()`
 (*wass2s.was_cca.WAS_CCA_base* *method*),
 125
`preprocess_forecast_data()`
 (*wass2s.was_cca_2.WAS_KernelCCA_base*
 method), 129
`preprocess_test_data()`
 (*wass2s.was_cca.WAS_CCA method*), 122
`preprocess_test_data()`
 (*wass2s.was_cca.WAS_CCA_base* *method*),
 124
`preprocess_test_data()`

(wass2s.was_cca.WAS_CCA_op method), 126
 preprocess_test_data() (wass2s.was_cca.WAS_CCA_strict method), 127
 preprocess_test_data() (wass2s.was_cca_2.WAS_KernelCCA_base method), 129
 proceed_seasonal_daily_bias_correction() (in module wass2s.utils), 58
 process_datasets_for_mme() (in module wass2s.was_mme), 175
 process_model_for_other_params() (in module wass2s.utils), 57
 process_variable() (in module wass2s.was_seasonal_analysis), 211
Q
 Q_HIGH (wass2s.was_cca_2.WAS_KernelCCA_base attribute), 129
 Q_LOW (wass2s.was_cca_2.WAS_KernelCCA_base attribute), 129
R
 rainf_zone() (wass2s.was_compute_predictand.WAS_compute_cessation method), 67
 rainf_zone() (wass2s.was_compute_predictand.WAS_compute_cessation_dry_spell method), 69
 rainf_zone() (wass2s.was_compute_predictand.WAS_compute_onset method), 64
 rainf_zone() (wass2s.was_compute_predictand.WAS_compute_onset_dry_spell method), 65
 random_forest_merging() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90
 ratio_to_average() (wass2s.was_verification.WAS_Verification method), 173
 ReanalysisName() (wass2s.was_download.WAS_Download method), 40
 regression_kriging() (wass2s.was_merge_predictand.WAS_Merging method), 88
 regression_kriging() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90
 reliability_diagram() (wass2s.was_verification.WAS_Verification method), 168
 reliability_score_grid() (wass2s.was_verification.WAS_Verification method), 167
 resolution_and_reliability_over_all_grid() (wass2s.was_verification.WAS_Verification method), 167
 resolution_and_reliability_over_all_grid_per_category() (wass2s.was_verification.WAS_Verification method), 168
 resolution_and_reliability_per_category_grid() (wass2s.was_verification.WAS_Verification method), 167
 resolution_score_grid() (wass2s.was_verification.WAS_Verification method), 166
 retrieve_several_zones_for_PCR() (in module wass2s.utils), 54
 retrieve_single_zone_for_PCR() (in module wass2s.utils), 55
 reverse_standardize() (in module wass2s.utils), 51
 revert_grid_index() (wass2s.ceac_agro.CAF_AgroClimateBase method), 207
 revert_to_index() (wass2s.ceac_agro.CAF_AgroClimateBase method), 207
 root_mean_square_error() (wass2s.was_verification.WAS_Verification method), 161
S
 score() (wass2s.was_mme.HPELMWrapper method), 201
 set_hyperparameters() (wass2s.was_cca_2.WAS_KernelCCA_base method), 130
 set_params() (wass2s.was_mme.HPELMWrapper method), 201
 shift_gridded_data() (wass2s.ceac_agro.CAF_AgroClimateBase method), 207
 SHIFT_OFFSET (wass2s.ceac_agro.CAF_AgroClimateBase attribute), 207
 show() (wass2s.was_seasonal_analysis.BOMViewer method), 212
 show() (wass2s.was_seasonal_analysis.C3SViewer method), 211
 simple_bias_adjustment() (wass2s.was_merge_predictand.WAS_Merging method), 88
 simple_bias_adjustment() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90
 SOM() (wass2s.was_analog.WAS_Analog method), 154
 SOM() (wass2s.was_analog.WAS_Analog__ method), 158
 spearman_corr() (wass2s.was_verification.WAS_Verification method), 161

split() (wass2s.was_cross_validate.CustomTimeSeriesSplit method), 92

standardize_timeseries() (in module wass2s.utils), 50

standardize_timeseries() (wass2s.was_analog.WAS_Analog method), 154

standardize_timeseries() (wass2s.was_analog.WAS_Analog__ method), 158

start_date(wass2s.was_compute_predictand.HeatWaveDefinition attribute), 82

summary() (wass2s.was_mme.WAS_mme_ExpertKnowledge method), 206

summary() (wass2s.was_mme.WAS_mme_NGR_Gaussian method), 192

summary() (wass2s.was_mme.WAS_mme_NGR_NonGaussian method), 193

svp() (in module wass2s.utils), 59

T

taylor_diagram() (wass2s.was_verification.WAS_Verification method), 162

to_iridl_lat() (in module wass2s.utils), 48

to_iridl_lon() (in module wass2s.utils), 48

transform() (wass2s.was_eof.WAS_EOF method), 91

transform() (wass2s.was_mme.WAS_mme_MVA method), 193

transform() (wass2s.was_mme.WAS_mme_NGR_Gaussian method), 192

transform_and_shift_cdt() (wass2s.ceac_agro.CAF_AgroClimateBase method), 207

transform_cdt() (wass2s.was_compute_predictand.WAS_compute_cessation static method), 67

transform_cdt() (wass2s.was_compute_predictand.WAS_compute_cessation_dry_spell static method), 69

transform_cdt() (wass2s.was_compute_predictand.WAS_compute_onset static method), 63

transform_cdt() (wass2s.was_compute_predictand.WAS_compute_onset_dry_spell static method), 65

transform_cdt() (wass2s.was_compute_predictand.WAS_count_dry_spells static method), 71

transform_cdt() (wass2s.was_compute_predictand.WAS_count_rainy_days static method), 76

transform_cdt() (wass2s.was_compute_predictand.WAS_count_wet_spells static method), 74

transform_cdt() (wass2s.was_compute_predictand.WAS_PrecipIndices static method), 81

transform_cdt() (wass2s.was_compute_predictand.WAS_TempPercentileIndices static method), 79

transform_cdt() (wass2s.was_merge_predictand.WAS_Merging method), 87

transform_cdt() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90

transform_cpt() (wass2s.was_compute_predictand.WAS_count_dry_spell static method), 72

transform_cpt() (wass2s.was_merge_predictand.WAS_Merging method), 87

transform_cpt() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90

transform_score() (wass2s.was_mme.WAS_mme_Weighted method), 177

truncated_normal_mean_var() (in module wass2s.was_mme), 191

V

VALID_FAMILIES(wass2s.was_mme.WAS_mme_NGR_NonGaussian attribute), 193

VALID_OCC(wass2s.was_mme.WAS_mme_NGR_NonGaussian attribute), 193

validate_merging_methods() (wass2s.was_merge_predictand.WAS_Merging_Extended method), 90

VAR_CONFIG (in module wass2s.was_seasonal_analysis), 208

verify_station_network() (in module wass2s.utils), 58

W

WAS_AdaBoost_Model (class in wass2s.was_machine_learning), 150

WAS_Analog (class in wass2s.was_analog), 152

WAS_Analog__ (class in wass2s.was_analog), 156

WAS_bias_correction (class in wass2s.was_bias_correction), 86

WAS_CCA (class in wass2s.was_cca), 122

WAS_CCA_op (class in wass2s.was_cca), 125

WAS_CCA_predict (class in wass2s.was_cca), 127

WAS_compute_cessation (class in wass2s.was_compute_predictand), 66

WAS_compute_cessation_dry_spell (class in wass2s.was_compute_predictand), 68

WAS_compute_onset (class in wass2s.was_compute_predictand), 62

WAS_compute_onset_dry_spell (class in wass2s.was_compute_predictand), 64

WAS_count_dry_spells (class in wass2s.was_compute_predictand), 70

WAS_count_rainy_days (class in wass2s.was_compute_predictand), 75

WAS_count_wet_spells (class in wass2s.was_compute_predictand), 73

WAS_Cross_Validator (class in wass2s.was_cross_validate), 93

WAS_Download (class in wass2s.was_download), 39
 WAS_Download_AgroIndicators ()
 (wass2s.was_download.WAS_Download
 method), 42
 WAS_Download_AgroIndicators_daily ()
 (wass2s.was_download.WAS_Download
 method), 41
 WAS_Download_CHIRPSv3_Daily ()
 (wass2s.was_download.WAS_Download
 method), 44
 WAS_Download_CHIRPSv3_Seasonal ()
 (wass2s.was_download.WAS_Download
 method), 45
 WAS_Download_ERA5Land ()
 (wass2s.was_download.WAS_Download
 method), 43
 WAS_Download_ERA5Land_daily ()
 (wass2s.was_download.WAS_Download
 method), 43
 WAS_Download_Models ()
 (wass2s.was_download.WAS_Download
 method), 40
 WAS_Download_Models_Daily ()
 (wass2s.was_download.WAS_Download
 method), 42
 WAS_Download_Reanalysis ()
 (wass2s.was_download.WAS_Download
 method), 43
 WAS_Download_Reanalysis_ ()
 (wass2s.was_download.WAS_Download
 method), 43
 WAS_Download_TAMSAT_Daily ()
 (wass2s.was_download.WAS_Download
 method), 46
 WAS_Download_TAMSAT_Seasonal ()
 (wass2s.was_download.WAS_Download
 method), 45
 WAS_ElasticNet_Model (class
 wass2s.was_linear_models), 108
 WAS_EOF (class in wass2s.was_eof), 91
 WAS_EOF_KCCA (class in wass2s.was_cca_2), 130
 WAS_ExtraTrees_Model (class
 wass2s.was_machine_learning), 150
 WAS_GradientBoosting_Model (class
 wass2s.was_machine_learning), 150
 WAS_HeatWaveIndices (class
 wass2s.was_compute_predictand), 82
 WAS_HurdleNB_Model (class
 wass2s.was_machine_learning), 149
 WAS_KCCA (class in wass2s.was_cca_2), 130
 WAS_KernelCCA_base (class in wass2s.was_cca_2),
 129
 WAS_KGCCA (class in wass2s.was_cca_2), 130
 WAS_KNN_Model (class in
 wass2s.was_machine_learning), 151
 WAS_Lasso_Model (class in
 wass2s.was_linear_models), 103
 WAS_LassoLars_Model (class in
 wass2s.was_linear_models), 113
 WAS_LightGBM_Model (class in
 wass2s.was_machine_learning), 151
 WAS_LinearRegression_Model (class in
 wass2s.was_linear_models), 94
 WAS_LogisticRegression_Model (class in
 wass2s.was_machine_learning), 132
 WAS_MARS_Model (class in
 wass2s.was_machine_learning), 144
 WAS_Merging (class in wass2s.was_merge_predictand),
 87
 WAS_Merging_Extended (class in
 wass2s.was_merge_predictand), 89
 WAS_Min2009_ProbWeighted (class in
 wass2s.was_mme), 181
 WAS_mme_ELR (class in wass2s.was_mme), 185
 WAS_mme_ExpertKnowledge (class in
 wass2s.was_mme), 205
 WAS_mme_FastBMA (class in wass2s.was_mme), 194
 WAS_mme_FullBMA (class in wass2s.was_mme), 195
 WAS_mme_gaussian_process (class in
 wass2s.was_mme), 188
 WAS_mme_hpELM (class in wass2s.was_mme), 202
 WAS_mme_logistic (class in wass2s.was_mme), 184
 WAS_mme_MLP (class in wass2s.was_mme), 203
 WAS_mme_MVA (class in wass2s.was_mme), 193
 WAS_mme_NGR_Gaussian (class in wass2s.was_mme),
 192
 WAS_mme_NGR_NonGaussian (class in
 wass2s.was_mme), 192
 WAS_mme_PCR (class in wass2s.was_mme), 204
 WAS_mme_RF (class in wass2s.was_mme), 196
 WAS_mme_Weighted (class in wass2s.was_mme), 177
 WAS_mme_xcELM (class in wass2s.was_mme), 188
 WAS_mme_xcELR (class in wass2s.was_mme), 183
 WAS_mme_XGBoosting (class in wass2s.was_mme),
 199
 WAS_NegativeBinomial_Model (class in
 wass2s.was_machine_learning), 149
 WAS_PCR (class in wass2s.was_pcr), 119
 WAS_PoissonRegression (class in
 wass2s.was_machine_learning), 139
 WAS_PolynomialRegression (class in
 wass2s.was_machine_learning), 134
 WAS_PrecipIndices (class in
 wass2s.was_compute_predictand), 80
 WAS_ProbWeighted (class in wass2s.was_mme), 180
 WAS_Qmap (class in wass2s.was_bias_correction), 84
 WAS_RandomForest_Model (class in
 wass2s.was_machine_learning), 150

WAS_Ridge_Model	(class	in	(wass2s.was_analog.WAS_Analog__	static
	wass2s.was_linear_models), 99		method), 158	
WAS_SVR_Model	(class	in	weibull_shape_solver()	
	wass2s.was_machine_learning), 151		(wass2s.was_cca.WAS_CCA	static method),
WAS_TempPercentileIndices	(class	in	123	
	wass2s.was_compute_predictand), 78		weibull_shape_solver()	
WAS_TransformData	(class	in	(wass2s.was_cca.WAS_CCA_base	static method),
	wass2s.was_transformdata), 61		125	
WAS_Verification	(class in wass2s.was_verification),		weibull_shape_solver()	
	160		(wass2s.was_cca.WAS_CCA_op	static method),
WAS_XGBoost_Model	(class	in	126	
	wass2s.was_machine_learning), 150		weibull_shape_solver()	
WAS_ZINB_Model	(class	in	(wass2s.was_cca.WAS_CCA_strict	static
	wass2s.was_machine_learning), 149		method), 128	
wass2s.ceac_agro			weibull_shape_solver()	
	module, 206		(wass2s.was_cca_2.WAS_KernelCCA_base	
wass2s.utils			static method), 130	
	module, 47		weibull_shape_solver()	
wass2s.was_analog			(wass2s.was_linear_models.WAS_ElasticNet_Model	
	module, 151		static method), 111	
wass2s.was_bias_correction			weibull_shape_solver()	
	module, 84		(wass2s.was_linear_models.WAS_Lasso_Model	
wass2s.was_cca			static method), 106	
	module, 122		weibull_shape_solver()	
wass2s.was_cca_2			(wass2s.was_linear_models.WAS_LassoLars_Model	
	module, 129		static method), 116	
wass2s.was_compute_predictand			weibull_shape_solver()	
	module, 62		(wass2s.was_linear_models.WAS_LinearRegression_Model	
wass2s.was_cross_validate			static method), 97	
	module, 92		weibull_shape_solver()	
wass2s.was_download			(wass2s.was_linear_models.WAS_Ridge_Model	
	module, 38		static method), 102	
wass2s.was_eof			weibull_shape_solver()	
	module, 91		(wass2s.was_machine_learning.WAS_MARS_Model	
wass2s.was_linear_models			static method), 146	
	module, 94		weibull_shape_solver()	
wass2s.was_machine_learning			(wass2s.was_machine_learning.WAS_PoissonRegression	
	module, 131		static method), 141	
wass2s.was_merge_predictand			weibull_shape_solver()	
	module, 87		(wass2s.was_machine_learning.WAS_PolynomialRegression	
wass2s.was_mme			static method), 137	
	module, 175		weibull_shape_solver()	
wass2s.was_pcr			(wass2s.was_mme.WAS_mme_hpELM	static
	module, 119		method), 203	
wass2s.was_seasonal_analysis			weibull_shape_solver()	
	module, 208		(wass2s.was_mme.WAS_mme_MLP	static
wass2s.was_transformdata			method), 204	
	module, 61		weibull_shape_solver()	
wass2s.was_verification			(wass2s.was_mme.WAS_mme_PCR	static
	module, 159		method), 205	
weibull_shape_solver()			weibull_shape_solver()	
	(wass2s.was_analog.WAS_Analog	static	(wass2s.was_mme.WAS_mme_RF	static method),
	method), 154		198	
weibull_shape_solver()			weibull_shape_solver()	

```

        (wass2s.was_mme.WAS_mme_Weighted    static
        method), 178
weibull_shape_solver()
        (wass2s.was_mme.WAS_mme_xcELM      static
        method), 189
weibull_shape_solver()
        (wass2s.was_mme.WAS_mme_XGBoosting
        static method), 200
weibull_shape_solver()
        (wass2s.was_verification.WAS_Verification
        static method), 170
weighted_gcm_forecasts()
        (wass2s.was_verification.WAS_Verification
        method), 172
WSDI (wass2s.was_compute_predictand.HeatWaveMetric
      attribute), 82
wsdi() (wass2s.was_compute_predictand.ETCCDIHeatWaveIndices
        static method), 83

```

X

```

xgboost_merging()
        (wass2s.was_merge_predictand.WAS_Merging_Extended
        method), 90

```